



WAIOTECH

CANONICAL DOCUMENTATION

Waiotech V1 Alpha

Product Authority, Engineering Authority, Experience Contracts, and canonical Reference for one wastewater plant.

PLANT MODEL

Functional Locations · Assets
· Process Units · Process
Streams · Measurement
Points

OPERATIONAL DOMAINS

Process · Maintenance · In-
ventory · Reliability

GENERATED SOURCE DIGEST

de07a344b11b96669fa1a549...

Contents

This cumulative edition follows the canonical numeric order of the maintained documentation. Page references point to the exact source-document boundary.

Documentation

Waiotech documentation 15

Product

Product.....	17
Product foundations	18
Tenant and operational scope	19
Domain and plant model.....	21
Product boundaries	24
Alpha Product Edition	27
Identity and access	31
Users, Memberships, and Teams	32
Permissions and Access Profiles	34
Responsibility and actors	39
Integration Principals	41
Plant model	43
Functional Locations.....	44
Assets and installation	47
Asset classification and Operational Criticality	50
Process Units and Streams	52
Measurement Points.....	55
External identifiers and Data Sources	58
Process Operations	61
Process Readings and Operational Observations	62
Process Routines and Rounds.....	65
Process Conditions	68
Operational Actions, outcomes, and responsibility	71
Cross-domain Process interactions.....	73
Maintenance	75
Maintenance classification and control	76
Accepted maintenance need.....	85
Work Requests	87
Procedures and Maintenance Plans.....	100

Governance

Governance.....	8
Authority and precedence.....	9
Documentation rules.....	12

Engineering

Engineering.....	233
Engineering foundations	234
Repository and source ownership	235
System architecture	240
Canonical engineering model.....	244
Data and definitions	248
Persistence and Tenant isolation	249
Revisions and publication.....	254
Plan target resolution.....	259
Consistency, concurrency, and correction....	265
Catalogues and generated reference data...	268
Actions and contracts.....	274
Commands and idempotency	275
Audit, events, and transactional outbox	280
API contracts	283
API queries, files, batches, and revisions	289
OpenAPI and generated clients.....	293
Security	295
Authorization architecture.....	296
Authorization enforcement.....	299
Authentication, sessions, and secrets	302
Background processing	305
Background work	306
Report and Notification processing	310
Application surfaces.....	313
Application surface architecture	314
Public Website engineering	316
Android installation and offline authority	322
Android synchronization and release	327

Scheduled Work	110
Work Order lifecycle and Readiness	117
Work Requirements and scope control	127
Completion, verification, and closeout	131
Findings	141
Inventory	144
Items, categories, and quantities	145
Stock Locations and Inventory Movements ..	150
Reservations and availability	156
Maintenance materials, tools, services, and cost	159
Process material usage	167
Replenishment Requests	169
Stocktaking	171
Reliability	175
Failure Events	176
Restoration, cause, and recurrence	179
Reliability analysis	182
Evidence and communications	184
Reports	185
Notifications	192
Attachments and evidence	196
Audit browsing	204
Policy and Decisions	206
Tenant Control Policy	207
Decisions, waivers, and separation of duties	213
Reason codes	220
Tenant lifecycle and portability	222
Tenant lifecycle	223
Tenant portability	226
Preservation and destruction	229

Experience

Experience	392
Product surfaces	393
Public Website	396
Tenant Dashboard and Platform Admin	399
Android Work App	402
Offline field packages	410

Documentation, help, and learning engineering	331
Browser application contracts	338
Browser interaction, accessibility, and localization	341
Files and artifacts	347
File ingestion and storage	348
File access, integrity, and lifecycle	353
Generated artifacts and file recovery	357
Delivery and operation	359
Deployment and migrations	360
Backup and recovery	366
Observability	370
Operational security	376
Testing and software release	380

Reference

Reference	441
Canonical terminology	442
Canonical product model	452
Lifecycle summary	458

Documentation and Help Center.....	415
Learning environment.....	417
Tenant Dashboard experience.....	422
Dashboard design conventions.....	433

Source manifest

Generated from 119 maintained Markdown sources. Source digest: [de07a344b11b96669-fa1a54952f97b4d197839be5e3dea69b20040756db498a4](#). Generated output is not itself authority.

1. 00-governance/000-index.md
2. 00-governance/010-authority-and-precedence.md
3. 00-governance/020-documentation-rules.md
4. 000-index.md
5. 10-product/000-index.md
6. 10-product/10-foundations/000-index.md
7. 10-product/10-foundations/010-tenant-and-operational-scope.md
8. 10-product/10-foundations/020-domain-and-plant-model.md
9. 10-product/10-foundations/030-product-boundaries.md
10. 10-product/10-foundations/040-alpha-product-edition.md
11. 10-product/20-identity-and-access/000-index.md
12. 10-product/20-identity-and-access/010-users-memberships-and-teams.md
13. 10-product/20-identity-and-access/020-permissions-and-access-profiles.md
14. 10-product/20-identity-and-access/030-responsibility-and-actors.md
15. 10-product/20-identity-and-access/040-integration-principals.md
16. 10-product/30-plant/000-index.md
17. 10-product/30-plant/010-functional-locations.md
18. 10-product/30-plant/020-assets-and-installation.md
19. 10-product/30-plant/030-asset-classification-and-criticality.md
20. 10-product/30-plant/040-process-units-and-streams.md
21. 10-product/30-plant/050-measurement-points.md
22. 10-product/30-plant/060-external-identifiers-and-data-sources.md
23. 10-product/35-process/000-index.md
24. 10-product/35-process/010-readings-and-observations.md
25. 10-product/35-process/015-process-routines-and-rounds.md
26. 10-product/35-process/020-process-conditions.md
27. 10-product/35-process/030-actions-outcomes-and-responsibility.md
28. 10-product/35-process/040-cross-domain-interactions.md
29. 10-product/40-maintenance/000-index.md
30. 10-product/40-maintenance/010-maintenance-classification-and-control.md
31. 10-product/40-maintenance/020-accepted-maintenance-need.md
32. 10-product/40-maintenance/030-work-requests.md
33. 10-product/40-maintenance/040-procedures-and-maintenance-plans.md
34. 10-product/40-maintenance/050-scheduled-work.md
35. 10-product/40-maintenance/060-work-order-lifecycle-and-readiness.md
36. 10-product/40-maintenance/070-work-requirements-and-scope.md
37. 10-product/40-maintenance/080-completion-verification-and-closeout.md
38. 10-product/40-maintenance/090-findings.md
39. 10-product/50-inventory/000-index.md
40. 10-product/50-inventory/010-items-categories-and-quantities.md
41. 10-product/50-inventory/020-stock-locations-and-movements.md
42. 10-product/50-inventory/030-reservations-and-availability.md
43. 10-product/50-inventory/040-maintenance-materials-tools-services-and-cost.md
44. 10-product/50-inventory/045-process-material-usage.md
45. 10-product/50-inventory/050-replenishment.md
46. 10-product/50-inventory/060-stocktaking.md
47. 10-product/55-reliability/000-index.md
48. 10-product/55-reliability/010-failure-events.md
49. 10-product/55-reliability/020-restoration-cause-and-recurrence.md
50. 10-product/55-reliability/030-reliability-analysis.md
51. 10-product/60-evidence-and-communications/000-index.md
52. 10-product/60-evidence-and-communications/010-reports.md
53. 10-product/60-evidence-and-communications/020-notifications.md
54. 10-product/60-evidence-and-communications/030-attachments-and-evidence.md
55. 10-product/60-evidence-and-communications/040-audit-browsing.md
56. 10-product/60-policy-and-decisions/000-index.md
57. 10-product/60-policy-and-decisions/010-tenant-control-policy.md
58. 10-product/60-policy-and-decisions/020-decisions-waivers-and-separation-of-duties.md
59. 10-product/60-policy-and-decisions/030-reason-codes.md
60. 10-product/70-tenant-lifecycle/000-index.md
61. 10-product/70-tenant-lifecycle/010-tenant-lifecycle.md
62. 10-product/70-tenant-lifecycle/020-tenant-portability.md
63. 10-product/70-tenant-lifecycle/030-preservation-and-destruction.md
64. 20-engineering/000-index.md
65. 20-engineering/10-foundations/000-index.md

- [66. 20-engineering/10-foundations/010-repository-and-source-ownership.md](#)
- [67. 20-engineering/10-foundations/020-system-architecture.md](#)
- [68. 20-engineering/10-foundations/030-canonical-engineering-model.md](#)
- [69. 20-engineering/20-data/000-index.md](#)
- [70. 20-engineering/20-data/010-persistence-and-tenant-isolation.md](#)
- [71. 20-engineering/20-data/020-revisions-and-publication.md](#)
- [72. 20-engineering/20-data/030-plan-target-resolution.md](#)
- [73. 20-engineering/20-data/040-consistency-concurrency-and-correction.md](#)
- [74. 20-engineering/20-data/050-catalogues-and-generated-reference-data.md](#)
- [75. 20-engineering/30-actions-and-contracts/000-index.md](#)
- [76. 20-engineering/30-actions-and-contracts/010-commands-and-idempotency.md](#)
- [77. 20-engineering/30-actions-and-contracts/020-audit-events-and-outbox.md](#)
- [78. 20-engineering/30-actions-and-contracts/030-api-contracts.md](#)
- [79. 20-engineering/30-actions-and-contracts/040-api-queries-files-and-batches.md](#)
- [80. 20-engineering/30-actions-and-contracts/050-openapi-and-generated-clients.md](#)
- [81. 20-engineering/40-security/000-index.md](#)
- [82. 20-engineering/40-security/010-authorization-architecture.md](#)
- [83. 20-engineering/40-security/020-authorization-enforcement.md](#)
- [84. 20-engineering/40-security/030-authentication-sessions-and-secrets.md](#)
- [85. 20-engineering/50-background-processing/000-index.md](#)
- [86. 20-engineering/50-background-processing/010-background-work.md](#)
- [87. 20-engineering/50-background-processing/020-report-and-notification-processing.md](#)
- [88. 20-engineering/60-applications/000-index.md](#)
- [89. 20-engineering/60-applications/010-application-surface-architecture.md](#)
- [90. 20-engineering/60-applications/020-public-website.md](#)
- [91. 20-engineering/60-applications/030-android-installation-and-offline-authority.md](#)
- [92. 20-engineering/60-applications/040-android-synchronization-and-release.md](#)
- [93. 20-engineering/60-applications/050-documentation-help-and-learning.md](#)
- [94. 20-engineering/60-applications/060-browser-application-contracts.md](#)
- [95. 20-engineering/60-applications/070-browser-interaction-accessibility-and-localization.md](#)
- [96. 20-engineering/70-files/000-index.md](#)
- [97. 20-engineering/70-files/010-file-ingestion-and-storage.md](#)
- [98. 20-engineering/70-files/020-file-access-integrity-and-lifecycle.md](#)
- [99. 20-engineering/70-files/030-generated-artifacts-and-recovery.md](#)
- [100. 20-engineering/80-delivery/000-index.md](#)
- [101. 20-engineering/80-delivery/010-deployment-and-migrations.md](#)
- [102. 20-engineering/80-delivery/020-backup-and-recovery.md](#)
- [103. 20-engineering/80-delivery/030-observability.md](#)
- [104. 20-engineering/80-delivery/040-operational-security.md](#)
- [105. 20-engineering/80-delivery/050-testing-and-release.md](#)
- [106. 30-experience/000-index.md](#)
- [107. 30-experience/010-product-surfaces.md](#)
- [108. 30-experience/020-public-website.md](#)
- [109. 30-experience/030-tenant-dashboard-and-platform-admin.md](#)
- [110. 30-experience/040-android-work-app.md](#)
- [111. 30-experience/050-offline-field-packages.md](#)
- [112. 30-experience/060-documentation-and-help.md](#)
- [113. 30-experience/070-learning-environment.md](#)
- [114. 30-experience/080-tenant-dashboard-experience.md](#)
- [115. 30-experience/090-dashboard-design-conventions.md](#)
- [116. 70-reference/000-index.md](#)
- [117. 70-reference/010-canonical-terminology.md](#)
- [118. 70-reference/020-canonical-product-model.md](#)
- [119. 70-reference/030-lifecycle-summary.md](#)

00-GOVERNANCE

Governance

Authority, precedence, and documentation rules.

3 source documents

Governance

Authority, precedence, writing, change, and generated-output rules for the documentation library.

Contents

- [Authority and precedence](#)
- [Documentation rules](#)

Authority and precedence

Product Authority defines Waiotech's business truth. Engineering Authority defines the technical guarantees required to preserve it.

Authority scopes

Product Authority owns:

- product concepts and canonical terminology;
- authoritative facts and relationships;
- lifecycles, actions, guards, and outcomes;
- responsibility, authority meaning, and separation of duties;
- required evidence and historical interpretation;
- cross-domain behavior and product boundaries;
- observable product obligations.

Engineering Authority owns:

- architecture and module boundaries;
- persistence, transactions, consistency, and recovery;
- commands, APIs, events, background work, and integrations;
- authentication, authorization enforcement, and operational security;
- application, file, deployment, observability, and testing standards.

Product documents do not prescribe code structure, HTTP shapes, database design, repository layout, or framework conventions. Engineering documents do not redefine product meaning or weaken product guarantees.

Concern ownership

Each material concern has one authoritative owner. Other documents may summarize the concern or coordinate its actions, but they must link to the owner instead of creating a competing definition.

Domain documents own their entities, facts, lifecycles, actions, guards, responsibility, Decisions, evidence, exceptions, and end-of-life behavior. Cross-domain workflows coordinate domain-owned actions without redefining them. Experience contracts define observable interaction behavior without granting authority absent from Product Authority.

Documentation classes

A document's numbered path establishes its class:

- `00-governance/` — authority and documentation rules;

- `10-product/` — Product Authority;
- `20-engineering/` — Engineering Authority;
- `30-experience/` — Experience Contracts;
- `70-reference/` — consolidated references;
- `90-generated/` — generated output.

Documents do not carry classification banners, document IDs, owner labels, or status panels. Numbering exists for stable ordering and navigation, not for display inside the content.

Precedence

Conflicts are resolved in this order:

1. Product Authority;
2. Engineering Authority;
3. accepted bounded architecture decisions within delegated scope;
4. Experience Contracts and operational standards within their delegated scope;
5. generated references derived from authoritative sources;
6. implementation;
7. tests and operational observations;
8. superseded or legacy guidance.

A lower-order source cannot weaken, broaden, or reinterpret a higher-order source. Existing implementation behavior does not become authoritative because it is old, widely used, or difficult to change.

Product meaning changes

A change to a product concept, ownership rule, lifecycle, action, evidence requirement, authority meaning, catalogue meaning, cross-domain consequence, Tenant rule, or user-facing obligation requires an explicit Product Authority change.

Engineering may identify ambiguity, contradiction, infeasibility, or missing meaning. It must not resolve those issues through schema shape, feature flags, framework behavior, configuration, compatibility aliases, or undocumented conventions.

Normative language

`Must` and `must not` state mandatory requirements and prohibitions. `May` states a permitted option. `Should` states the canonical rule where a mandatory form would be unnecessarily rigid; departure requires an explicit bounded decision or authority change.

Examples never override a rule. A list or example is either a required canonical minimum, a governed extension point, or non-normative illustration. Its meaning must be clear from the surrounding text.

Timeless authority

Authoritative documents state enduring contracts. They do not contain delivery status, implementation percentages, roadmap phases, release dates, task ownership, remediation tracking, or migration progress.

Runtime time semantics remain explicit through terms such as effective time, recorded time, occurrence time, cutoff, and evaluation time.

Conformance

A conforming implementation preserves Product Authority, Engineering Authority, accepted bounded technical decisions, generated-contract parity, and mandatory verification. Conformance is determined from observable guarantees, not directory names, framework labels, or implementation intent.

Implementation cannot override authority. A known dependency on non-conforming behavior is a defect to correct, not a reason to preserve competing product meaning.

Related documents

- [Documentation rules](#)
- [Repository and source ownership](#)
- [Canonical terminology](#)

Documentation rules

Waiotech documentation is a numbered, subject-oriented library. It defines Waiotech directly and can be combined into one deterministic Markdown document.

Numbering and order

Every maintained directory contains `000-index.md`. Every maintained document uses a zero-padded numeric prefix.

Numeric path order is canonical reading order. New documents use available numbering gaps instead of renaming unrelated files.

Example:

```
10-product/  
└─ 40-maintenance/  
    ├── 000-index.md  
    ├── 010-maintenance-classification-and-control.md  
    ├── 020-accepted-maintenance-need.md  
    ├── 030-work-requests.md  
    └─ 040-procedures-and-maintenance-plans.md
```

Subject-oriented writing

Documents begin with the subject itself. They do not reproduce Q&A history, deliberation, rejected alternatives, extraction labels, document metadata panels, or commentary about how the document should be read.

A document includes only sections needed by its subject. Operational aggregates, evidence records, configuration entities, projections, engineering mechanisms, and experience contracts use structures appropriate to their concerns rather than one rigid template.

Granularity

A separate document is appropriate when a subject has distinct ownership, lifecycle, security meaning, substantial rules, or a stable independent audience.

Closely related rules remain together when splitting them would produce shallow files, repeated introductions, or competing authority.

Definitions and fields

Product documents define product-significant facts only. A fact is product-significant when changing it can alter meaning, permitted actions, lifecycle outcomes, responsibility, authorization, Decisions, required evidence, historical interpretation, cross-domain behavior, or user-facing obligations.

Storage columns, framework properties, transport shapes, repository paths, and generated-client mechanics belong to Engineering Authority.

Lifecycles, actions, and workflows

Entity lifecycles and named actions live with the owning domain. Workflow documents coordinate those actions and may contain a concise linked summary, but they do not restate complete transition tables or introduce alternate states and guards.

Lifecycle, action, permission, responsibility, approval, and separation-of-duty matrices are generated or verified from their owners. Hand-maintained summaries cannot become a second authority source.

Revisions

Immutable revisions preserve exact historical meaning for governed definitions. Ordinary users interact with stable objects through edit, review, publish, discard, history, and restore operations.

Restoring historical content creates new unpublished changes. It does not edit, reactivate, or rewrite a prior published revision.

Cross-document navigation

A document may end with `Related documents` when direct links materially improve navigation. Links point to prerequisites, coordinated workflows, technical realizations, or experience contracts. The section is omitted when it adds no value.

Related links do not repeat the target document's rules.

Architecture decisions and deviations

Architecture decisions select bounded technical mechanisms delegated by Engineering Authority. They cannot redefine product meaning, create alternate authority paths, weaken mandatory guarantees, or authorize permanent unbounded exceptions.

A deviation is narrow, time-bounded, risk-assessed, approved, observable, and removable. Repeated renewal does not make a deviation permanent.

Generated output

`just docs` validates the maintained library and writes the cumulative Markdown file in canonical numeric order.

Generated output:

- is deterministic;
- contains a cover and table of contents;
- identifies every ordered source file;
- marks source-document boundaries;
- rewrites internal document links to cumulative anchors;
- excludes `90-generated/` from its source set;
- carries a generated-file warning;
- must not be edited manually.

`just verify-docs` fails when validation fails or generated output differs from the maintained sources.

Prohibited content

Maintained documentation must not contain:

- Q&A headings or `Answer` and `Decision` extraction labels;
- document IDs, classification banners, owner banners, or canonical-status panels;
- task markers, unresolved placeholders, or empty sections;
- delivery progress, roadmap, phase, or implementation-status language;
- obsolete terminology retained as an alternate canonical vocabulary;
- manually edited generated output;
- unresolved internal Markdown links.

A genuine unresolved product or engineering rule is written as `SPEC GAP` at the exact point where the missing authority blocks a complete statement.

Related documents

- [Authority and precedence](#)
- [Canonical terminology](#)

Waiotech documentation

Waiotech is the professional operating system for one wastewater plant. It connects a coherent Plant Model with Process Operations, Maintenance, Inventory, Reliability, evidence, and accountable plant decisions so staff can understand what is happening, act with confidence, and preserve what the plant learned.

This documentation is the canonical authority for Waiotech product meaning, engineering guarantees, and user experience.

Sections

- [Governance](#)
- [Product](#)
- [Engineering](#)
- [Experience](#)
- [Reference](#)

Product

Canonical product meaning, ownership, lifecycles, evidence, and boundaries.

Product

Canonical product meaning, domain ownership, lifecycles, actions, evidence, boundaries, and operational guarantees.

Sections

- [Product foundations](#)
- [Identity and access](#)
- [Plant model](#)
- [Process Operations](#)
- [Maintenance](#)
- [Inventory](#)
- [Reliability](#)
- [Evidence and communications](#)
- [Policy and Decisions](#)
- [Tenant lifecycle and portability](#)

Product foundations

The foundations define Waiotech's operating boundary, shared product model, external-system boundary, and complete Alpha Product Edition.

Contents

- Tenant and operational scope
- Domain and plant model
- Product boundaries
- Alpha Product Edition

Tenant and operational scope

A Tenant is one independently governed wastewater plant and Waiotech's highest boundary for operational data, configuration, security, administration, time interpretation, and isolation.

Use Tenant as the plant boundary

Waiotech must use Tenant as the only top-level operating scope.

A Tenant represents one wastewater plant or one independently governed wastewater operating installation. Waiotech does not model a subordinate Site concept inside a Tenant.

Every Tenant-owned product entity belongs to exactly one Tenant. Ordinary product relationships, authority, operational activity, process data, work, Inventory, reliability evidence, and configuration must not cross Tenant boundaries.

Tenant is not a commercial subscription, billing account, or legal-customer model. Commercial account concepts require separate Product Authority.

Distinguish operational and learning Tenants explicitly

Every Tenant has exactly one immutable purpose:

- `operational` ;
- `learning` .

An operational Tenant represents a real plant. A learning Tenant is a private synthetic plant used only for guided learning and practice.

An operational Tenant cannot become a learning Tenant, and a learning Tenant cannot become an operational Tenant. Operational records are not promoted into learning Tenants and learning records are never promoted into operational authority.

Learning Tenants are excluded from real external integrations, operational communication, cross-Tenant operational reporting, support queues that represent live plant work, and other externally consequential behavior. Guided learning may simulate those experiences without creating real consequences.

Keep one authoritative Tenant time zone

Tenant owns one authoritative operational time zone.

Business evidence must preserve an unambiguous instant. Operational dates, due states, schedules, occurrence windows, readings, observations, failures, and histories are interpreted and displayed using the Tenant time zone unless a user explicitly selects another display context.

A user display preference must not change authoritative operational time meaning.

Keep Tenant-wide governance explicit

Tenant remains the ordinary scope for:

- Membership and Team participation;
- Access Profiles and Permissions;
- Tenant Control Policy;
- plant configuration;
- Process, Maintenance, Inventory, and Reliability authority;
- operational Reports and Notifications;
- Tenant export, preservation, and destruction.

Plant structure does not create hidden security scopes. Functional Location, Process Unit, Asset, or Inventory relationships must not grant authority by themselves.

Related documents

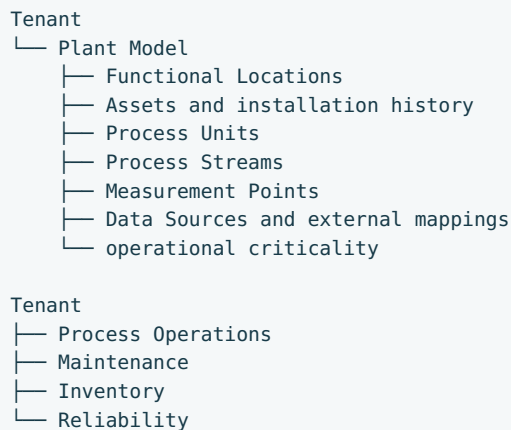
- [Domain and plant model](#)
- [Tenant lifecycle](#)
- [Learning environment](#)

Domain and plant model

WaioTech represents one plant through several explicit, connected truths rather than one universal hierarchy. Plant structure, process structure, operational work, material custody, and reliability remain distinct authorities that reference one another without duplicating ownership.

The high-level product model

The canonical high-level model is:



The Plant Model describes what the plant is and how its stable operational subjects relate. Process Operations describes what is happening and what Operations does about it. Maintenance describes accepted physical work. Inventory describes material custody. Reliability describes actual functional failure, restoration, cause assessment, recurrence, and derived learning.

Keep plant dimensions orthogonal

WaioTech must not force physical placement, physical object composition, process function, process flow, and measurement meaning into one tree.

The canonical questions are:

- **Functional Location:** at which stable installed position or maintainable place?
- **Asset:** which individually identified physical object or assembly?
- **Process Unit:** which stable process function?
- **Process Stream:** what logical process medium flows between process functions?
- **Measurement Point:** what operational quantity can be observed, and on which explicit plant subject?

These concepts may be related but are never aliases for one another.

Keep relationships explicit and product-owned

Waiotech must use explicit relationship meaning instead of a universal Plant Object or arbitrary relationship engine.

Canonical relationships include:

- Functional Location contains child Functional Location;
- Asset physically contains child Asset;
- Asset is installed at Functional Location;
- Process Unit contains child Process Unit;
- Process Stream flows from and/or to Process Unit;
- Functional Location serves Process Unit;
- Measurement Point observes one supported plant subject;
- Measurement Point may be implemented by an instrument Asset;
- a supported plant object may have governed external mappings.

A generic `entity_type + entity_id + relationship_type` product model must not become the source of business meaning.

Keep work, process, and reliability targets explicit

Maintenance Work Targets are Functional Locations or Assets.

Process Conditions concern Process Units or Process Streams and may reference related Assets, Functional Locations, Measurement Points, Readings, and evidence.

Failure Events concern one primary Functional Location or Asset and may reference process consequences and supporting Process evidence.

These target sets are deliberately different because each domain answers a different business question.

Keep human-facing identity separate from immutable identity

Every canonical Plant Model entity has immutable product identity.

Functional Locations, Assets, Process Units, Process Streams, and Measurement Points have a required human-readable name and may have a human-facing plant code. Names are not required to be unique. When a code is present, it is unique case-insensitively within the Tenant and that entity category.

The same familiar plant code may intentionally exist in different categories, for example a Functional Location position code and the physical Asset installed there. User interfaces must always preserve the entity type so this does not become ambiguous.

Changing a retained code is an explicit attributable recode action that preserves prior code history. A code that has identified a retained entity must not later be silently reused for a different entity in the same code scope.

External identifiers never replace canonical identity. Machine-readable labels and integration contracts should resolve immutable Waiotech identity or a governed external mapping rather than depending on mutable display names.

Related documents

- [Functional Locations](#)
- [Assets and installation](#)
- [Process Units and Streams](#)
- [Measurement Points](#)

Product boundaries

Waiotech is authoritative for the connected operational meaning, accountable decisions, work, material custody, selected process evidence, failure history, and outcomes required to operate and maintain one wastewater plant. Specialist external systems retain their specialist authority.

Waiotech is authoritative for

Waiotech owns the authoritative records required by its product domains, including:

- the canonical Plant Model used by Waiotech;
- human and machine Actors within Waiotech authority;
- Process Readings accepted into Waiotech, Operational Observations, Process Conditions, Operational Actions, and outcomes;
- Work Requests, Findings, Procedures, Maintenance Plans, Scheduled Work, Work Orders, Requirements, execution evidence, completion, and maintenance verification;
- Inventory Items, Stock Locations, Movements, Reservations, replenishment, Stocktaking, and material-use meaning where defined by the consuming domain;
- Failure Events, restoration history, Cause Assessments, recurrence relationships, and reliability interpretation;
- Attachments, Reports, Notifications, audit access, policy Decisions, and Tenant portability required by Product Authority.

Preserve external authority instead of copying it

Waiotech may preserve source references, accepted values, snapshots, files, identifiers, and evidence needed to explain a Waiotech decision or history. Preservation does not transfer authority over the complete external source system.

Waiotech must make source, provenance, effective time, and correction meaning explicit whenever external evidence can materially affect interpretation.

Process control and continuous telemetry

Waiotech is not SCADA and must not control the plant.

SCADA, PLC, DCS, or equivalent control systems remain authoritative for control commands, live control state, control logic, alarm configuration, and the operational guarantees required for process control.

Waiotech is not a general historian. A historian or source process system may remain authoritative for high-frequency continuous telemetry and long-retention raw time series.

Waiotech may accept selected Process Readings for configured Measurement Points when those readings are useful for plant operation, maintenance, reliability, evidence, or governed trends. Human entry is a normal first-class Process workflow. Machine ingestion supplements human operation and must use governed Data Sources and Integration Principals.

Waiotech must not require a plant to integrate SCADA or a historian before Process Operations is usable.

Laboratory systems

Waiotech may preserve or reference laboratory results needed as operational evidence, but it is not the authoritative laboratory information management system unless separate Product Authority defines that responsibility.

A laboratory result used in Waiotech must preserve its source and must not be silently reclassified as a Waiotech Process Reading unless the Measurement Point and source contract explicitly support that meaning.

Engineering drawings, GIS, and digital-twin visuals

Waiotech may display plant diagrams, simple P&ID-oriented views, maps, or imported references when they improve navigation and operational understanding.

Waiotech is not an engineering CAD or P&ID authoring system, a full GIS authority, a 3D modelling platform, or a process simulator.

Visuals are views over or references into authoritative product data. They must not create an alternate plant model.

Safety and qualification

Waiotech may record governed safety requirements, qualifications, permits, attestations, and evidence defined by Product Authority. It does not replace statutory safety management, legal certification, or regulatory authority that belongs outside Waiotech.

The interface must never imply that a software action itself makes unsafe work safe.

Procurement and financial systems

Inventory owns physical custody and availability inside Waiotech. Procurement, purchasing, invoicing, accounting, payroll, and general-ledger authority remain outside Waiotech unless separate Product Authority explicitly adopts them.

Waiotech may create replenishment demand and preserve procurement references without becoming the commercial system of record.

External systems and integrations

External integration is explicit and bounded.

An Integration Principal is a machine Actor owned by one Tenant. It receives only explicitly granted machine authority. In Alpha, the principal external-write use case is submitting Process Readings for configured Measurement Points through governed Data Sources.

Integration Principal authority must not imply ordinary human authority, arbitrary Maintenance actions, Inventory posting, or platform administration.

Platform operation

Platform Admin owns platform provisioning, Tenant lifecycle control, support and recovery functions explicitly defined by Product Authority. It is not an alternate path for ordinary plant work and cannot impersonate Tenant Users.

Android boundary

Android is a governed plant-work surface, not an alternate database.

It supports defined Process and Maintenance field workflows, evidence capture, bounded Inventory interactions, and offline authority where Product Authority and Experience Contracts explicitly permit them. Configuration, publication, broad administration, and unrestricted machine ingestion remain outside Android.

Related documents

- [Alpha Product Edition](#)
- [Responsibility and actors](#)
- [Integration Principals](#)
- [Measurement Points](#)
- [Process Readings and Observations](#)

Alpha Product Edition

Alpha is a complete coherent Waiotech product for one wastewater plant. Every required capability must preserve the same Product Authority, security model, plant identity, evidence semantics, and operational history across its supported application surfaces.

Application surfaces

Server and workers

Server owns canonical product authority, commands, queries, transactions, background obligations, generated API contracts, and integration ingress.

Tenant Dashboard

Tenant Dashboard is the primary professional workspace for plant configuration, Process Operations, Maintenance planning and control, Inventory, Reliability, Reports, and Tenant administration.

Platform Admin

Platform Admin is limited to platform-owned provisioning, Tenant lifecycle, support, preservation, recovery, and other explicitly platform-owned actions. It is not a plant-operations interface.

Android Work App

Android supports governed field execution for Maintenance and defined Process field workflows, including human Readings, Operational Observations, Process Condition interaction where authorized, evidence capture, and bounded Inventory use. Offline operation exists only through explicitly issued authority.

Public Website

Public Website explains Waiotech truthfully and remains outside authenticated plant authority.

Required Alpha capabilities

Alpha must include the following coherent capabilities.

Plant Model

- Functional Location hierarchy and lifecycle;
- physical Asset identity, classification, lifecycle, composition, installation, replacement, and history;
- Operational Criticality for stable Functional Locations;

- Process Unit hierarchy;
- Process Streams;
- explicit Functional-Location-to-Process-Unit service relationships;
- Measurement Points with quantity, canonical unit, supported subject, optional instrument Asset, and lifecycle;
- Data Sources, effective Measurement Source Mappings with at most one active machine source per Measurement Point, and governed external mappings;
- connected Plant Explorer with Functional Location hierarchy, Process Unit hierarchy, Process Stream flow, installation context, Measurement Points, and cross-domain navigation.

Process Operations

- human-entered Process Readings as a primary workflow;
- governed machine Process Readings for configured Measurement Points;
- Reading provenance, quality, unit, time, correction, and source evidence;
- Operational Observations;
- Process Routines with immutable published revisions, scheduled or ad-hoc Process Rounds, and low-friction human Reading/Observation collection;
- Process Conditions with explicit attention, accountable responsibility, and lifecycle;
- Operational Actions and outcome assessment;
- Process evidence, attachments, and operational history;
- focused trends and process context based on configured Measurement Points;
- Process-to-Maintenance escalation and linked maintenance response;
- Process evidence and consequence relationships to Reliability;
- Process Material Usage with explicit Inventory custody impact when tracked operational material is consumed.

Maintenance

- Work Requests and triage;
- accepted maintenance need represented by Work Orders;
- Findings;
- Procedures and immutable published Procedure Revisions;
- Maintenance Plans and immutable published Plan Revisions;
- calendar and Measurement Point reading triggers;
- Scheduled Work;
- Work Order planning, Requirements, Readiness, assignment, release, execution, completion, verification, closeout, cancellation, correction, and emergency-start behavior;
- Work Targets using Functional Locations or Assets;
- maintenance measurements and evidence whose meaning belongs to execution context;
- material requirements, Reservations, issue, return, and usage reconciliation;
- explicit relationships to Process Conditions and Failure Events.

Inventory

- Items and Item Categories;
- Stock Locations and balances;
- Movements, receiving, issue, return, transfer, adjustment, and disposal where defined;
- Reservations and availability;
- replenishment;
- Stocktaking;
- Maintenance material demand and usage;
- defined Process material usage;
- full quantity, unit, source, reason, custody, and audit semantics.

Reliability

- Failure Events against Functional Locations or Assets;
- operational failure and temporary-restoration state;
- authoritative failure, detection, restoration, and resolution timing;
- consequence;
- proportional investigation;
- immutable Cause Assessments;
- recurrence and duplicate relationships;
- links to Process Conditions, Readings, Findings, Work Requests, and Work Orders;
- derived reliability analytics from preserved source facts.

Identity, policy, evidence, and communications

- Users, Memberships, Teams, Permissions, Access Profiles, Access Grants, and Team Access Profile Assignments;
- Integration Principals with narrow machine authority;
- Tenant Control Policy and governed Decisions where defined;
- Attachments and evidence;
- Reports;
- Notifications;
- audit browsing;
- Tenant export, import, preservation, destruction, and recovery obligations defined by Product Authority;
- English and Persian experience parity.

Product exclusions

Alpha does not include:

- Site or multi-plant hierarchy inside one Tenant;
- SCADA or PLC control;
- a general-purpose historian or unrestricted raw telemetry archive;

- generic alarm-management replacement;
- generic dashboard or workflow builders;
- 3D digital-twin authoring;
- engineering CAD or P&ID authoring;
- full GIS authority;
- process simulation;
- unrestricted formula, rules, or anomaly-detection engines;
- full procurement, accounting, payroll, or ERP replacement;
- arbitrary machine authority;
- universal Plant Object, Event, Measurement, Condition, or Relationship engines.

These exclusions define product boundaries; they do not permit incomplete implementation of required Alpha capabilities.

Release completeness

An Alpha capability is complete only when its authoritative meaning, Server command and query behavior, authorization, audit, correction, evidence, generated contract, required Dashboard or Android experience, localization, loading and failure states, tests, export behavior, and operational recovery are mutually consistent.

A technical CRUD screen or partial endpoint does not satisfy a required capability.

Related documents

- [Domain and plant model](#)
- [Product boundaries](#)
- [Tenant Dashboard experience](#)

Identity and access

Identity, human authority, responsibility, and machine Actors inside one Tenant.

Contents

- [Users, Memberships, and Teams](#)
- [Permissions and Access Profiles](#)
- [Responsibility and actors](#)
- [Integration Principals](#)

Users, Memberships, and Teams

Users, Memberships, and Teams define attributable human participation and responsibility without creating alternate authority paths.

A User

A User is one platform-wide human identity. The User owns authentication and account state but does not by itself grant access to any Tenant.

A Membership

A Membership represents one User's participation in one Tenant. A User may have Memberships in several Tenants. Every ordinary Tenant action uses exactly one active Membership in the selected Tenant.

Membership is the exclusive human participation boundary. Waiotech does not use support sessions, impersonation, shared human identities, or alternate Tenant actor paths.

A Membership preserves Tenant, User, lifecycle, effective periods, reason, history, and action attribution. Ending a Membership removes ordinary future Tenant authority without deleting historical identity or evidence.

A Team

A Team is a stable Tenant-owned group used for operational responsibility and human authority assignment. Team Membership records which Memberships participate and during which effective period.

Access Profile authority is assigned to Teams through explicit, time-bounded Team Access Profile Assignments. Users and Memberships do not receive Access Profiles directly.

Human authorization

Human authorization follows:

```
User
-> active Membership in selected Tenant
-> effective Team Membership
-> active Team
-> effective Team Access Profile Assignment
-> published Access Profile Revision
-> Access Grants
-> Permission
-> action-specific product guards
```


Responsibility and authorization remain separate. Owning or being assigned work does not automatically grant Permission, and Permission does not automatically assign responsibility.

Browser onboarding boundary

User invitation, Membership establishment, credential setup, expiry, resend, and recovery require complete governed contracts. No application may invent those workflows or present local success before the Server accepts them.

Related documents

- [Permissions and Access Profiles](#)
- [Responsibility and actors](#)
- [Authorization architecture](#)

Permissions and Access Profiles

Human authorization follows one deterministic path through Team Membership, Team Access Profile Assignment, published Access Profile Revisions, Access Grants, and stable Permission codes.

Prohibit direct User-to-Access Profile and Membership-to-Access Profile relationships

Waiotech must prohibit direct User-to-Access Profile and Membership-to-Access Profile relationships.

Direct assignments would create a second human authorization path and make access harder to review and govern.

All human access must flow through Teams. A person who needs different access must join an appropriate Team or the Tenant must create a separate Team with the required Access Profiles.

Remove Role from normative vocabulary

Waiotech must remove Role from normative vocabulary.

Role is too ambiguous because it can mean a job title, organizational position, responsibility, or permission collection.

Use:

- **Access Profile** for a named collection of Access Grants;
- **Team position or job title** for organizational description;
- **Responsibility type** for operational responsibility.

Application logic must not check Access Profile names, Team names, or job titles.

A Permission

Waiotech must authorize protected actions through stable Permission codes, never through Team or Access Profile names.

A Permission is a stable code representing one protected product capability.

Examples include:

- `plant.asset.read ;`
- `plant.asset.create ;`
- `process.reading.record ;`
- `process.round.execute ;`
- `process.condition.resolve ;`

- `inventory.stock.read` ;
- `inventory.movement.issue` ;
- `maintenance.work_order.release` .

Permissions should be atomic enough to evaluate clearly and stable enough to remain meaningful across implementation changes.

Product modules define their protected Permissions. IAM maintains the catalogue and evaluates the authority granted through them.

An Access Grant

Waiotech must keep Access Grants atomic, allow-only, and Tenant-wide.

An Access Grant includes one Permission in an Access Profile Revision.

The canonical model is allow-only. An Access Grant does not contain deny rules, arbitrary conditions, priorities, expressions, or domain-specific subjects.

The effective Tenant comes from the acting Membership and Team context.

Permissions apply Tenant-wide within the effective Tenant

Permissions apply Tenant-wide within the effective Tenant. Generic fine-grained IAM scope is outside Waiotech Product Authority.

Generic scope designs such as resource-specific selectors, untyped free-form structures, or expression languages would couple IAM to every product domain and create inconsistent enforcement.

Requirements such as allowing access only to mechanical Assets affect related work, reporting, exports, classifications, and information disclosure. They require a complete scoped-authorization design rather than a generic field.

IAM should not reference Functional Locations, Assets, Process Units, Stock Locations, Work Orders, or other domain entities.

An Access Profile

Waiotech must use Access Profiles as reusable named collections of exact Permissions.

An Access Profile is a stable named authority policy containing an exact collection of Access Grants through its effective published revision.

Examples may include:

- Inventory Storekeeper;
- Inventory Supervisor;
- Maintenance Technician;
- Maintenance Planner;
- Plant Administrator;
- Process Operator;

- Process Supervisor;
- Read-only Auditor.

The name explains the intended use but grants no authority by itself. Authorization is determined only by the Access Grants in the effective revision.

Replace generic Access Assignment with Team Access Profile Assignment for human authorization

Waiotech must replace generic Access Assignment with Team Access Profile Assignment for human authorization.

Use a narrow **Team Access Profile Assignment**.

It binds:

- one Team;
- one Access Profile;
- effective start;
- optional effective end;
- assigned by;
- assignment reason;
- revocation evidence.

It must not contain arbitrary assignee types, domain subjects, or resource scopes.

Changes preserve history through expiration, revocation, or supersession rather than rewriting previous assignments.

Keep immutable published Access Profile Revisions with one effective revision at a time

Waiotech must keep immutable published Access Profile Revisions with one effective revision at a time.

Revisions preserve the exact Access Grants that were effective at a particular time without preventing immediate access changes.

An Access Profile is the stable object assigned to Teams. Its published revision contains the effective Access Grants.

When a new revision is published:

- it becomes effective according to its publication rule;
- all Teams assigned to the Access Profile receive the new grants;
- Team assignments do not need to be recreated;
- the previous revision remains immutable for audit and restoration.

This allows administrators to change access once while preserving a reliable history of what authority existed before the change.

Present Access Profile changes as one simple edit-and-publish workflow backed by immutable revisions

Waiotech must present Access Profile changes as one simple edit-and-publish workflow backed by immutable revisions.

The user edits the stable Access Profile, reviews the Permission changes, and publishes them.

Unpublished changes have no authorization effect.

When changes are published:

- Waiotech creates an immutable underlying Access Profile Revision;
- the new revision becomes effective according to the publication rule;
- the previous effective revision becomes superseded;
- all Teams assigned to the Access Profile receive the new effective grants;
- historical authority remains interpretable.

The product may show change history and effective times, but ordinary users do not manage revision numbers, branches, merges, or supersession actions.

Represent meaningful access differences through separate Access Profiles, not hidden Team exceptions

Waiotech must represent meaningful access differences through separate Access Profiles, not hidden Team exceptions.

Changing a shared Access Profile affects every Team assigned to it.

When one Team needs different access:

1. copy the existing Access Profile into a new profile;
2. change and publish the new profile;
3. assign the new profile to that Team;
4. remove the previous assignment when appropriate.

Team-specific exceptions must not be embedded inside a shared Access Profile.

Use templates and grouped Permission selection to simplify profile creation without changing the grant model

Waiotech must use templates and grouped Permission selection to simplify profile creation without changing the grant model.

Keep the underlying Access Grant model exact, but provide structured authoring assistance.

Waiotech should provide product-managed Access Profile templates such as Process Operator, Inventory Storekeeper, or Maintenance Planner. Creating from a template copies its Access Grants into a normal Tenant Access Profile draft.

The Permission catalogue should also support:

- grouping by module and capability;
- selecting or clearing a whole group;
- search;
- showing selected Permissions only;
- comparison against a template;
- clear descriptions of each Permission.

Templates and catalogue groups are authoring aids. They do not grant authority and are not alternative authorization concepts.

Support governed Permission dependencies with visible authoring assistance and publication validation

WaioTech must support governed Permission dependencies with visible authoring assistance and publication validation where an action cannot function safely without supporting Permissions.

For example, issuing stock may require visibility of Items, Stock Locations, and stock balances.

When an administrator selects a Permission, WaioTech should show and add its mandatory dependencies. Publishing must reject a profile that omits required dependencies.

Dependencies should remain narrow and explicit. They must not silently add large unrelated capability groups.

Related documents

- [Users, Memberships, and Teams](#)
- [Responsibility and external actors](#)
- [Authorization architecture](#)

Responsibility and actors

Waiotech distinguishes identity, authority, operational responsibility, and attribution. Human Users and machine Integration Principals are both Actors, but their authority paths and permitted responsibilities remain explicit.

Users and Memberships

A User is a person identity. A Membership places that User inside one Tenant.

Human operational actions require an active Membership in the effective Tenant and the Permissions and domain guards required by the action.

Teams and responsibility

A Team is the ordinary durable unit for operational ownership and responsibility.

Domains may define explicitly named responsibility relationships such as owner Team, responsible User, planner, verifier, stock custodian, Process Condition owner, or investigator. A relationship records business responsibility; it does not grant Permission by itself.

Team membership likewise does not make every Team responsibility an authorization rule. Authorization remains derived through the IAM grant model.

Actor attribution

Every consequential accepted action must preserve an attributable Actor.

Canonical Actor categories are:

- human User acting through an active Membership;
- Integration Principal acting through its machine authority;
- product-defined internal system Actor for an exact Waiotech-owned job;
- platform User acting through a platform-owned command.

The Actor category, effective Tenant, command or accepted event, time, reason where required, and causation must remain distinguishable in audit evidence.

A machine or system Actor must never be presented as though a human performed the action.

Keep responsibility separate from source provenance

A Process Reading may be entered by a User or submitted by an Integration Principal. Its recording Actor does not by itself establish operational responsibility for a resulting Process Condition.

A Process Condition has explicit accountable responsibility independent of who first recorded evidence.

Likewise, a Work Order requester, assignee, executor, verifier, and approving Actor may be different people or Teams according to their owning Maintenance rules.

Platform operation

Platform authority remains separate from Tenant authority.

A platform User may perform only explicitly platform-owned commands. Platform support cannot silently become an ordinary Tenant Actor and cannot impersonate a Tenant User.

Separation of duties

Where Product Authority requires independent approval, verification, or review, the action must evaluate the actual Actor and relevant prior Actors. Team membership, machine identity, or platform authority must not bypass separation-of-duty rules.

Related documents

- [Users, Memberships, and Teams](#)
- [Permissions and Access Profiles](#)
- [Integration Principals](#)

Integration Principals

An Integration Principal is a Tenant-owned machine Actor used for narrowly authorized external automation. It supplements human operation and never inherits ordinary User authority.

An Integration Principal

An Integration Principal has:

- immutable identity;
- Tenant;
- name and operational description;
- active or revoked state;
- explicit machine Permissions;
- attributable credential history managed by Engineering Authority;
- optional association with one or more governed Data Sources;
- creation, revocation, and correction evidence.

Integration Principal identity is not a User, Membership, Team, service-account impersonation, or platform User.

Keep machine authority narrow

Machine authority must be allow-only and explicit.

Alpha must support the machine authority required to submit Process Readings for configured Measurement Points through authorized Data Sources. Additional machine-write capabilities require explicit Product Authority in the owning domain.

An Integration Principal must not acquire Maintenance, Inventory, Process Condition, Reliability, or Tenant-administration authority merely because it can submit Readings.

Bind submitted evidence to source authority

A machine-submitted Process Reading must identify the Integration Principal and the Data Source authority under which the Reading was accepted.

The principal may submit only for Measurement Points whose active Measurement Source Mapping permits that Data Source and whose lifecycle permits new Readings.

Source authorization is re-evaluated for every accepted submission. A previously valid credential or mapping must not grant authority after revocation or retirement.

Preserve machine attribution and revocation history

Revocation prevents future machine actions but does not rewrite previously accepted evidence.

Historical Readings continue to identify the Integration Principal and Data Source that submitted them.

Credential replacement, secret rotation, and transport authentication mechanics belong to Engineering Authority and must preserve this product identity.

Related documents

- [Responsibility and actors](#)
- [External identifiers and Data Sources](#)
- [Process Readings and Observations](#)

Plant model

The Plant Model is Waiotech's canonical structural understanding of one Tenant plant. It separates stable installed positions, physical Assets, process function, process flow, and measurement meaning while connecting them explicitly.

Contents

- [Functional Locations](#)
- [Assets and installation](#)
- [Asset classification and Operational Criticality](#)
- [Process Units and Streams](#)
- [Measurement Points](#)
- [External identifiers and Data Sources](#)

Functional Locations

A Functional Location is a stable installed position, structure, or maintainable place in the plant whose identity may continue while physical Assets are installed, removed, or replaced.

A Functional Location

A Functional Location represents where installed plant responsibility exists.

Examples include:

- a building, room, gallery, basin, or fixed structure that needs independent history;
- a pump position such as `P-101` ;
- an MCC position;
- an equipment bay;
- a maintainable fixed plant place.

A Functional Location is not a physical Asset and is not a Process Unit. It may correspond closely to a physical or process concept in ordinary plant language, but Waiotech keeps the identities separate because their histories and relationships differ.

Use one recursive hierarchy without fixed levels

A Functional Location may have one parent Functional Location in the same Tenant. The hierarchy has no fixed level taxonomy and must not contain cycles.

The hierarchy expresses stable installed containment only. It must not be used to express Asset assembly, Process Unit containment, process flow, Team authority, or Inventory custody.

A Tenant may have several root Functional Locations.

Keep Functional Location identity stable

A Functional Location has immutable identity, required name, and optional human-facing plant code according to the Plant Model code contract.

Its identity survives ordinary changes to installed physical Assets. A pump replacement at `P-101` must not require replacing the Functional Location identity `P-101`.

Name, code, and hierarchy corrections are governed changes. A parent change must preserve effective hierarchy history when earlier Process, Maintenance, Inventory-reference, or Reliability evidence depends on the previous containment. A change that represents a genuinely different stable plant position requires retirement and a new Functional Location rather than identity reuse.

The Functional Location lifecycle

Functional Location uses:

active → retired

Active means the stable plant position or place remains available for ordinary current use.

Retired prevents new normal targeting and installation while preserving all history. Retirement must evaluate active installed Assets, open work, active Maintenance Plan coverage, Measurement Points, unresolved Failure Events, and other required relationships. Owning domains determine how those obligations are resolved or reassigned.

Retirement does not delete or rewrite historical work, failures, readings, installations, or evidence.

Functional Locations as Work Targets

A Maintenance Work Order may use one Functional Location as its primary Work Target when the work concerns the stable position or place rather than one specific physical Asset.

Examples include:

- building or area repair;
- cleaning or fixed infrastructure work;
- a pump position whose installed failed object is not yet known;
- inspection of the installed function at a stable position.

Before release or emergency start, a Work Order must have a governed primary Work Target according to Maintenance Product Authority.

Keep Stock Location separate

A Stock Location is an Inventory custody node and remains distinct from Functional Location.

A Stock Location may reference a Functional Location when its physical placement is useful, but Inventory balances and Movements must always use Stock Location identity.

Logical and mobile custody such as quarantine, goods in transit, or technician custody may have no Functional Location.

Keep authority Tenant-scoped

Functional Location hierarchy does not create security or policy inheritance.

Teams, Permissions, policy, and responsibility remain governed by their owning domains.

Related documents

- [Assets and installation](#)
- [Asset classification and Operational Criticality](#)
- [Work Order lifecycle and Readiness](#)

Assets and installation

An Asset is an individually identified physical object or assembly whose own lifecycle, maintenance, reliability, or installation history matters. Asset identity is separate from the stable Functional Location where the Asset may be installed.

Use one physical Asset concept without mandatory structural types

Waiotech must use one Asset concept.

Asset does not have mandatory structural types such as Unit, Equipment, or Component. What an Asset technically is belongs to governed Asset Classification. Where physical decomposition is useful, an Asset may contain child Assets.

Asset parentage means physical composition only.

It must not be used for:

- Functional Location containment;
- Process Unit hierarchy;
- Process Stream topology;
- organizational ownership;
- responsibility;
- Inventory custody.

Create Asset identity only when independent history matters

A physical item should be an Asset when Waiotech needs its own durable identity and history, for example because it is maintained, inspected, installed, removed, replaced, failed, traced, or reported independently.

A consumable bearing or gasket normally remains an Inventory Item when its individual serialized history is not needed. A serialized motor, pump, blower, valve, instrument, gearbox, generator, or replaceable assembly may be an Asset.

Physical composition

An Asset may have one effective parent Asset in the same Tenant when it is physically part of that parent assembly. The composition graph must not contain cycles and one Asset may have at most one effective physical parent at one instant.

Physical composition has no fixed depth and no mandatory category sequence.

Composition changes preserve effective history. Attaching, detaching, or moving a child Asset to another physical parent must not overwrite which assembly contained it when earlier work, failures, or evidence occurred.

A child Asset retains its own identity, lifecycle, history, and classification.

Asset installation

Installation is an explicit historical relationship between one root physical Asset and one Functional Location.

At any effective instant, an Asset with no effective physical parent may be directly installed at no more than one Functional Location. A Functional Location may contain several directly installed root Assets when the stable position legitimately contains several assemblies or objects.

An Asset that has an effective physical parent does not carry a competing direct installation. Its effective installed Functional Location is inherited through the nearest physically containing ancestor with a direct installation. This keeps one authoritative placement path for a composed assembly.

An installation relationship preserves:

- directly installed Asset;
- Functional Location;
- installation effective time;
- installing Actor;
- reason or source where required;
- removal effective time when ended;
- removing Actor and reason;
- relevant evidence.

Changing composition or installation must preserve enough effective history to resolve an Asset's installed Functional Location at a retained historical instant. Installation history must not be represented by overwriting one current-location field without history.

Replacement

Replacing a physical Asset does not replace the Functional Location.

The outgoing Asset is removed from the Functional Location and the incoming Asset is installed through explicit actions. Both Asset histories remain intact.

A direct Asset-to-Asset replacement relationship may be preserved when it adds useful traceability, but it must not substitute for installation history or allow the new Asset to inherit the old Asset identity.

The Asset lifecycle

Asset administrative lifecycle uses:


```
inactive → active  
active → inactive  
inactive / active → decommissioned
```

- **Inactive:** retained physical identity is not available for ordinary current operational use.
- **Active:** Asset is available for ordinary operational use, whether installed or held as a managed spare according to its context.
- **Decommissioned:** Asset is permanently retired from operational use.

Decommissioning is terminal and preserves all history.

Asset lifecycle is separate from installation and from temporary operating condition. An Asset may be active but uninstalled. An installed Asset may be stopped, isolated, degraded, or failed without changing its administrative lifecycle.

Do not create one universal Asset-condition status

Operational condition may be represented by Process evidence, Findings, Failure Events, Work Order evidence, and other domain-owned records.

Waiotech must not collapse operating, stopped, isolated, degraded, failed, unavailable, and similar dimensions into one mutable Asset status.

Asset codes and identity

Asset identity is immutable. Asset has a required human-readable name and may have a human-facing Asset code under the Plant Model code contract.

A Functional Location code and an Asset code are different namespaces and may intentionally resemble one another where the plant uses familiar tag conventions. The interface must preserve which identity is being shown.

Manufacturer serial number, barcode, ERP identifier, and similar source identities are not silently substituted for Asset identity or Asset code; they are preserved as governed technical facts or external mappings according to their owning contract.

Related documents

- [Functional Locations](#)
- [Asset classification and Operational Criticality](#)
- [Failure Events](#)

Asset classification and Operational Criticality

Asset Classification describes what a physical Asset is. Operational Criticality describes the consequence of losing the stable function represented by a Functional Location. The concepts must not be conflated.

Asset Classification

Waiotech must use governed Asset Classification to describe technical Asset meaning without creating mandatory structural Asset types.

Classification may represent equipment family, subtype, manufacturer-oriented technical category, or other governed taxonomy required by Maintenance, Inventory compatibility, Reporting, or Reliability.

Classification facts must be relational and historically interpretable. Free-text labels must not silently become authority for behavior that depends on classification.

Asset Classification does not determine physical parentage or Process Unit membership.

Keep classification governance explicit

Where a classification uses governed dimensions, allowed values, hierarchy, applicability, retirement, and required assignments must be defined by their catalogue owner.

Published Maintenance Plans or historical Failure Events must preserve canonical Asset identity and required snapshots rather than depending on later reinterpretation of mutable classification labels.

Operational Criticality belongs to the stable function

Operational Criticality represents the potential plant consequence of losing the function associated with a Functional Location.

Criticality belongs to Functional Location rather than the replaceable physical Asset because the operational consequence normally survives equipment replacement at the same stable position.

An installed Asset may display the Operational Criticality of its effective Functional Location as context. That context must not be rewritten as intrinsic Asset identity.

Keep criticality separate from current consequence and work risk

Operational Criticality is a relatively stable assessment of potential consequence.

It is distinct from:

- actual consequence recorded by a Failure Event;
- current Process attention;
- Work Order urgency;
- Work Order priority;
- execution risk;
- deferral consequence.

Criticality may influence planning, prioritization, investigation requirements, readiness, escalation, and analysis only where the owning rule explicitly uses it.

Preserve criticality history where it affects decisions

When a criticality value materially affects a governed decision, Waiotech must preserve the effective assessment or equivalent reproducible evidence used at that time.

Changing current criticality must not rewrite the meaning of earlier decisions.

Related documents

- [Functional Locations](#)
- [Assets and installation](#)
- [Maintenance classification and control](#)
- [Failure Events](#)

Process Units and Streams

Process Units describe stable wastewater-process functions. Process Streams describe directional logical flow between those functions. Together they provide process topology without turning physical plant hierarchy into process hierarchy or making Waiotech an engineering P&ID authoring system.

A Process Unit

A Process Unit is a stable functional boundary in which a defined part of the wastewater process occurs.

Examples include:

- preliminary treatment;
- screening;
- grit removal;
- primary clarification;
- aeration train;
- aeration basin;
- secondary clarification;
- sludge dewatering;
- disinfection.

A Process Unit is not a Functional Location and is not an Asset.

Use one recursive Process Unit hierarchy

A Process Unit has immutable identity, required name, and optional human-facing code under the Plant Model code contract.

A Process Unit may contain child Process Units in the same Tenant. The hierarchy must not contain cycles.

Parent changes are explicit governed changes and preserve effective hierarchy history when earlier operational evidence depends on the prior process decomposition. A materially different process function is represented by retiring the old Process Unit and creating a new identity rather than reusing the old identity.

The hierarchy expresses functional process decomposition only. Waiotech must not require fixed levels such as area, stage, train, unit, and step.

A plant may therefore model only the depth that is operationally useful while retaining one clear meaning for Process Unit.

The Process Unit lifecycle

Process Unit uses:

active → retired

Retirement prevents new normal Process activity and configuration against the Process Unit while preserving historical Conditions, Readings, Observations, Failure Event consequence links, Reports, and other evidence.

Retirement must evaluate active child Process Units, Process Streams, Measurement Points, and open Process Conditions.

A Process Stream

A Process Stream is a stable logical flow of process medium between Process Units or between a Process Unit and the plant boundary.

Examples include:

- influent wastewater;
- effluent;
- return activated sludge;
- waste activated sludge;
- internal recycle;
- process air;
- chemical feed;
- service water;
- bypass flow.

A Process Stream is not a pipe, channel, valve, or other physical object. Physical infrastructure is represented through Functional Locations and Assets when Waiotech needs that physical history.

Represent direction explicitly

A Process Stream records direction.

A Stream may have:

- an upstream Process Unit and downstream Process Unit;
- only a downstream Process Unit when it enters from outside the modelled plant boundary;
- only an upstream Process Unit when it leaves the modelled plant boundary.

At least one Process Unit endpoint is required. Branching and merging are represented by several explicit Streams rather than arbitrary graph expressions hidden inside one Stream.

The Process Stream lifecycle and identity

A Process Stream has immutable identity, required name, and optional human-facing code under the Plant Model code contract.

Process Stream uses:

active → retired

Retirement prevents new ordinary Process use while preserving historical Readings, Conditions, Failure consequence links, and topology evidence.

The process medium and directional endpoints define the Stream's stable operational meaning. Once the Stream has retained operational evidence, a materially different medium or endpoint path requires retirement and a new Process Stream rather than silently rewriting the earlier flow meaning. Name or code corrections remain governed identity-preserving changes.

Keep stream medium governed but simple

A Process Stream may use a governed medium classification where operational interpretation depends on it. The classification must not become a substitute for process topology or an unrestricted engineering fluid-property model.

Relate stable installed function to Process Units explicitly

A Functional Location may serve one or more Process Units when the stable installed function supports those process functions. One Process Unit may be served by several Functional Locations.

This relationship is independent of Functional Location containment and Process Unit containment. It must preserve effective history when changing the relationship would alter the interpretation of retained Process, Maintenance, or Reliability evidence.

A physical Asset normally acquires current process context through its effective installation at a Functional Location. Replacing the Asset therefore does not require rebuilding the stable process relationship.

Waiotech must not infer process service solely from physical containment or naming similarity.

Do not make the diagram authoritative

A P&ID-oriented or process-flow view may render Process Units, Streams, Measurement Points, Functional Locations, and Assets.

The view is a projection over canonical relationships. Editing a visual diagram must not become an alternate ungoverned source of plant identity or topology.

Related documents

- [Domain and plant model](#)
- [Measurement Points](#)
- [Process Conditions](#)

Measurement Points

A Measurement Point is the stable semantic identity of an operational quantity that Waiotech understands can be observed on one explicit plant subject. It separates plant meaning from individual readings, instruments, and external tag names.

A Measurement Point

A Measurement Point records:

- immutable identity;
- Tenant;
- required name and optional code under the Plant Model code contract;
- measured quantity;
- canonical unit;
- exactly one supported subject;
- optional instrument Asset;
- active or retired state;
- applicable manual-entry behavior;
- governed Measurement Source Mappings where machine submission is supported.

Supported subjects are exactly one of:

- Functional Location;
- Asset;
- Process Unit;
- Process Stream.

Adding another subject category requires explicit Product Authority.

Keep Measurement Point separate from Reading

Measurement Point answers what can be observed. Process Reading records an observed value.

Replacing an instrument Asset, changing a SCADA tag, or rotating an Integration Principal must not require changing Measurement Point identity when the operational meaning remains the same.

Use a canonical quantity and unit

Every Measurement Point has one governed measured quantity and one canonical unit.

A Reading submitted in a supported compatible unit may be normalized according to governed unit-conversion rules, but Waiotech must preserve the reported value and unit when they differ from the canonical representation.

An incompatible quantity or unit must be rejected rather than silently coerced.

Preserve semantic identity after operational use

Measurement Point identity means one quantity on one supported subject in one canonical unit context. After the first accepted Reading or publication of a Maintenance Plan Revision that binds the point, changing the subject or measured quantity requires retirement and a new Measurement Point.

A compatible canonical-unit correction may occur only through an explicit governed change that preserves how earlier reported and normalized values remain interpreted. A change that would alter the physical quantity or reinterpret historical values requires a new Measurement Point.

Instrument Asset and Measurement Source Mappings may change over time without changing Measurement Point identity because they describe how the same semantic quantity is observed. Their effective history preserves prior provenance.

Manual entry is first-class

A Measurement Point may allow human entry when plant staff legitimately observe or record the quantity manually.

The Process experience must not require an external Data Source for such a Measurement Point.

Where manual entry is prohibited for a specific Measurement Point because only a governed machine source is meaningful, that restriction must be explicit configuration and visible to Users.

Optional instrument Asset

A Measurement Point may identify the instrument Asset that physically measures the quantity.

The instrument relationship is optional because a Measurement Point may be manually observed, calculated externally, or measured by an instrument not modelled as a Waiotech Asset.

Replacing the instrument Asset does not replace the Measurement Point when the semantic observation remains unchanged.

The Measurement Point lifecycle

Measurement Point uses:

active → retired

Retirement prevents new ordinary Readings while preserving all historical Readings, Process evidence, Maintenance trigger evidence, Reports, and Measurement Source Mappings.

A retired Measurement Source Mapping must not be silently repurposed to represent a different operational quantity.

Keep Measurement Source Mapping explicit

Machine-submitted Readings require the one active effective Measurement Source Mapping from the Measurement Point to a governed Data Source. Alpha permits at most one active machine-ingress mapping for one Measurement Point at one effective instant.

The mapping preserves the external source key or tag, effective history, source-specific unit or quality semantics where required, and whether machine submission is enabled. It does not replace Measurement Point identity.

A general External Identifier Mapping to a Measurement Point is useful for lookup but does not by itself authorize or define machine Reading ingestion.

Related documents

- [External identifiers and Data Sources](#)
- [Process Readings and Observations](#)
- [Procedures and Maintenance Plans](#)

External identifiers and Data Sources

External mappings connect Waiotech's canonical plant identities to source-scoped external identities. Data Sources identify external systems that may provide accepted Process Readings or evidence without becoming Waiotech's canonical Plant Model.

A Data Source

A Data Source represents one governed external source or source namespace relevant to plant data.

It preserves:

- immutable identity;
- Tenant;
- name;
- source kind or description where needed;
- active or retired state;
- source namespace semantics;
- authorized Integration Principals where machine submission is enabled.

Examples may include one SCADA system, historian namespace, laboratory feed, or another explicit source.

Data Source is not an Integration Principal. The source identifies where data comes from; the Integration Principal identifies the machine Actor authorized to submit it.

Data Source lifecycle

Data Source uses:

active → retired

Retirement prevents new ordinary machine submissions or new active mappings while preserving historical source identity, mappings, credentials history required by security evidence, and accepted Reading provenance.

A source namespace must not be silently redefined after accepted evidence exists. If an external system or namespace changes meaning materially, Waiotech creates a new Data Source identity.

External identifier mappings

A governed external mapping associates a source-scoped external identifier with one supported canonical Plant Model object.

Supported mapping targets include:

- Functional Location;
- Asset;
- Process Unit;
- Process Stream;
- Measurement Point.

The mapping preserves source, external identifier, canonical target, effective start, retirement time where applicable, creating or correcting Actor, and reason where required. Mapping changes are effective-dated; they do not rewrite what an external identifier resolved to in retained historical evidence.

Within one Tenant and source namespace, one retained external identifier must not ambiguously identify several canonical targets at the same effective time.

Never replace canonical identity with external identity

External identifiers:

- do not become canonical Waiotech identity;
- must not silently reassign to another retained object;
- may be retired or corrected through explicit history-preserving actions;
- may resolve lookup and integration input to canonical identity where Product Authority permits it.

Published Maintenance Plan coverage and accepted operational records preserve canonical Waiotech target identity even when an external identifier was used for lookup.

Measurement Source Mapping

A Measurement Source Mapping is the governed relationship that allows one Data Source signal or key to provide machine Readings for one Measurement Point.

It preserves:

- Measurement Point;
- Data Source;
- external source key or tag;
- effective start and retirement;
- machine-ingress enablement;
- source-specific unit, quality, or sequence interpretation required by the accepted contract;
- creating or correcting Actor and reason where required.

A machine Reading submission must resolve through one active Measurement Source Mapping and an authorized Integration Principal.

A Measurement Point may preserve several historical source mappings, but Alpha permits at most one active machine-ingress Measurement Source Mapping at one effective instant. Changing machine source ends the prior effective mapping and establishes the new mapping without rewriting earlier Reading provenance.

Human Reading entry remains independent of the active machine source where the Measurement Point permits manual entry. WaioTech must not choose among competing active machine sources by arrival order.

The source key may change while Measurement Point identity remains stable. Source changes preserve historical mappings instead of rewriting prior Reading provenance.

Measurement Source Mapping is distinct from a general External Identifier Mapping. A lookup alias does not automatically grant machine-ingress semantics.

Preserve external evidence without becoming the external system

WaioTech may retain selected accepted values, source identifiers, snapshots, and source links required to explain operational history.

It does not claim authority over the complete source-system history merely because a mapping exists.

Related documents

- [Measurement Points](#)
- [Integration Principals](#)
- [Product boundaries](#)

Process Operations

Process Operations records selected plant operating evidence, meaningful operational conditions, accountable operational actions, and outcomes. It is independently useful to Operations and connects explicitly to Maintenance, Inventory, and Reliability without taking ownership of their facts.

Contents

- Process Readings and Operational Observations
- Process Routines and Rounds
- Process Conditions
- Operational Actions, outcomes, and responsibility
- Cross-domain Process interactions

Process Readings and Operational Observations

Process Readings preserve structured operational values at configured Measurement Points. Operational Observations preserve qualitative or contextual evidence. Human entry is a first-class Process workflow; machine submission supplements it through explicit source authority.

A Process Reading

A Process Reading is immutable operational evidence that one value was observed for one Measurement Point.

It preserves at least:

- Measurement Point;
- reported value and unit;
- canonical normalized value and unit where normalization is required;
- effective time when the value applies or was observed;
- recorded time when Waiotech accepted the Reading;
- recording Actor;
- provenance source;
- Data Source and source identity where machine-submitted;
- quality;
- correction relationship where applicable.

A Reading is not a Process Condition and does not by itself establish that action is required.

Human entry is primary and normal

An authorized User may record a Reading manually for a Measurement Point that permits human entry.

The interface should default known context such as Measurement Point, canonical unit, current time, Tenant, and User rather than asking the User to reconstruct it.

Manual Reading does not mean low-quality Reading. Its provenance remains human and its quality is explicit where uncertainty exists.

Machine submission uses the same business record

A machine-submitted value accepted through an Integration Principal creates the same Process Reading concept with different provenance.

Waiotech must not maintain separate human-reading and machine-reading truths for the same Measurement Point merely because the ingestion path differs.

The Integration Principal, Data Source, external source identity, and effective time remain attributable.

Reading quality

Canonical Reading quality values are:

- good ;
- uncertain ;
- bad ;
- unknown .

Quality describes confidence in the observed value, not whether the operational condition is desirable.

A bad Reading may remain valuable evidence but must not drive automated Maintenance trigger recognition. Uncertain or unknown quality may influence automated use only where the consuming Product Authority explicitly permits it.

Human review may still use lower-quality evidence with visible provenance.

Preserve reported and canonical units

When a supported Reading is entered in a compatible unit different from the Measurement Point canonical unit, Waiotech must preserve the original reported value and unit and the deterministic normalized value used by Waiotech.

Unit conversion must not silently reinterpret an incompatible quantity.

Correct Readings additively

Accepted Readings are not overwritten.

An authorized correction identifies the prior Reading, reason, correcting Actor, and correction time. A correction may invalidate the prior value and may establish a replacement Reading. The original evidence remains historically visible.

A corrected or invalidated Reading must not continue to drive derived current views or automated trigger behavior as though it remained accepted current evidence.

An Operational Observation

An Operational Observation is immutable qualitative or contextual evidence about plant operation.

It may record:

- narrative observation;
- effective time;

- recording Actor;
- exactly one primary operational subject: Process Unit, Process Stream, Functional Location, or Asset;
- related Process Units, Streams, Functional Locations, Assets, Measurement Points, or Readings where useful;
- attachments and evidence;
- correction relationship where applicable.

The primary subject uses one of the explicit supported subject types above, not an arbitrary untyped identifier or generic subject engine.

Keep Observation separate from Condition

An Observation records what was noticed. A Process Condition records an accepted operational situation that requires active handling, monitoring, or resolution.

Most Readings and Observations should never need a Process Condition.

Keep Process evidence separate from Maintenance execution measurements

A technician measurement recorded as part of a Work Order task may remain Maintenance execution evidence when its meaning belongs only to that task.

When the observed value is intentionally part of the plant's ongoing operational history, it should be recorded against a configured Measurement Point as a Process Reading and may also be referenced by Maintenance.

The product must not force all numeric evidence into one universal Measurement aggregate.

Related documents

- [Measurement Points](#)
- [Integration Principals](#)
- [Process Conditions](#)

Process Routines and Rounds

Process Routines define repeatable human operating rounds through immutable published revisions. Process Rounds are accountable executions of those routines. They make routine human data collection fast and structured without creating a generic form engine or duplicating Process Readings and Operational Observations.

A Process Routine

A Process Routine is the stable identity of a reusable operating round or log-sheet definition.

Process Routine uses:

```
inactive → active
active → paused → active
inactive / active / paused → retired
```

An active Routine requires one effective published Process Routine Revision. Only active Routines generate ordinary scheduled Rounds. Pause suspends new scheduled occurrences for the explicit pause interval without cancelling Rounds that already exist. Resume applies prospectively; paused intervals do not create retroactive Round obligations unless an explicit correction establishes that the Routine was paused incorrectly.

Retirement permanently prevents new ordinary Rounds while preserving all published revisions and historical Round evidence.

A Process Routine Revision

A Process Routine Revision is immutable published content defining the exact human collection method for future Rounds.

A revision may define:

- purpose and concise instructions;
- default responsible Team where applicable;
- ordered collection entries;
- optional calendar recurrence and execution window;
- evidence expectations;
- other explicitly defined human-round behavior.

Process Routine Revision uses:

```
draft → published → superseded
draft → discarded
```

Publishing creates a new immutable revision and supersedes the previously effective revision for subsequent Round creation. It never edits the definition used by an existing or historical Round.

Use only explicit collection entry types

Alpha Process Routine entries are exactly:

- **Reading entry** — record one Process Reading for one configured Measurement Point;
- **Observation entry** — record one Operational Observation against one explicit supported Process subject.

An entry defines its subject, instruction, required or optional meaning, and any reference guidance needed by the operator.

A Reading entry may present revision-owned reference guidance such as expected context or a compatible numeric reference range. Such guidance helps the User interpret the entry; it does not automatically create a Process Condition, Failure Event, Maintenance work, or machine alarm.

Waitech must not turn Process Routine entries into an unrestricted form, formula, script, or workflow builder.

A Process Round

A Process Round is the accountable execution record for one Process Routine Revision at one nominal occurrence or one authorized ad-hoc start.

It preserves:

- Process Routine and exact Revision;
- nominal occurrence time where scheduled;
- execution window where defined;
- responsible Team and executing User where applicable;
- start and completion times;
- required-entry completion;
- linked Readings and Operational Observations;
- evidence and cancellation reason where applicable.

Readings and Observations created during the Round remain canonical Process-owned evidence. The Round references them; it does not copy their values into a second authority.

Process Round lifecycle

Process Round uses:

```
open → in_progress → completed  
open → cancelled  
in_progress → cancelled
```

- **Open:** the Round exists and execution has not started.
- **In progress:** accountable human execution has started.
- **Completed:** every required entry and required evidence has been accepted.
- **Cancelled:** the Round will not be completed under this occurrence; reason and Actor are required.

Due, overdue, and approaching are derived conditions from the nominal occurrence, execution window, current state, and Tenant time. They are not lifecycle states.

Keep Round completion separate from operational resolution

Completing a Round means the required collection work was performed. It does not assert that the plant is healthy or that an abnormal condition was resolved.

A Reading or Observation recorded during a Round may support creation of a Process Condition, Work Request, Finding, or Failure Event according to the owning domain criteria. No such record is created automatically merely because a value is outside reference guidance.

Support scheduled and ad-hoc Rounds

A Process Routine Revision may define deterministic calendar recurrence or may be available only for authorized ad-hoc execution.

Scheduled occurrence identity must remain deterministic and duplicate-safe. Repeated generation, retry, device synchronization, or publication of a later Revision must not create several Rounds for one logical occurrence.

Preserve context and minimize entry effort

When a Round entry already identifies Measurement Point or operational subject, Waiotech must carry that context into the Reading or Observation interaction. Users must not re-select known Tenant, subject, unit, Routine, or Round context.

The Round experience should support rapid sequential entry while keeping each accepted Reading or Observation individually attributable and historically interpretable.

Related documents

- [Process Readings and Operational Observations](#)
- [Process Conditions](#)
- [Android Work App](#)

Process Conditions

A Process Condition is the authoritative Process record of a meaningful operational situation that requires active intervention, continued monitoring, or explicit resolution. It is an interpretation of plant evidence, not a copy of every alarm or out-of-range Reading.

A Process Condition

A Process Condition has one primary operational subject:

- Process Unit; or
- Process Stream.

It may reference supporting:

- Readings;
- Operational Observations;
- Measurement Points;
- Functional Locations;
- Assets;
- attachments and external evidence;
- related Process Conditions;
- Maintenance and Reliability records according to their owning contracts.

It preserves a concise operational description, effective onset or first-known time where available, creation time, Process attention, accountable owner Team, optional responsible User, current handling state, and resolution evidence.

Create a Condition only when operational meaning exists

A Process Condition should be established when plant staff determine that a situation requires accountable operational handling or monitoring.

A threshold breach, alarm, unusual Reading, or isolated Observation does not automatically become a Process Condition.

Waiotech Alpha does not use a generic alarm-to-condition or anomaly engine as Product Authority. A human may establish a Condition from machine or human evidence.

Process attention

Every active or monitoring Process Condition has one current attention level:

- `routine`;

- `elevated` ;
- `urgent` ;
- `critical` .

Attention describes the required operational response intensity and timeliness for this Condition. It does not assert Failure Event consequence, Maintenance urgency, Asset condition, or regulatory severity.

Changing attention is an explicit attributable action that preserves previous value, new value, Actor, time, and reason. Lowering attention requires the same accountability as increasing it.

A critical Process Condition requires immediate visible attention and accountable coordination, but it does not automatically create a Failure Event or Maintenance Work Order; those domains apply their own acceptance criteria.

Preserve uncertainty instead of forcing diagnosis

A Condition may begin before cause is known.

The Condition description must state the operational situation without requiring premature diagnosis. Cause or explanation may develop through later Observations, Operational Actions, Maintenance findings, or Failure Event investigation.

The Process Condition lifecycle

Process Condition uses:

```
active → monitoring → resolved
active → resolved
monitoring → active
resolved → active
```

- **Active:** immediate operational intervention, decision, or coordination remains required.
- **Monitoring:** no immediate intervention is currently required, but continued observation and an explicit review obligation remain.
- **Resolved:** no active Process obligation remains for the recorded condition.

Reopening a resolved Condition preserves the earlier resolution and records new evidence and reason.

Require accountable responsibility

An active or monitoring Process Condition must have one owner Team. It may also identify one responsible User.

Changing ownership is an explicit attributable action and preserves history.

Monitoring must include an explicit next review time or another Product Authority-defined review condition. Monitoring must not become indefinite silent deferral.

Resolution

Resolving a Process Condition requires:

- resolution time;
- resolving Actor;
- resolution summary;
- evidence sufficient to explain why the active Process obligation ended;
- disposition of required follow-up.

Resolution may be supported by new Readings, Observations, Operational Actions, Maintenance results, or another governed operational outcome.

A Maintenance Work Order being completed does not automatically resolve a Process Condition. Operations must establish the operational outcome when Process recovery matters.

Duplicate and related conditions

Waiotech should avoid several open Process Conditions representing the same operational situation.

When duplicate reporting occurs, one Condition remains primary and the duplicate evidence is linked or corrected without inflating operational queues. Distinct simultaneous conditions remain separate even when they share the same Process Unit.

Related documents

- [Process Readings and Operational Observations](#)
- [Operational Actions, outcomes, and responsibility](#)
- [Cross-domain Process interactions](#)

Operational Actions, outcomes, and responsibility

Operational Actions record what Operations deliberately did in response to plant context or a Process Condition. Outcome Assessments record what happened afterward. Both preserve chronological meaning without turning Process into Maintenance work or a generic workflow engine.

An Operational Action

An Operational Action is immutable evidence of an intentional operations action.

It has exactly one primary Process Unit or Process Stream context and may belong to one Process Condition when the action is part of handling that Condition. Routine intentional operating changes may be recorded without creating a Condition when no accountable abnormal-condition obligation exists.

It preserves:

- Actor;
- effective time;
- recorded time;
- concise description of what was changed or done;
- primary Process Unit or Process Stream;
- relevant Assets, Functional Locations, Measurement Points, or settings where useful;
- reason or objective;
- attachments or supporting evidence.

Examples include changing equipment duty, redistributing flow, changing a process setting, adjusting a dose, starting parallel equipment, changing temporary configuration, or increasing monitoring.

Operational Action does not become a Work Order merely because equipment is involved. Physical inspection, repair, replacement, and maintainable work belong to Maintenance.

An Outcome Assessment

An Outcome Assessment is immutable evidence of the observed result attributable to one Operational Action.

Canonical outcome values are:

- `improved` ;
- `no_meaningful_change` ;
- `worsened` ;
- `inconclusive` .

The assessment preserves the assessed Operational Action, Actor, time, rationale, and supporting Readings or Observations. An Operational Action may have several chronological Outcome Assessments when its effect develops over time.

An outcome is not the same as Process Condition resolution. A Condition may improve but remain active or monitoring.

Preserve chronology rather than editing the story

Operational Actions and Outcome Assessments are append-only evidence. A later action or assessment supplements prior history.

Corrections identify and explain earlier incorrect evidence instead of silently rewriting it.

Keep operational responsibility explicit

The Process Condition owner Team remains responsible for the active operational obligation until explicit reassignment or resolution.

Recording an Operational Action does not automatically transfer responsibility to the person who performed it.

Treat handover as continuity over authoritative records

WaioTech does not use a separate generic Handover aggregate in Alpha.

Shift and team handover experiences are projections over:

- open Process Conditions;
- owner Team and responsible User;
- current handling state;
- latest Observations and Readings;
- Operational Actions already taken;
- next review obligations;
- related Maintenance or Reliability work.

Handover must not duplicate or detach these facts into an alternate log.

Related documents

- [Process Conditions](#)
- [Tenant Dashboard experience](#)

Cross-domain Process interactions

Process Operations, Maintenance, Inventory, and Reliability have independent authority but must compose one operational story. Cross-domain relationships preserve source ownership and must not copy or silently convert one domain record into another.

Process and Maintenance

A Process Condition may support a new or existing Work Request when Operations determines that physical maintenance investigation or work is required.

The Process Condition remains Process authority. The Work Request and resulting Work Order are Maintenance authority.

Creating Maintenance work from Process must carry forward relevant context such as Process Unit, related Assets or Functional Locations, supporting Readings, Observations, and condition narrative without forcing the User to re-enter known facts.

Maintenance completion does not automatically resolve the Process Condition. When operating recovery matters, Operations records or confirms Process evidence and resolves the Condition explicitly.

Maintenance and Process evidence

A Work Order may reference Process Readings and Conditions as diagnosis, planning, execution, or verification context.

Maintenance may also request post-work Process monitoring.

Maintenance technical verification answers whether the maintenance work achieved its defined technical acceptance criteria. Process resolution answers whether the operational condition no longer requires Process handling. Both may be true at different times.

Process and Reliability

A Process Condition, Reading, or Observation may reveal or support a Failure Event when the functional-failure definition is met.

Failure Event remains Reliability authority. Process evidence remains linked source evidence and must not be rewritten as Failure Event fields when its original meaning matters.

A Failure Event may record Process consequence by reference to affected Process Units, Streams, Conditions, and relevant Readings.

Process and Inventory

Process Operations may consume Inventory-controlled material when an operational action uses stock such as treatment chemical or another governed plant supply.

Inventory owns physical custody and Movement. Process owns why the material was used and the operational action or condition to which the usage relates.

Process material usage must not directly mutate Inventory balances outside Inventory authority.

Do not introduce generic cross-domain ownership

Waiotech must not solve cross-domain composition with a generic Activity, Event, Subject, Relationship, or universal attribution entity that becomes business authority.

Each relationship must state the owning records and business meaning explicitly.

Related documents

- [Work Requests](#)
- [Failure Events](#)
- [Stock Locations and Movements](#)

Maintenance

Maintenance owns proposed and accepted physical work, planning, readiness, execution, Findings, maintenance evidence, technical completion, and verification. It references Plant, Process, Inventory, and Reliability authority without taking ownership of those domains.

Contents

- Maintenance classification and control
- Accepted maintenance need
- Work Requests
- Procedures and Maintenance Plans
- Scheduled Work
- Work Order lifecycle and Readiness
- Work Requirements and scope
- Completion, verification, and closeout
- Findings

Maintenance classification and control

Every accepted maintenance objective uses one Work Order and exactly one primary maintenance class: preventive, corrective, inspection, or improvement. Trigger, origin, urgency, priority, discipline, execution risk, deferral consequence, and planning route remain separate dimensions.

Keep one Work Order model and express differences through classification and conditional rules

Waiotech must keep one Work Order model and express differences through classification and conditional rules.

Waiotech has one canonical Work Order model for all accepted maintenance work.

Preventive, corrective, inspection, and improvement work may require different planning, evidence, approval, and completion rules, but they remain classifications of one Work Order rather than separate Work Order entities.

Use Work Order as the only representation of accepted maintenance work

Waiotech must use Work Order as the only representation of accepted maintenance work.

A Work Order represents maintenance work that Waiotech has accepted, authorized, or generated for execution.

It is the authoritative unit for:

- planning;
- responsibility;
- scheduling;
- readiness;
- execution;
- evidence;
- verification;
- costing;
- closeout.

There must be no accepted maintenance need outside a Work Order.

Keep the four maintenance classes as a closed product catalogue

Waiotech must keep the four maintenance classes as a closed product catalogue.

Waiotech uses exactly four canonical maintenance classes:

- preventive;
- corrective;
- inspection;
- improvement.

The classes are defined by the intent of the work, not by the physical activity being performed.

Preventive maintenance

Waiotech must classify work as preventive when its primary intent is failure prevention or preservation of required function.

Preventive maintenance preserves required function or reduces failure likelihood before a known failure or unacceptable condition requires restoration.

Preventive work may be time-based, usage-based, condition-informed, or generated by another governed maintenance rule.

Examples include planned lubrication, scheduled replacement, calibration, cleaning, and periodic servicing when their purpose is preventing deterioration or failure.

Corrective maintenance

Waiotech must classify work as corrective when its primary intent is restoring or correcting an existing unacceptable condition.

Corrective maintenance restores required function or corrects a known failure, defect, or unacceptable condition.

Corrective work may be planned or unplanned. It may originate from a Failure Event, Finding, Work Request, inspection result, or direct authorized creation.

Inspection maintenance

Waiotech must classify work as inspection when its primary intent is determining or verifying condition.

Inspection maintenance determines, measures, or verifies Asset condition, performance, or compliance without an already accepted corrective objective.

Inspection work may include:

- visual checks;
- measurements;
- tests;

- condition assessments;
- compliance checks;
- structured inspection rounds.

An inspection may discover Findings or Failure Events, but the inspection itself remains complete when the required examination and evidence have been properly recorded.

Improvement maintenance

Waiotech must classify work as improvement when its primary intent is an intentional enhancement rather than restoration.

Improvement maintenance changes design, capability, maintainability, reliability, safety, or performance beyond restoring the previously required condition.

An improvement may be triggered by repeated failures, operational experience, safety concerns, obsolescence, or performance opportunities.

Require exactly one maintenance class per Work Order

Waiotech must require exactly one maintenance class per Work Order.

Every Work Order has exactly one primary maintenance class.

Tasks and Procedures inside the Work Order may include different activity types, but they do not create additional maintenance classes.

When one proposed job contains independently managed objectives with different maintenance intent, it should be split into linked Work Orders.

Represent activity types separately from maintenance class

Waiotech must represent activity types separately from maintenance class.

They describe the activity, method, Procedure, or job type rather than the maintenance intent.

For example:

- scheduled lubrication may be preventive;
- leakage testing may be inspection;
- calibration after an identified error may be corrective;
- equipment redesign during overhaul may be improvement.

Allow ordinary class changes before execution and governed correction afterward

Waiotech must allow ordinary class changes while work is still being clarified or planned and require governed correction after execution begins.

Before first execution begins, an authorized Actor may correct the maintenance class.

After execution begins, changing the class requires an explicit correction with a reason because the original classification may already have affected planning, reporting, controls, and evidence.

Represent predictive information through trigger and origin, not maintenance class

Waiotech must represent predictive information through trigger and origin, not maintenance class.

Predictive describes an analytical trigger or detection method.

The resulting Work Order may be:

- inspection, when further assessment is required;
- preventive, when intervention is performed before unacceptable degradation;
- corrective, when a known unacceptable condition must be restored.

Represent emergency through urgency and emergency-start rules, not maintenance class

Waiotech must represent emergency through urgency and emergency-start rules, not maintenance class.

Emergency describes required response speed and the use of a governed exception path.

An emergency Work Order still has one of the four maintenance classes, commonly corrective or inspection.

Trigger

Waiotech must record the primary business trigger independently of maintenance class.

Trigger is the business condition that caused the maintenance need to require attention.

Examples include:

- scheduled interval reached;
- Measurement Point threshold reached;
- defect observed;
- failure occurred;
- predicted degradation;
- compliance obligation;
- improvement proposal.

Trigger explains why the need arose.

Origin

WaioTech must keep trigger and origin separate, and use Scheduled Work as the origin of Plan-generated Work Orders.

Origin identifies the authoritative source record or creation route through which the Work Order entered WaioTech.

Canonical origins include:

- Scheduled Work;
- Work Request;
- Failure Event;
- Finding;
- direct authorized creation;
- product-defined accepted-work integration contract.

For Plan-generated work, Scheduled Work is the direct Work Order origin. The related Maintenance Plan and Plan Revision remain traceable through that occurrence.

Origin explains where the Work Order came from.

Require clear trigger and origin before normal release

WaioTech must require clear trigger and origin before normal release.

Both facts must be complete and valid before the Work Order is released for execution.

During early drafting, either may be incomplete while the need is being clarified.

Trigger and origin may have different values. For example:

- maintenance class: corrective;
- trigger: predicted degradation;
- origin: external analytics event.

Urgency

WaioTech must use routine, urgent, and emergency as the closed urgency catalogue.

Urgency expresses how quickly the need requires response.

The canonical urgency values are:

- **Routine:** may proceed through normal planning and scheduling.
- **Urgent:** requires expedited evaluation or execution within a governed response expectation.
- **Emergency:** requires immediate action to control serious existing or imminent consequence.

Urgency does not determine whether work is ready or authorized.

Allow governed urgency changes with preserved evidence

WaioTech must allow governed urgency changes with preserved evidence.

Conditions may improve or deteriorate.

Urgency changes must preserve:

- previous urgency;
- new urgency;
- changed by;
- changed at;
- reason.

Lowering urgency requires the same accountability as increasing it because it may delay work.

Priority

WaioTech must keep priority separate from urgency and allow it to change with recorded reason.

Priority is the relative scheduling order among work that is otherwise eligible to proceed.

Priority may consider:

- urgency;
- deferral consequence;
- Operational Criticality;
- production opportunity;
- dependencies;
- regulatory timing;
- resource availability.

Priority does not override readiness, safety controls, or authorization.

Work-execution risk

WaioTech must use work-execution risk for controls governing how the work is performed.

Work-execution risk is the evaluated hazard and exposure created by performing the specific work under conditions at evaluation time.

It may control:

- planning depth;
- permits;
- isolations;
- approvals;
- execution controls;
- evidence requirements;
- verification;

- emergency exceptions.

Deferral consequence

Waiotech must keep deferral consequence separate from work-execution risk.

Deferral consequence is the expected consequence of not completing the work within the required period.

It may include effects on:

- safety;
- environment;
- production;
- quality;
- compliance;
- reliability;
- cost.

Deferral consequence informs urgency, scheduling, escalation, and temporary operating controls.

Operational Criticality

Waiotech must keep Operational Criticality as an input, not a substitute for job-specific evaluation.

Operational Criticality is the relatively stable potential plant consequence of losing the stable function represented by a Functional Location. When work targets an installed Asset, the effective Operational Criticality comes from that Asset's effective Functional Location context rather than from intrinsic Asset identity.

Criticality may inform:

- planning;
- prioritization;
- maintenance strategy;
- inspection frequency;
- risk evaluation;
- approval requirements.

It does not determine urgency, execution risk, or deferral consequence by itself.

Derive planned or unplanned status from the release and execution sequence

Waiotech must derive planned or unplanned status from the release and execution sequence.

Planning route is derived from what actually happened, not selected as a label by a planner.

Canonical values:

- **Planned:** the Work Order was normally released before first execution began.

- **Unplanned:** first execution began through an emergency or unreleased exception.

Planning route is recorded when first execution begins and must remain immutable after first execution begins.

A corrective Work Order may be planned, and preventive work may become unplanned if execution begins through an exception.

Treat emergency as urgency plus a governed exception-start path

WaioTech must treat emergency as urgency plus a governed exception-start path.

Emergency urgency alone does not bypass normal controls.

Emergency execution requires a named emergency-start action performed by an authorized Actor.

The action must record:

- why immediate execution is necessary;
- which normal readiness requirements are incomplete;
- temporary controls;
- responsible Team and coordinator where required;
- retrospective obligations;
- required review and evidence.

Bypassed requirements remain visible and must be resolved according to the emergency workflow.

Use one primary discipline with optional supporting disciplines, governed outside IAM

WaioTech must use one primary discipline with optional supporting disciplines, governed outside IAM.

Discipline supports technical routing, planning, and responsibility, but it does not define maintenance meaning or IAM authority.

A Work Order has:

- one primary discipline;
- optional supporting disciplines.

Examples include:

- mechanical;
- electrical;
- instrumentation;
- civil;
- operations support.

A discipline label does not grant Permissions.

Use one product-managed Maintenance discipline catalogue across all Tenants

Waiotech must use one product-managed Maintenance discipline catalogue across all Tenants.

Maintenance discipline is product-fixed.

Waiotech owns one stable discipline catalogue for every Tenant. Tenants may select applicable disciplines but cannot:

- create new discipline codes;
- rename canonical disciplines;
- change their meaning;
- retire them;
- attach custom product behavior to them.

Display labels may be localized without changing the canonical code or meaning.

Adding, changing, or retiring a discipline requires a governed Waiotech product change. A discipline code that has been used remains permanently interpretable in historical records.

Related documents

- [Accepted maintenance need](#)
- [Work Requests](#)
- [Work Order lifecycle and Readiness](#)

Accepted maintenance need

Accepted maintenance need is represented by Work Order authority. Work Request, Finding, Process Condition, Failure Event, inspection evidence, and direct authorized creation may all provide evidence or origin without becoming alternate accepted-work records.

Define acceptance through Work Order representation

A maintenance need becomes accepted work only when one or more Work Orders authoritatively represent the accepted maintenance response.

There must be no observable state in which a Work Request is marked accepted but its accepted maintenance need is not represented by a Work Order.

The same principle applies when accepted work originates from Process or Reliability: Process Condition and Failure Event remain authoritative source records, while Work Order owns accepted maintenance work.

Keep evidence in its owning context

Waiotech deliberately uses several evidence concepts with distinct meaning:

- Process Reading records a structured operational value at a Measurement Point;
- Operational Observation records qualitative Process evidence;
- Inspection Result records structured inspection evidence;
- Finding records a discovered maintainable condition requiring explicit Maintenance disposition;
- Failure Event records actual functional failure and consequence;
- Work Request records an unaccepted proposed maintenance need;
- Work Order records accepted maintenance work and execution evidence.

These records may reference one another but must not be silently converted or rewritten as one generic Observation or maintenance-need aggregate.

The minimum maintenance need model

```
Process Condition
  → no maintenance / monitor / Work Request / related Work Order

Inspection Result
  → no Finding / Finding

Finding
  → resolved here / no action / monitor / Work Request / Work Order / duplicate / invalid or corrected

Work Request
```

→ triage → rejected / cancelled / deferred / more information required / accepted into Work Order

Failure Event

→ zero or more Work Requests and one or more Work Orders as response requires

Direct authorized creation

→ Work Order

The distinctions are:

- evidence may establish that something happened without establishing accepted maintenance work;
- Work Request permits accountable triage before acceptance;
- Work Order is the only representation of accepted maintenance work;
- source records remain historically intact after Maintenance responds.

Related documents

- [Work Requests](#)
- [Findings](#)
- [Process Conditions](#)
- [Failure Events](#)

Work Requests

A Work Request preserves a reported maintenance concern before Waiotech accepts it for execution. Triage clarifies the report and records a Decision to accept, request information, reject, cancel, defer, or identify a duplicate.

Work Request purpose

Waiotech must use Work Request only for intake and triage of unaccepted maintenance needs.

A Work Request captures a reported candidate maintenance need that has not yet been accepted as work.

It supports:

- reporting;
- evidence submission;
- clarification;
- triage;
- classification;
- duplicate detection;
- acceptance or rejection.

A Work Request is not:

- a lightweight Work Order;
- authorization to execute work;
- an accepted maintenance backlog item;
- a scheduling or material-reservation record.

Keep Work Request as an optional intake route, not a mandatory precursor to Work Order

Waiotech must keep Work Request as an optional intake route, not a mandatory precursor to Work Order.

Authorized maintenance personnel, Scheduled Work occurrences, Failure Events, Findings, and product-defined accepted-work integrations may create Work Orders directly when the need is already understood and accepted.

Creating a Work Request first would add unnecessary workflow ceremony in those cases.

Direct Work Order creation must still record the applicable origin, trigger, target, classification, creator, accountable Team, and reason.

Preserve requester claims separately from maintenance assessment

Waiotech must preserve requester claims separately from maintenance assessment.

Waiotech must preserve the report as submitted, including:

- requester identity;
- reported time;
- original description;
- reported observations;
- submitted evidence;
- proposed Asset;
- proposed Functional Location or reported area;
- proposed urgency;
- proposed maintenance class;
- proposed discipline.

These are authoritative as reported facts, even when triage subsequently determines that some proposals were incorrect.

Triage must not overwrite the original report.

Distinguish reported facts from assessed maintenance facts

Waiotech must distinguish reported facts from assessed maintenance facts.

The following requester-supplied facts are proposals until triage confirms or corrects them:

- Asset;
- Functional Location;
- urgency;
- maintenance class;
- discipline;
- interpretation of the condition;
- requested action.

The observation, description, requester, reported time, and submitted evidence remain authoritative as reported.

Record triage conclusions without rewriting requester evidence

Waiotech must record triage conclusions without rewriting requester evidence.

Triage may determine:

- confirmed Work Target;
- assessed urgency;

- assessed maintenance class;
- assessed discipline;
- whether more information is required;
- whether the request duplicates an existing need;
- whether the need should become accepted work;
- whether the request should be rejected, cancelled, or deferred;
- which Work Orders represent the accepted need.

Triage conclusions must preserve the reviewer, time, rationale, and any difference from the original report.

Do not require Asset identification at submission

Waitech must not require Asset identification at submission.

The reporter may not know the Asset, may identify the wrong Asset, or may report an unidentified item or general symptom.

A proposed Asset may be added during submission or triage, but it is not required to submit the request.

Allow Functional Location-based Work Requests

Waitech must allow Functional Location-based Work Requests.

The need may genuinely concern a place rather than an Asset.

Examples include:

- building fabric;
- lighting;
- signage;
- access infrastructure;
- cleanup;
- area inspection.

The Functional Location remains proposed until triage confirms it as the correct target.

Permit unidentified Work Requests while requiring a governed Work Target before Work Order execution

Permit unidentified Work Requests while requiring a governed Work Target before Work Order execution.

The reporter must provide enough description, evidence, or reported area for triage to understand and investigate the need.

Free text is acceptable for intake. Before accepted work is released or started, the resulting Work Order must satisfy the authoritative Work Target rules.

Waitech must not force users to create false Assets or unnecessary Functional Locations merely to submit a request.

The Work Request lifecycle

Waiotech must require review before acceptance, rejection, or duplicate disposition and keep terminal outcomes immutable.

The lifecycle is:

```
submitted → under_review

under_review → more_information_required
more_information_required → under_review

under_review → deferred
deferred → under_review

under_review → accepted
under_review → rejected
under_review → duplicate

submitted / under_review /
more_information_required / deferred
→ cancelled
```

Terminal states are:

- accepted;
- rejected;
- duplicate;
- cancelled.

More information required and deferred are non-terminal states.

Make acceptance and resulting Work Order representation one completed product action

Waiotech must make acceptance and resulting Work Order representation one completed product action.

No intermediate accepted state is allowed.

A Work Request becomes accepted only when every accepted part of the maintenance need is already represented by at least one linked Work Order.

No observable product state may contain an accepted Work Request whose accepted need is not represented by Work Order.

Use `accept_work_request` rather than `accept_and_create_work_order`

Waiotech must use `accept_work_request` rather than `accept_and_create_work_order`.

Use a named `accept_work_request` action.

The action chooses exactly one resolution mode:

- create one or more new Work Orders; or
- link the request to one existing compatible Work Order.

Alpha does not combine existing and newly created Work Orders in one acceptance command. That restriction keeps resolution ownership and atomic evidence unambiguous while still allowing one reported need to be split into several newly created Work Orders.

The action records the Triage Decision and establishes all required Work Order links before the Work Request enters `accepted`.

Allow acceptance into existing Work Orders

Waitech must allow acceptance into existing Work Orders.

A reviewer may link the request to one existing Work Order when that Work Order already represents the need. The Work Order must belong to the same Tenant, have the same confirmed target, be visible to the Actor, be non-terminal, and not already contain the same Work Request link.

This avoids duplicate work when:

- several people report the same condition;
- the need was already accepted through another route;
- the request belongs to a larger intervention already being managed.

Allow one Work Request to be accepted into multiple Work Orders

Waitech must allow one Work Request to be accepted into multiple Work Orders.

One Work Request may contain several independently managed maintenance objectives requiring separate:

- Work Targets;
- disciplines;
- planning;
- ownership;
- execution;
- scheduling;
- acceptance;
- costing;
- closeout.

Each accepted part of the request must be represented by a linked Work Order.

Allow several Work Requests to support one Work Order without merging away their reporting history

Waiotech must allow several Work Requests to support one Work Order without merging away their reporting history.

Several reports may describe the same maintenance need or contribute additional evidence to one accepted intervention.

Each Work Request retains its own:

- requester;
- original description;
- reported time;
- evidence;
- triage outcome.

Replace `converted` with `accepted`

Waiotech must replace `converted` with `accepted`.

The term is inaccurate when the request is linked to existing work or split into several Work Orders.

Use `accepted` as the terminal state.

Accepted has a strict meaning: the accepted maintenance need is represented by linked Work Orders.

A Triage Decision

Waiotech must preserve explicit decision evidence for material triage outcomes.

A Triage Decision records an authorized conclusion about a Work Request.

It should preserve:

- outcome;
- decision maker;
- decision time;
- rationale;
- assessed urgency;
- assessed maintenance class;
- assessed discipline;
- confirmed target where applicable;
- Work Order links for acceptance;
- duplicate link for duplicate disposition.

Acceptance, rejection, and duplicate classification require a Triage Decision.

Reviewer-driven cancellation should also preserve a decision. Requester withdrawal may be recorded as a cancellation action.

Keep `more_information_required` as an active, accountable waiting state

Waiotech must keep `more_information_required` as an active, accountable waiting state.

This state means triage cannot yet determine the correct disposition and has requested specific additional information.

It must record:

- information requested;
- responsible person or party;
- requested by;
- requested time;
- optional response deadline;
- received response and responder.

When sufficient information is received, the request returns to `under_review`.

The reviewer may also resume review without the requested response when other evidence becomes sufficient.

Require explicit resolution of unanswered information requests

Waiotech must require explicit resolution of unanswered information requests.

The request must not remain indefinitely hidden in `more_information_required`.

An authorized reviewer may:

- proceed using available evidence;
- defer it with an explicit review condition;
- reject it because the need cannot be substantiated;
- cancel it when the report is no longer valid.

The chosen action must record its rationale.

Keep controlled deferral and prohibit indefinite deferral without a review trigger

Waiotech must keep controlled deferral and prohibit indefinite deferral without a review trigger.

Deferred means assessment of the unaccepted need has been intentionally postponed until a defined time or condition.

Deferral must record:

- reason;
- responsible Team or Membership;
- review date or explicit review condition;

- decided by;
- decision time.

The request must return to active review when the date or condition is met.

Do not use Work Request states to manage accepted backlog

Waitech must not use Work Request states to manage accepted backlog.

Work Request deferral postpones the decision about whether the reported need should become accepted work.

Once the need is accepted, scheduling, postponement, deferral consequence, and execution timing belong to the Work Order.

Keep duplicate terminal while preserving and linking the original report

Waitech must keep duplicate terminal while preserving and linking the original report.

A duplicate disposition means another record already represents substantially the same reported maintenance need.

The duplicate Work Request remains preserved with its original report and evidence.

It must link to:

- the primary Work Request while the need remains under triage; or
- the Work Order that already represents the accepted need.

Use maintenance-need equivalence, not simple target matching, for duplicate determination

Waitech must use maintenance-need equivalence, not simple target matching, for duplicate determination.

Two requests are duplicates when they report substantially the same condition or maintenance need, not merely because they concern the same Asset or Functional Location.

Triage should consider:

- symptom or condition;
- target;
- reported time;
- operational consequence;
- existing accepted work;
- whether separate action or acceptance outcomes are required.

Additional evidence from a duplicate request should remain available to the primary request or linked Work Order.

Keep rejection and cancellation as separate terminal outcomes

WaioTech must keep rejection and cancellation as separate terminal outcomes.

Rejection means an authorized reviewer evaluated the reported need and determined that it should not become maintenance work.

Examples include:

- condition is acceptable;
- no maintenance intervention is required;
- report is unsupported after review;
- the matter belongs outside maintenance.

Cancellation means the request was withdrawn, entered in error, superseded before assessment, or otherwise became invalid without a negative maintenance determination.

Examples include:

- requester withdraws an incorrect report;
- the wrong subject was reported;
- the condition no longer exists before review and no investigation is required.

Use the dedicated duplicate disposition

WaioTech must use the dedicated duplicate disposition.

Duplicate is a distinct outcome because the reported need may be valid even though another record already represents it.

Calling it rejected would incorrectly imply that maintenance determined no work was required.

Preserve rejection as terminal and represent materially new consideration through a linked Work Request

WaioTech must preserve rejection as terminal and represent materially new consideration through a linked Work Request.

A rejected Work Request remains terminal and its Triage Decision must not be rewritten.

When new evidence materially changes the situation, create a new linked Work Request that references the rejected request and preserves the reason for renewed triage.

Minor factual corrections may be appended through the normal evidence-correction mechanism, but they do not reopen the rejected lifecycle.

Do not use Work Request cancellation to alter accepted work

WaioTech must not use Work Request cancellation to alter accepted work.

After acceptance, the Work Request is a historical intake record linked to accepted Work Orders.

Any decision to cancel, stop, or replace the accepted work belongs to the Work Order lifecycle.

Keep Work Request triage responsibility separate from Work Order responsibility

WaioTech must keep Work Request triage responsibility separate from Work Order responsibility.

A Work Request may have one responsible triage Team and, where needed, an assigned reviewer.

Triage ownership is accountability for:

- review;
- clarification;
- escalation;
- timely disposition.

It does not grant authority and does not automatically become ownership of resulting Work Orders.

Each resulting Work Order receives responsibility according to its own authoritative rules.

Preserve the submitted report and represent subsequent changes as additions or explicit corrections

WaioTech must preserve the submitted report and represent subsequent changes as additions or explicit corrections.

The original submitted report must not be silently rewritten.

The requester may add:

- clarifications;
- corrections;
- additional evidence;
- responses to information requests.

Material corrections should preserve both the original statement and the corrected information.

Prohibit execution against Work Request

WaioTech must prohibit execution against Work Request.

A Work Request does not authorize:

- labor execution;

- material issue;
- scheduling as accepted work;
- operational verification;
- maintenance closeout.

The need must first be represented by a Work Order.

Allow Findings to use Work Request when triage is needed, but do not require it universally

Waitech must allow a Finding to use a Work Request when a discovered condition requires triage before becoming accepted work, but must not require that route universally.

A Finding may also:

- require no action;
- be monitored;
- link directly to existing work;
- support direct Work Order creation when an authorized workflow already accepts the need.

Do not force Failure Events through Work Request

Waitech must not force Failure Events through Work Request. A Work Request is used only when the required maintenance response still needs triage.

A Failure Event may instead create or link directly to Work Orders when the maintenance need is already accepted, especially during urgent or emergency response.

Process-origin Work Requests

A Process Condition may create or support a Work Request when Operations determines that physical maintenance investigation or work is needed.

The Work Request must preserve the originating Process Condition relationship and carry forward relevant plant context and evidence without copying ownership of the Condition. Triage may refine the proposed Functional Location, Asset, urgency, class, or discipline, but it must not rewrite the Process record.

A Process Condition may instead link directly to an existing Work Request or Work Order when the maintenance response already exists.

Keep intake requirements low while requiring enough information for accountable triage

Waitech must keep intake requirements low while requiring enough information for accountable triage.

Submission requires enough information to establish a meaningful report:

- requester or reporting Actor;

- reported time;
- description of the observed need or condition;
- Tenant context.

Asset, Functional Location, urgency, maintenance class, discipline, and supporting evidence may be optional at submission.

The minimum Work Request model

Waiotech must keep Work Request focused on truthful reporting, accountable triage, and explicit disposition.

The minimum Work Request model contains:

- immutable identity;
- Tenant;
- reporting Actor;
- reported time;
- original description and observations;
- submitted evidence;
- optional proposed Asset or Functional Location;
- optional proposed urgency, maintenance class, and discipline;
- lifecycle status;
- triage Team and optional reviewer;
- information requests and responses;
- deferral condition;
- Triage Decisions;
- linked Work Orders;
- duplicate relationship;
- preserved history.

A Work Request

Waiotech must keep Work Request as the intake concept for unaccepted maintenance needs.

A Work Request represents a proposed maintenance need that has not yet been accepted as executable work.

A Work Request may originate from:

- a person;
- a Finding;
- a Process Condition;
- a Failure Event;
- an external system;
- another governed workflow.

A Work Request may be rejected, marked duplicate, clarified, or accepted into one or more Work Orders.

Avoid duplicate work by allowing acceptance into existing or multiple Work Orders

Avoid duplicate work by allowing acceptance into existing or multiple Work Orders.

Acceptance may:

- create a new Work Order;
- link the Work Request to an existing Work Order;
- link several duplicate Work Requests to one Work Order;
- split one Work Request into several Work Orders when it contains independently managed objectives.

Every accepted part of the need must be represented by a linked Work Order.

Related documents

- [Accepted maintenance need](#)
- [Work Order lifecycle and Readiness](#)
- [Findings](#)

Procedures and Maintenance Plans

Procedures define reusable governed maintenance methods through immutable published revisions. Maintenance Plans define recurring maintenance intent, exact published coverage, deterministic triggers, timing, and generation defaults through immutable published revisions. Historical work must always remain interpretable from the exact method and recurring policy that governed it.

A Procedure

A Procedure is the stable identity of a reusable governed maintenance method.

Procedure lifecycle is:

active → retired

An active Procedure is available for ordinary planning only when it has an effective published Procedure Revision. A retired Procedure remains available for historical interpretation but cannot be newly applied.

Retirement does not alter Procedure Revisions already referenced by Maintenance Plans or Work Orders.

A Procedure Revision

A Procedure Revision contains one exact version of the governed maintenance method.

It may define:

- scope and purpose;
- ordered Task templates;
- instructions and cautions;
- prerequisites;
- Work Requirements;
- required discipline, competence, tool, or service context where defined;
- material expectations;
- measurement expectations;
- evidence expectations;
- acceptance criteria;
- completion requirements;
- verification expectations.

Procedure Revision lifecycle is:

draft → published → superseded
draft → discarded
published → withdrawn

Draft revisions are editable and have no operational effect. Published revisions are immutable.

Publishing a new revision supersedes the previously effective published revision for new application. It does not alter an earlier revision or the work that used it.

Preserve exact execution-method history

Procedure content may change while historical and active work must retain the exact method that applied.

Without immutable revisions, changing instructions, Requirements, measurements, evidence expectations, or acceptance criteria could rewrite the apparent basis of earlier or already-planned work.

Waiotech therefore preserves the exact Procedure Revision wherever the method is operationally applied.

Present Procedure changes through stable identity and publication

Users manage the stable Procedure and its proposed unpublished changes rather than manually managing revision numbers or version graphs.

Before publication, the experience must make the proposed method understandable and reviewable. Publication establishes a new immutable Procedure Revision.

Restoring earlier content creates new unpublished changes. It never edits or reactivates the old published revision.

Withdraw an unsafe or materially invalid published revision explicitly

An authorized Actor may withdraw a published Procedure Revision when it must no longer be used for new work because it is unsafe, materially incorrect, or otherwise invalid.

Withdrawal:

- prevents new application of that revision;
- preserves the revision and withdrawal evidence historically;
- does not rewrite completed or accepted execution evidence;
- requires affected unstarted or active Work Orders to be made visible for governed review;
- does not itself retire the stable Procedure.

A replacement method requires another reviewed and published Procedure Revision.

Snapshot exact method meaning into Work Orders

Applying a Procedure Revision to a Work Order creates Work Order-owned execution content derived from that exact revision.

The snapshot may include:

- Tasks;
- instructions;
- Requirements;
- expected evidence;
- measurements;
- acceptance criteria;
- verification requirements.

The Work Order preserves source Procedure, exact Procedure Revision, generated content, application time, and attributable application basis.

The snapshot is not a live mirror. Later Procedure publication, supersession, withdrawal, or retirement must not silently change accepted Work Order content.

Allow job-specific additions without changing the source Procedure

A planner may add job-specific content where authorized, including additional Tasks, contextual instructions, Requirements, evidence expectations, or acceptance criteria.

Job-specific additions remain distinguishable from content originating from the Procedure Revision.

Adding work-specific content does not modify the source Procedure or Procedure Revision.

Govern variation from Procedure-derived content

Controlled Procedure-derived content must not be changed through an unexplained ordinary edit.

Where Product Authority permits job-specific variation, the Work Order preserves:

- original Procedure-derived content;
- changed content;
- reason for deviation;
- authorized Actor;
- change time;
- approval or Decision where required.

The source Procedure Revision remains unchanged.

Keep maintenance execution measurements contextual

Procedure Tasks may require numeric measurements whose business meaning belongs to Maintenance execution, such as torque, clearance, alignment, insulation resistance, or inspection values.

Those values remain Maintenance execution evidence unless the action intentionally records an operational value against a configured Measurement Point as a Process Reading.

Waiotech does not create one universal Measurement aggregate merely because several domains record numeric evidence.

A Maintenance Plan

A Maintenance Plan is the stable identity of governed recurring maintenance intent.

It provides continuity across published revisions, activation, pause and resume, and later policy changes while keeping each historical occurrence tied to the exact rules that governed it.

Maintenance Plan lifecycle is:

```
inactive → active  
active → paused → active  
inactive / active / paused → retired
```

- **Inactive:** ordinary new occurrence recognition and generation are not performed.
- **Active:** the effective published Plan Revision may recognize occurrences and generate work.
- **Paused:** ordinary new generation is suspended while Plan identity, trigger history, already-recognized occurrences, and obligations reached during the pause remain governed.
- **Retired:** no new ordinary occurrences or work are generated after retirement; historical policy and occurrences remain preserved.

A Plan cannot become active without an effective published Plan Revision.

Pause and retirement are prospective controls. They never rewrite Scheduled Work or Work Orders that already exist.

A Maintenance Plan Revision

A Maintenance Plan Revision is one immutable published version of the recurring-maintenance policy.

It defines:

- maintenance class;
- exact Procedure Revision where applicable;
- exactly one trigger definition;
- exactly one target kind;
- target-authoring definition and exact resolved target set;
- timing window and generation horizon;
- primary and supporting discipline where applicable;

- default responsibility;
- urgency and Work Order defaults;
- occurrence-handling rules;
- other product-defined deterministic generation facts.

Maintenance Plan Revision lifecycle is:

draft → published → superseded
draft → discarded

Draft revisions have no occurrence-generation authority. Published revisions are immutable.

Preserve recurring-policy history through Plan Revisions

Changes to method, target coverage, trigger, frequency or threshold, timing tolerance, classification, responsibility, discipline, or Work Order defaults can change the maintenance obligation.

Historical Scheduled Work and Work Orders must therefore preserve the exact Plan Revision that established their basis.

Changing the stable Plan never rewrites the apparent rules that produced earlier occurrences.

Pin every Plan Revision to an exact Procedure Revision

Where a Maintenance Plan uses a Procedure, the published Plan Revision references one exact published Procedure Revision.

A Plan must not silently begin using a newer Procedure Revision merely because the Procedure has another effective revision.

Adopting a different Procedure Revision requires reviewed Plan changes and publication of a new Plan Revision.

A withdrawn Procedure Revision blocks new publication or application according to its withdrawal contract and requires affected existing work to be reviewed rather than silently mutated.

Make Plan publication a business-impact review

Users manage proposed Plan changes through the stable Maintenance Plan. Before publication, Waiotech must make the resulting method, trigger, timing, defaults, target coverage, trigger prerequisites, and prospective occurrence impact reviewable.

Users do not manually manage revision numbers, branches, or supersession.

Publication establishes the immutable Plan Revision only after the protected publication action verifies the reviewed meaning.

Use Functional Location or Asset Plan targets

One Plan Revision has exactly one target kind:

- Functional Location; or
- Asset.

A Plan may cover several resolved targets of that one kind when they genuinely share the same maintenance intent, method, trigger contract, timing, classification, discipline, and default responsibility.

Each resolved target receives its own Scheduled Work occurrence and normally its own Work Order. Multi-target Plan coverage must not combine independent physical work into one executable record merely because the targets share a Plan.

Functional Location targets are appropriate when recurring work concerns a stable installed position, fixed place, or function whose identity survives physical Asset replacement.

Asset targets are appropriate when recurring maintenance must follow the identified physical Asset itself.

A recurring programme requiring both target kinds uses separate Plans unless Product Authority explicitly defines another typed contract.

Support explicit target selection and governed selectors

Plan authoring may use:

- 1. Explicit selection:** specifically selected canonical Functional Locations or Assets of the Plan target kind.
- 2. Governed selector:** one product-defined typed selector used to resolve a proposed target set.

The selector is an authoring mechanism. It is never a continuously evaluated source of operational coverage.

Publication always preserves one exact resolved target set.

Keep selectors typed, narrow, and product-governed

Waiotech must not expose arbitrary field/operator/value expressions, SQL-like queries, scripts, executable formulas, user-defined metadata predicates, or generic expression builders for Plan target coverage.

Supported selector dimensions may include only Product Authority-defined plant facts.

Functional Location selectors

Supported dimensions may include:

- exact Functional Location branch;
- active lifecycle;
- Operational Criticality where defined;
- explicitly supported external-mapping criteria.

Asset selectors

Supported dimensions may include:

- Asset Classification;
- physical Asset subtree where meaningful;
- active lifecycle;
- current installation within a Functional Location branch where explicitly supported;
- explicitly supported external-mapping criteria.

Maintenance discipline, Team names, display text, or naming convention must not substitute for governed Plant facts.

Preview and publish exact target coverage

A target preview is non-authoritative and must make the exact proposed coverage understandable before publication.

The preview includes, as applicable:

- normalized authoring definition;
- evaluation time;
- target kind;
- proposed canonical targets;
- exclusions and reasons;
- ineligible targets;
- missing trigger prerequisites;
- target additions and removals relative to the effective published revision;
- resolved target-to-Measurement-Point bindings for Measurement Point triggers;
- count and deterministic reviewed-content digest.

Publication re-resolves selector-based coverage inside the protected publication action. If the reviewed meaning or digest changed, publication is rejected and a new review is required.

The published Plan Revision owns the immutable resolved target set by canonical Functional Location or Asset identity.

Preserve explainable omissions and all-or-nothing publication

A selector candidate excluded from publication because required facts are missing or invalid must have an explainable reason in the reviewed result.

Publication must not silently drop unresolved or invalid targets and continue with a different coverage set than the Actor reviewed.

A required target or trigger prerequisite that cannot be resolved blocks publication unless Product Authority explicitly defines a reviewed exclusion path.

Do not let changing Plant facts silently change published coverage

After publication, changes to Functional Location hierarchy, Asset composition, Asset Classification, Asset installation, lifecycle, Operational Criticality, Measurement Points, or external mappings do not add or remove published Plan targets.

Waiotech may derive target drift by comparing current selector resolution with published coverage. Drift is awareness and review input, not mutation.

Coverage changes require a new reviewed and published Plan Revision.

Keep current eligibility separate from historical coverage

A target remains part of the historical published Plan Revision even when it later becomes retired, decommissioned, uninstalled, otherwise ineligible, or unable to satisfy a trigger prerequisite.

At occurrence evaluation time, Waiotech evaluates current eligibility according to the Plan and Scheduled Work contracts.

An ineligible target creates a visible planning, reconciliation, or disposition obligation where required. It does not silently disappear from the historical Plan definition.

Existing Scheduled Work and Work Orders remain governed by their own records and actions.

Generate only from immutable published coverage

Scheduled Work recognition uses the exact targets and trigger bindings preserved by the effective published Plan Revision.

A selector may be re-evaluated to preview proposed changes, detect drift, or prepare unpublished Plan changes. It must not act as a live filter that silently changes generation coverage.

Support calendar and Measurement Point triggers

Alpha Maintenance Plans support exactly two deterministic trigger kinds:

- calendar trigger;
- Measurement Point trigger.

Analytical judgment, alarms, Process Conditions, predictive models, and anomaly detection are not arbitrary Plan trigger expressions. They may create or support Work Requests or direct accepted work only through explicit owning-domain actions.

Calendar trigger

A calendar trigger defines deterministic recurrence and timing facts governed by the Plan Revision.

The product preserves nominal occurrence identity separately from recognition, generation, due, execution, and completion times.

Waiotech must not expose an unrestricted cron expression as user-facing Product Authority.

Measurement Point trigger

A Measurement Point trigger defines a deterministic recurring maintenance obligation from accepted Process Readings.

The Plan Revision defines:

- measured quantity and canonical unit required by the trigger;
- trigger direction and threshold-sequence semantics;
- threshold increment or exact governed threshold rule;
- planning or generation horizon where supported;
- reset, rollover, replacement, or correction behavior when required by the measured quantity;
- accepted Reading-quality requirements;
- any other deterministic facts needed to prevent ambiguous occurrence recognition.

For every resolved Plan target, the published Plan Revision preserves exactly one compatible trigger Measurement Point.

That Measurement Point must observe the resolved target directly as its supported subject and must satisfy the trigger quantity and unit contract.

Authoring may resolve a trigger Measurement Point automatically only when exactly one eligible Measurement Point exists for the target. Zero or several candidates block publication until the author explicitly resolves the binding or changes the target coverage.

The reviewed target preview shows the resolved Measurement Point for every target, and the reviewed-content digest includes those bindings.

The published target-to-Measurement-Point binding is immutable with the Plan Revision. Later Measurement Point creation, retirement, instrument replacement, source remapping, Asset replacement, or other Plant Model change does not silently change the trigger source for existing published coverage.

Maintenance does not create a separate meter or usage-source registry. Measurement Point identity belongs to Plant; Process Reading evidence belongs to Process.

Human and machine Readings may satisfy the trigger when their provenance is permitted by the Plan rule and their quality is acceptable.

A **bad** Reading must never recognize a maintenance occurrence. **uncertain** or **unknown** quality may affect automated recognition only when the Plan trigger contract explicitly permits it.

Preserve cumulative, reset, rollover, and correction semantics explicitly

Measurement Point triggers must distinguish cumulative quantities from ordinary instantaneous values.

Where a counter can reset, roll over, be replaced, or be corrected, the trigger contract must define deterministic sequence ownership and occurrence recognition.

Waiotech must not infer a new maintenance threshold merely because a later value is lower than an earlier one.

When a Reading used for recognition is later corrected or invalidated, Scheduled Work reconciliation follows its own governed contract. Historical occurrence or accepted work must not be silently erased.

Keep exactly one trigger per Plan Revision

Combined trigger expressions are outside the Alpha contract.

A maintenance programme requiring different trigger logic uses separate Plans unless Product Authority explicitly introduces one typed combined-trigger contract.

Arbitrary logical expressions remain prohibited.

Preserve obligations through pause and retirement

Pausing a Plan does not erase already-recognized Scheduled Work or trigger evidence that becomes relevant during the pause.

On resume, Waiotech evaluates preserved trigger history according to the Scheduled Work contract so obligations reached during the pause receive explicit recognition or disposition rather than disappearing.

Retirement prevents new ordinary post-retirement occurrences while preserving all earlier Plan Revisions, Scheduled Work, Work Orders, trigger evidence, and publication history.

Plan publication does not rewrite existing work

Publishing a new Plan Revision affects subsequent occurrence recognition according to its effective rules.

Existing Scheduled Work and Work Orders preserve their original Plan Revision, Procedure Revision, target, trigger facts, and evidence unless an explicit governed reconciliation action changes a fact that Product Authority permits to be reconciled.

Definition publication itself never edits accepted work.

Related documents

- [Scheduled Work](#)
- [Measurement Points](#)
- [Process Readings and Operational Observations](#)
- [Revisions and publication](#)
- [Plan target resolution](#)

Scheduled Work

Scheduled Work is the durable identity of one recognized Maintenance Plan occurrence for one published target and one nominal trigger instance. It preserves why planned maintenance became applicable before or alongside Work Order generation without becoming a second execution model.

A Scheduled Work occurrence

Scheduled Work belongs to:

- one stable Maintenance Plan;
- one Plan Revision effective for recognition;
- one resolved Functional Location or Asset target;
- one nominal calendar or Measurement Point trigger instance.

It records the recognized recurring obligation and its original trigger meaning.

Scheduled Work is not executable maintenance. Responsibility for planning, execution, evidence, completion, verification, and closeout belongs to the resulting Work Order.

Require a Work Order before execution

A Scheduled Work occurrence becomes represented as accepted maintenance work when its Work Order is generated.

Before Work Order generation, Scheduled Work is a governed recurring obligation awaiting generation or explicit disposition. It cannot itself be started, completed, or used as a substitute execution record.

The relationship between occurrence and Work Order remains preserved permanently.

Recognize occurrences early enough for planning while preserving original trigger facts

An occurrence may be recognized within the Plan Revision's governed planning or generation horizon.

For calendar work, recognition may occur before the nominal due time.

For Measurement Point work, recognition occurs when accepted Process Reading evidence establishes the governed threshold instance or, where Product Authority explicitly supports it, when a deterministic planning horizon identifies an approaching threshold without changing which threshold remains authoritative.

Recognition time is not the nominal trigger time and must not rewrite the original trigger facts.

The Scheduled Work lifecycle

Scheduled Work uses explicit reversible deferral and distinct terminal outcomes:

```
recognized → deferred → recognized  
recognized / deferred → generated  
recognized / deferred → skipped  
recognized / deferred → cancelled
```

- **Recognized:** the occurrence exists and awaits Work Order generation or explicit disposition.
- **Deferred:** review or Work Order generation has been intentionally postponed.
- **Generated:** a Work Order represents the occurrence as accepted maintenance work.
- **Skipped:** the occurrence was valid and applicable but an authorized decision determined it will not be performed.
- **Cancelled:** the occurrence should no longer be treated as an applicable valid obligation for an explicit governed reason.

Generated, skipped, and cancelled are terminal Scheduled Work states.

Deferral preserves the obligation

Deferral is available only before Work Order generation.

Deferral preserves the original Plan Revision, target, nominal trigger, timing, and due meaning while postponing review or generation.

It records:

- deferral reason;
- one accountable active Team or responsible active User according to the governing responsibility contract;
- explicit review time or review condition;
- deciding Actor;
- decision time;
- additional evidence required by policy.

Deferral does not move the nominal occurrence or make overdue work appear current.

After a Work Order has been generated, postponement belongs to Work Order planning and control rather than Scheduled Work.

Keep skipped and cancelled distinct

Skipped and cancelled are different terminal decisions.

Skipped means the occurrence was valid and applicable, but an authorized decision determined that the maintenance will intentionally not be performed for this occurrence.

Examples may include:

- equivalent maintenance was already performed under another accepted work record;
- the maintenance opportunity is intentionally omitted under an accepted Decision;
- the consequence of skipping this one occurrence is explicitly accepted.

Cancelled means the occurrence should not remain an applicable maintenance obligation.

Examples may include:

- duplicate occurrence identity;
- incorrect recognition caused by invalid trigger evidence;
- invalid Plan configuration affecting this occurrence;
- the target ceased to be applicable before the nominal obligation took effect under a governed rule.

Both outcomes preserve deciding Actor, time, reason, evidence, and any related occurrence or work identity required to explain the decision.

A generic close action must not erase this distinction.

Preserve original calendar trigger and execution-window facts

Calendar Scheduled Work preserves the Plan-defined facts needed to interpret timeliness, including:

- nominal occurrence date or time;
- earliest allowed start where defined;
- due date or time;
- latest acceptable completion where defined;
- recognition time;
- Work Order generation time where applicable;
- deferral and disposition history.

These values retain their original meaning when the occurrence later becomes overdue, is deferred, or is represented by a Work Order.

Preserve Measurement Point trigger facts

Measurement Point Scheduled Work preserves:

- resolved trigger Measurement Point;
- measured quantity and canonical unit;
- nominal threshold instance or governed threshold-sequence identity;
- accepted Process Reading or deterministic accepted Reading sequence used to recognize the occurrence;
- Reading effective time;
- Reading quality and provenance required for traceability;
- recognition time;

- reset, rollover, replacement, or sequence evidence where relevant;
- optional forecasted calendar date where a governed forecast exists;
- latest acceptable threshold where the Plan defines one;
- Work Order generation time;
- deferral and disposition history.

A forecasted calendar date is informative unless the Plan explicitly assigns it another governed meaning. The authoritative trigger remains the configured Measurement Point threshold and accepted Process Reading evidence.

Make occurrence identity independent of Plan Revision

Logical occurrence identity is based on:

- stable Maintenance Plan;
- resolved target;
- nominal trigger instance.

For calendar work, the trigger instance is the governed nominal occurrence sequence or date.

For Measurement Point work, the trigger instance is the governed threshold sequence or threshold identity.

The Plan Revision is preserved for traceability but does not permit another occurrence to be created for the same logical Plan, target, and trigger instance merely because a new revision was published.

Require duplicate-safe recognition and Work Order generation

Re-evaluating the same Plan, target, and nominal trigger instance must return the existing Scheduled Work.

Repeated generation commands must return or reference the Work Order already representing the occurrence when one exists.

Retries, repeated machine Readings, duplicate manual recognition attempts, synchronization retries, or Worker retries must not create duplicate Scheduled Work or Work Orders.

A later correction that invalidates trigger evidence requires explicit governed reconciliation. It must not silently delete a Scheduled Work or Work Order that has already entered operational history.

Apply Plan changes prospectively

Publishing a new Plan Revision does not rewrite Scheduled Work or Work Orders that already exist.

Existing occurrences preserve:

- original Plan Revision;
- original resolved target;
- original trigger identity and facts;

- Procedure Revision basis where applicable;
- disposition and Work Order relationship.

Occurrences recognized under the new effective policy use the new Plan Revision without duplicating an occurrence whose stable identity was already established.

If a Plan change makes an existing ungenerated occurrence operationally inconsistent with the new policy, the occurrence is handled through explicit generation, deferral, skip, cancellation, or another defined reconciliation action rather than silent migration.

Preserve obligations reached during Plan pause

Pausing a Maintenance Plan suspends ordinary generation behavior according to the Plan contract.

Existing Scheduled Work remains visible and governed.

Calendar occurrences or Measurement Point thresholds reached during the pause must not silently disappear. On resume, Waiotech evaluates preserved trigger history and establishes the applicable missed obligations so each can be generated, deferred, skipped, cancelled, or otherwise explicitly reconciled according to Product Authority.

A pause action may intentionally include an explicit Decision about defined occurrences. It must not imply that all obligations reached during the pause vanished.

Make Plan retirement prospective and history-preserving

Retiring a Maintenance Plan prevents ordinary new occurrences for post-retirement trigger instances.

Retirement does not delete or rewrite:

- prior Plan Revisions;
- historical Scheduled Work;
- existing ungenerated Scheduled Work;
- generated Work Orders;
- Procedure snapshots;
- trigger evidence;
- completed maintenance evidence.

Existing Scheduled Work and Work Orders continue through their own governed lifecycles and dispositions.

Derive missed and overdue meaning from original facts

Missed and overdue are derived conditions, not manually assigned lifecycle states.

They are evaluated from:

- current evaluation time or accepted Measurement Point evidence;
- original nominal trigger facts;
- execution window or latest acceptable completion/threshold where defined;
- Scheduled Work state;

- resulting Work Order state where one exists.

Missed work remains visible. Waiotech must not silently cancel it, merge it into the next occurrence, regenerate it under a new nominal identity, or move its original due facts forward merely to make the queue appear current.

Keep recurring occurrences independently traceable

Each nominal trigger instance has its own durable Scheduled Work identity.

A later recurring occurrence does not remove or merge an earlier incomplete occurrence.

Where one Work Order legitimately satisfies several occurrences under an explicitly governed reconciliation contract, every original Scheduled Work identity and its disposition remains preserved. A shared Work Order must not erase the individual obligations.

Keep the recurring-maintenance concepts distinct

The minimum authority chain is:

```

Procedure
├─ Procedure Revision
│   └─ Work Order-owned Task / Requirement / evidence snapshots
└─ Maintenance Plan
    ├─ Maintenance Plan Revision
    │   ├── exact Procedure Revision where applicable
    │   ├── exact trigger rule
    │   ├── exact resolved targets and trigger bindings
    │   └─ generation defaults
    └─ Maintenance Plan Revision
        ├── Scheduled Work per target and nominal trigger instance
        └─ Work Order
  
```

The distinctions are:

- Procedure owns reusable method identity.
- Procedure Revision owns one exact method version.
- Maintenance Plan owns stable recurring intent.
- Plan Revision owns one exact recurring policy.
- Scheduled Work owns one recognized occurrence identity.
- Work Order owns accepted executable maintenance work.

Treat Plan-generated maintenance as accepted only through Work Order

A published Maintenance Plan authorizes governed recognition of recurring obligations under its effective rules. It does not itself make every possible occurrence accepted executable maintenance.

The recurring need becomes accepted maintenance work when a Scheduled Work occurrence generates or is explicitly linked to its Work Order under the owning contract.

Scheduled Work remains preserved because occurrence identity, due meaning, deferral, skip/cancellation decision, and recurring compliance cannot be reconstructed reliably from the Work Order alone.

Related documents

- [Procedures and Maintenance Plans](#)
- [Work Order lifecycle and Readiness](#)
- [Process Readings and Operational Observations](#)

Work Order lifecycle and Readiness

A Work Order is the authoritative record of one accepted maintenance objective. Its lifecycle separates preparation, release, execution, completion submission, and closeout, while Readiness remains a derived action-specific evaluation.

A Work Order

Waitech must use one canonical Work Order model for all accepted maintenance execution.

A Work Order is the authoritative record of one accepted maintenance objective and its controlled progression from preparation through execution, acceptance, and closeout.

It owns the work-specific context for:

- scope and objective;
- primary Work Target;
- maintenance classification;
- responsibility;
- Tasks;
- applied Procedure content;
- Work Requirements;
- planning and execution;
- completion evidence;
- verification and acceptance;
- Findings and follow-up;
- direct-cost attribution;
- lifecycle history.

The Work Order lifecycle

Waitech must use explicit lifecycle states for preparation, authorization, execution, completion submission, and final closeout.

The normal lifecycle is:

```
draft → prepared → released → in_progress → completed → closed
```

Additional governed transitions are:

```
in_progress → on_hold → in_progress  
completed → in_progress
```

```
released → prepared
prepared → draft

draft → cancelled
prepared → cancelled
released → cancelled
in_progress → cancelled
on_hold → cancelled
```

Emergency start may move a Work Order from `draft`, `prepared`, or `released` directly to `in_progress` when minimum emergency requirements are satisfied.

Use `prepared` for lifecycle state and reserve planned or unplanned for the derived planning route

Waiotech must use `prepared` for lifecycle state and reserve planned or unplanned for the derived planning route.

`Planned` already describes the derived planning route of a Work Order:

- planned;
- unplanned.

Using the same word as a lifecycle state would create two different meanings.

`Prepared` means the Work Order has reached the required planning result and contains the information needed for release evaluation.

Use `released` as the lifecycle state and evaluate readiness separately for each protected action

Waiotech must use `released` as the lifecycle state and evaluate readiness separately for each protected action.

Release is a historical authorization action. Readiness is a derived condition evaluated from authoritative facts that may change after the evaluation.

A Work Order may remain released while temporarily blocked because:

- an execution window has not opened;
- a permit expired;
- material became unavailable;
- an isolation is incomplete;
- a requirement satisfaction expired;
- authority changed after release.

The release remains part of history even when the Work Order is not ready at evaluation time to start.

draft

Waiotech must use draft as the flexible planning state.

Draft means the accepted maintenance objective exists, but the Work Order has not yet reached the minimum planning standard required for preparation.

Broad controlled editing is allowed in draft, including changes to:

- target;
- scope;
- classification;
- Tasks;
- Procedure;
- responsibility;
- timing;
- Requirements;
- expected evidence.

A draft Work Order cannot begin normal execution.

Make `prepare_work_order` a named action that validates the minimum planning contract

Waiotech must make `prepare_work_order` a named action that validates the minimum planning contract.

Before preparation is complete, the Work Order must contain:

- one primary Work Target;
- clear objective;
- defined scope boundary;
- maintenance class;
- trigger and origin;
- urgency and priority;
- primary discipline;
- accountable owner Team;
- timing expectations;
- at least one Task;
- applicable Procedure content;
- applicable Work Requirements;
- expected outcome;
- expected evidence;
- known execution-risk and control needs.

The depth of planning may remain proportional to the work, but required facts cannot be replaced by vague narrative.

Require at least one Task instead of supporting a separate taskless execution path

Waiotech must require at least one Task instead of supporting a separate taskless execution path.

Every Work Order must contain at least one Task before release.

A draft Work Order may temporarily have no Tasks while planning is incomplete.

For simple work that does not require meaningful decomposition, the Work Order may contain one broad Task representing the objective.

Example:

Inspect and correct the reported coupling guard issue

prepared

Waiotech must treat prepared as a meaningful validated state rather than a label that survives arbitrary edits.

Prepared means the Work Order satisfies the minimum planning contract and is eligible for release evaluation.

Prepared does not mean:

- authorized for execution;
- ready at evaluation time to start;
- scheduled;
- free from temporary blockers.

Material changes after preparation invalidate that planning result and require an explicit return to draft.

Preserve the meaning of prepared by requiring explicit lifecycle regression after material change

Waiotech must preserve the meaning of prepared by requiring explicit lifecycle regression after material change.

A material planning change requires `reopen_preparation`, returning the Work Order from `prepared` to `draft`.

Material changes include changes to:

- primary Work Target;
- core objective;
- scope boundary;
- maintenance class;
- execution method;
- required Procedure content;

- execution-risk assessment;
- mandatory controls;
- acceptance criteria;
- accountable responsibility;
- substantial Task structure.

Minor corrections that do not change planning meaning may remain allowed with audit evidence.

Release

Waitech must define release as authorization of the prepared execution package, not permanent confirmation that work can start at every subsequent execution attempt.

Release means an authorized Actor approves the prepared execution package for operational use under its defined scope, method, Tasks, controls, and Requirements.

Release confirms that release-blocking conditions are satisfied. It does not guarantee that every condition required to start execution will remain satisfied at execution time.

Release records:

- releasing Actor;
- release time;
- applicable authorization;
- evaluated release readiness;
- material Decisions;
- approved execution package identity.

Readiness

Waitech must make readiness action-specific and derived from authoritative facts at evaluation time.

Readiness is the authoritative derived evaluation of whether all Requirements and core conditions blocking a specified action are satisfied at evaluation time.

Readiness is evaluated separately for actions such as:

- prepare;
- release;
- start;
- complete;
- verify;
- close.

There is no single permanent Work Order readiness value.

A readiness blocker

Return explicit actionable blockers rather than only a ready or not-ready result.

A readiness blocker is a derived explanation of why a requested action is not allowed at evaluation time.

A blocker identifies:

- blocked action;
- unsatisfied Requirement or core condition;
- reason;
- authoritative source;
- responsible party where known;
- required resolution.

A blocker is not independently edited or manually closed.

Evaluate start Readiness through authoritative online facts or an explicitly bounded offline Work Package

Waiotech must evaluate start Readiness through authoritative online facts or an explicitly bounded offline Work Package.

Release does not create permanent start Readiness.

An online `start_work` or applicable resume action evaluates effective:

- Requirements;
- execution window;
- permits;
- isolations;
- material availability;
- responsibility;
- authority;
- restrictions effective at evaluation time;
- other start-blocking conditions.

An offline start or resume is permitted only through a governed offline Work Package whose issuance contract proves that every required condition is fixed for the package period, represented by an immutable snapshot, or verifiable locally through a defined field input. Synchronization preserves the package basis and evaluates conflicts and revocation explicitly.

Use `released → prepared` for release withdrawal and `prepared → draft` only through explicit reopening of preparation

Waiotech must use `released → prepared` for release withdrawal and `prepared → draft` only through explicit reopening of preparation.

Release should be withdrawn when a material change means the previously approved execution package is no longer valid.

Examples include changes to:

- primary target;
- core objective;
- method;
- mandatory Procedure content;
- execution risk;
- permit or isolation boundary;
- shutdown scope;
- acceptance criteria;
- required authority.

`withdraw_release` records the reason and returns the Work Order from `released` to `prepared`.

When the required replanning is substantial, a separate `reopen_preparation` action then returns it from `prepared` to `draft`.

A temporary blocker alone does not require release withdrawal.

Make the primary Work Target immutable after first execution starts

WaioTech must make the primary Work Target immutable after first execution starts.

Before release, the primary target may be corrected through controlled planning changes.

After release but before execution starts, changing the target requires release withdrawal and renewed preparation or release.

After first execution starts, the primary Work Target cannot change because doing so would reinterpret:

- Tasks;
- evidence;
- labor;
- material;
- Findings;
- costs;
- failure relationships;
- maintenance history.

Work concerning a different primary target must use a linked Work Order.

Require accountable Team responsibility and add named individual responsibility only where operationally required

Waiotech must require accountable Team responsibility and add named individual responsibility only where operationally required.

Not universally.

Every Work Order requiring accountable progression must have an owner Team before preparation.

A named coordinator, lead executor, planner, verifier, or closer is required only where the applicable lifecycle stage, work organization, or policy requires that responsibility.

Crew-based and contractor work may use Team accountability, a coordinator, and separately recorded contributing Executors.

Support one governed emergency-start path without creating a separate emergency Work Order model

Waiotech must support one governed emergency-start path without creating a separate emergency Work Order model.

Emergency start may occur from:

- draft;
- prepared;
- released.

It moves the Work Order to `in_progress`.

Emergency start is allowed only when immediate action is necessary and normal preparation or release requirements cannot all be completed first.

Require a rigid emergency minimum, including an executable Task, while allowing explicitly recorded bypass of normal controls

Waiotech must require a rigid emergency minimum, including an executable Task, while allowing explicitly recorded bypass of normal controls.

Emergency start requires at least:

- primary Work Target;
- immediate objective;
- at least one immediate Task, which may be broad when emergency evidence is incomplete;
- emergency reason;
- authorized Actor;
- accountable Team;
- execution coordinator where required;
- known immediate hazards;

- minimum immediate controls;
- record of bypassed normal Requirements;
- required retrospective obligations.

Emergency start must not permit execution against an empty or meaningless draft.

Preserve the actual exception path and require retrospective control without rewriting history

Waiotech must preserve the actual exception path and require retrospective control without rewriting history.

Emergency start:

- sets the Work Order to `in_progress` ;
- establishes planning route as `unplanned` ;
- records bypassed Requirements;
- creates retrospective planning and review obligations;
- preserves the emergency justification;
- requires unresolved controls and documentation to be completed according to the emergency workflow.

Release after emergency start must not rewrite the Work Order as planned.

`in_progress`

Waiotech must use in progress as the active execution state.

In progress means authorized execution has begun and the Work Order is accumulating operational evidence.

During this state:

- Tasks are executed;
- labor and material evidence is recorded;
- Inspection Results and Findings may be created;
- scope variations are governed;
- completion may be submitted when all completion blockers are resolved.

Planning facts cannot be casually rewritten after execution begins.

`on_hold`

Waiotech must use on hold only for temporary suspension after execution has started.

On hold means execution began but is temporarily suspended with remaining work expected to resume.

Placing work on hold records:

- hold reason;
- Actor and time;
- responsible Team;
- safety and restoration condition at evaluation time;
- resume condition or review time;
- unresolved blockers.

On hold does not erase actual execution, complete Tasks, or imply that the Asset is restored.

Recheck applicable resume conditions rather than treating hold as a passive pause

Recheck applicable resume conditions rather than treating hold as a passive pause.

`resume_work` returns the Work Order to `in_progress`.

Before resuming, Waiotech evaluates applicable readiness at evaluation time, authority, controls, and expired Requirements.

A prior start does not automatically authorize resumption without reevaluation.

Related documents

- [Work Requirements and scope control](#)
- [Completion, verification, and closeout](#)
- [Work Requests](#)

Work Requirements and scope control

Work Requirements express governed conditions that must be satisfied, waived where permitted, or remain visible as blockers. Material scope changes are explicit and may invalidate preparation, release, or issued field authority.

A Work Requirement

Waitech must use explicit Work Requirements for action-blocking obligations without creating a generic policy engine.

A Work Requirement is a Work Order-owned obligation that must be satisfied, waived where explicitly allowed, or declared not applicable before a named Work Order action.

A Work Requirement records:

- governed requirement type;
- description;
- source;
- action or actions it blocks;
- responsible party where known;
- satisfaction evidence;
- validity or expiry where applicable;
- whether waiver is permitted;
- applicable waiver Decision.

Keep Requirement sources closed and governed

Waitech must keep Requirement sources closed and governed.

Work Requirements may come from:

- an applied Procedure Revision;
- a Maintenance Plan Revision;
- an authorized planner;
- an explicit product rule;
- a governed Decision or variation.

IAM, Asset classification, or other modules must not inject arbitrary requirements through generic references.

Additional sources should be added only through explicit product contracts.

Use a small closed catalogue and expand it only for distinct blocking semantics

Waitech must use a small closed catalogue and expand it only for distinct blocking semantics.

The governed catalogue includes:

- information;
- Task definition;
- execution responsibility;
- material;
- tool or service;
- Procedure or document;
- permit;
- isolation;
- access;
- safety control;
- execution window;
- approval;
- completion evidence;
- verification;
- follow-up.

Detailed module-specific meaning may exist beneath these types without creating a new universal requirement type for every case.

Use explicit requirement outcomes with preserved evidence

Waitech must use explicit requirement outcomes with preserved evidence.

A Work Requirement may be:

- unsatisfied;
- satisfied;
- waived;
- not applicable;
- expired.

Satisfaction, waiver, and not-applicable outcomes preserve:

- Actor;
- time;
- evidence;
- reason;
- validity period where relevant.

Expired satisfaction causes the Requirement to become an active blocker again when applicable.

Make waiver explicit, exceptional, and controlled per Requirement

Waitech must make waiver explicit, exceptional, and controlled per Requirement.

Waiver is available only when the Requirement explicitly permits it.

A waiver requires:

- authorized Decision;
- reason;
- deciding Actor;
- decision time;
- temporary controls where required;
- effective period or expiry where applicable.

Mandatory product, safety, permit, or isolation Requirements cannot be waived merely because an Actor has broad administrative authority.

Combine Requirements conservatively and prohibit silent weakening

Combine Requirements conservatively and prohibit silent weakening.

Requirements are additive unless an explicit governed rule states that one replaces or supersedes another.

A Work Order may add stricter Requirements.

It must not silently remove mandatory Requirements originating from:

- the Procedure Revision;
- the Maintenance Plan Revision;
- product rules;
- approved Decisions.

Conflicting Requirements must be resolved explicitly before the blocked action becomes ready.

Preserve material variation explicitly rather than rewriting the original scope

Waitech must preserve material variation explicitly rather than rewriting the original scope.

A scope variation records a material change discovered or required after release or execution start.

Minor variation may be allowed within explicitly defined tolerance.

A material variation requires:

- effective work to pause where necessary;
- preservation of safe condition;
- description of the original and revised scope;

- reason;
- reassessment of risk and Requirements;
- authorization;
- renewed release where applicable;
- linked Work Order when the new objective is independently manageable.

Establish non-negotiable variation boundaries

Establish non-negotiable variation boundaries.

The following are always material:

- primary Work Target;
- core objective;
- maintenance class where the meaning of work changes;
- mandatory Procedure content;
- execution-risk classification;
- permit or isolation boundary;
- shutdown boundary;
- acceptance criteria;
- required authority;
- separation-of-duty requirements.

The primary target cannot change after execution begins.

Related documents

- [Work Order lifecycle and Readiness](#)
- [Completion, verification, and closeout](#)
- [Decisions, waivers, and separation of duties](#)

Completion, verification, and closeout

Completion records execution claims and evidence. Verification evaluates that claim when required, and closeout confirms that required acceptance, follow-up, cost, and evidence obligations are resolved.

completed

WaioTech must treat completion as submission of execution results, not final acceptance.

Completed means the responsible execution side declares field execution finished and submits the work for acceptance.

Completion records:

- actual execution result;
- Task outcomes;
- supporting evidence;
- unresolved limitations;
- restoration condition;
- Findings;
- temporary repair information;
- completion Actor and time.

Completed does not mean independently verified, accepted, or administratively final.

A Completion Record

WaioTech must preserve every completion attempt rather than overwriting the latest result.

A Completion Record is the immutable evidence of one completion attempt.

It preserves:

- completing Actor;
- completion time;
- Task results;
- execution narrative;
- evidence;
- restoration condition;
- limitations;
- Findings;
- temporary-repair declarations.

A Work Order may have multiple Completion Records when completion is withdrawn or work is returned for rework.

Support voluntary withdrawal of submitted completion without deleting evidence

WaioTech must support voluntary withdrawal of submitted completion without deleting evidence.

`withdraw_completion` may be used before completion has been accepted.

It:

- requires a reason;
- preserves the prior Completion Record;
- returns the Work Order to `in_progress` ;
- allows additional execution or correction.

Represent rework as another execution cycle on the same Work Order when the objective remains the same

WaioTech must represent rework as another execution cycle on the same Work Order when the objective remains the same.

Rework is represented by `return_for_rework` , moving the Work Order from `completed` to `in_progress` .

It records a Decision containing:

- reason acceptance failed;
- required correction;
- deciding Actor;
- decision time;
- relevant evidence.

Prior Tasks, execution evidence, Completion Records, and Decisions remain preserved.

The difference between completion withdrawal and rework

WaioTech must keep voluntary withdrawal and negative acceptance as separate actions.

`withdraw_completion` is initiated by the completing side before acceptance to correct or continue the work voluntarily.

`return_for_rework` is initiated by the verifier or closer because submitted completion was not accepted.

Both return the Work Order to `in_progress` , but their authority and decision meaning are different.

Require verification only through explicit governed rules

Waiotech must require verification only through explicit governed rules.

Verification is required only when established by:

- a Work Requirement;
- Procedure Revision;
- Maintenance Plan Revision;
- execution risk;
- Operational Criticality rule;
- temporary repair;
- authorized Decision;
- other explicit product rule.

Low-control work may be accepted during closeout without a separate verification step.

Valid verification outcomes

Waiotech must represent verification through explicit Decisions with evidence and rationale.

Valid verification outcomes are:

- accepted;
- accepted with controlled follow-up;
- returned for rework.

Verification is an immutable Decision, not a mutable boolean.

Permit conditional acceptance only with explicit controlled obligations

Permit conditional acceptance only with explicit controlled obligations.

This outcome requires an explicit unresolved condition that is safe and authorized to remain temporarily.

It must establish:

- limitation or Finding;
- accountable owner;
- linked Work Request or Work Order where needed;
- due date or review condition;
- Decision rationale;
- operating constraints where applicable.

It must not be used to hide incomplete work without accountable follow-up.

Establish acceptance through verification when required and through governed closeout otherwise

Establish acceptance through verification when required and through governed closeout otherwise.

When separate verification is not required, the authorized `close_work_order` action accepts the submitted Completion Record as part of closeout.

Policy may permit the completing Actor to close low-control work.

Where separation of duties applies, another authorized Actor must perform closeout.

Preserve separate planning, execution, acceptance, and finalization times

Waiotech must preserve separate planning, execution, acceptance, and finalization times.

A Work Order should distinguish:

- earliest permitted start;
- target due time;
- latest acceptable completion;
- scheduled execution window;
- first actual execution start;
- hold and resume times;
- completion-attempt times;
- final accepted completion time;
- effective close time;
- cancellation time where applicable.

These facts must not be collapsed into one generic due or completion timestamp.

Keep closeout focused on final control and reconciliation

Waiotech must keep closeout focused on final control and reconciliation.

Closeout performs only final control checks.

It confirms that:

- completion is accepted;
- required verification is established;
- required follow-up is linked and owned;
- temporary limitations are governed;
- material and Reservation consequences are reconciled;
- final outcome classification is recorded;
- no closure-blocking Requirements remain.

Closeout must not require users to repeat the completion narrative.

closed

Waiotech must make closed operationally terminal and preserve historical truth.

Closed means the Work Order has completed its operational lifecycle and all required acceptance and closeout controls are satisfied.

Closed is terminal for operational execution.

New maintenance work uses a linked Work Order. Historical errors are handled through:

- annotations;
- explicit corrections;
- supplemental evidence;
- governed financial or inventory corrections in the owning modules.

Define cancellation transitions explicitly

Waiotech must define cancellation transitions explicitly.

Cancellation is allowed from:

- draft;
- prepared;
- released;
- in progress;
- on hold.

A completed Work Order cannot be cancelled because execution has already been declared finished. It must instead be accepted, returned for rework, corrected, or closed.

Work Order cancellation

Waiotech must treat cancellation as terminal cessation, never deletion or erasure.

Cancellation means the accepted maintenance objective will not continue to normal completion under this Work Order.

Cancellation requires:

- reason;
- authorized Actor;
- cancellation time;
- execution and restoration condition at evaluation time;
- consequences for Tasks;
- responsibility resolution;
- material and Reservation reconciliation;
- Scheduled Work disposition where applicable;

- Findings and follow-up handling.

Cancellation after execution begins preserves all actual labor, material, evidence, costs, and completed activity.

Keep Work Order and Scheduled Work lifecycles distinct

WaioTech must keep Work Order and Scheduled Work lifecycles distinct.

Cancelling a generated Work Order does not silently cancel or skip its Scheduled Work occurrence.

The originating occurrence must receive its own explicit governed disposition according to why the Work Order was cancelled.

Possible outcomes include:

- regenerate replacement work;
- mark the occurrence skipped;
- cancel the occurrence;
- retain the relationship to the cancelled Work Order and create a linked replacement.

Editable in each lifecycle state

Tie editability to lifecycle meaning and prohibit silent historical rewriting.

Draft

Broad controlled editing is allowed.

Prepared

Only non-material corrections are allowed. Material planning changes require `reopen_preparation`.

Released

Only non-material edits are allowed. Material changes require `withdraw_release`.

In progress

Execution evidence and permitted job-specific additions are recorded. Material planning changes require governed variation, pause, renewed authorization, or linked work.

On hold

Evidence about the hold and resolution may be added. Scope changes remain governed as variations.

Completed

Submitted evidence is reviewable. Changes require completion withdrawal, correction, or return for rework.

Closed or cancelled

Only annotation, supplemental evidence, and explicit correction actions are allowed.

Use Work Request for unaccepted follow-up and Work Order for accepted follow-up

Waiotech must use Work Request for unaccepted follow-up and Work Order for accepted follow-up.

A Finding, limitation, or unresolved condition may create:

- a Work Request when triage is still needed;
- a Work Order when an authorized maintenance Actor has already accepted the need;
- a link to existing work when the need is already represented.

Follow-up must preserve its source relationship to the originating Work Order and evidence.

Govern temporary repair through completion limitations and follow-up without a separate top-level entity

Waiotech must govern temporary repair through completion limitations and follow-up without a separate top-level entity.

Temporary repair must be explicitly declared in completion evidence.

It records:

- temporary restoration achieved;
- remaining limitation;
- operating constraints;
- accountable owner;
- inspection, review, or expiry condition;
- permanent follow-up need;
- linked follow-up Work Order where required.

The Work Order cannot close unless:

- permanent follow-up is linked and owned; or
- an authorized Decision accepts the result as permanent.

Use Work Order as the operational cost aggregation context, not as the source ledger for labor, stock, usage, services, or accounting

Waiotech must use Work Order as the operational cost aggregation context, not as the source ledger for labor, stock, usage, services, or accounting.

A Work Order aggregates direct-cost evidence attributed to the maintenance work.

This may include:

- labor evidence;
- material cost evidence;
- external service or contractor cost evidence;
- other direct costs.

The underlying source records remain authoritative for their own facts:

- labor evidence owns recorded effort, rate basis, and labor-cost facts;
- posted Inventory Movements own stock custody and quantity consequences;
- Material Usage owns what was consumed, installed, applied, or otherwise used by the work;
- service and contractor cost evidence owns external direct-charge facts.

The Work Order owns the relationship of those costs to the work and presents historical cost snapshots.

Preserve historical direct-cost meaning through source-owned snapshots and governed corrections

Waiotech must preserve historical direct-cost meaning through source-owned snapshots and governed corrections.

Historical work should retain the applicable labor rates, material valuations, service costs, and other cost facts used when the cost evidence was recorded or finalized.

Changing rates or valuations effective after the historical record must not silently rewrite historical Work Order cost.

Corrections require explicit actions in the authoritative source records.

The minimum Work Order model

Waiotech must keep Work Order as the complete controlled execution aggregate without duplicating authority owned by IAM, Inventory, Procedures, or financial systems.

The minimum model contains:

- immutable identity;
- Tenant;
- lifecycle state;
- primary Work Target;

- objective and scope;
- maintenance classification;
- urgency and priority;
- planning route;
- discipline;
- owner Team and applicable individual responsibilities;
- Tasks;
- applied Procedure snapshots;
- Work Requirements;
- readiness evaluations and blockers;
- release evidence;
- execution and hold evidence;
- scope variations;
- Completion Records;
- Verification Decisions;
- Findings and follow-up;
- cancellation or closeout evidence;
- direct-cost attribution;
- preserved action history.

Use inspection-class Work Orders rather than a separate Inspection aggregate

WaioTech must use inspection-class Work Orders rather than a separate Inspection aggregate.

Normal maintenance inspections are represented by Work Orders with maintenance class `inspection`.

A separate Inspection aggregate would duplicate planning, scheduling, execution, evidence, verification, and closeout already owned by Work Order.

A separate Inspection aggregate is outside WaioTech Product Authority. Introducing it requires a Product Authority amendment defining independent inspection identity, lifecycle, authority, evidence, and maintenance relationships.

An Inspection Result

WaioTech must use structured Inspection Results beneath Work Order execution.

An Inspection Result is structured evidence produced while executing an inspection Work Order, Task, or Procedure step.

It may record:

- pass, fail, or not applicable;
- measured value and unit;
- selected condition;
- observation;

- acceptance limits;
- supporting evidence.

An Inspection Result belongs to the execution context that produced it.

Separate inspection execution completion from inspection outcome

Waiotech must separate inspection execution completion from inspection outcome.

Completing an inspection Work Order means the required inspection was performed and recorded correctly. It does not mean the Asset passed the inspection.

Unacceptable results must produce explicit Findings or Failure Events and the required follow-up.

Related documents

- [Work Order lifecycle and Readiness](#)
- [Findings](#)
- [Maintenance materials, tools, services, and cost](#)

Findings

A Finding records a specific maintainable condition discovered through governed work or inspection that requires explicit maintenance disposition. It preserves discovery context without becoming accepted maintenance work or a Reliability Failure Event.

A Finding

A Finding is created when a discovered condition is significant enough that it must not disappear inside general notes or execution narrative.

Examples include:

- defect;
- failed inspection criterion;
- maintainable abnormal condition;
- compliance issue requiring maintenance disposition;
- temporary limitation discovered during work;
- improvement opportunity;
- missing or contradictory maintenance information.

Routine observations that need no independent decision remain ordinary execution or inspection evidence.

Preserve discovery context

A Finding preserves:

- discovering Actor;
- discovery effective time and recorded time where distinct;
- governing Work Order, Task, Procedure step, inspection result, verification activity, Failure Event investigation, or other supported source context;
- affected Functional Location or Asset where applicable;
- concise description;
- supporting evidence;
- classification and first significance assessment where defined.

Later disposition must not rewrite what was originally discovered.

Keep Finding distinct from Process Condition and Failure Event

A Finding is a Maintenance concept.

- **Process Condition** records an operational situation requiring Process handling or monitoring.
- **Finding** records a maintainable condition requiring maintenance disposition.
- **Failure Event** records actual loss or materially unacceptable degradation of required function.
- **Work Order** records accepted maintenance work.

The same real situation may involve several of these records. Waiotech links them and preserves ownership instead of converting one record into another.

Finding disposition

Every Finding requiring independent tracking receives one explicit disposition:

- resolved within governing work;
- no action required;
- monitor;
- Work Request created;
- follow-up Work Order created;
- linked to an existing Work Order;
- duplicate;
- invalid or corrected.

Disposition preserves deciding Actor, time, reason, and related evidence or follow-up identity.

A generic `closed` state or bare `accepted` disposition is prohibited because it does not explain how the condition was controlled.

Resolve within governing work only when scope remains authorized

A Finding may be resolved inside the governing Work Order when the corrective action remains within the authorized objective, scope, controls, Requirements, and competence of that work.

Resolution preserves:

- action taken;
- Task or evidence that resolved the Finding;
- resolving Actor;
- resolution time;
- resulting condition.

When correction materially expands scope, changes the primary objective, or requires separate planning or acceptance, the Finding creates or links follow-up maintenance work instead.

Follow-up maintenance

A Finding may:

- create a Work Request when triage remains necessary;
- create a Work Order directly when an authorized maintenance Actor already accepts the need;
- link to compatible existing work.

Creating or linking work does not erase the Finding or discovery evidence.

Defects remain Finding classification

A defect that has not crossed the Failure Event threshold is represented as a Finding with the applicable defect classification.

Waiotech does not use a separate Defect aggregate. Introducing one would require independent Product Authority for identity, lifecycle, ownership, ageing, prioritization, evidence, reporting, and resolution.

Related documents

- [Completion, verification, and closeout](#)
- [Failure Events](#)
- [Work Requests](#)
- [Process Conditions](#)

Inventory

Inventory owns material identity, custody, availability, Reservations, Movements, replenishment, Stocktaking, and stock history. Consuming domains own why material is required or used.

Contents

- Items, Categories, and quantities
- Stock Locations and Movements
- Reservations and availability
- Maintenance materials, tools, services, and cost
- Process material usage
- Replenishment
- Stocktaking

Items, categories, and quantities

An Item is the governed identity of a material, consumable, spare part, tool, or other inventory-relevant thing. Item policy, categorization, units, conversions, and tracking granularity remain explicit.

An Item

Waiotech must use **Item** as the canonical entity name and keep its meaning limited to physical supplies.

An Item is the stable catalogue identity of a material, spare part, consumable, or other physical supply used by operations.

An Item may be:

- tracked in stock;
- purchased without being stocked;
- planned or recorded as material used by a Work Order.

Tools, services, labor, contractors, vehicles, and rental resources are not Items.

Separate catalogue identity from stock-tracking behavior

Waiotech must separate catalogue identity from stock-tracking behavior.

Item policy determines whether Waiotech tracks stock custody and quantity for the Item.

A non-stock Item may still be:

- included in a Material Requirement;
- purchased externally;
- recorded as Work Order material usage;
- associated with operational cost.

It does not participate in Stock Balances, Reservations, or stock-changing Movements.

Keep Item policy small, explicit, and inventory-specific

Waiotech must keep Item policy small, explicit, and inventory-specific.

Item policy defines only concrete inventory behavior:

- stock-tracked or non-stock;
- stocking unit;
- allowed unit conversions;
- lot or batch tracking requirement;

- expiry tracking requirement;
- stock-condition tracking requirement;
- whether Reservation is allowed;
- maximum quantity decimal places, from 0 through 12.

Item policy must not become an open-ended rule engine.

Replace fixed Category, Family, and Item Type levels with one governed recursive Item Category model

Waitech must replace fixed Category, Family, and Item Type levels with one governed recursive Item Category model.

A fixed-depth hierarchy will not fit every material catalogue.

Use one recursive Item Category hierarchy:

```
Item Category
└─ Item Category
    └─ Item
```

Examples include:

```
Mechanical Spares
└─ Bearings
    └─ Rolling-element Bearings
```

The hierarchy may have any required depth.

Use one primary taxonomy and prohibit overlapping classification systems without an explicit Product Authority amendment

Waitech must use one primary taxonomy and prohibit overlapping classification systems without an explicit Product Authority amendment.

Item Category follows these rules:

- each Item has one primary category;
- categories may contain child categories;
- cycles are forbidden;
- category identity and codes remain stable after use;
- moving an Item between categories does not change the Item identity;
- category does not independently determine stock policy or authorization.

Additional Item classification dimensions require a Product Authority amendment defining their meaning, governance, relationships, and reporting consequences.

Use bounded catalogue lookup and explicit hierarchy scope

Waiotech must expose bounded, paginated catalogue lookup rather than loading the complete Tenant catalogue into a client.

Item Category listing may select exactly one hierarchy scope:

- `root_only=true` returns root categories;
- `parent_id=<id>` returns direct children of one category;
- omitting both returns the ordinary paginated registry.

`root_only` and `parent_id` are mutually exclusive.

Item Category and Item listing may accept `q` as a case-insensitive literal match against code or name. Percent and underscore characters are ordinary search characters, not wildcard authority. Empty or whitespace-only search values are invalid. Search remains Tenant-scoped, bounded, and paginated.

Use one stocking unit per stock-tracked Item with governed Item-specific conversions

Waiotech must use one stocking unit per stock-tracked Item with governed Item-specific conversions.

Every stock-tracked Item has one stocking unit.

Examples include:

- each;
- metre;
- litre;
- kilogram.

Purchasing, issue, and usage may use other allowed units only when the Item defines an explicit conversion.

Preserve Item-specific conversion meaning

Waiotech must preserve Item-specific conversion meaning.

Conversions are Item-specific when physical meaning may vary.

For example:

```
Stocking unit: each
Purchasing unit: box
Conversion: 1 box = 24 each
```

Conversions between unlike dimensions, such as volume and mass, require Item-specific physical basis and must never be assumed globally.

Make quantity precision explicit and reject silent rounding

WaioTech must make quantity precision an explicit Item policy fact.

Every Item declares the maximum number of decimal places accepted for entered and normalized quantity. The policy applies consistently to receipts, issues, transfers, reservations, counts, corrections, material use, and all Item-specific unit conversions.

The API must require the precision when creating or replacing Item policy. Domain and Application validation reject quantities that exceed it. Repositories and PostgreSQL must not silently round an authoritative quantity.

Quantity precision becomes immutable after the first posted Movement because changing it would reinterpret historical and future balance meaning. Cost and currency rounding remain separately governed accounting facts.

Preserve both entered and normalized quantity facts

WaioTech must preserve both entered and normalized quantity facts.

A Movement should preserve:

- entered quantity;
- entered unit;
- equivalent stocking quantity;
- conversion basis applied;
- applicable conversion revision or effective rule.

Conversion changes effective after the historical Movement must not rewrite historical Movement quantities.

Use simple active and retired semantics for catalogue records

WaioTech must use simple active and retired semantics for catalogue records.

Item and Item Category use:

active → retired

Retirement prevents new ordinary selection while preserving all historical Movements, Requirements, Usage, Reservations, and reporting meaning.

An Item or Category cannot be retired while active obligations require it unless those relationships are resolved or explicitly governed.

Related documents

- [Stock Locations and Inventory Movements](#)

- Maintenance materials, tools, services, and cost

Stock Locations and Inventory Movements

Stock Locations are Inventory custody nodes. Posted Inventory Movements are immutable authoritative stock facts, and stock balances are derived from their posted quantity effects.

A Stock Location

Waiotech must require all tracked stock custody to use a Stock Location.

A Stock Location is an Inventory-controlled custody node where stock may be held, received, issued, returned, transferred, counted, or disposed.

It is distinct from a Functional Location.

A Stock Location may optionally reference a Functional Location, but it may also represent logical or mobile custody such as:

- quarantine;
- goods in transit;
- technician custody;
- damaged stock;
- external holding.

Use bounded Stock Location lookup

Waiotech must expose Stock Location lookup as a Tenant-scoped, bounded, paginated query.

The query may accept `q` as a case-insensitive literal match against Stock Location code or name. Percent and underscore characters are ordinary search characters, not wildcard authority. Empty or whitespace-only search values are invalid.

This lookup supports operational selectors and registries; it does not authorize client-side loading of the complete Tenant custody model.

The authoritative stock fact

Waiotech must derive stock quantity from immutable posted Inventory Movements.

Posted Inventory Movements are the authoritative stock facts.

Replenishment Request receipt attribution is established only by the governed Replenishment receive action, and Stocktaking Run attribution is established only by the governed Stocktaking post action. Generic Movement preparation must reject both relationships so custody evidence cannot diverge from the owning aggregate lifecycle.

A Stock Balance is a projection calculated from posted Movements across the applicable stock dimensions.

Balance rows must not be independently edited as stock truth.

Make posting the only action that changes authoritative stock

Waiotech must make posting the only action that changes authoritative stock.

An Inventory Movement affects stock only when it is posted.

A Movement may be prepared, validated, or approved before posting, but those stages do not change stock.

Posting establishes the governed stock consequence.

Make posted Movements immutable and preserve correction history

Waiotech must make posted Movements immutable and preserve correction history.

After posting, the following facts cannot be silently changed:

- Item;
- quantity;
- source Stock Location;
- destination Stock Location;
- lot or batch;
- stock condition;
- effective time;
- Work Order or source attribution.

Corrections require a governed reversal or compensating Movement linked to the original Movement.

Keep Movement types product-defined and closed

Waiotech must keep Movement types product-defined and closed.

The closed catalogue includes:

- receipt;
- issue;
- return;
- transfer;
- adjustment;
- disposal.

These describe the primary stock consequence.

More detailed operational meaning belongs in governed reasons, source relationships, and evidence.

Use one adjustment type with explicit governed reasons

Waiotech must use one adjustment type with explicit governed reasons.

Both are adjustments with different governed reasons and source evidence.

Adjustment reasons may include:

- stocktake variance;
- opening balance;
- discovered stock;
- missing stock;
- condition reclassification;
- authorized correction;
- other product-governed reason.

Correct posted stock through linked compensating evidence rather than editable correction records

Correct posted stock through linked compensating evidence rather than editable correction records.

They are governed correction actions.

A correction action creates a reversing or compensating Movement that:

- references the original Movement;
- records the reason;
- preserves the correcting Actor and time;
- restores the intended stock truth.

The original posted Movement remains visible.

Treat transfer as one indivisible product action

Waiotech must treat transfer as one indivisible product action.

Normatively, transfer is one business action with one identity and balanced source and destination consequences.

A transfer records:

- Item;
- quantity;
- source Stock Location;
- destination Stock Location;
- lot or condition where applicable;

- Actor;
- effective time;
- reason.

The source decrease and destination increase become effective together. No partial transfer may be presented as successful.

Use explicit Stock Location custody for in-transit stock

Waiotech must use explicit Stock Location custody for in-transit stock. A dedicated transfer lifecycle requires a Product Authority amendment.

An instantaneous transfer should not pretend to represent prolonged custody in transit.

When goods remain in transit for a meaningful period, use an explicit in-transit Stock Location or require a Product Authority amendment defining a dedicated transfer lifecycle.

Forbid negative stock

Forbid negative stock under Waiotech Product Authority. Any exception requires a Product Authority amendment defining authority, evidence, reconciliation, and reporting.

A stock-decreasing Movement cannot be posted when it would make the applicable on-hand quantity negative.

When physical stock and recorded stock disagree, users must first reconcile the difference through an authorized adjustment or stocktaking process.

Work Order evidence may record externally supplied or uncatalogued material without inventing an Inventory issue.

Derive balances at the full governed tracking granularity

Waiotech must derive balances at the full governed tracking granularity.

A stock position is identified by:

- Item;
- Stock Location;
- lot or batch when tracking is required;
- stock condition when tracking is required;
- applicable expiry facts where needed.

Every posted Movement must identify the exact stock position or positions it changes.

Use stock condition only where it has real custody and availability meaning

Waiotech must use stock condition only where it has real custody and availability meaning.

Stock condition exists only through a typed Product Authority contract defining custody, availability, transitions, evidence, and posting consequences.

Canonical examples include:

- available;
- quarantine;
- damaged;
- expired;
- awaiting inspection.

Stock condition affects whether quantity is available for Reservation or issue.

Changing condition must occur through a governed Movement or equivalent stock consequence, not by editing a Balance projection.

Inventory tracks quantity, lot, expiry, and condition

Inventory tracks quantity, lot, expiry, and condition. Serialized Item lifecycle is outside this Product Authority.

Waiotech supports:

- quantity by Item and Stock Location;
- optional lot or batch tracking;
- optional expiry tracking;
- optional stock condition.

The following are outside Waiotech Product Authority:

- individual serial custody;
- rotatable repair lifecycle;
- exchange pools;
- calibrated-tool history;
- serialized issue and return;
- individual-part genealogy.

Retire Stock Locations only after active custody obligations are resolved

Waiotech must retire Stock Locations only after active custody obligations are resolved.

Stock Location uses:

active → retired

A Stock Location cannot be retired while it contains on-hand stock, active Reservations, unresolved Stocktaking Runs, or pending governed custody obligations.

Retirement preserves all historical Movement and count evidence.

Related documents

- [Items, categories, and quantities](#)
- [Reservations and availability](#)
- [Functional Locations](#)

Reservations and availability

A Reservation binds an Item quantity at a Stock Location to an authorized demand without moving stock. Availability is derived from authoritative balances, reservations, condition, and applicable Inventory rules.

A Reservation

WaioTech must keep Reservation as an operational aggregate separate from Movement.

A Reservation is a governed commitment of available stock to a Work Order or Material Requirement.

It protects availability but does not change on-hand quantity or physical custody.

A Reservation has its own identity and history.

Support bounded operational Reservation lookup

Reservation listing must remain Tenant-scoped, bounded, and paginated. It may be filtered by lifecycle state, Item, Work Order, or Stock Location so custody and maintenance workflows can inspect the obligations relevant to their current context.

These filters expose existing Reservation facts only. They do not create a second availability calculation or transfer custody ownership out of Inventory.

Reserve exact governed stock availability for a defined maintenance demand

Reserve exact governed stock availability for a defined maintenance demand.

A Reservation binds:

- Work Order or Material Requirement;
- Item;
- Stock Location;
- reserved quantity in stocking unit;
- optional lot or condition constraints;
- effective time;
- optional expiry;
- reserving Actor.

Preserve Reservation quantity history rather than one mutable remaining value alone

Waiotech must preserve Reservation quantity history rather than one mutable remaining value alone.

Reservation should preserve:

- originally reserved quantity;
- quantity consumed through issue;
- quantity released;
- quantity expired or cancelled;
- remaining reserved quantity.

Remaining quantity may be derived from preserved Reservation and issue facts.

The Reservation lifecycle

Waiotech must use a small explicit Reservation lifecycle.

The lifecycle is:

```
active  
→ fulfilled  
→ released  
→ expired  
→ cancelled
```

- **Active:** quantity remains protected.
- **Fulfilled:** the reserved quantity has been fully consumed through issue.
- **Released:** remaining quantity was intentionally made available.
- **Expired:** the Reservation ended at its governed expiry.
- **Cancelled:** the underlying demand or Reservation was invalidated.

Terminal outcomes preserve reason, Actor, and time.

Reserve only governed stock available at reservation time

Reserve only governed stock available at reservation time under Waiotech Product Authority.

Creating or increasing a Reservation must not reduce available stock below zero.

A shortage remains visible as an unfulfilled Material Requirement rather than an impossible Reservation.

Never treat Reservation as custody or consumption evidence

Waiotech must never treat Reservation as custody or consumption evidence.

Reservation changes available quantity only.

Physical issue requires a posted Inventory Movement.

A Movement used to fulfil a Reservation must address the same stock position and Work Order. When the Reservation is bound to a Material Requirement, the Movement must address that same requirement. The total fulfilment quantity attributed to one Movement across all Reservations must not exceed that Movement's posted issue quantity. One Reservation records at most one fulfilment event against the same Movement; later fulfilment requires another posted issue Movement.

Available stock

WaioTech must derive availability from posted Movements, Reservations, and governed stock condition.

Available stock is derived from on-hand stock less active Reservations and other explicitly unavailable stock conditions.

It is a projection, not independently maintained truth.

Related documents

- [Stock Locations and Inventory Movements](#)
- [Maintenance materials, tools, services, and cost](#)

Maintenance materials, tools, services, and cost

Maintenance owns its requirements and usage evidence; Inventory owns stock custody and movement. Tools, service requirements, substitutions, returns, and direct cost attribution preserve their distinct meanings.

A Material Requirement

Waiotech must keep technical material demand in Maintenance, not Inventory.

A Material Requirement is structured maintenance demand owned by the Work Order.

It may define:

- required Item or free-text material;
- required quantity and unit;
- acceptable alternatives;
- required-by action or time;
- linked Reservation;
- fulfillment state;
- issued and used material references.

It is a specialized form of the Work Order's material Requirement.

Separate technical demand from physical custody

Waiotech must separate technical demand from physical custody.

Inventory owns:

- Stock Location;
- stock availability;
- Reservation;
- custody;
- Inventory Movement.

Maintenance owns:

- Material Requirement;
- technical suitability;
- actual Work Order usage;
- substitution acceptance;
- completion evidence.

The difference between material issue and material usage

WaioTech must not infer that all issued material was consumed.

Material issue records stock leaving Inventory custody for a Work Order or another governed destination.

Material usage records what was actually consumed, installed, applied, or otherwise used by the Work Order.

They may differ.

Example:

```
10 seals issued
7 seals installed
3 seals returned
```

Issue is authoritative for stock custody. Material Usage is authoritative for work attribution.

Require explicit disposition of issued material where applicable

WaioTech must require explicit disposition of issued material where applicable.

Work Order material reconciliation should distinguish:

- issued quantity;
- used or installed quantity;
- returned quantity;
- disposed or damaged quantity;
- unresolved difference.

The Work Order cannot close when a material reconciliation Requirement remains unresolved.

Keep custody and work attribution aligned as one successful product action

WaioTech must keep custody and work attribution aligned as one successful product action.

A successful issue must establish both:

- its posted Inventory Movement;
- its Work Order attribution.

No valid product state may show stock issued to work without the corresponding Work Order relationship, or Work Order-issued stock without the posted Movement.

Engineering Authority determines how this consistency is enforced.

Support explicit alternatives and governed technical substitution without automatic equivalence inference

WaioTech must support explicit alternatives and governed technical substitution without automatic equivalence inference.

Substitution has two supported paths.

Pre-approved alternative

A Material Requirement or Item declares specific Items approved as alternatives.

Unapproved substitution

A technical Decision is required and records:

- originally required Item;
- selected substitute;
- compatibility basis;
- reason;
- deciding Actor;
- quantity;
- effect on Procedure, safety, warranty, or acceptance where applicable.

The original requirement and actual material used remain preserved.

Allow truthful non-stock material evidence without pretending it belongs to Inventory

WaioTech must allow truthful non-stock material evidence without pretending it belongs to Inventory.

Free-text material may represent:

- uncatalogued supply;
- externally provided material;
- locally purchased material;
- incidental consumable.

It should record:

- description;
- quantity;
- unit;
- source;
- Actor;
- time;
- cost where known;
- evidence where applicable.

It does not affect stock and does not automatically create an Item.

Preserve historical reporting meaning

Waiotech must preserve historical reporting meaning.

Not silently.

When a catalogue Item is created after the historical usage, historical free-text usage remains as originally recorded unless an authorized correction explicitly establishes the relationship.

Treat return as a new custody event, not reversal of usage evidence

Waiotech must treat return as a new custody event, not reversal of usage evidence.

Return is a separate posted Movement.

It records:

- originating issue where available;
- returning Work Order;
- Item;
- quantity;
- destination Stock Location;
- stock condition;
- Actor;
- time;
- assessment evidence where required.

Require condition-aware returns

Waiotech must require condition-aware returns.

Unused serviceable material may return to ordinary available stock.

Opened, contaminated, damaged, expired, or suspect material must enter an appropriate unavailable Stock Location or stock condition until its disposition is determined.

Do not assume removed components are stock returns

Waiotech must not assume removed components are stock returns.

Not automatically.

Removed material may be:

- scrap;

- repairable material;
- failed evidence;
- externally owned property;
- rotatable stock after return to Inventory.

It may enter Inventory only through an explicit receipt or return action whose custody meaning is valid.

A tool in Waitech

Waitech must represent tool needs through Work Requirements. A Tool or Resource aggregate requires a Product Authority amendment.

A tool is represented as a Work Requirement describing equipment needed to perform work.

It may record:

- description or catalogue reference;
- quantity;
- required capability;
- provider;
- provision evidence.

Waitech does not claim:

- tool custody;
- issue and return;
- availability;
- calibration control;
- serial tracking.

Keep tools outside the Item and stock model

Waitech must keep tools outside the Item and stock model.

Tools are governed by their distinct custody, availability, calibration, assignment, and return semantics, not merely because they are physical objects.

Stock Items represent consumable or supply custody. Controlled tools have different lifecycle, availability, calibration, assignment, and return semantics.

A dedicated Tool or Resource model is outside Waitech Product Authority and requires an amendment defining identity, lifecycle, custody, availability, evidence, and cost.

A Service Requirement

Manage required service provision without introducing procurement or supplier-management workflows.

A Service Requirement is a Work Order Requirement for an external or internal service needed to perform work.

Examples include:

- crane support;
- specialist contractor;
- machining;
- external inspection;
- temporary utilities.

It may record:

- required service;
- provider;
- provision time;
- delivered scope;
- acceptance evidence;
- direct operational cost.

Keep procurement outside this Product Authority

Waitech must keep procurement outside this Product Authority.

Waitech may:

- establish material demand;
- authorize replenishment demand;
- send demand to ERP or procurement;
- retain external references;
- receive fulfillment information;
- record Inventory receipt.

The following remain external. Adding any of them requires an explicit Product Authority amendment:

- supplier selection;
- quotations;
- purchase orders;
- invoice approval;
- payment;
- tax;
- accounting liability.

Provide traceable operational cost evidence without claiming accounting authority

Waitech must provide traceable operational cost evidence without claiming accounting authority.

Inventory may provide operational cost snapshots for stock Movements and Work Order history.

Where available, preserve:

- unit cost;
- total cost;
- currency;
- cost source;
- costing basis or method;
- effective time;
- external reference.

Possible sources include:

- ERP-provided valuation;
- receipt cost;
- maintained operational standard cost;
- authorized manual estimate.

Keep operational costing and accounting valuation separate

Waitech must keep operational costing and accounting valuation separate.

Inventory cost evidence must not be presented as automatically equal to:

- general-ledger value;
- tax basis;
- invoice value;
- replacement cost;
- financial inventory valuation.

Accounting systems remain authoritative for financial valuation.

Attribute Inventory cost to Work Orders without transferring source authority

Attribute Inventory cost to Work Orders without transferring source authority.

A Work Order may aggregate the historical material cost snapshots associated with its issued, used, returned, or disposed Items.

Inventory Movement remains authoritative for the underlying custody and cost facts. Material Usage remains authoritative for how material contributed to the work.

Preserve historical operational cost meaning

Waitech must preserve historical operational cost meaning.

Historical Movements and Work Order cost attribution retain the cost source and basis applied when the evidence was recorded or finalized.

Corrections require explicit source-owned actions and must not silently rewrite history.

Related documents

- [Completion, verification, and closeout](#)
- [Stock Locations and Inventory Movements](#)
- [Reservations and availability](#)

Process material usage

Process Operations may use Inventory-controlled material during an Operational Action. Inventory owns physical custody and Movement; Process owns the operational reason, action, and outcome associated with that use.

A Process Material Usage

Process Material Usage is Process-owned evidence that an Operational Action consumed or otherwise used an Inventory Item.

It may preserve:

- exactly one Operational Action;
- related Process Condition where the Action belongs to one;
- Item;
- actual used quantity and unit;
- relevant Process Unit or Stream context;
- recording Actor;
- operational reason;
- linked Inventory Movement;
- correction evidence.

Process Material Usage does not own stock balance or Stock Location custody.

Post custody through Inventory

When Process use changes tracked stock, the physical quantity effect must be represented through an Inventory Movement under Inventory authority.

Process must not directly decrement stock or create an alternate Process balance.

The accepted cross-domain action must preserve both meanings:

- Process: why the material was used and what operational action it supported;
- Inventory: what physical quantity moved from which Stock Location under which Actor and reason.

Keep chemical-process meaning bounded

Waiotech may use Process Material Usage for treatment chemicals and other governed operational supplies where the plant tracks them through Inventory.

Process Material Usage is not a dosing-control system, laboratory ledger, purchasing record, or financial accounting journal.

Correction

Correcting Process usage and correcting Inventory custody are related but distinct actions.

A Process correction must not silently reverse a posted Inventory Movement. Where physical custody also requires correction, Inventory records the governed corrective Movement explicitly.

Related documents

- [Operational Actions, outcomes, and responsibility](#)
- [Stock Locations and Movements](#)

Replenishment Requests

A Replenishment Request records a governed need to increase stock availability. It does not make Waiotech a procurement system or claim external purchasing fulfillment.

A Replenishment Request

Waiotech must use **Replenishment Request** instead of the ambiguous term Restock.

A Replenishment Request is authorized demand to replenish an Item at a destination Stock Location.

It may record:

- Item;
- destination Stock Location;
- requested quantity and unit;
- required-by time;
- reason;
- requester;
- approving Actor where required;
- external procurement reference.

It is not a purchase order and does not prove that stock has arrived.

Keep replenishment demand separate from receipt evidence

Waiotech must keep replenishment demand separate from receipt evidence.

A Replenishment Request is fulfilled through one or more posted receipt Movements linked to it.

Partial fulfillment may remain visible until:

- the requested quantity is received;
- the remaining demand is cancelled;
- an authorized Decision accepts the shortfall.

Start with Replenishment Request and add Recommendation only for a concrete automation workflow

Start with Replenishment Request and add Recommendation only for a concrete automation workflow.

Only when automated reorder suggestions are in scope.

A Replenishment Recommendation would be an unaccepted system suggestion. A Replenishment Request is authorized demand.

The two concepts should not be introduced unless the distinction is operationally required.

Keep replenishment demand active until fulfilled or explicitly cancelled

WaioTech must keep replenishment demand active until fulfilled or explicitly cancelled.

Replenishment Request uses:

```
active → fulfilled  
active → cancelled
```

Partial fulfillment is derived from linked receipt quantities and does not require another lifecycle state.

Cancellation preserves received quantity, external references, Actor, time, and reason.

Related documents

- [Items, categories, and quantities](#)
- [Stock Locations and Inventory Movements](#)
- [Product boundaries](#)

Stocktaking

A Stocktaking Run governs observed counts, frozen submissions, variance evaluation, approval, and posting through explicit Inventory Movements.

A Stocktaking Run

Waitech must use Stocktaking Run as the controlled source of stocktake adjustments.

A Stocktaking Run is a governed process for counting stock positions, reviewing differences, authorizing variances, and posting resulting adjustments.

It does not directly edit Stock Balances.

A Count Observation

Waitech must preserve every physical count attempt.

A Count Observation is immutable evidence of one physical count.

It should preserve:

- Stock Location;
- Item;
- lot or condition where applicable;
- counted quantity and unit;
- counter;
- count time;
- evidence;
- relationship to a prior observation when a recount supersedes it.

A recount creates another Count Observation rather than editing the original.

A frozen stocktake submission

Waitech must keep the reviewed count set stable.

Submission establishes the exact set of Count Observations presented for review.

After submission, those observations cannot be silently added, removed, or edited.

Corrections require reopening where allowed, recounting, or creating an explicit revised submission.

Preserve count-time variance and rebase its adjustment against stock at posting time

Waiotech must preserve count-time variance and rebase its adjustment against stock at posting time.

Variance is calculated against expected stock at the Count Observation time.

Movements occurring after the count but before posting must not be treated as count variance.

Example:

```
Expected at count time: 100  
Counted: 97  
Variance: -3
```

```
Receipt after count: +20  
Stock before posting: 120  
Posted adjustment: -3  
Result: 117
```

Keep approval and posting as distinct actions

Waiotech must keep approval and posting as distinct actions.

Approval accepts the reviewed variance and authorizes its stock consequence.

Posting creates the resulting adjustment Movements.

Tenant policy may allow the same Actor to approve and post low-risk stocktakes. Separation of duties is not universally mandatory.

Preserve consistency between approved variance and posted stock consequences

Waiotech must preserve consistency between approved variance and posted stock consequences.

No unexplained partial result may remain visible.

A posting action must either establish all approved adjustment consequences or report failure without presenting the Stocktaking Run as fully posted.

Engineering Authority determines transaction and recovery mechanisms.

The Stocktaking Run lifecycle

Waiotech must separate counting, review, approval, and posting as durable Stocktaking conditions.

The canonical lifecycle is:

```

counting → submitted → approved → posted
submitted → returned_for_recount → counting
counting / submitted / returned_for_recount → cancelled

```

- **Counting:** Count Observations may be recorded.
- **Submitted:** the exact count set is frozen for review.
- **Returned for recount:** review requires new Count Observations.
- **Approved:** variance is authorized but has not yet changed stock.
- **Posted:** all approved adjustment consequences are represented by posted Inventory Movements.
- **Cancelled:** the run ended without posting stock consequences.

The minimum Inventory and material model

Waiotech must keep Inventory focused on stock custody, availability, movement, and operational cost evidence while keeping tools, services, procurement, serialized resources, and accounting outside its scope defined by the governing record.

The minimum model is:

```

Item
├─ Item Category
├─ Item Policy
└─ allowed Unit Conversions

Stock Location

Inventory Movement
├─ receipt
├─ issue
├─ return
├─ transfer
├─ adjustment
└─ disposal

Reservation

Work Order
├─ Material Requirement
├─ Material Usage
├─ Tool Requirement
└─ Service Requirement

Replenishment Request
└─ Receipt Movements

Stocktaking Run
├─ Count Observations
├─ variance review
├─ approval
└─ Adjustment Movements

```

The governing distinctions are:

- Item identifies a physical supply.
- Posted Inventory Movement owns stock truth.

- Stock Balance and availability are derived.
- Reservation protects availability without moving stock.
- Material Requirement records technical demand.
- Inventory issue records custody.
- Material Usage records actual work consumption.
- Replenishment Request records authorized demand, not procurement.
- Inventory cost is operational evidence, not accounting valuation.

Related documents

- [Stock Locations and Inventory Movements](#)
- [Decisions, waivers, and separation of duties](#)

Reliability

Reliability records actual functional failure, restoration, investigation, cause assessment, and recurrence, then derives learning across the Plant Model, Process Operations, Maintenance, and Inventory.

Contents

- Failure Events
- Restoration, cause, and recurrence
- Reliability analysis

Failure Events

A Failure Event is the authoritative Reliability record of an actual loss or materially unacceptable degradation of required plant function. It records what failed and its operational consequence; Maintenance records the accepted physical work performed in response.

A Failure Event

A Failure Event has exactly one primary failure target:

- Functional Location; or
- Asset.

Use Functional Location when the stable installed function failed and the responsible physical object is unknown, secondary, or replaceable without changing the failed operational position.

Use Asset when one identified physical Asset itself lost or materially degraded its required function and that physical identity matters to reliability history.

The event preserves:

- failed or degraded required function;
- occurrence or effective failure time where known;
- detection time where different;
- failure mode where known;
- operational consequence;
- operational condition;
- investigation condition;
- restoration and terminal-resolution evidence;
- related Process evidence;
- supporting Findings;
- linked Work Requests and Work Orders;
- Cause Assessments;
- recurrence and duplicate relationships;
- required follow-up.

Reserve Failure Event for actual functional failure

A Failure Event should be created when required function is lost or materially unacceptable, including when:

- the stable installed function cannot be provided;
- an Asset cannot perform its required function;
- performance is outside an accepted functional limit to a materially unacceptable degree;

- the failure causes a material operational interruption or consequence.

A Failure Event should not be created merely because there is:

- a defect;
- an alarm;
- an abnormal Reading;
- a Process Condition;
- a Finding;
- a failed inspection result;
- a corrective Work Order.

When required function remains acceptable, Process Condition, Finding, or Work Request may be appropriate instead.

Permit direct creation and evidence-based creation

A Failure Event may be created directly when failure is evident. It may also be established from a Process Condition, Finding, operator report, or other preserved evidence when the failure threshold is met.

No precursor record is mandatory.

Keep Process Condition, Finding, Failure Event, and Work Order distinct

- Process Condition records a meaningful operational situation requiring Process handling or monitoring.
- Finding records a maintainable condition discovered in Maintenance or inspection context and requiring maintenance disposition.
- Failure Event records actual functional failure and consequence.
- Work Order records accepted maintenance work.

One real-world situation may legitimately involve all four records. Waiotech connects them instead of collapsing them.

Use separate operational and investigation conditions

Failure Event does not use one lifecycle status.

It has independent operational and investigation state dimensions because plant function may be restored while investigation remains open.

The operational condition is:

```
active_failure → temporarily_restored → operationally_resolved
active_failure → operationally_resolved
temporarily_restored → active_failure
```

The investigation condition is:

```
not_required → required  
required → in_progress → concluded  
concluded → in_progress
```

An overall open or resolved view may be derived but must not become independently edited truth.

Preserve authoritative failure timing

Failure occurrence, detection, maintenance execution, temporary restoration, renewed failure, full restoration, and terminal operational resolution are different facts.

Reliability timing must not be inferred solely from Work Order start and completion times.

Related documents

- [Restoration, cause, and recurrence](#)
- [Findings](#)
- [Process Conditions](#)

Restoration, cause, and recurrence

Failure Event restoration, investigation, Cause Assessments, duplicate handling, and recurrence preserve the difference between returning function, understanding why failure occurred, and learning whether the event repeats.

Temporary restoration

Temporary restoration means acceptable function is available under an explicit temporary limitation while permanent resolution remains outstanding.

It must preserve:

- restoration time;
- restoring Work Order or other authorized basis;
- restoration evidence;
- remaining limitation;
- operating constraints;
- accountable owner;
- inspection, review, or expiry obligation;
- permanent follow-up.

Temporary restoration does not erase the Failure Event or represent permanent resolution.

A renewed failure returns the operational condition to `active_failure` and preserves the entire temporary-operation interval.

Terminal operational resolution

A Failure Event may become `operationally_resolved` when the active operational obligation ends through an exact outcome such as:

- required function restored;
- physical Asset replaced;
- target decommissioned or retired from required service;
- another authorized terminal resolution.

Replacement or decommissioning must not be falsely described as restoration.

Investigation proportionality

Investigation may be required because of consequence, recurrence, Operational Criticality, safety or environmental significance, compliance, repeated temporary repair, management Decision, or another explicit policy rule.

The decision that investigation is required or not required preserves deciding Actor, time, rationale, and governing rule where applicable.

New evidence may escalate a prior `not_required` conclusion to required investigation without rewriting the earlier decision.

A concluded investigation may be reopened to `in_progress` when material new evidence requires further investigation. Reopening preserves the prior conclusion, reopening Actor, time, reason, and new evidence; it does not edit the earlier investigation history.

Cause Assessments

A Cause Assessment is immutable evidence of one accepted reliability conclusion about why the Failure Event occurred.

It may preserve:

- failure mode;
- assessed primary cause;
- contributing factors;
- confidence;
- method;
- assessor;
- evidence;
- assessment time;
- explicit supersession of an earlier assessment.

Waiotech must not force false certainty. Valid conclusions include confirmed, probable, multiple contributing causes, inconclusive, or no cause identified after proportionate investigation.

Keep mode, cause, and consequence distinct

- **Failure mode:** how required function was lost or degraded.
- **Cause:** why the failure occurred.
- **Consequence:** what operational effect resulted.

Narrative may supplement these facts but must not collapse their meaning.

Duplicate evidence

When several reports describe the same failure occurrence, Waiotech preserves source evidence without counting several Failure Events.

One event may be designated primary and another candidate linked as duplicate with source, Actor, time, evidence, and duplicate decision preserved.

Duplicate determination is based on the same occurrence, not merely the same target or failure mode.

Recurrence relationships

A new functional failure after operational resolution normally creates a new Failure Event.

A Failure Event may link to prior events using explicit recurrence meaning such as:

- repeat failure;
- continuation of unresolved failure;
- common-cause event;
- related distinct occurrence.

The relationship preserves rationale, deciding Actor, and decision time.

Closure

A Failure Event is considered resolved only when:

- the operational condition is `operationally_resolved`;
- required investigation is concluded or explicitly not required;
- mandatory follow-up is represented by governed records with accountable ownership;
- unresolved temporary limitations are explicitly controlled;
- no Reliability closure blocker remains.

Linked follow-up Work Orders do not always need to be closed unless the governing rule requires them before Reliability resolution.

Related documents

- [Failure Events](#)
- [Reliability analysis](#)

Reliability analysis

Reliability analysis derives learning from authoritative Plant Model, Process, Maintenance, Inventory, and Failure Event evidence. Derived measures never replace or rewrite the source facts from which they are calculated.

Derive reliability rather than maintaining duplicate truth

Waiotech may derive measures such as:

- failure recurrence;
- mean time between failures where the data semantics support it;
- mean time to restoration;
- downtime;
- repeat failure rate;
- dominant failure modes;
- cause patterns by Asset Classification;
- temporary-restoration frequency;
- failure consequence trends;
- Process Conditions or Reading patterns associated with failures;
- maintenance response and verification patterns;
- restoration delay associated with unavailable material;
- failure concentration by Functional Location or Asset.

Derived analytics must identify their time window, population, source rules, and evaluation time sufficiently to support correct interpretation.

Connect operating condition to failure and recovery

Reliability views may relate Process evidence before, during, and after a Failure Event.

Examples include:

- selected Readings before failure;
- open Process Conditions at detection;
- operating consequence;
- maintenance response;
- Readings after return to service;
- whether Process recovery followed technical repair.

The full historian need not be copied into Waiotech for these relationships to be meaningful.

Preserve the stable-position and physical-Asset distinction

Analysis must distinguish:

- repeated failure of the same Functional Location across several installed Assets;
- repeated failure of one physical Asset across several installations;
- failure of a child Asset within a larger assembly;
- repeated failure affecting one Process Unit through different physical causes.

This distinction is a principal reason Functional Location and Asset identities are separate.

Do not present weak data as precise reliability truth

A metric must not imply confidence that the underlying evidence does not support.

Missing failure occurrence time, unknown restoration, uncertain Reading quality, incomplete installation history, or insufficient observation windows must remain visible in interpretation where they materially affect a result.

Related documents

- [Failure Events](#)
- [Assets and installation](#)
- [Process Readings and Operational Observations](#)

Evidence and communications

Reports, Notifications, Attachments, operational evidence, and audit browsing.

Contents

- [Reports](#)
- [Notifications](#)
- [Attachments and evidence](#)
- [Audit browsing](#)

Reports

Report Types define governed report meaning and temporal semantics. Report Runs evaluate current authority, produce immutable artifacts, and preserve completion, failure, cancellation, and retrieval evidence.

Alpha Report Type catalogue

The Alpha catalogue is closed and contains:

- Work Order backlog;
- overdue Scheduled Work;
- maintenance cost;
- Process Condition history;
- Process Round history;
- Failure Event history;
- stock availability;
- Inventory Movement history.

Each Report Type owns its filters, temporal meaning, columns, grouping, sorting, sensitivity, CSV/PDF support, retention, and maximum execution limits. Adding a Report Type requires a Product Authority amendment and complete implementation.

A Report Type

Waiotech must keep Report Types product-defined and governed.

A Report Type is a product-defined governed analytical view.

It defines:

- authoritative source facts;
- supported measures;
- supported filters;
- supported grouping and sorting;
- temporal and as-of semantics;
- sensitivity classification;
- available output formats;
- export controls.

Examples may include:

- Work Order backlog;
- maintenance cost;

- Failure Event history;
- Process Round history;
- overdue Scheduled Work;
- stock availability;
- Inventory Movement history.

Tenants cannot create arbitrary joins, calculations, or source queries.

A Saved Report Configuration

WaioTech must allow reusable report configuration without introducing a general-purpose query builder.

A Saved Report Configuration is a reusable selection of options supported by a Report Type.

It may contain:

- filters;
- selected columns;
- grouping;
- sorting;
- date range;
- output format;
- display preferences.

It does not define new business meaning, joins, calculations, or authoritative facts.

A configuration may belong to a Tenant or individual User according to its visibility rules.

Provide controlled analytical flexibility rather than unrestricted querying

WaioTech must provide controlled analytical flexibility rather than unrestricted querying.

An unrestricted builder could create:

- inconsistent calculations;
- unsupported joins;
- alternate definitions of product metrics;
- access-control mistakes;
- misleading reports presented as authoritative.

WaioTech may provide flexible filtering, grouping, supported calculations, and governed analytical views within each Report Type.

Treat reports as reproducible derived outputs, not alternate truth sources

WaioTech must treat reports as reproducible derived outputs, not alternate truth sources.

A report result is a derived view of authoritative records under a defined:

- Report Type;
- revision;
- parameter set;
- Tenant context;
- temporal rule;
- data cutoff;
- access context.

The underlying domain records remain authoritative.

A report artifact may prove that a particular derived view was produced at a particular time, but it does not replace the source facts.

Define time and as-of meaning separately for each Report Type

WaioTech must define time and as-of meaning separately for each Report Type.

Every Report Type must define how time affects its result.

Possible semantics include:

- state at execution time;
- lifecycle state at a historical cutoff;
- events effective during a selected period;
- actions recorded during a selected period;
- transactions posted before a cutoff;
- historical cost attributed during a period.

A generic date range must not be assumed to mean the same thing for every report.

A Report Run

WaioTech must make every generated report reproducible and traceable to its exact semantics.

A Report Run is one execution of a Report Type using an exact configuration and parameter set.

It preserves:

- Report Type and exact product version;
- configuration snapshot;
- actual parameters;
- requesting Actor;

- Tenant context;
- request time;
- start and completion times;
- as-of rule;
- data cutoff;
- output format;
- sensitivity classification;
- result artifact;
- failure evidence where applicable.

The Report Run lifecycle

Waiotech must use a small execution lifecycle with truthful cancellation semantics.

The lifecycle is:

```
queued → running → completed
queued → cancelled
queued / running → failed
running → cancelled
```

A running Report Run enters `cancelled` only when execution is successfully stopped and no completed artifact is produced.

A completed Report Run cannot be cancelled.

Do not label a completed run as cancelled merely because cancellation was requested

Waiotech must not label a completed run as cancelled merely because cancellation was requested.

Cancellation is successful when the run stops without producing a completed report artifact.

For queued work, cancellation prevents execution.

For running work, cancellation requests termination. When termination cannot be completed and the report finishes successfully, the run remains `completed`.

Keep failed report evidence clear without exposing unsafe technical details

Waiotech must keep failed report evidence clear without exposing unsafe technical details.

A failed Report Run should preserve:

- failure classification;
- safe explanatory message;

- failure time;
- retry relationship where applicable;
- technical diagnostic reference where appropriate;
- absence or invalidity of any partial artifact.

Partial output must not be presented as a completed report unless the Report Type explicitly supports partial results.

Avoid unnecessary approval workflows for ordinary reporting

Avoid unnecessary approval workflows for ordinary reporting.

Ordinary report generation requires:

- applicable Permission;
- access to the requested information at execution time;
- valid supported parameters;
- compliance with Report Type sensitivity controls.

Approval is required only when a specific sensitive report or export mode explicitly requires it.

Checked when a report is generated

Waiotech must authorize the requested report at execution time.

At generation time, Waiotech evaluates:

- requesting Actor;
- Tenant context;
- applicable Permissions;
- access to the requested data;
- selected parameters;
- Report Type sensitivity;
- export or approval requirements.

The resulting artifact contains only the data authorized for that run.

Freeze report content, not access rights at retrieval

Waiotech must freeze report content, not access rights at retrieval.

The report content is frozen at completion, but access to that artifact is evaluated whenever it is retrieved or exported.

A User whose access ends after artifact creation must not retain retrieval authority merely because they generated or accessed the report previously.

Checked when a Report Artifact is retrieved

Recheck authority at retrieval and export.

Retrieval should evaluate:

- Tenant participation effective at retrieval;
- Permission effective at retrieval;
- Report Type sensitivity controls effective at retrieval;
- access to the artifact's governing report context at retrieval;
- export-specific controls where applicable;
- artifact availability and integrity.

Access is not inherited permanently from the Actor who generated the report.

Keep completed report content immutable

WaioTech must keep completed report content immutable.

A completed Report Artifact represents the exact result produced by that Report Run.

Changes to source records, policy, access, Report Type version, or saved configuration do not mutate the existing artifact.

A new result requires a new Report Run.

Define export controls explicitly per Report Type and output mode

WaioTech must define export controls explicitly per Report Type and output mode.

Sensitivity is defined by the Report Type and output mode.

Sensitive exports may include:

- personal information;
- security or access configuration;
- Failure Event investigations;
- authorization evidence;
- bulk operational records;
- commercially restricted information.

A sensitive export may require:

- additional Permission;
- reason;
- destination classification;
- approval;
- identifying metadata or watermark;
- expiry;

- export audit evidence.

Do not treat every generated report as permanent business evidence

WaioTech must not treat every generated report as permanent business evidence.

Not automatically.

A Report Artifact is normally a reproducible derived output.

It becomes governed evidence only when an owning domain explicitly links it as evidence for a Decision, action, investigation, or other controlled subject.

The underlying source records remain authoritative regardless.

The relationship between Report Artifact and Attachment

WaioTech must separate report-output meaning from file identity while keeping a one-to-one artifact file relationship.

Report Artifact owns the business meaning of one completed report output. Attachment owns the accepted file identity, integrity, quarantine status, and storage availability.

The relationship is:

```
graph TD; A[Report Run] --- B[Report Artifact]; B --- C[exactly one Attachment];
```

Replacing the file requires a new Report Artifact and a new Attachment. The original remains preserved.

Related documents

- [Notifications](#)
- [Attachments and evidence](#)
- [Report and Notification processing](#)

Notifications

A Notification is a recipient-specific product communication created from a governed Notification Type. Reading and external delivery are communication facts and do not directly mutate business state.

Alpha Notification Type catalogue

The Alpha catalogue contains:

- Process Condition assigned or reassigned;
- Process Condition attention raised to urgent or critical;
- Process Condition monitoring review due;
- Process Round assigned, due, or overdue where product-defined delivery is warranted;
- Work Order assigned, released, and placed on hold;
- Work Request information required, accepted, and rejected;
- independent verification required;
- Scheduled Work overdue;
- Failure Event investigation or restoration action requiring attention;
- replenishment or Inventory shortfall requiring attention;
- Report completed and failed;
- Tenant Export completed and failed;
- Tenant Import completed and failed;
- Android synchronization conflict requiring User action.

Alpha delivery channels are in-app delivery in Dashboard and Admin, in-app delivery in Android, and Android push as a delivery hint. Email, SMS, arbitrary Tenant-defined triggers, and webhook delivery are not Alpha channels. Opening a Notification navigates to the normal authorized workflow and never performs the action.

A Notification Type

WaioTech must use product-defined Notification Types rather than configurable notification rules.

A Notification Type is a product-defined communication contract for a governed product event.

It defines:

- triggering event;
- recipient-resolution rule;
- content structure;
- mandatory or optional nature;
- available delivery channels;
- deduplication behavior;

- related subject;
- available navigation or action behavior.

Tenants cannot define arbitrary event triggers, audience queries, scripts, or business actions.

A Notification Preference

Waiotech must allow preferences only within product-defined bounds.

A Notification Preference records the recipient's or Tenant's allowed choices for a Notification Type.

Preferences may control:

- enabled optional notifications;
- permitted channels;
- digest versus immediate delivery;
- quiet or delivery periods where supported.

Preferences cannot disable mandatory Notification Types or weaken required product communication.

Keep mandatory product communication non-optional

Waiotech must keep mandatory product communication non-optional.

Each Notification Type declares whether it is:

- mandatory;
- channel-configurable;
- optional;
- user-subscribable.

A mandatory Notification must remain available through at least its required product channel.

Users may configure optional channels only where permitted.

A Notification

Waiotech must use Notification as a communication record, not as a duplicate workflow state.

A Notification is the recipient-visible record that a governed product event requires their awareness or attention.

It preserves:

- recipient;
- Notification Type;
- related subject;
- creation time;
- rendered content or rendering context;
- read state at evaluation time;

- action or navigation destination.

Keep Notification state minimal

Waiotech must keep Notification state minimal under Waiotech Product Authority.

The recipient-visible lifecycle is:

unread → read

States such as approved, completed, acted, or resolved belong to the related business subject, not the Notification.

Keep delivery mechanics separate from recipient-visible business meaning

Waiotech must keep delivery mechanics separate from recipient-visible business meaning.

Email, push, or other channel delivery may have technical outcomes such as:

- queued;
- delivered;
- failed;
- bounced;
- retried.

Those outcomes do not redefine:

- whether the Notification exists;
- whether the business event occurred;
- whether the Notification is read.

Use Notifications as entry points to normal authorized actions

Waiotech must use Notifications as entry points to normal authorized actions.

A Notification may link to or invoke a normal domain action, but that action must re-evaluate:

- lifecycle state at evaluation time;
- Permission effective at retrieval;
- separation-of-duty conditions at evaluation time;
- Requirements effective at evaluation time;
- blockers at evaluation time;
- policy effective at evaluation time.

A Notification does not preserve authority from the time it was created.

Related documents

- [Reports](#)
- [Attachments and evidence](#)
- [Android Work App](#)

Attachments and evidence

An Attachment is an immutable accepted file identity linked to governed subjects through explicit roles. Access, integrity, quarantine, correction, unavailability, and preservation remain explicit.

An Attachment

WaioTech must keep file identity separate from domain meaning.

An Attachment is the stable identity of one accepted file.

It owns file-level facts such as:

- Tenant;
- filename;
- media type;
- size;
- integrity identifier;
- uploader;
- upload time;
- classification;
- integrity and quarantine status;
- storage availability.

Attachment does not define why the file matters to the business.

Maintenance-specific upload and linking

Maintenance and Process upload are governed workflows. Supported evidence subjects include Work Requests, Work Orders, Tasks and completion evidence, Findings, Process Conditions, Operational Observations, Operational Actions, Failure Events, Procedure Revisions, and Android offline evidence.

```
local file
-> Upload Session
-> private storage
-> integrity verification
-> safety and quarantine evaluation
-> accepted Attachment
-> module-owned Attachment Link
-> audited result
```

A local or uploaded file is not presented as accepted evidence before Attachment acceptance and successful domain-specific linking. The link command verifies the selected Tenant, subject authority, allowed role, classification, sensitivity, file availability, duplicate link, and action-specific evidence requirement in one transaction. Quarantined, rejected, expired, or unavailable files cannot satisfy a Requirement.

An Attachment Link

Let the owning domain link define business meaning.

An Attachment Link connects an Attachment to a governed product subject and defines the file's business role.

Possible roles include:

- completion evidence;
- inspection evidence;
- Finding evidence;
- Procedure instruction;
- Decision evidence;
- Report Artifact;
- reference document;
- supporting photograph.

The link preserves:

- subject;
- role;
- added by;
- added at;
- description;
- governing status;
- correction or supersession relationship where applicable.

Require explicit governed evidence linkage

Waitech must require explicit governed evidence linkage.

A file satisfies an evidence Requirement only when:

- it is linked through the correct governed evidence role;
- the relevant Requirement accepts that evidence type;
- the file is available;
- integrity is established;
- it is not blocked by quarantine;
- required metadata is present.

Uploading a file somewhere on a Work Order does not automatically satisfy a Requirement.

The governing subject of an Attachment

Waitech must use one governing subject to prevent access bypass through weaker links.

Every Attachment must have one governing subject that determines:

- minimum access;
- business classification;
- sensitivity;
- primary evidence meaning;
- retention dependency under a Product Authority amendment defining retention.

Additional links may reference the file for navigation or context, but they cannot weaken the governing subject's access controls.

Permit controlled reuse without access inheritance from weaker links

Permit controlled reuse without access inheritance from weaker links.

An Attachment may support several related subjects where that reuse is legitimate.

However:

- its governing subject remains authoritative for minimum access;
- additional links cannot broaden access;
- each link must state its own business role;
- sensitive evidence must not become accessible through a less restricted subject.

Recheck governing-subject and file-level authority on every retrieval

Recheck governing-subject and file-level authority on every retrieval.

Access requires:

- access to the governing subject at retrieval;
- applicable Attachment or evidence Permission where required;
- compliance with sensitivity classification;
- acceptable integrity status;
- acceptable quarantine status;
- Tenant context at retrieval.

Access through an additional link must never bypass these checks.

Correct evidence by addition and supersession, never in-place replacement

Correct evidence by addition and supersession, never in-place replacement.

A file accepted under one Attachment identity must remain historically stable.

Correction creates:

- a new Attachment;
- a new Attachment Link;
- correction or supersession relationship;
- correction reason;
- correcting Actor;
- correction time.

The original file and its historical role remain visible.

Prevent metadata edits from rewriting evidence meaning

Waiotech must prevent metadata edits from rewriting evidence meaning.

Material metadata corrections must preserve history.

Minor display corrections may be allowed where they do not change business meaning.

Changes to any of the following require an explicit correction action or new link rather than silent editing:

- evidence role;
- governing subject;
- sensitivity;
- uploader attribution;
- accepted file identity;
- integrity facts.

File integrity

Waiotech must make verifiable file identity mandatory for governed evidence.

File integrity means Waiotech can establish whether the retrieved file is the same file accepted under that Attachment identity.

Product Authority requires reliable integrity verification.

Engineering Authority defines the storage, hashing, and verification mechanisms.

When integrity cannot be established, the Attachment cannot satisfy governed evidence Requirements.

Quarantine

Waiotech must treat quarantine as restricted file usability, not silent deletion.

Quarantine means the Attachment is unavailable for normal use because its safety, integrity, or content status is unresolved or unacceptable.

Possible reasons include:

- suspected malware;

- integrity mismatch;
- unsafe content;
- failed security review;
- unsupported or unresolved content processing.

A quarantined file:

- cannot be normally viewed or downloaded;
- cannot satisfy an evidence Requirement;
- remains visible as a restricted historical reference;
- may be released after governed review or permanently rejected through a governed review.

Separate file usability from evaluation of the underlying business claim

Waiotech must separate file usability from evaluation of the underlying business claim.

Quarantine means the file cannot be trusted at evaluation time or safely used.

It does not by itself prove that the related observation, action, or business claim is false.

The owning domain may require replacement evidence, review, or another corrective action.

Support explicit quarantine review and release

Waiotech must support explicit quarantine review and release when an authorized review establishes that the file is safe, intact, and acceptable.

Release from quarantine must preserve:

- reviewing Actor;
- review time;
- evidence;
- resulting status.

A file that remains unsafe or unverifiable may be permanently rejected from normal use while its historical link remains visible.

Preserve file identity and evidence history even when binary content is unavailable

Waiotech must preserve file identity and evidence history even when binary content is unavailable.

When a file cannot be retrieved because of technical loss or corruption, the Attachment record and domain links must remain visible.

The product should clearly distinguish:

- available;

- quarantined;
- unavailable;
- rejected from normal use.

Unavailability must not be presented as though the Attachment never existed.

Configurable retention and physical destruction are outside this Product Authority and require a complete Product Authority contract

Configurable retention and physical destruction are outside this Product Authority and require a complete Product Authority contract.

Retention and physical destruction require a dedicated contract covering:

- evidence classes;
- legal or contractual holds;
- privacy;
- deletion;
- anonymization;
- backups;
- integration copies;
- destruction authority;
- historical proof.

A simple Tenant-configurable retention period would be incomplete and unsafe.

Preserve governed evidence by default

Waiotech must preserve governed evidence by default. Any destruction or configurable retention requires a complete Product Authority contract.

Attachments and governed evidence remain preserved while required by their owning records and mandatory product-integrity rules.

Ordinary users cannot physically destroy governed evidence.

A file may become quarantined or technically unavailable, but its Attachment identity, governing link, and historical meaning remain preserved.

Allow report outputs to support evidence without changing source authority

Waiotech must allow report outputs to support evidence without changing source authority.

A completed Report Artifact may be linked as evidence to a Decision, investigation, Work Order, or other governed subject.

The link must preserve:

- Report Run;
- Report Type version;
- parameters;
- as-of semantics;
- artifact identity;
- evidence role.

The artifact remains derived evidence and does not replace its underlying source records.

The minimum Report, Notification, and Attachment model

Waiotech must keep reporting, communication, and file evidence governed without introducing unrestricted querying, configurable event rules, access bypass, or premature retention behavior.

The minimum model is:

```

Report Type
├─ Saved Report Configuration
│   └─ Report Run
│       └─ Report Artifact

Notification Type
├─ Notification Preference
└─ Notification
    └─ normal authorized domain action

Attachment
├─ Attachment Link
│   ├── governing subject
│   ├── evidence
│   ├── instruction
│   ├── Report Artifact
│   └─ reference

```

The governing distinctions are:

- Report Type defines a product-governed analytical view.
- Saved Report Configuration selects supported options.
- Report Run creates one frozen derived result.
- Report Artifact is not an authoritative source record.
- Notification communicates a product event but does not own its workflow.
- Attachment owns file identity and integrity.
- Attachment Link defines business meaning.
- One governing subject controls minimum access.
- Evidence correction adds new records and preserves originals.
- Configurable retention and physical destruction are outside Waiotech Product Authority.

Related documents

- [Reports](#)
- [Tenant portability](#)
- [File ingestion and storage](#)

Audit browsing

WaioTech records material actions as immutable audit evidence and provides separate Tenant and platform browsing experiences.

Audit Event

An Audit Event preserves at least:

- stable event identity;
- effective Tenant where applicable;
- real Actor kind and identity;
- Membership or platform authority basis where applicable;
- action code and owning module;
- subject type and identity;
- effective time and recorded time;
- request, operation, correlation, and causation identities;
- reason and Decision references where required;
- outcome and safe failure classification;
- bounded before/after or evidence references required by the action.

Audit is business and governance evidence, not a copy of application logs or raw HTTP payloads. Audit Events are append-only and correction or explanatory events never rewrite prior evidence.

Tenant audit browsing

Authorized Tenant Users may browse only Audit Events belonging to their selected Tenant. Filters include time range, Actor, action, module, subject, outcome, correlation, and reason code. Results use stable cursor pagination and preserve a deterministic order.

Tenant browsing never exposes another Tenant, platform-only security details, secrets, raw tokens, storage keys, unrestricted personal data, or unredacted request bodies.

Platform audit browsing

Authorized platform Users may browse platform-owned Audit Events and narrowly inspect Tenant audit metadata through dedicated platform contracts. Platform browsing does not create Tenant Permission or permit ordinary Tenant actions. Every sensitive audit query is itself audited with purpose and affected Tenant.

Access and redaction

Audit access is checked for every query and export. Redaction is defined by the owning event schema and remains stable across UI, API, Reports, and export. Authorization changes may remove future access without altering historical audit content.

Retention and export

Audit retention follows preservation policy and legal obligations. Audit may be included in Tenant Export through its versioned portability contract. Audit browsing does not provide arbitrary bulk extraction outside governed Report or Export actions.

Related documents

- [Audit events and outbox](#)
- [Authorization enforcement](#)
- [Tenant portability](#)

[10-PRODUCT/60-POLICY-AND-DECISIONS/000-INDEX.MD](#)

Policy and Decisions

Tenant controls, immutable Decisions, waivers, separation of duties, and governed reasons.

Contents

- [Tenant Control Policy](#)
- [Decisions, waivers, and separation of duties](#)
- [Reason codes](#)

Tenant Control Policy

Tenant Control Policy selects and configures product-defined controls for one Tenant through immutable published revisions. It cannot weaken Product Authority or create an unrestricted rule engine.

Policy

WaioTech must use policy only for configurable product-defined controls.

Policy means governed Tenant configuration that selects values for product-defined controls.

Policy may determine matters such as:

- when verification is required;
- whether separation of duties applies;
- approval thresholds;
- permitted tolerances;
- evidence requirements;
- escalation timing;
- exception conditions.

Policy does not create arbitrary business behavior, Permissions, workflow states, scripts, or domain rules.

Do not support user-defined rules, scripts, expressions, or generic workflow configuration

WaioTech must not support user-defined rules, scripts, expressions, or generic workflow configuration.

A generic rule engine would require arbitrary conditions, scopes, precedence, conflict resolution, historical evaluation, and cross-domain references.

That would introduce unnecessary complexity and make product behavior difficult to understand and enforce consistently.

A Tenant Control Policy

WaioTech must use one Tenant Control Policy per Tenant.

Tenant Control Policy is the stable identity of the Tenant's governed control configuration.

Each Tenant has one Tenant Control Policy.

Its effective published revision contains values for the product-defined controls defined by WaioTech Product Authority.

Examples may include:

- verification required for specified Operational Criticality levels;
- independent approval required for protected Access Profile changes;
- maximum Work Request deferral period;
- whether the same Actor may approve and post a low-risk Stocktaking Run;
- whether defined temporary-repair conditions require independent verification.

Keep the control catalogue product-owned and closed

WaioTech must keep the control catalogue product-owned and closed.

WaioTech defines the available controls, their meaning, value types, permitted ranges, dependencies, and operational effects.

Tenant administrators choose permitted values.

Tenants cannot create new control types or change the meaning of existing controls.

A Tenant Control Policy Revision

WaioTech must use immutable published revisions for historically significant control configuration.

A Tenant Control Policy Revision contains one exact version of all configured Tenant control values.

A revision preserves:

- control values;
- created by and created at;
- published by and published at;
- effective time;
- change reason;
- required approval evidence where applicable.

Its lifecycle is:

draft → published → superseded
draft → discarded

Draft revisions have no operational effect. Published revisions are immutable.

Preserve policy history through immutable revisions

WaioTech must preserve policy history through immutable revisions.

Policy settings may change while historical actions and Decisions must retain the exact controls that governed them.

Without revisions, configuration changes effective after an action would alter the apparent basis of previous:

- releases;
- starts;
- waivers;
- verifications;
- approvals;
- stock adjustments;
- platform recovery Decisions.

Present policy configuration as controlled edit, impact review, and publication backed by immutable revisions

Waiotech must present policy configuration as controlled edit, impact review, and publication backed by immutable revisions.

An authorized user edits the stable Tenant Control Policy, reviews the affected controls and their operational impact, and publishes the changes.

Waiotech manages the underlying Tenant Control Policy Revision automatically.

The user does not manage revision numbers, branches, merge operations, or supersession directly. Change history and the exact policy that governed a historical action remain available for evidence and interpretation.

Keep the policy identity permanent and version only its configuration

Waiotech must keep the policy identity permanent and version only its configuration.

Each Tenant always has one stable Tenant Control Policy identity.

Only its revisions change. The policy identity is not activated, paused, or retired.

A product-provided default revision may apply until the Tenant publishes another revision.

Keep policy Tenant-wide and add narrower typed behavior only through explicit product controls

Waiotech must keep policy Tenant-wide and add narrower typed behavior only through explicit product controls.

Tenant Control Policy applies within one Tenant.

The canonical model does not support generic policy scope by:

- Asset;
- Functional Location;

- Work Order;
- Stock Location;
- Team;
- explicit subject;
- arbitrary classification;
- user-defined expression.

A specific product control may include typed parameters such as an Operational Criticality threshold or monetary value threshold where that parameter is part of the control's defined meaning.

Allow only explicitly designed control parameters

Waitech must allow a control parameter only when that dimension is explicitly defined by the specific product control.

For example:

```
Verification requirement:  
- never  
- for critical Functional Locations  
- for high and critical Functional Locations  
- always
```

This is a typed product setting, not a generic scope or expression system.

The precedence between product rules and Tenant policy

Waitech must use explicit product ownership instead of generic policy merging.

Precedence is simple:

1. Mandatory product rules always apply.
2. Tenant policy selects values only within product-permitted bounds.
3. Procedures, Plans, Work Orders, and Decisions may add stricter Requirements.
4. They cannot silently remove mandatory Requirements.
5. A waivable Requirement may be bypassed only through a valid Waiver Decision.

There is no generic "more specific rule wins" or configurable allow-versus-deny precedence model.

Make non-configurable product guarantees non-waivable through policy

Waitech must make non-configurable product guarantees non-waivable through policy.

Mandatory product rules protect product integrity, security, evidence, safety, and historical meaning.

A control may permit Tenant choice only within the bounds defined by the product.

Permit additive control while prohibiting silent weakening

Permit additive control while prohibiting silent weakening.

A Work Order, Procedure Revision, Plan Revision, or authorized Decision may add stricter Requirements where operational context requires them.

They must not silently remove mandatory product, policy, Procedure, or Plan Requirements.

Waiotech is not a generic workflow or policy engine

Waiotech is not a generic workflow or policy engine.

Tenant administrators may configure product-defined controls and parameters.

They cannot create:

- arbitrary conditions;
- scripts;
- expressions;
- custom workflow actions;
- custom Permission logic;
- generic approval rules;
- arbitrary resource scopes.

New control types require product design and an authoritative product contract.

Apply controls effective at evaluation time to new action evaluations

Waiotech must apply controls effective at evaluation time to new action evaluations.

New evaluations use the effective Tenant Control Policy Revision.

Protected actions such as release, start, verification, closeout, waiver, or approval must evaluate the controls effective at evaluation time.

Preserve historical control context

Waiotech must preserve historical control context.

Historical actions and Decisions retain the policy revision that governed them.

A policy change effective after the historical action must not rewrite whether a historical action was valid under the controls effective at that time.

Define existing-work impact separately for every control

Waiotech must define existing-work impact separately for every control.

Each product-defined control must declare its effect on existing Work Orders.

A policy change may:

- affect only newly created Work Orders;
- add a blocker effective at evaluation time to release, start, verification, or closeout;
- require review of released work;
- require release withdrawal when the approved execution basis is invalidated;
- have no effect on in-progress or historical work.

A blanket migration rule must not be assumed.

Distinguish invalidated approval from action readiness at evaluation time

WaioTech must distinguish invalidated approval from action readiness at evaluation time.

Release withdrawal is required only when the policy change materially invalidates the previously approved execution package.

A newly applicable temporary start blocker may prevent execution without invalidating release.

Configurable retention is outside this Product Authority and requires a dedicated Product Authority contract

Configurable retention is outside this Product Authority and requires a dedicated Product Authority contract.

Retention affects legal obligations, evidence classes, privacy, correction, deletion, anonymization, integrations, and backup behavior.

A simple configurable retention period would not adequately define those consequences.

Related documents

- [Decisions, waivers, and separation of duties](#)
- [Revisions and publication](#)

Decisions, waivers, and separation of duties

A Decision is a domain-owned immutable outcome with explicit subject, authority, reason, effect, evidence, and temporal meaning. Waivers and independent review use the same bounded Decision semantics without becoming a universal lifecycle engine.

A Decision

Waiotech must use Decisions for governed judgment, authorization, acceptance, and exception outcomes.

A Decision is immutable evidence that an authorized Actor reached a governed conclusion about a specific subject.

Examples include:

- Work Request Triage Decision;
- Work Order Verification Decision;
- Work Requirement Waiver Decision;
- Stocktaking Approval Decision;
- Failure Event investigation requirement Decision;
- temporary-repair acceptance Decision.

Standardize Decision meaning without forcing one universal entity

Standardize Decision meaning without forcing one universal entity.

Product Authority should define shared Decision semantics, but it does not require one universal polymorphic aggregate.

Each domain owns its Decisions.

For example:

- Work Request owns Triage Decisions;
- Work Order owns Verification and Waiver Decisions;
- Inventory owns Stocktaking Approval Decisions;
- Failure Event owns investigation Decisions.

Engineering Authority determines whether shared semantics use one implementation or several domain-owned implementations.

The minimum Decision contract

Waiootech must use one shared minimum semantic contract for all Decisions.

A Decision should preserve:

- decision type;
- subject;
- outcome;
- deciding Actor;
- decision time;
- recorded time where different;
- authority basis;
- applicable policy revision;
- rationale;
- supporting evidence;
- effective time;
- validity or expiry where applicable;
- prior Decision superseded or revoked where applicable.

Decision-specific contracts may require additional facts.

Preserve every effective Decision as historical evidence

Waiootech must preserve every effective Decision as historical evidence.

An effective Decision is immutable.

A subsequent action may:

- supersede it;
- revoke it where revocation is meaningful;
- reopen the governed process;
- record a correction.

The original Decision remains visible.

Decision supersession

Waiootech must use supersession when governed judgment changes.

Supersession means a newer Decision replaces an earlier Decision for interpretation after supersession.

The earlier Decision remains valid for the period in which it applied.

The newer Decision must reference the Decision it supersedes and explain the changed conclusion.

Decision revocation

Waiotech must use revocation only for Decisions with continuing effect.

Revocation ends the continuing effect of a Decision without claiming the original Decision was incorrect when made.

Examples may include:

- ending a Waiver before expiry;
- revoking an approval after its conditions cease to be valid;
- terminating temporary authority.

Revocation records:

- revoking Actor;
- revocation time;
- reason;
- resulting obligations.

Decision expiry

Waiotech must preserve time-bounded authority explicitly.

Expiry is the planned end of a Decision's validity.

After expiry, any Requirement or blocker previously controlled by the Decision becomes applicable again where still relevant.

Expiry does not delete or invalidate the historical Decision.

Share Decision structure, not one generic outcome catalogue

Share Decision structure, not one generic outcome catalogue.

Each Decision type defines its own governed outcomes.

Examples include:

- Verification: accepted, accepted with controlled follow-up, returned for rework.
- Triage: accepted, rejected, duplicate.
- Waiver: granted, denied.
- Stocktaking approval: approved, returned for recount, rejected.
- Investigation requirement: required, not required.

Represent approval through Decisions and named actions rather than adding approval states everywhere

Waiotech must represent approval through Decisions and named actions rather than adding approval states everywhere.

Usually no.

Approval is normally a Decision or an authorization action associated with an entity.

Examples include:

- release authorizes a Work Order execution package;
- verification evaluates completion;
- stocktake approval authorizes variance posting;
- waiver permits a Requirement exception.

An `approved` lifecycle state should be added only when the entity itself remains in a durable approved condition over time.

A waiver

Waiotech must use waiver only for Requirements whose contract explicitly permits the `waived` outcome.

A Waiver Decision places one explicitly waivable Work Requirement in the `waived` outcome for a named blocked action under defined conditions.

A waiver does not delete, change, or satisfy the Requirement. It preserves a distinct exception outcome and its continuing conditions.

Make every waiver narrow, traceable, and time-bounded where appropriate

Waiotech must make every waiver narrow, traceable, and time-bounded where appropriate.

A Waiver Decision must record:

- exact Work Requirement;
- blocked action;
- outcome;
- deciding Actor;
- authority basis;
- reason;
- supporting evidence;
- compensating controls;
- effective time;
- expiry;
- affected Work Order state;
- revocation where applicable.

Apply expiry prospectively without rewriting valid historical actions

WaioTech must apply expiry prospectively without rewriting valid historical actions.

When a waiver expires, the Work Requirement becomes an active blocker again if it remains applicable and unsatisfied.

Historical actions completed while the waiver was valid remain governed by that valid historical Decision.

Make non-waivable requirements explicit

WaioTech must make non-waivable requirements explicit.

A Requirement is waivable only when its authoritative contract explicitly permits waiver.

Mandatory product, safety, permit, isolation, security, and integrity rules cannot be waived merely because an Actor has broad authority.

Do not collapse exceptions into one generic entity

WaioTech must not collapse exceptions into one generic entity.

Different exceptions have different business meaning, authority, lifecycle, and consequences.

Examples include:

- emergency start;
- Work Requirement waiver;
- scope variation;
- temporary repair;
- controlled follow-up;
- platform recovery action.

Each must use its own named domain action and Decision evidence.

Apply separation of duties only to explicitly protected decisions

WaioTech must apply separation of duties only to explicitly protected decisions.

Separation of duties is required only when an explicit product rule or Tenant control requires independent judgment.

Examples may include:

- verification of controlled work;
- publication of protected Access Profile revisions;

- approval and posting of high-risk Stocktaking Runs;
- high-impact Waiver Decisions.

Routine work must not inherit unnecessary approval chains.

Define independence per Decision type

Waiotech must define independence per Decision type.

Each protected Decision type must define which prior Actor relationships are disallowed.

Examples include:

- verifier must not be the completing Actor;
- protected Access Profile publisher must not be its sole author;
- Stocktaking approver must not be the counter where independence is required;

Independence is based primarily on Actor identity, not merely different Team names.

Keep authority and independence as separate requirements

Waiotech must keep authority and independence as separate requirements.

An Actor must satisfy both:

- the Permission required to make the Decision;
- the applicable independence requirement.

Being a different person, senior employee, owner Team member, or manager does not independently grant Decision authority.

Treat unavailable independence as a real control blocker

Waiotech must treat unavailable independence as a real control blocker.

The protected Decision remains blocked unless its authoritative rule defines an explicit escalation or exception path.

Waiotech must not silently weaken separation of duties because the Tenant lacks available personnel.

Possible governed responses may include:

- escalation to another authorized Team;
- temporary authorized external reviewer;
- a specifically defined emergency Decision;
- postponement until independence is available.

Define change protection per control rather than using a vague sensitive-policy category

Waiotech must define change protection per control rather than using a vague sensitive-policy category.

Only when the affected product-defined control declares it.

Each control should specify whether changing it requires:

- ordinary publication authority;
- independent approval;
- stronger separation of duties;
- delayed effective time;
- additional evidence.

Examples may include controls affecting:

- verification independence;
- protected Access Profile publication;
- Stocktaking approval separation;
- high-impact waiver authority.

Keep policy configuration separate from IAM authority

Waiotech must keep policy configuration separate from IAM authority.

IAM owns Permissions, Access Grants, Access Profiles, Team assignments, and authorization evaluation.

Tenant Control Policy may require a particular approval, verification, or separation-of-duty condition, but it cannot create Permissions or alter Access Grants.

Related documents

- [Tenant Control Policy](#)
- [Reason codes](#)
- [Work Requirements and scope control](#)

Reason codes

Reason codes provide governed, stable meaning for actions that require structured explanation. Free text may add context but cannot replace a required canonical reason.

Reason codes

Waitech must use reason codes where recurring reasons affect reporting, control, or interpretation.

Reason codes are governed catalogue values that provide consistent meaning for recurring operational actions and Decisions.

Examples include:

- Work Order cancellation reasons;
- Work Request rejection reasons;
- hold reasons;
- Inventory adjustment reasons;
- release-withdrawal reasons;
- waiver reasons;
- skipped Scheduled Work reasons.

Keep reason catalogues domain-specific

Waitech must keep reason catalogues domain-specific.

Each action family should own its reason-code catalogue because cancellation, stock adjustment, rejection, and waiver have different meanings.

Use controlled meaning with optional explanatory narrative

Waitech must use controlled meaning with optional explanatory narrative.

Where a governed reason code is required, an Actor must select one.

Free text may supplement the controlled meaning.

An **other** reason should require an explanation.

Preserve reason-code meaning over time

Waitech must preserve reason-code meaning over time.

Reason-code identity and meaning must remain stable after historical use.

A code may be retired from new selection while remaining understandable in historical records.

A replacement code must use a new identity.

The minimum policy and Decision model

Waiotech must keep policy concrete, Tenant-scoped, revisioned, and product-defined without introducing a generic rule or workflow engine.

The minimum model is:

```

Mandatory Product Rules
└─ non-configurable product guarantees

Tenant Control Policy
└─ Tenant Control Policy Revision
   └─ product-defined typed controls

Work Requirement
├─ unsatisfied
├─ satisfied
├─ waived
├─ not applicable
└─ expired

Domain-owned Decision
├─ subject
├─ governed outcome
├─ Actor and authority
├─ rationale and evidence
├─ applicable policy revision
├─ effective period
└─ supersession or revocation where applicable

Domain-specific Reason Codes
  
```

The governing distinctions are:

- Product rules define non-negotiable behavior.
- Tenant policy configures only product-defined controls.
- Work Requirements express action-blocking obligations.
- Decisions preserve authorized judgment.
- Waivers are narrow Decisions against explicitly waivable Requirements.
- Exceptions remain named domain actions.
- Reason codes provide controlled operational meaning.

Related documents

- [Decisions, waivers, and separation of duties](#)
- [Canonical terminology](#)

Tenant lifecycle and portability

Suspension, deactivation, export, preservation, and destruction boundaries.

Contents

- [Tenant lifecycle](#)
- [Tenant portability](#)
- [Preservation and destruction](#)

Tenant lifecycle

Tenant lifecycle controls when a plant may operate, how its authority is suspended or ended, and how its evidence remains preserved.

The Tenant lifecycle

```
provisioning -> active -> suspended -> active
provisioning -> deactivated
active -> deactivated
suspended -> deactivated
```

Deactivation is terminal in the ordinary product model. Recovery from operator error uses a protected platform recovery contract and preserved evidence rather than a normal lifecycle action.

Provisioning

A provisioning Tenant may receive validated configuration, imported data, and protected platform setup. Ordinary Membership authority and operational work are blocked until activation succeeds.

Tenant Import creates a new provisioning or inactive Tenant. It never merges into an existing Tenant and never exposes partially validated data as active.

Suspension

Suspension blocks ordinary operational mutation while preserving all Tenant data, history, audit, scheduled obligations, attachments, and identity relationships. Only explicit suspension-safe actions may run, including protected inspection, export, remediation, and reactivation where authorized.

Background processing must not fabricate successful work during suspension. Notifications and Android synchronization follow explicit suspension behavior; queued facts remain durable and are reconciled without rewriting history.

Reactivation

Reactivation resumes ordinary evaluation from current authoritative facts. It does not pretend that the suspension period did not occur. Overdue work, expired authority, missed schedules, stale offline field packages, and pending Notifications are recalculated or reconciled through their owning contracts.

Deactivation

Deactivation ends ordinary operation and new Membership authority. It preserves historical identity, audit, maintenance, Inventory, Reports, Notifications, Attachments, exports, imports, and lifecycle evidence according to retention and preservation rules.

Deactivation revokes browser and mobile sessions, Android installations, outstanding offline field packages, delegated download URLs, and other Tenant-scoped credentials. It does not delete historical Memberships, assignments, audit, or business evidence.

Offboarding checks

Deactivation requires one protected assessment that makes unfinished plant obligations explicit without forcing them into false terminal states.

The assessment must identify at least:

- open Process Conditions and monitoring obligations;
- unfinished Work Orders and Scheduled Work;
- unresolved Findings and Failure Events;
- Inventory on-hand custody, active Reservations, replenishment, and Stocktaking obligations;
- queued or running Reports, Notifications, and Tenant portability operations;
- open file upload or accepted-file processing obligations;
- Android installations, sessions, offline field packages, unsynchronized evidence, and synchronization obligations;
- active Integration Principals and machine credentials;
- active Data Sources and machine-ingestion obligations;
- retention, preservation, and required export obligations.

Open business records do not have to be falsely closed before deactivation. The protected deactivation action records the unresolved counts, reviewing platform Actor, reason, assessment time, and explicit acceptance that ordinary Tenant operation is ending with those facts preserved.

Deactivation must not proceed while an atomic or externally consequential operation is in a state that cannot be safely stopped or preserved, including a queued or running Tenant import/export cutover, an in-progress destructive storage action, or another explicitly defined protected operation. Such operations must complete, fail safely, or be cancelled according to their owning contract.

Except for a provisioning Tenant that never entered ordinary operation, deactivation requires one completed Tenant Export whose declared cutoff is at least as recent as the latest authoritative Tenant activity at the start of the protected deactivation assessment. Activity created by the deactivation action itself does not invalidate that export requirement.

The deactivation transaction revokes ordinary browser and mobile authority, Android installations and offline field packages, Integration Principal credentials, delegated Tenant file URLs, and other Tenant-scoped credentials according to their engineering contracts. Historical identities and evidence remain preserved.

Related documents

- [Tenant portability](#)

- Preservation and destruction
- Authentication, sessions, and secrets

Tenant portability

Tenant Export and protected Tenant Import are required Waiotech Alpha capabilities. They use a versioned, integrity-verifiable Waiotech package and never expose an undocumented database copy.

Tenant Export

A Tenant Export is a governed asynchronous operation that produces one immutable Export Artifact for one Tenant, contract version, cutoff, and authorized request.

The lifecycle is:

```
queued -> running -> completed
queued -> cancelled
running -> failed
running with expired lease -> running retry
```

Cancellation is truthful: a running operation is not labelled cancelled merely because cancellation was requested.

Every export preserves request Actor, Tenant, reason, contract version, cutoff, parameters, authority evidence, start and completion times, outcome, artifact identity, digest, encryption metadata where used, and audit correlation.

Export content

The Alpha package contract is `waiotech-tenant-export-v1`. It is an explicit, closed contract generated from the accepted persistence model and reviewed in source. Every Tenant-scoped table is classified as included or excluded; new tables and new columns are not exported implicitly.

One package contains:

- `manifest.json` with package identity, source product and Tenant, cutoff, included domains, counts, SHA-256 digests, relationship digest, and file inventory;
- `contract.json` with the exact component and field contract and exclusions;
- `tenant.json` and `participants.json`;
- one canonical JSON record file per included component;
- accepted Attachment binaries under stable package paths.

The database snapshot uses one PostgreSQL repeatable-read, read-only transaction and one transaction timestamp as the cutoff. Record files are deterministic and Attachment binaries are checked against accepted size, media type, and SHA-256 evidence before and during packaging. The completed ZIP is written to a deterministic private object identity, verified again after storage, and only then marked completed.

The contract excludes cached projections, command receipts, internal domain/outbox events, worker item failures, bootstrap state, live object-storage references, export worker leases, and request authority internals. It never includes password credentials, refresh sessions, login-attempt state, CSRF material, signing or encryption keys, storage credentials, or one-time secrets. Participants contain only portable identity and account-state facts required to reconcile User references.

Export access

Platform initiation, inspection, cancellation, and retrieval are separate explicit Permissions and actions. A Tenant may have at most one queued or running export. Only a queued export can be cancelled; running work reports its real outcome. Learning Tenants are excluded from portability.

Retrieval rechecks current durable platform authority, requires recent platform authentication, a fresh idempotency key, and an explicit reason, then records immutable retrieval evidence and audit before streaming the verified artifact with `no-store` headers. Alpha retrieval streams through the authenticated API. A short-lived delegated or one-time download URL is not part of the Alpha portability contract.

Export artifacts are preserved by default. Configurable retention, expiry, encryption, and physical destruction remain separate governed contracts rather than hidden cleanup.

Tenant Import

Tenant Import is a protected Platform Admin operation. It accepts only a complete `waio-tech-tenant-export-v1` package. The source ZIP is first written to a private, digest-derived Tenant-pre-fixed quarantine identity under the target Tenant ID. The stable identity makes repeated HTTP delivery safe without creating parallel import operations or files. Ordinary Tenant sessions cannot access that object.

The lifecycle is:

```
received -> validating -> ready -> importing -> completed
received|validating|importing -> failed
received|ready -> cancelled
failed|cancelled -> received (explicit retry)
```

Receiving a package requires recent platform authentication, explicit import authority, an idempotency key, a reason, the source byte count, and the source SHA-256 digest. The Server enforces compressed-size, expanded-size, record-count, file-count, path, duplicate entry, symbolic-link, and encrypted-entry limits before trusting package content.

Validation is asynchronous and non-mutating. It requires the exact supported component set and verifies the manifest, contract, Tenant metadata, participant identities, canonical JSON digests, record shapes, primary-key uniqueness, Tenant scope, foreign-key closure, supported Permission codes, relationship digest, Attachment inventory, bytes, media evidence, and SHA-256 digests. Unknown fields, missing components, extra ZIP entries, conflicting package identities, and changed source bytes are rejected rather than guessed.

A ready import starts only through a separate recent-authenticated action. Import always creates one new `provisioning` Tenant with a deterministic new technical identity. It never merges into or overwrites an existing Tenant. Source Tenant and record UUIDs are remapped deterministically under the import operation identity, preserving internal relationships while preventing collision with live platform records.

Users are reconciled only by exact canonical phone number. An existing matching global User is reused. A missing participant becomes a disabled global User with no password, session, platform grant, or other credential. Imported Memberships and Tenant authority continue to reference the reconciled User. Activation readiness therefore remains blocked until at least one imported administrator User is enabled and the imported IAM foundation is otherwise valid.

All accepted database records are written in one protected import transaction. Source audit facts are preserved as immutable imported audit evidence linked to the import and target Tenant; they are not inserted into the live command, event, or platform-audit streams. Accepted Attachment bytes are copied to deterministic target-Tenant object identities and verified again. Retries reuse those identities instead of creating parallel files.

Completion leaves the new Tenant in [provisioning](#). Activation is a separate protected, readiness-gated action. A failed import never exposes a partially active Tenant. Any files prepared before a failed database transaction remain private and unreachable from ordinary Tenant APIs until a governed retry or the file-lifecycle cleanup contract resolves them.

Passwords, password hashes, sessions, mobile credentials, Android installation secrets, signing keys, encryption keys, storage credentials, one-time credentials, export worker state, import worker state, command receipts, and outbox state are never imported. Repeating the same source digest returns the existing operation; a conflicting package identity is rejected. A failed or cancelled operation can be explicitly retried. Retry clears prior validation evidence and runs the complete validation stage again before the operator can restart import.

Portability integrity

The package manifest identifies format version, source product version, source Tenant identity, export cutoff, included domains, record counts, file inventory, content digests, relationship digest, and required SHA-256 verification. Version 1 records encryption as [none](#). Adding package encryption requires an explicit versioned contract and never replaces integrity verification.

The package and its resulting operations remain auditable. Retention and physical destruction follow the preservation policy without rewriting the fact that an export or retrieval occurred.

Excluded behavior

Waiotech does not support ordinary cross-Tenant record movement, merging two Tenants, importing arbitrary third-party database dumps, or restoring live security secrets.

Related documents

- [Tenant lifecycle](#)
- [Preservation and destruction](#)
- [Generated artifacts and recovery](#)

Preservation and destruction

Operational Tenant records remain preserved after deactivation unless a complete governed destruction contract explicitly authorizes physical destruction. Learning-purpose Tenant reset follows its separate synthetic-data contract.

Physical destruction is outside this Product Authority and requires a complete retention and destruction contract

Physical destruction is outside this Product Authority and requires a complete retention and destruction contract.

Physical Tenant destruction is outside Waiotech Product Authority.

A correct destruction capability requires a dedicated retention and destruction model covering:

- legal and contractual holds;
- evidence classes;
- privacy requirements;
- financial and safety records;
- backups and replicas;
- external-system references;
- partial failure;
- authorization;
- irreversible proof.

Deactivation alone never destroys data.

Keep learning reset as a narrow synthetic-data exception to operational preservation

Waiotech must keep learning reset as a narrow synthetic-data exception to operational preservation.

A learning-purpose Tenant contains explicitly synthetic disposable state under the learning-environment contract. Its deterministic reset contract may replace or remove that synthetic state.

The exception does not apply to operational Tenants, imported operational evidence, onboarding progress, or any record whose source is not safely synthetic.

Preserve governed Tenant evidence by default

Waiotech must preserve governed Tenant evidence by default. A retention or destruction amendment must define every exception.

Retention cannot be safely represented through one configurable duration because different records and evidence classes may have different obligations and restrictions.

Preserve deactivated Tenant history by default

Waiotech must preserve deactivated Tenant history by default.

Tenant-owned records, histories, Decisions, and Attachments remain preserved after deactivation according to mandatory product-integrity rules.

Ordinary Users cannot physically destroy governed evidence.

The Tenant remains unavailable for normal operation while its historical records remain interpretable.

The minimum Tenant offboarding and portability model

Waiotech must keep Tenant lifecycle, export, and protected import explicit. Tenant cloning, configurable retention, and physical destruction are outside this Product Authority.

The minimum model is:

```
Tenant
├─ active
├─ suspended
└─ deactivated

Tenant Export
├─ product-defined export type
├─ versioned export contract
├─ execution lifecycle
├─ declared cutoff
├─ frozen artifact
├─ integrity evidence
└─ effective-access retrieval

Tenant Import
├─ versioned package validation
├─ integrity and safety checks
├─ new inactive Tenant
├─ identity reconciliation
├─ staged evidence restoration
└─ atomic activation
```

Capabilities outside Waiotech Product Authority are:

```
Tenant Cloning
Tenant Migration
Configurable Retention
Physical Destruction
```

The governing distinctions are:

- Suspension temporarily blocks ordinary operation.
- Deactivation permanently ends normal operation.

- Neither state deletes or rewrites data.
- Export provides governed portability of Tenant-owned facts and evidence.
- User identity remains platform-wide.
- Credentials and security secrets are excluded.
- Projections are rebuildable and non-authoritative.
- Tenant Import restores a valid Waiotech package only into a new inactive Tenant and activates it atomically after complete validation.
- Physical destruction requires a separate Product Authority amendment.

Related documents

- [Tenant lifecycle](#)
- [Tenant portability](#)
- [Learning environment](#)

20-ENGINEERING

Engineering

Architecture and technical guarantees that preserve Product and Experience Authority.

42 source documents

Engineering

Engineering Authority defines how Waiotech implements Product and Experience Authority through modular domain ownership, authoritative persistence, explicit Application orchestration, generated contracts, security, field synchronization, files, workers, deployment, testing, and recovery.

Sections

- [Engineering foundations](#)
- [Data and definitions](#)
- [Actions and contracts](#)
- [Security](#)
- [Background processing](#)
- [Application surfaces](#)
- [Files and artifacts](#)
- [Delivery and operation](#)

Engineering foundations

Repository ownership, modular architecture, and the canonical engineering model.

Contents

- [Repository and source ownership](#)
- [System architecture](#)
- [Canonical engineering model](#)

Repository and source ownership

The repository contains one coordinated Waiotech implementation with explicit deployables, module ownership, dependency direction, generated outputs, scripts, configuration examples, and review responsibility.

The repository

Waiotech must treat the repository as implementation evidence, not as an alternate source of product meaning.

The repository is the source-controlled realization of Product Authority and Engineering Authority. It contains application source, migrations, generated sources, tests, documentation, deployment definitions, and verification tooling required to build one identifiable Waiotech release.

Use a coordinated monorepository unless a reviewed replacement preserves every contract boundary

Waiotech must use a coordinated monorepository unless a reviewed replacement preserves every contract boundary.

Waiotech uses one coordinated repository containing:

```
server/  
dashboard/  
admin/  
website/  
android/  
docs/  
deploy/  
scripts/  
tests/
```

The repository may use additional bounded directories when ownership remains explicit. Splitting repositories requires an ADR proving preserved release identity, generated-contract parity, security, and recoverability.

Retain the typed Python and PostgreSQL platform unless an ADR proves an equivalent replacement

Waiotech must retain the typed Python and PostgreSQL platform unless an ADR proves an equivalent replacement.

The approved backend platform is Python 3.11 or a compatible supported Python release, FastAPI, Pydantic, async SQLAlchemy, asyncpg, Alembic, PostgreSQL, Redis, and ARQ. Typer may provide controlled CLI entry points. Approved security and observability libraries remain replaceable behind explicit engineering boundaries.

Use explicit stable platforms suited to browser administration, public content, and native field work

Waiotech must use explicit stable platforms suited to browser administration, public content, and native field work.

Tenant Dashboard and Platform Admin use the approved TypeScript and React stack. Public Website uses Astro and TypeScript with static-first content generation. Android Work App uses native Kotlin, stable Jetpack Compose and AndroidX components, Room, WorkManager, CameraX, Android Keystore, and generated API contracts.

Every surface uses accessible platform-appropriate components without transferring product behavior into a UI framework.

Make module and layer ownership visible in source structure

Waiotech must make module and layer ownership visible in source structure.

Backend source is organized by product module and layer. A module owns its Domain, Application, ports, infrastructure adapters, API registration, events, catalogues, migrations, and tests. Shared technical packages provide narrow primitives and cannot own product behavior.

Principal backend module ownership must remain visible for `plant`, `process`, `maintenance`, `inventory`, `reliability`, `iam`, `policy`, evidence/communications, and portability. Compatibility import aliases that preserve competing product-module vocabularies are prohibited.

Align surface source with its responsibility without merging authority or runtime platforms

Align surface source with its responsibility without merging authority or runtime platforms.

Dashboard and Platform Admin separate application composition, module-aligned features, shared technical UI, generated contracts, and tests. Public Website separates Astro application composition, bilingual governed content, SEO generation, public forms, and tests. Android separates Compose UI, mobile use cases, local persistence, synchronization, platform adapters, generated contracts, and tests.

Each surface remains a distinct build and release tree.

Shared code contents

Share stable mechanisms and keep product meaning module-owned.

Shared code may contain technical primitives such as identifiers, clocks, transaction interfaces, error envelopes, logging context, HTTP transport helpers, accessibility primitives, formatting, and test infrastructure.

Shared code must not contain a generic lifecycle engine, universal selector, universal policy evaluator, unowned business repository, generic reason catalogue, or cross-domain entity service.

Keep dependency direction explicit and mechanically verified

Waitech must keep dependency direction explicit and mechanically verified.

Domain depends only on language-level and domain-owned types. Application depends on Domain and ports. Infrastructure implements ports. API and Worker entry points invoke Application contracts. Frontends depend on generated API contracts and presentation libraries. Architecture tests reject reverse and cross-module private dependencies.

Permit committed generated artifacts only with mandatory drift verification

Permit committed generated artifacts only with mandatory drift verification.

Generated source required for deterministic builds may be committed when the release process verifies parity. Generated runtime references may also be produced during build. Every generated file identifies its source and regeneration command.

Use disposition labels only to state the required canonical outcome

Waitech must use disposition labels only to state the required canonical outcome.

Static architectural assessment may classify a component as:

- Retain;
- Refactor;
- Replace;
- Retire.

These labels describe required architectural disposition, not project status. A normative document does not include completion percentages, dates, assignees, or delivery-cycle state.

Keep source snapshots and remediation tracking outside Engineering Authority

Waitech must keep source snapshots and remediation tracking outside Engineering Authority.

A repository assessment is protected engineering evidence bound to a source commit or release identity. It may record findings, controlling authority, observed implementation, severity, and required disposition. It is separate from the timeless authority document.

Treat scripts as governed engineering entry points

Waitech must treat scripts as governed engineering entry points.

Repository scripts must have one documented purpose, safe argument handling, explicit environment selection, controlled credentials, idempotent behavior where applicable, and tests for high-impact operations. A script must not become a hidden authorization or migration path.

Keep configuration technical, validated, and secret-free in source control

Waitech must keep configuration technical, validated, and secret-free in source control.

Example configuration contains no real secret and states which values are required, optional, security-sensitive, or environment-specific. Defaults must be safe. Product behavior cannot be hidden in deployment variables.

Make third-party software identifiable and reproducible

Waitech must make third-party software identifiable and reproducible.

Runtime and development dependencies are declared, resolved through lock files, scanned, and reproducibly installed. Broad unbounded version ranges are prohibited for release resolution. Dependency updates receive the same tests as affected source changes.

Make ownership explicit without embedding individual staffing status in authority

Waitech must make ownership explicit without embedding individual staffing status in authority.

Every module, shared package, migration area, deployment component, and authority document has an owning review group. High-impact changes require review by the applicable product, security, data, or operations owner.

Prevent repository archaeology from becoming dual authority

Waitech must prevent repository archaeology from becoming dual authority.

Obsolete canonical paths are removed in the change that establishes their replacement. Retained historical migrations and evidence remain immutable but are not executable alternate product models, import targets, or alternate authorities.

Make repository conformance executable

Waiotech must make repository conformance executable.

Checks include formatting, linting, strict typing, architecture rules, generated parity, secret scanning, dependency scanning, migration graph validation, frontend API discipline, documentation generation, container builds, and mandatory tests.

Related documents

- [System architecture](#)
- [Canonical engineering model](#)
- [Authority and precedence](#)

System architecture

WaioTech uses a modular monolith with explicit API, Application, Domain, and Infrastructure layers. Product modules own their facts and actions, layers depend inward, and cross-domain workflows are coordinated through declared contracts rather than private data access.

Adopt a modular monolith

WaioTech must use one coordinated modular monolith rather than distributed services or an undifferentiated application.

The product contains strongly related Plant, Process, Maintenance, Inventory, Reliability, IAM, policy, evidence, and portability behavior with shared Tenant isolation and frequent cross-domain consistency requirements. One PostgreSQL authority and one transactional backend preserve those guarantees while module boundaries remain explicit.

Canonical product modules

The backend must make these principal product modules explicit:

```
plant
process
maintenance
inventory
reliability
iam
policy
attachments
reporting
notifications
portability
```

Tenancy and shared platform concerns remain explicit according to source ownership. The exact directory tree may evolve, but module ownership must remain mechanically visible and product-aligned.

plant owns Functional Locations, Assets and installation, Asset Classification, Operational Criticality, Process Units, Process Streams, Measurement Points, Data Sources, Measurement Source Mappings, and plant external mappings.

process owns Process Readings, Operational Observations, Process Routines and immutable revisions, Process Rounds, Process Conditions, Operational Actions, Outcome Assessments, and Process material-usage meaning.

maintenance owns Work Requests, Findings, Procedures, Maintenance Plans, Scheduled Work, Work Orders, Requirements, execution, completion, and maintenance verification.

inventory owns Items, Stock Locations, Movements, Reservations, replenishment, Stocktaking, and custody projections.

reliability owns Failure Events, restoration, Cause Assessments, recurrence, and reliability projections.

Separate runtime responsibilities without distributing ownership

API and Worker run as separate processes. Public Website, Tenant Dashboard, Platform Admin, and Android Work App are separately built application artifacts. Migration and controlled engineering commands use explicit entry points.

Runtime separation must not create alternate product ownership.

Canonical backend layers

Within each product module, dependencies point inward:

```
graph TD; A[API or Worker entry point] --> B[Application]; B --> C[Domain]; C --> D[Ports]; E[Infrastructure adapters] --> D;
```

Infrastructure types must not leak into Domain contracts.

API layer

API owns HTTP routing, transport authentication integration, request parsing, transport validation, idempotency and concurrency metadata extraction, correlation metadata, Application invocation, response serialization, and safe error mapping.

API does not own product lifecycle or authorization decisions.

Application layer

Application is the authoritative use-case and cross-domain orchestration boundary.

It owns named commands and queries, Actor and Tenant context, authorization, transaction orchestration, module coordination, Domain invocation, audit coordination, Command Receipts, outbox creation, and typed results.

Cross-domain workflows such as Process Condition → Work Request, Work Order material issue, Failure Event restoration, or Measurement Point-triggered maintenance are coordinated here.

A Domain aggregate must not call another product module's private Domain or repository directly.

Domain layer

Domain owns product-significant invariants, lifecycle, Decisions, value objects, calculations, evidence meaning, correction semantics, and action guards assigned by Product Authority.

Domain code must remain independent from HTTP, ORM, React, Android, queues, and provider SDKs.

Ports and Infrastructure

Ports are narrow typed capabilities required by Application or Domain behavior. Infrastructure implements those capabilities using PostgreSQL, Redis, ARQ, file storage, cryptography, external delivery, integration authentication, observability, and framework adapters.

Infrastructure may share technical mechanisms but cannot weaken product semantics.

Cross-domain contracts

A module may depend on another module only through:

- public Application command;
- public query;
- module-owned event;
- narrow typed read port explicitly preserving owner semantics.

Private repositories, ORM models, internal tables, and infrastructure adapters are not cross-domain contracts.

Atomic cross-domain actions

Where Product Authority requires one accepted cross-domain action to be atomic, one Application orchestrator may coordinate several module-owned ports through one transaction manager.

Examples include posting Inventory custody while accepting Process or Maintenance material usage, or creating a Maintenance Work Request from a Process Condition with the relationship established atomically.

Shared transaction coordination does not transfer business ownership.

Keep the module dependency graph acyclic

Circular module dependencies must be decomposed through Application orchestration, events, neutral shared value types, or ownership correction.

Mutual private imports are prohibited.

Explicit product contracts instead of meta-models

Waiotech must not use universal business engines for Plant Objects, arbitrary relationships, generic life-cycle, generic Conditions, generic Activities, generic Measurement, unrestricted selectors, arbitrary domain JSON, or generic entity CRUD.

Technical helpers may operate on typed product-owned definitions without becoming their authority.

Human and machine authority

Human Users, Integration Principals, product-defined internal system Actors, and platform Users remain distinct authority paths.

Machine Reading ingestion reaches the same Process Application and Domain Reading behavior after authentication and source authorization; it must not create a parallel machine-data domain.

Related documents

- [Repository and source ownership](#)
- [Canonical engineering model](#)
- [Authorization architecture](#)

Canonical engineering model

Waiotech is a Product-Authority-driven modular monolith whose implementation favors explicit ownership, exact history, least authority, deterministic contracts, professional user experience, and operational recovery over generic extensibility.

Complete authority chain

Implementation remains subordinate to authority:

```
Product Authority
→ Engineering Authority
→ bounded ADRs and Experience Contracts
→ module-owned canonical definitions
→ generated contracts and references
→ implementation
→ tests, release evidence, and operational evidence
```

A lower layer must not resolve a higher-layer ambiguity by inventing alternate product meaning.

Complete runtime model

```
Waiotech
├─ API
├─ Worker
├─ Tenant Dashboard
├─ Platform Admin
├─ Android Work App
├─ Public Website
├─ Documentation and Help Center
├─ learning Tenant experience
├─ migration entry point
└─ controlled engineering CLI
```

PostgreSQL owns durable structured truth. Governed file storage owns accepted immutable bytes. Redis provides non-authoritative delivery and transient coordination.

Canonical module model

```
Plant
Process
Maintenance
Inventory
Reliability
IAM
```

Policy
Evidence and communications
Portability

Module ownership follows Product Authority rather than existing table names or route layout.

Strongest architecture rules

Waiotech must preserve:

- module-owned facts and actions;
- inward layer dependencies;
- Application-owned cross-domain orchestration;
- one truthful transaction boundary for atomic accepted actions;
- durable outbox obligations for external effects;
- additive correction of accepted evidence;
- explicit Tenant isolation;
- no generic business meta-models.

Strongest authority rules

Human authority flows through Membership, Team Access Profile Assignment, Access Profile Revision, Access Grant, Permission, and domain action guards.

Integration Principals use separate narrow machine authority. Internal system Actors are exact product-defined jobs. Platform authority is explicit and cannot become ordinary Tenant authority.

No authority path may impersonate another Actor category.

Strongest history rules

Published revisions, posted Inventory Movements, accepted Process Readings and Observations, Process Condition history, Operational Actions, Failure Event evidence, Cause Assessments, accepted attachments, audit, and other Product Authority-defined evidence are immutable or additively corrected.

Functional Location and Asset installation history must preserve physical replacement without rewriting stable-position history.

Measurement Source Mapping changes and machine source changes must preserve prior Reading provenance.

Strongest contract rules

Commands and queries are typed. APIs expose named domain actions. OpenAPI is canonical transport authority for browser and machine clients. Generated Dashboard and Admin clients must remain in parity. Android contracts must remain version-compatible with the supported release policy.

Clients never recreate Server authority.

Strongest experience rules

Tenant Dashboard must implement Product Authority as an operational workspace rather than storage-shaped CRUD.

Purpose-specific read models, available actions, connected context, fast Process Reading entry, master-detail workspaces, operational timelines, and consistent Plant navigation are engineering obligations where Experience Contracts require them.

Frontend architecture must not force each Domain aggregate into a generic registry/detail/edit template.

Strongest operational rules

Background obligations are recoverable from PostgreSQL. Workers use durable claims and bounded leases. Machine ingress is authenticated, idempotent where required, rate-limited, attributable, and source-authorized. Deployments use immutable artifacts. Backup correctness requires tested restoration. Observability remains protected and non-authoritative.

Prohibited generic mechanisms

Waiotech must not introduce universal Product Authority through:

- Role authorization;
- direct profile grants outside the canonical human path;
- generic IAM deny/scope expressions;
- universal lifecycle engines;
- universal Plant Object or relationship engines;
- generic Measurement or Event authorities;
- unrestricted selectors or workflow scripts;
- arbitrary core-domain JSON;
- browser-side authorization;
- Redis business truth;
- external mutation of derived state;
- in-place replacement of accepted files or evidence.

Canonical migration baseline and retained history

Disposable development databases and migration histories that have never governed retained external data may be regenerated from the canonical persistence model.

Once a migration has governed retained external data, it is immutable history. Subsequent schema evolution is forward-only and history-preserving according to Delivery Authority.

Conformance

A conforming implementation uses one canonical vocabulary, explicit module ownership, enforced Tenant isolation, canonical human and machine authority, duplicate-safe commands, immutable evidence, recoverable background work, typed APIs, generated-client parity, non-authoritative clients, governed offline authority, bilingual experience parity, forward-only migration for retained data, layered tests, and verified recovery.

Related documents

- [System architecture](#)
- [Canonical product model](#)
- [Tenant Dashboard experience](#)

Data and definitions

Persistence, Tenant isolation, revisions, target resolution, consistency, and governed catalogues.

Contents

- Persistence and Tenant isolation
- Revisions and publication
- Plan target resolution
- Consistency, concurrency, and correction
- Catalogues and generated reference data

Persistence and Tenant isolation

PostgreSQL is the authoritative structured data store. Tenant ownership, purpose, identifiers, timestamps, quantities, canonical codes, immutability, and deletion semantics are explicit in schema and query boundaries.

The authoritative structured data store

Waiotech must use PostgreSQL as the source of durable structured truth.

PostgreSQL is the authoritative store for structured Waiotech business state, evidence metadata, audit, revisions, assignments, Command Receipts, outbox records, process records, integration obligations, and reconciliation.

Keep caches and queues non-authoritative

Waiotech must keep caches and queues non-authoritative.

Redis and process-local caches may hold delivery, coordination, rate-limit, and rebuildable read data. Cache loss or Redis loss must not erase accepted commands, lifecycle, process obligations, or evidence.

Make persistence ownership correspond to module ownership

Waiotech must make persistence ownership correspond to module ownership.

Each product module owns its tables, constraints, indexes, migrations, and repository mappings. Shared platform tables have an explicit platform owner. Table location does not transfer product ownership.

Make operational and learning purpose explicit and immutable in authoritative persistence

Waiotech must make operational and learning purpose explicit and immutable in authoritative persistence.

Tenant purpose uses a product-owned constrained code with exactly `operational` and `learning`. It is required, immutable after Tenant creation, included in Tenant-context loading, and covered by database and architecture tests.

Never infer learning isolation from naming convention

Waiotech must never infer learning isolation from naming convention.

Side-effect fencing, reporting exclusion, reset authority, and content visibility use the authoritative purpose field.

Enforce purpose isolation in every cross-Tenant operational query

Waiotech must enforce purpose isolation in every cross-Tenant operational query.

Ordinary Tenant isolation through `tenant_id` always applies. Platform-wide operational queries, report selection, integration scans, billing scans, support queues, and delivery workers also apply the required purpose predicate or use an owned query contract that excludes learning Tenants.

Represent Tenant scope explicitly in authoritative persistence

Waiotech must represent Tenant scope explicitly in authoritative persistence.

Every Tenant-owned row contains a non-null Tenant identifier or belongs through an immutable parent relationship whose Tenant is enforced by database constraints. Ordinary cross-Tenant relationships are prohibited.

A Tenant-aware foreign key

Waiotech must use database constraints to prevent cross-Tenant relationships.

A Tenant-aware foreign key verifies both referenced identity and Tenant equality, ordinarily through a composite key such as `(tenant_id, referenced_id)`. Application checks supplement but do not replace applicable database constraints.

Require explicit Tenant identity wherever it improves isolation and verification

Waiotech must require explicit Tenant identity wherever it improves isolation and verification.

Every independently queried or related Tenant-owned table should contain `tenant_id`. A tightly owned child may inherit Tenant through a constrained parent when direct storage would add no verification value. The choice must not weaken query safety or foreign-key enforcement.

Never use an absent Tenant as an implicit superuser or platform path

Waiotech must never use an absent Tenant as an implicit superuser or platform path.

Platform-scoped records have an explicit platform scope and cannot be confused with a missing Tenant. Nullable Tenant identifiers are prohibited unless a typed scope discriminator and constraints make the distinction unambiguous.

Separate immutable technical identity from human operational codes

Waiotech must separate immutable technical identity from human operational codes.

Durable entities use opaque UUID identifiers generated by trusted application or database mechanisms. Human-facing operational codes are separate, Tenant-scoped where applicable, and never substitute for technical identity.

Make business codes stable and interpretable

Waiotech must make business codes stable and interpretable.

A code contract defines scope, normalization, case sensitivity, uniqueness, immutability, retirement, and correction. Code reuse after historical use is prohibited unless Product Authority explicitly defines equivalent identity.

For optional Plant Model codes, persistence preserves the entered display form and also stores or derives a deterministic comparison key by trimming surrounding whitespace, applying Unicode NFKC normalization, and Unicode case folding. Empty values and control characters are rejected. Uniqueness is enforced on `(tenant_id, plant_entity_category, normalized_code)` for active and historically reserved code ownership as required by Product Authority.

Preserve the distinction between instants and local calendar meaning

Waiotech must preserve the distinction between instants and local calendar meaning.

Recorded instants use timezone-aware PostgreSQL timestamps representing unambiguous instants. Local dates, local times, and Tenant timezone identifiers use separate typed fields when they are product facts. Naive timestamps are prohibited for recorded instants.

Preserve decimal meaning across persistence and APIs

Waiotech must preserve decimal meaning across persistence and APIs.

Quantities, rates, costs, and money use exact PostgreSQL numeric types with contract-defined precision and scale. Floating-point storage is prohibited for authoritative exact values.

Keep persistence language-independent and code-stable

Waiotech must keep persistence language-independent and code-stable.

Lifecycle, catalogue, action, event, and reason identities use stable machine codes. Display labels are not stored as identity. Database constraints or reference tables enforce permitted values where compatible with retirement and migration rules.

Preserve orthogonal Plant Model identity in persistence

Waiotech must preserve Functional Location, Asset, Process Unit, Process Stream, and Measurement Point as distinct persisted identities.

Persistence must not encode them as one universal entity table whose discriminator becomes the real source of domain meaning. Shared technical infrastructure is permitted, but owning tables, constraints, repositories, and relationships must preserve the explicit Product Authority concepts.

Functional Location containment, Asset physical composition, Asset installation, Process Unit containment, Process Stream direction, Functional-Location-to-Process-Unit service relationships, and Measurement Point subject relationships each require constraints appropriate to their own semantics.

Where Product Authority requires effective history, persistence stores non-overlapping effective periods instead of overwriting the prior relationship. Asset physical composition has at most one effective parent per Asset and must remain acyclic. A composed child Asset cannot carry a competing direct Functional Location installation; its installed context is resolved through the effective containing-Asset chain. Measurement Point subject and quantity changes that would alter semantic identity are represented by retirement and new identity rather than row mutation.

Preserve Asset installation history rather than overwriting current placement

Asset installation is effective-dated authoritative history.

Persistence prevents overlapping active installations for one Asset and preserves removal and replacement history. A current-installation projection may be derived, but it cannot replace the historical relationship as authority.

Persist Process Routines and Rounds without duplicating Process evidence

Published Process Routine Revisions are immutable revision records. Scheduled Process Round occurrence identity is protected by a deterministic uniqueness constraint over Routine and nominal occurrence identity.

A Process Round stores execution state and references accepted Process Readings and Operational Observations by canonical identity. It must not copy those values into mutable Round-entry columns that could diverge from Process evidence.

Persist Process Readings as append-only operational evidence

Accepted Process Readings are append-oriented records with explicit Measurement Point, effective time, recorded time, Actor, provenance, quality, reported value and unit, normalized canonical representation where applicable, and source identity.

Machine-submitted source identity and idempotency must permit duplicate-safe retry without creating duplicate accepted Readings. Correction uses additive relationships rather than in-place rewriting.

Protect historical meaning through immutable persistence patterns

Waiotech must protect historical meaning through immutable persistence patterns.

Published revisions, audit, posted Inventory Movements, accepted Decisions, completed artifacts, event envelopes, and other immutable evidence have no ordinary update or delete path. Constraints, repository APIs, and architecture tests enforce append-oriented correction.

Related documents

- [Revisions and publication](#)
- [Consistency, concurrency, and correction](#)
- [Authorization architecture](#)

Revisions and publication

Stable governed definitions use one editable working draft and immutable published revisions. Publication, supersession, withdrawal, restoration, concurrency, APIs, and migration preserve exact historical meaning without exposing version-control mechanics to ordinary users.

Use revisions only for governed definitions whose exact content must remain historically interpretable

Waiotech must use revisions only for governed definitions whose exact content must remain historically interpretable.

Procedure, Maintenance Plan, Access Profile, and Tenant Control Policy use stable definition identities with immutable published revisions. Another concept uses this model only when Product Authority assigns publication and historical interpretation to it.

The canonical revision structure

Waiotech must separate stable identity from immutable content history.

The structure is:

```
stable definition
├─ optional working draft
├─ immutable published revision history
└─ one published revision effective at evaluation time
```

The stable definition owns identity and lifecycle. Revisions own exact content.

Keep definition editing simple and conflict-explicit

Waiotech must keep definition editing simple and conflict-explicit.

One working draft may exist for the definition. Concurrent editors use optimistic concurrency on that draft. A separate branching and merge model is outside Engineering Authority.

Keep unpublished changes operationally inactive

Waiotech must keep unpublished changes operationally inactive.

It grants no Permission, generates no Scheduled Work, controls no Tenant action, and does not replace the published definition. It is accepted unpublished content only.

Make draft persistence explicit and non-effective

Waiotech must make draft persistence explicit and non-effective.

`save_changes` validates the editable draft schema, checks concurrency, preserves the editing Actor and recorded time, and updates only the working draft. It does not create a published revision or event asserting publication.

Review meaning rather than storage-level diffs

Review meaning rather than storage-level diffs.

Review compares the working draft with the published revision effective at evaluation time. It presents semantic additions, removals, changed facts, authority impact, target impact, control impact, and publication blockers.

Make publication the only route from unpublished content to effective definition

Waiotech must make publication the only route from unpublished content to effective definition.

Publication is one protected idempotent transaction that:

- revalidates draft content;
- checks expected concurrency;
- verifies authority and separation of duties;
- verifies referenced identities;
- computes required impact and target snapshots;
- creates an immutable published revision;
- makes it effective;
- preserves the superseded revision;
- clears or advances the working draft;
- records audit and events.

Keep published revisions immutable

Waiotech must keep published revisions immutable.

A correction creates new unpublished changes and another publication. Typographical, security, or migration corrections do not authorize in-place mutation of published content.

Make effective content unambiguous and queryable

Waiotech must make effective content unambiguous and queryable.

The stable definition references the published revision effective at evaluation time through an explicit relation or effective-period contract. Numeric ordering alone does not determine effectivity.

Freeze the definition basis of accepted operational records

Waiotech must freeze the definition basis of accepted operational records.

Where exact historical meaning matters, the operational record references the applicable immutable revision rather than only the stable definition. Examples include Work Orders referencing Procedure Revisions and Scheduled Work referencing Plan Revisions.

Keep content history and stable-definition lifecycle separate

Waiotech must keep content history and stable-definition lifecycle separate.

Supersession makes another published revision effective. Withdrawal prevents ordinary new application of the stable definition while preserving all revisions and historical references. Retirement prevents ordinary use according to the Product Authority lifecycle.

Restore content through forward change

Restore content through forward change.

Restoration copies selected historical content into new unpublished changes. The historical revision remains unchanged and is not reactivated. Review and publication remain mandatory.

Make deletion of unpublished content explicit without altering effective history

Waiotech must make deletion of unpublished content explicit without altering effective history.

`discard_changes` removes the working draft through a protected idempotent action, preserves audit where required, and leaves published revisions unchanged.

Prevent two publications from claiming the same predecessor

Waiotech must prevent two publications from claiming the same predecessor.

The publication command locks or compare-and-swaps the stable definition and working draft, verifies the expected published revision, and rejects a stale publication. Only one transaction can establish the next effective revision.

Keep revision identity available without exposing version-control mechanics

Waiotech must keep revision identity available without exposing version-control mechanics.

A revision may have a stable internal sequence or identifier for evidence and support. User workflows do not depend on manual revision-number selection. Labels are presentation metadata and not authority.

Preserve truthful publication provenance

Waiotech must preserve truthful publication provenance.

The owning module emits fact events such as `maintenance.maintenance_plan.published` or `iam.access_profile.published`, with envelope schema version. Migration-origin revisions use migration-specific provenance and do not fabricate a historical human publication event.

Expose stable-object workflows rather than revision CRUD

Waiotech must expose stable-object workflows rather than revision CRUD.

Ordinary APIs address the stable definition and expose named actions such as:

```
save-changes
review-changes
publish-changes
discard-changes
restore-content
change-history
```

Published revisions are read-only evidence resources.

Keep user experience aligned with product change obligations

Waiotech must keep user experience aligned with product change obligations.

The user sees published content, unpublished changes, semantic comparison, blockers, impact, publication, discard, history, and restoration. The interface does not expose branching, merging, manual supersession, or direct activation.

Create truthful canonical history without inventing product actions

Waiotech must create truthful canonical history without inventing product actions.

Migration establishes stable identity, immutable migration-origin revisions, effective content supported by source facts, and a working draft only when unpublished content is evidenced. Migration provenance remains separate from historical author or publisher attribution.

Related documents

- [Procedures and Maintenance Plans](#)
- [Tenant Control Policy](#)
- [Plan target resolution](#)

Plan target resolution

Maintenance Plan targets are authored through explicit canonical selections or typed governed selectors, previewed as non-authoritative candidate coverage, resolved atomically during protected publication, and preserved as immutable published targets and trigger bindings.

Keep target kind and trigger interpretation unambiguous

One Maintenance Plan Revision has exactly one target kind:

- Functional Location;
- Asset.

The target kind is part of revision meaning and cannot vary row by row inside one published revision.

A Plan requiring both kinds is represented by separate Plans unless Product Authority introduces another explicit typed contract.

Measurement Point-triggered Plans additionally preserve one compatible trigger Measurement Point for every resolved target. Trigger identity is therefore part of published target resolution rather than a later lookup against mutable Plant data.

Explicit target selection

Explicit selection accepts canonical target identities of the Plan target kind.

Application validation must establish at publication time:

- Tenant ownership;
- target kind;
- identity existence;
- duplicate elimination under canonical identity;
- lifecycle and publication eligibility rules;
- trigger prerequisites;
- any Product Authority-defined exclusion or blocking condition.

An external identifier may be used for lookup, but published coverage stores canonical Waiotech target identity.

Typed selector

A selector is a versioned Product Authority-defined structure. It is not a generic field/operator/value language.

The selector schema explicitly defines which governed facts may participate, normalization, validation, and interpretation.

Functional Location selector dimensions

Permitted dimensions may include only explicitly supported facts such as:

- exact Functional Location branch;
- lifecycle;
- Operational Criticality;
- governed external-mapping criteria.

Asset selector dimensions

Permitted dimensions may include only explicitly supported facts such as:

- Asset Classification;
- physical Asset subtree where Product Authority permits it;
- lifecycle;
- current installation within a Functional Location branch where explicitly supported;
- governed external-mapping criteria.

Selectors must not expose arbitrary SQL columns, JSON paths, free-form field names, executable expressions, browser-defined predicates, Team names as Plant semantics, or internal ORM structure.

Keep selector behavior reviewable and deterministic

For one normalized selector, Tenant state, evaluation instant, catalogue/reference-data revision, and selector contract version, target resolution must be deterministic.

Ordering used for review and digest generation is canonical and stable.

Selector evaluation must be Tenant-scoped before any other predicate can broaden the candidate set.

Preview is non-authoritative

A preview is a review artifact, not published coverage.

It includes at least:

- target kind;
- normalized selector or explicit-selection summary;
- selector contract version where applicable;
- evaluation time;
- included canonical target identities;
- excluded or ineligible candidates and typed reasons;
- missing trigger prerequisites;
- target-to-Measurement-Point binding where applicable;

- additions and removals relative to effective published coverage where applicable;
- deterministic reviewed-content digest;
- bounded count and truncation/limit meaning where relevant.

The browser must not construct its own candidate target set from partial registry data.

Resolve operational coverage only during protected publication

Publication re-evaluates the proposed selection inside the authoritative Application transaction using the same typed resolution contract.

For selector-based authoring, the publication result must match the reviewed digest. If it does not, publication fails with an explicit target-drift result and returns or makes available a new reviewable preview.

The Server must never publish a target set materially different from the one the Actor reviewed merely because Plant facts changed between preview and publication.

Prevent partially published target sets

Target resolution and revision publication are atomic.

If any required target, trigger binding, or publication prerequisite cannot be established, the Plan Revision is not partially published unless Product Authority explicitly defines a reviewed exclusion rule whose result was part of the reviewed digest.

Database rows for some valid candidates do not justify silently dropping invalid candidates.

A resolved Plan target

A resolved target is immutable published evidence owned by one Plan Revision.

It preserves at least:

- Plan Revision;
- target kind;
- canonical target identity;
- resolution provenance as explicit selection or selector result;
- publication-time relationship to the reviewed target-set digest;
- Product Authority-required snapshot facts needed for interpretation;
- resolved trigger Measurement Point where the trigger kind requires one.

Resolved target identity is not recalculated during ordinary occurrence generation.

Resolve Measurement Point trigger bindings per target

For a Measurement Point-triggered Plan, every target must resolve exactly one trigger Measurement Point before publication.

The candidate Measurement Point must:

- belong to the same Tenant;
- be active for publication use;
- observe that resolved target directly as its supported subject;
- use the measured quantity required by the trigger;
- have a canonical unit compatible with the trigger contract;
- satisfy any additional Product Authority-defined trigger eligibility rule.

Automatic selection is allowed only when exactly one eligible candidate exists.

Zero eligible candidates and several eligible candidates are both publication blockers until the author explicitly resolves the ambiguity or changes Plan coverage/configuration.

The preview and deterministic digest include each target-to-Measurement-Point binding. Publication persists the binding with the Plan Revision.

A later Measurement Point creation, retirement, source remapping, instrument replacement, Asset replacement, or Plant-model change never substitutes another trigger point inside an already-published revision.

Preserve published coverage immutably

After publication, Functional Location hierarchy, Asset composition, Asset Classification, Asset installation, lifecycle, Operational Criticality, Measurement Point configuration, and external mappings may change without rewriting the Plan Revision's published target set.

Historical operation uses the resolved target identity and trigger binding preserved by the Plan Revision.

Target drift is a derived comparison

Target drift compares current selector resolution with one published resolved target set at an explicit evaluation time.

It may identify:

- newly matching targets;
- targets that no longer match;
- retired or decommissioned targets;
- changed installation context;
- classification changes;
- missing or ambiguous trigger prerequisites;
- external-mapping differences;
- other Product Authority-defined causes.

Drift is review information only. It never mutates published coverage or creates/removes Scheduled Work.

A coverage change requires a new draft and reviewed Plan publication.

Separate historical coverage from current action eligibility

A target may remain historical published coverage while becoming currently ineligible for new occurrence recognition or generation.

Occurrence evaluation therefore checks current eligibility through the owning Plan/Scheduled Work contract without altering the historical target row.

An ineligible published target produces an explicit planning, reconciliation, or disposition result where required. It must not silently disappear.

Do not rewrite accepted work through definition publication

Publishing or superseding a Plan Revision never changes existing Scheduled Work or Work Orders.

Those records retain their original Plan Revision, Procedure Revision, resolved target, trigger identity, and evidence.

Where a business correction is required, it occurs through the affected record's explicit reconciliation or lifecycle action rather than by changing the source definition.

Make omitted coverage explainable

Every candidate excluded from reviewed coverage because of missing facts, invalid lifecycle, trigger incompatibility, authorization, or another product rule must have a typed explainable reason when the exclusion is material to publication review.

Unknown or unresolved meaning must not be translated into silent exclusion.

Bound selector cost and protect Tenant isolation

Selector evaluation uses indexed Plant facts, deterministic ordering, bounded result size, explicit timeout/resource limits, and Tenant predicates enforced below Application code where persistence authority requires it.

A selector that exceeds supported cost or size fails with a typed reviewable result rather than degrading into an unbounded query.

Selector evaluation must not expose records or aggregate counts from another Tenant through results, errors, timing-sensitive diagnostics, or generated previews.

Version selector structure without making historical operation depend on it

Persisted selector definitions carry a contract version when interpretation can evolve.

A supported release may read earlier selector versions for review or migration according to Engineering Authority, but historical occurrence operation does not depend on re-running obsolete selector logic because published coverage is frozen as canonical resolved targets.

Selector normalization and digest calculation are deterministic and versioned together where required.

End live selector authority at publication

After publication, the selector remains provenance and authoring history. It is not operational authority for automatic target addition/removal.

Ordinary Scheduled Work recognition reads the immutable resolved target set and applicable trigger bindings.

Re-evaluation is limited to preview, drift analysis, preparation of new unpublished changes, and explicit governance functions.

Verify authoring, publication, and operation separately

Tests must distinguish:

- selector/explicit-selection authoring and normalization;
- preview and digest generation;
- concurrency between preview and protected publication;
- all-or-nothing target and trigger-binding persistence;
- drift detection;
- historical reproducibility after Plant changes;
- occurrence generation from immutable coverage;
- Tenant isolation and bounded selector cost.

A passing selector query test does not prove publication or operational correctness.

Related documents

- [Procedures and Maintenance Plans](#)
- [Scheduled Work](#)
- [Revisions and publication](#)
- [Measurement Points](#)

Consistency, concurrency, and correction

Authoritative writes use explicit transaction, lock, isolation, concurrency, projection, deletion, and additive-correction rules. Redis and replicas cannot become authoritative write paths.

Require explicit stale-write protection for mutable authoritative state

Waiotech must require explicit stale-write protection for mutable authoritative state.

Mutable aggregates carry a monotonic concurrency version. Commands update through a compare-and-swap condition and reject stale versions without overwriting intervening changes.

Use database locking only where it directly protects an invariant

Waiotech must use database locking only where it directly protects an invariant.

Row locks are used for contested records where the invariant depends on serialized access, including stock posting, unique process claims, publication contention, and bounded assignment transitions. Locks remain narrow and transactions remain short.

Do not use advisory locks as an unbounded global mutex

Waiotech must not use advisory locks as an unbounded global mutex.

Advisory locks are permitted for a defined coordination key when row ownership is unavailable or cross-row serialization is required. The key, acquisition order, timeout, and recovery behavior must be documented and tested.

Choose isolation from invariant requirements and test real concurrency

Choose isolation from invariant requirements and test real concurrency.

The command contract selects the lowest PostgreSQL isolation level that preserves its invariants. Serialization failures and deadlocks are classified and retried only when the command remains idempotent and safe.

Keep command truth on authoritative PostgreSQL

Waiotech must keep command truth on authoritative PostgreSQL.

Authoritative commands and Readiness evaluations use the primary authoritative database. Replica-backed queries are limited to contracts that declare tolerated lag and cannot affect protected action acceptance.

Keep projections rebuildable and subordinate

Waiotech must keep projections rebuildable and subordinate.

A projection declares its authoritative sources, derivation, freshness, rebuild behavior, and explicit non-authority. Loss or staleness does not alter source records. Commands do not depend on a projection unless its consistency contract proves authoritative equivalence.

Do not hide product lifecycle behind row deletion

Waiotech must not hide product lifecycle behind row deletion.

Product lifecycle actions use cancellation, retirement, revocation, withdrawal, or deactivation as defined by Product Authority. Physical deletion is limited to disposable technical data or a separate approved destruction contract.

Make data correction additive, attributable, and verifiable

Waiotech must make data correction additive, attributable, and verifiable.

A correction uses a typed protected command or migration that preserves original evidence, correction reason, Actor, effective and recorded time, affected relationships, audit, and resulting canonical state. Direct operational SQL is not an ordinary correction path.

Correct Process evidence without erasing provenance

Process Reading, Operational Observation, Operational Action, and immutable Reliability evidence use additive correction or supersession according to Product Authority.

A corrected Reading preserves the original reported value, source, effective time, quality, and Actor. Derived current views and Maintenance trigger evaluation exclude invalidated evidence according to the canonical correction state without deleting historical source evidence.

Coordinate cross-domain consistency in Application transactions

When one User intent affects several owning domains, the Application layer coordinates explicit domain commands inside the smallest safe transaction boundary or through a documented durable follow-up obligation.

No domain aggregate may mutate another domain's tables directly. For example, Process escalation may establish a Maintenance Work Request and relationship while Process retains Condition authority; Maintenance material use may establish Inventory custody movement while Maintenance retains usage meaning.

Use database constraints as part of the product-preserving implementation

Waiotech must use database constraints as part of the product-preserving implementation.

Migrations and continuous integration verify foreign keys, Tenant constraints, uniqueness, code domains, nullability, immutable relationships, revision effectivity, assignment integrity, Command Receipt uniqueness, outbox integrity, and artifact-to-Attachment relationships.

Related documents

- [Persistence and Tenant isolation](#)
- [Commands and idempotency](#)
- [Audit, events, and transactional outbox](#)

Catalogues and generated reference data

Product-owned catalogues use stable governed codes, canonical source definitions, localization, retirement, generation, API exposure, and migration rules. Tenant master data remains separate and cannot create executable product behavior.

A catalogue

Waiotech must treat catalogue values as product contracts rather than editable labels.

A catalogue is a governed set of stable coded values used to classify, constrain, authorize, or interpret product behavior. It defines code, meaning, owner, applicability, lifecycle, relationships, and presentation metadata.

A taxonomy

Waiotech must use taxonomy only when structure is part of the contract.

A taxonomy is a governed classification structure whose values have hierarchical or semantic relationships. Not every catalogue is hierarchical, and not every hierarchy is product-fixed.

Do not infer catalogue ownership from use in a dropdown or table

Waiotech must not infer catalogue ownership from use in a dropdown or table.

Tenant master data represents Tenant-owned operational identities such as Item, Item Category, Stock Location, Team, or another Product Authority concept. Product-owned catalogue values have one Waiotech-defined meaning across Tenants.

Declare governance explicitly in Product Authority

Declare governance explicitly in Product Authority.

Each governed value set is classified as:

1. product-fixed closed catalogue;
2. product-managed governed extension point;
3. Tenant-governed catalogue;
4. Tenant master data;
5. derived registry or reference.

Derive governance from Product Authority only

Waiotech must derive governance from Product Authority only.

A database table, active flag, CRUD component, or localization support does not grant Tenant authority. Source-code storage does not independently prove product ownership.

Keep Tenant purpose closed, stable, and product-owned

Waiotech must keep Tenant purpose closed, stable, and product-owned.

Tenant purpose is a product-fixed closed catalogue owned by the Tenant domain with exactly **operational** and **learning**. Tenants cannot add, rename, reinterpret, or retire purpose codes. Persistence, APIs, generated clients, reporting exclusions, side-effect fences, and tests derive from the same canonical definition.

Use one product-wide Maintenance discipline catalogue

Waiotech must use one product-wide Maintenance discipline catalogue.

One Waiotech catalogue defines every Maintenance discipline code and meaning. A Tenant may declare applicability to canonical codes but cannot create, rename, reinterpret, or retire disciplines.

Separate Tenant applicability from catalogue ownership

Waiotech must separate Tenant applicability from catalogue ownership.

A separate Tenant-to-canonical-code relationship controls ordinary selection. It does not create a Tenant-specific discipline or change historical records when applicability ends.

Keep catalogue meaning module-owned

Waiotech must keep catalogue meaning module-owned.

The product module owning the meaning owns the canonical definition. A shared generator validates and combines definitions but does not become business owner.

Share generation infrastructure without creating an unowned taxonomy domain

Share generation infrastructure without creating an unowned taxonomy domain.

Shared tooling may load, validate, localize, and generate catalogue representations. It cannot centralize unrelated product meanings into a universal catalogue service.

The canonical source format

Waiotech must maintain one reviewable source of truth for every product-owned catalogue.

A catalogue uses one source-controlled typed definition owned by its module. The source declares catalogue identity, code, canonical meaning, lifecycle, localization key, relationships, dependencies, and generation requirements. The wastewater taxonomy is owned by `server/reference-data/taxonomy/catalogue.json` and validated and generated by `server/scripts/generate_taxonomy.py`.

Generated representations include Server validation metadata, read-only PostgreSQL reference rows seeded by the canonical migration, Dashboard TypeScript, Android Kotlin where the Android contract consumes the canonical definition, localization coverage, and contract tests. Generated database rows have no runtime CRUD authority and must match the governed source digest.

Operational-code metadata is explicit. A taxonomy encoding token does not automatically become an entity-code token. Categories declare whether and how a value contributes to a module-owned operational-code policy. The Server allocates entity codes transactionally; browsers only display a non-reserving pattern.

Use language-independent stable machine identities

Waiotech must use language-independent stable machine identities.

Ordinary catalogue codes use stable lowercase ASCII `snake_case` within their owning catalogue. Permission and action codes use their module-qualified dotted standards. Codes do not contain labels, Tenant prefixes, or database sequence meaning.

Protect historical records, events, reports, exports, and integrations from code drift

Waiotech must protect historical records, events, reports, exports, and integrations from code drift.

A used code is permanent identity. A label may change when meaning remains unchanged. A new meaning requires a new code. A retired code remains reserved and interpretable.

Avoid a global catalogue version that obscures code-level compatibility

Avoid a global catalogue version that obscures code-level compatibility.

Stable codes and preserved definitions provide historical interpretation. Generated manifests may carry source commit, digest, generator identity, and schema version as provenance.

Separate identity and meaning from presentation language

Waiotech must separate identity and meaning from presentation language.

Localization uses stable keys derived from catalogue and code. APIs and persistence use canonical code. Missing translations use the governed fallback locale and fail verification when coverage is mandatory.

Treat addition as a governed product-contract change

Waiotech must treat addition as a governed product-contract change.

Addition requires Product Authority support, module-owned source change, stable code, behavior implementation where applicable, localization, generated output, persistence support, contract-version review, migrations, and tests.

Apply retirement prospectively and preserve history

Waiotech must apply retirement prospectively and preserve history.

Retirement prevents unsupported new use while preserving historical reads, audit, events, reports, exports, and integration interpretation. Retirement never deletes or reassigns the code.

Do not silently reinterpret historical codes

Waiotech must not silently reinterpret historical codes.

Only when Product Authority declares semantic equivalence for the affected records. Otherwise, conversion requires an explicit correction or migration preserving original and replacement evidence.

Surface unknown codes as integrity defects

Surface unknown codes as integrity defects.

Unknown input is rejected. Unknown persisted product-owned code is a data-integrity non-conformance and is not coerced to `other`, empty, or a similar value.

Prohibit generic escape values without a domain contract

Waiotech must prohibit generic escape values without a domain contract.

`other` exists only when Product Authority defines its meaning, supporting text, reporting effect, and behavioral limits.

Keep reasons tied to the action whose meaning they explain

Waiotech must keep reasons tied to the action whose meaning they explain.

Each action family owns its reason-code catalogue. A generic global reason table is prohibited. Each family declares ownership, extension authority, supporting text, lifecycle, reporting meaning, and integration behavior.

Keep Permission meaning module-owned and assembly centralized

Waiotech must keep Permission meaning module-owned and assembly centralized.

Product modules define protected Permissions. IAM assembles a generated catalogue with stable code, owner, description, dependencies, protected actions, sensitivity, and lifecycle. Tenants cannot create Permission codes.

Separate control definition from Tenant-configured value

Waiotech must separate control definition from Tenant-configured value.

Tenant Control types are product-owned typed definitions declaring code, value type, constraints, dependencies, publication validation, impact behavior, and sensitivity. Tenants configure values through Tenant Control Policy but cannot create control types.

Generate every repeated representation from canonical definitions

Waiotech must generate every repeated representation from canonical definitions.

Generation may produce backend types, validation metadata, database reference synchronization, OpenAPI values, TypeScript and Kotlin values, localization manifests, read-only catalogue responses, documentation, version manifests, and test fixtures.

Generate only representations required by the contract

Waiotech must generate only representations required by the contract.

Output depends on use. A small closed value may generate types. A rich Permission catalogue may generate metadata and queries. Tenant-owned master data uses ordinary persistence and APIs.

Use database tables as derived persistence representations

Waiotech may use database tables as derived persistence representations when relational constraints or rich metadata require them. Rows remain synchronized from canonical source and have no Tenant CRUD authority.

Prohibit direct editing of product-owned reference rows

Waiotech must prohibit direct editing of product-owned reference rows.

Changes occur through canonical source, approved migration or synchronization, verified generation, and release.

Keep API identity language-independent and governance-safe

Waiotech must keep API identity language-independent and governance-safe.

Commands submit canonical codes. Read responses may add localization key, label, description, owner, group, and active or retired condition. Product-fixed catalogue endpoints are read-only.

Model Tenant-governed values as real domain concepts

Waiotech must model Tenant-governed values as real domain concepts.

Each is an explicit module-owned entity and API contract defining Tenant identity, code scope, lifecycle, hierarchy, Permission, history, import, export, and retirement. A universal `custom_value` table is prohibited.

Prevent ungoverned behavior from attaching to configurable values

Waiotech must prevent ungoverned behavior from attaching to configurable values.

Only when Product Authority explicitly defines that behavior. A label or category cannot grant Permission, change lifecycle, alter Plan resolution, post Inventory, or invoke arbitrary automation.

Make catalogue drift release-blocking

Waiotech must make catalogue drift release-blocking.

Continuous integration verifies source validity, unique codes, no code removal or reassignment, dependency resolution, localization, generated parity, OpenAPI and client parity, database reference parity, and absence of forbidden Tenant CRUD.

Use evidence-backed mappings and normalize aliases before canonical storage

Waiotech must use evidence-backed mappings and normalize aliases before canonical storage.

Each value is classified as exact match, approved alias, distinct canonical code, Tenant master data, or unsupported ambiguity. Migration preserves original value, mapping, reason, provenance, affected records, and unresolved cases.

Related documents

- [Canonical terminology](#)
- [Plan target resolution](#)
- [OpenAPI and generated clients](#)

Actions and contracts

Commands, idempotency, audit, events, APIs, OpenAPI, and generated clients.

Contents

- [Commands and idempotency](#)
- [Audit, events, and transactional outbox](#)
- [API contracts](#)
- [API queries, files, batches, and revisions](#)
- [OpenAPI and generated clients](#)

Commands and idempotency

Every accepted mutation is a named command with explicit context, action code, authorization, guards, transaction boundary, idempotency, operation identity, and recoverable receipt semantics.

A command

Waitech must represent every product mutation through an explicit Application command.

A command is a named request to perform one product-significant mutation. It identifies the owned action, typed input, Actor context, Tenant or platform scope, authority, lifecycle guards, Readiness, transaction, result, audit, events, and retry behavior.

Use stable action identities across API, audit, events, and generated references

Waitech must use stable action identities across API, audit, events, and generated references.

Action codes use stable module-qualified identities such as `maintenance.work_order.release` or `inventory.movement.issue`. The code is independent from route names, labels, handler names, and implementation classes.

Establish authoritative execution context before command evaluation

Establish authoritative execution context before command evaluation.

The command context contains:

- authenticated principal;
- effective Tenant or platform scope;
- Actor type and identity;
- request identity;
- correlation identity;
- idempotency key where required;
- expected concurrency version where required;
- recorded time from the trusted clock;
- support or system-authority context where applicable.

Clients cannot assert Permission, effective Access Profile, lifecycle outcome, audit identity, or event identity.

The canonical command execution sequence

Waiotech must keep command acceptance atomic and ordered.

The sequence is:

```
authenticate
-> establish Tenant or platform context
-> acquire or inspect Command Receipt
-> authorize
-> load authoritative state
-> verify concurrency
-> evaluate lifecycle and Readiness
-> execute Domain behavior
-> persist all mandatory consequences
-> record Domain Events and required audit
-> create outbox obligations only for declared durable consumers
-> finalize Command Receipt
-> commit
-> return the accepted result
```

Make retry safety explicit for every mutation boundary

Waiotech must make retry safety explicit for every mutation boundary.

Every externally retryable mutation with business consequences requires idempotency. This includes browser actions, integration commands, artifact creation, publication, posting, assignment changes, support actions, and process creation. A purely internal transaction may omit a client key when duplicate invocation is impossible by contract.

The scope of an idempotency key

Waiotech must prevent key collision across authority and action boundaries.

The key is scoped by effective Tenant or platform scope, authenticated principal class, action code, and any additional stable boundary required by the command contract. It is opaque, bounded, contains no secret, and remains stable for one logical submission.

A request fingerprint

Waiotech must use fingerprints to distinguish replay from conflicting key reuse.

The fingerprint is a deterministic digest of the command's effective normalized input, including subject identities and business inputs but excluding transport noise such as request identity. Secret values are normalized without being stored in clear form.

A Command Receipt

Waiotech must use durable receipts to establish one result for one logical attempt.

A Command Receipt is the durable record of one idempotent logical command attempt. It preserves:

- receipt identity;
- scope;
- principal;
- action code;
- idempotency key digest;
- request fingerprint;
- processing claim;
- established result or rejection;
- operation identity;
- created and completed times;
- reconciliation condition where required.

The receipt does not become domain evidence unless Product Authority assigns that meaning.

Keep receipt state technical and separate from product lifecycle

Waitech must keep receipt state technical and separate from product lifecycle.

A receipt may represent bounded technical conditions such as:

```
processing
succeeded
rejected
failed_retryable
failed_internal
reconciliation_required
```

The state describes command execution, not the lifecycle of the subject.

Prevent abandoned receipts from blocking safe replay indefinitely

Waitech must prevent abandoned receipts from blocking safe replay indefinitely.

A processing claim has a bounded lease. When execution terminates before commit, the claim expires and the same logical command may resume. The claim does not permit a second concurrent executor while valid.

Replay one authoritative result rather than re-executing the command

Replay one authoritative result rather than re-executing the command.

WaioTech returns the established result without repeating accepted business consequences. The response may indicate idempotent replay.

Protect key identity from semantic reuse

WaioTech must protect key identity from semantic reuse.

The command is rejected with `platform.idempotency.conflict`. The established receipt and original result remain unchanged. The original payload is not exposed.

Make negative outcomes duplicate-safe without fabricating accepted actions

WaioTech must make negative outcomes duplicate-safe without fabricating accepted actions.

A deterministic business rejection may be stored in the Command Receipt so a retry returns the same rejection while the receipt contract applies. A rejection creates no partial domain consequence. Material security denials may create separate audit evidence.

Distinguish rollback, response loss, and cross-system uncertainty

WaioTech must distinguish rollback, response loss, and cross-system uncertainty.

When the transaction did not commit because of an explicitly classified transient infrastructure failure, the API returns `service.command_failed_retryable` with category `retryable_infrastructure`, `retryable: true`, and the established operation identity. PostgreSQL classification is limited to known connection, serialization, deadlock, lock-availability, and shutdown conditions; constraint failures and arbitrary exceptions are not transient. A deliberate retry reuses the same idempotency key and may reclaim the receipt. An unexpected programming, invariant, or mapping failure establishes `failed_internal`, returns a sanitized non-retryable `500`, and is not automatically re-executed by receipt replay. A connection failure raised while committing an otherwise successful command has an unknown outcome: the API returns `platform.command.reconciliation_required` with the operation identity and must not classify the command as a safe pre-commit retry. Reusing the same idempotency key then reconciles against any committed receipt without repeating accepted business consequences. When the transaction committed but the response was lost, replay returns the established result. When an external outcome is uncertain, the receipt or related process enters reconciliation-required handling.

Operation identity

WaioTech must use operation identity as the execution spine of a command.

Operation identity identifies one logical command execution across requests, receipt replay, audit, events, outbox delivery, Worker processing, and external reconciliation. The backend establishes it and preserves it across retries of the same logical attempt.

Keep execution identities distinct and propagate only those applicable to each boundary

Waiotech must keep execution identities distinct and propagate only those applicable to each boundary.

Request identity identifies one transport attempt. Correlation identity groups a broader flow. Operation identity identifies one logical command. Causation identity identifies the immediate command or event that produced another fact. Trace identity remains technical observability context.

Related documents

- [Audit, events, and transactional outbox](#)
- [API contracts](#)
- [Background work](#)

Audit, events, and transactional outbox

Required audit facts and durable domain events commit with the authoritative state change. The transactional outbox provides recoverable at-least-once delivery without turning delivery order into hidden business authority.

An audit record

Waiotech must record who exercised protected authority without copying unrestricted payloads.

An audit record is durable accountability evidence for a protected action or security change. It preserves the Actor, authority path, Tenant or platform scope, action code, subject, reason, accepted or denied outcome where required, recorded time, operation identity, and safe changed-fact summary.

Do not accept an auditable action without durable accountability

Waiotech must not accept an action requiring audit without durable accountability. Required audit is written in the same PostgreSQL transaction as the accepted action or denial it records.

Keep audit append-only and independently interpretable

Waiotech must keep audit append-only and independently interpretable.

Corrections are additive and reference the original record. Physical destruction requires a separate explicit legal and Product Authority contract.

A domain event

Emit facts rather than imperative integration commands from Domain ownership.

Persisting a Domain Event does not by itself create an asynchronous delivery obligation. An outbox destination is declared only when a declared internal consumer owns a required durable consequence. Waiotech does not create a default catch-all `internal` destination, and the outbox is not a second audit log.

A domain event is an immutable accepted fact owned by the module whose Domain action established it. It names what happened, not what a consumer should do.

Use stable fact-oriented event names

Waiotech must use stable fact-oriented event names.

Event types use `<module>.<subject>.<past_tense_fact>`, for example `maintenance.work_order.released`. A new business fact receives a new event type.

Version durable payload schemas without embedding versions in event type names

Version durable payload schemas without embedding versions in event type names.

Persisted, outbox, replayable, cross-module, or external messages may be processed after a deployment, retry, dead-letter recovery, or consumer upgrade. The envelope therefore carries `schema_version` so consumers can interpret the payload structure safely.

Separate payload compatibility from fact identity

Waiotech must separate payload compatibility from fact identity.

Compatible optional additions may retain the version when consumers are required to ignore unknown fields. Removing, renaming, changing requiredness, changing type, changing structure, or changing meaning requires another schema version. A different business fact requires another event type.

Use one stable event envelope for durable messaging

Waiotech must use one stable event envelope for durable messaging.

The envelope contains, where applicable:

- event identity;
- event type;
- schema version;
- source module;
- Tenant or platform scope;
- subject type and identity;
- occurred time;
- recorded time;
- Actor context;
- operation identity;
- Command Receipt identity;
- correlation identity;
- causation identity;
- typed payload.

It excludes secrets and unrestricted entity snapshots.

The transactional outbox

Guarantee that an accepted fact cannot lose its required asynchronous consequence.

The outbox is a PostgreSQL record created in the same transaction as the source fact. It preserves the event and destination-specific delivery obligations. External publication occurs only after commit.

Design every event consumer for duplicate delivery

WaioTech must design every event consumer for duplicate delivery.

Delivery is at least once. Consumers must be duplicate-safe through an inbox, processed-event record, unique constraint, or equivalent durable mechanism. Exactly-once transport claims do not replace application idempotency.

Preserve only necessary bounded ordering

WaioTech must preserve only necessary bounded ordering.

Ordering is declared only where product or integration meaning requires it. The contract identifies the ordering key, ordinarily one subject or stream, and consumers reject or reconcile impossible sequence. Global ordering is prohibited.

Avoid event payloads becoming uncontrolled replica models

Avoid event payloads becoming uncontrolled replica models. Events ordinarily contain the accepted fact and references needed by consumers rather than complete aggregate snapshots. A specific contract may require an immutable snapshot when replay or independent interpretation requires it. Consumers query owning modules through approved contracts when more data is required.

Apply versioning where compatibility risk exists

WaioTech must apply explicit schema versioning to any message that is persisted, replayable, or exchanged across an independently deployable boundary. Purely in-process ephemeral messages may omit schema version when they cannot survive or cross that boundary.

Related documents

- [Commands and idempotency](#)
- [Background work](#)
- [Observability](#)

API contracts

WaioTech exposes resource-oriented reads and named action commands through explicit, Tenant-scoped, typed HTTP contracts. Representations, errors, available actions, concurrency, idempotency, and diagnostics remain stable and precise.

Use resources for identity and named endpoints for product-significant mutations

WaioTech must use resources for identity and named endpoints for product-significant mutations.

WaioTech uses resource-oriented HTTP APIs with named business-action endpoints for protected transitions. Resources express stable concepts; actions express governed commands.

Align API ownership with product-module ownership

Align API ownership with product-module ownership.

The module owning the product concept owns routes, schemas, commands, queries, errors, action metadata, versioning, and OpenAPI representation. Shared API code owns only technical transport standards.

Keep HTTP transport subordinate to use-case contracts

WaioTech must keep HTTP transport subordinate to use-case contracts.

It parses and maps transport. Application and Domain own authority, lifecycle, Readiness, evidence, transactions, and events.

Version incompatible external commitments, not deployments

Version incompatible external commitments, not deployments.

One canonical major API base is used. Before any retained external deployment, an incorrect contract is corrected directly. After a retained external deployment, an incompatible commitment requires an explicit new major version with its own complete contract and migration plan; aliases, dual shapes, and hidden translation inside the canonical version are prohibited. Application releases do not create decorative API versions.

Use canonical Product Authority vocabulary in routes

Waiotech must use canonical Product Authority vocabulary in routes.

Paths use lowercase plural canonical nouns with hyphens, such as `/functional-locations`, `/assets`, `/process-units`, `/measurement-points`, `/process-routines`, `/process-rounds`, `/process-conditions`, `/work-orders`, `/failure-events`, `/replenishment-requests`, and `/access-profiles`. Obsolete or ambiguous terminology is prohibited.

Make generated clients and diagnostics unambiguous

Waiotech must make generated clients and diagnostics unambiguous.

OpenAPI path parameters use explicit names such as `{work_order_id}` rather than generic `{id}`.

Use shallow stable resource addressing

Waiotech must use shallow stable resource addressing.

Nesting is bounded to relationships whose meaning depends on the parent. Stable independently addressable entities receive their own canonical routes. Paths do not reproduce the database graph.

Map HTTP methods to product semantics rather than generic CRUD

Map HTTP methods to product semantics rather than generic CRUD.

GET performs safe reads. POST creates resources or invokes named commands. PATCH is limited to typed bounded edits of explicitly mutable non-lifecycle fields. PUT is used only for true complete replacement. DELETE is used only for disposable technical data or Product Authority physical deletion.

Prohibit storage-shaped mutation APIs

Waiotech must prohibit storage-shaped mutation APIs.

Each edit uses a typed request contract. Lifecycle, derived state, authority, and immutable evidence are never assignable through patch documents.

Never accept client assertions of authority or derived state

Waiotech must never accept client assertions of authority or derived state.

Clients submit legitimate business input, selected canonical identities, reasons, notes, expected version, idempotency key, and supported effective-time facts. The backend derives Actor, Tenant validity, Permission, lifecycle result, Readiness, audit, and event metadata.

Require one validated Tenant or platform context per operation

Waiotech must require one validated Tenant or platform context per operation.

One consistent explicit mechanism, such as a dedicated header or route boundary, identifies the requested Tenant. The backend validates it against the authenticated principal. Platform operations use explicit platform routes and scope.

Keep machine Process Reading ingress explicit

Machine Reading submission uses an explicit Process-owned API contract authenticated as an Integration Principal.

The transport may support bounded single and batch submission, but each accepted Reading is validated against canonical Measurement Point identity or a governed Measurement Source Mapping before Application invocation. The API must not accept arbitrary tag names as canonical plant identity.

Machine ingress preserves source event identity where available, effective time, reported value and unit, quality, Data Source, Integration Principal, and idempotency semantics required to make retries duplicate-safe.

Human Dashboard Reading entry uses ordinary human Tenant authority and may use a different transport route or request shape. Both paths converge on the same Process Application and Domain Reading semantics before acceptance.

Enforce one Tenant boundary per ordinary operation

Waiotech must enforce one Tenant boundary per ordinary operation.

Every identity, Attachment, reference, result, and consequence belongs to the effective Tenant. Cross-Tenant platform operations use dedicated contracts.

Use one stable language-independent wire naming convention

Waiotech must use one stable language-independent wire naming convention.

Wire properties use lowercase `snake_case`. Generated clients may map to language conventions internally without changing the wire contract.

Keep technical identity opaque

Waiotech must keep technical identity opaque.

Technical identifiers are opaque UUID strings. Operational codes are separate properties. Clients do not construct or infer meaning from UUIDs.

Exchange unambiguous time semantics

Exchange unambiguous time semantics.

Recorded instants use RFC 3339 with explicit offset and ordinarily normalize to UTC in responses. Local dates, local times, and timezone identifiers remain distinct fields.

Preserve exact values across clients

WaioTech must preserve exact values across clients.

Exact decimal values are serialized as strings when a JSON number could lose precision. The schema declares precision, scale, unit, and currency meaning.

Make partial input semantics explicit

WaioTech must make partial input semantics explicit.

Omission means the contract does not provide or change a value. `null` means explicit absence where the schema permits it. Edit contracts define this distinction per field.

Reject unrecognized command input by default

WaioTech must reject unrecognized command input by default unless a typed extension object is explicitly defined. Unknown fields indicate drift, typo, or unsupported meaning.

Permit additive enrichment without silent meaning change

Permit additive enrichment without silent meaning change.

Response fields may be added only when consumers are required to ignore unknown fields and the addition does not change interpretation. Semantically controlling fields require an explicit contract-version decision before release.

Expose product contracts rather than persistence models

WaioTech must expose product contracts rather than persistence models.

Representations contain product-significant identity, lifecycle, relationships, effective definition references, concurrency version, Readiness, blockers, available actions, and evidence references as applicable. They exclude secrets, ORM shape, private delivery state, and arbitrary metadata bags.

Use purpose-specific read models rather than one oversized entity schema

WaioTech must use purpose-specific read models rather than one oversized entity schema.

Summary, detail, history, command result, export, and administrative views may differ through explicit schemas.

Return the accepted product result rather than only transport acknowledgement

Return the accepted product result rather than only transport acknowledgement.

A command response contains the affected subject, operation and receipt identities, replay indicator, warnings, and authoritative concurrency version when the command establishes one. A purpose-specific response may add a typed bounded result such as a credential or upload instruction. Ordinary clients refetch the authoritative read model after mutation rather than receiving an arbitrary command payload bag.

Do not use 202 for non-durable receipt of a request

Waiotech must not use 202 for non-durable receipt of a request.

Only when the command transaction durably establishes a long-running process. The response identifies the process resource, state, operation, retrieval link, and cancellation action where supported.

Use status and typed errors consistently

Waiotech must use status and typed errors consistently.

HTTP status communicates broad transport category. Stable Waiotech error codes communicate exact meaning. Common mappings include 400 malformed input, 401 invalid authentication, 403 denied authority, 404 safe non-disclosure, 409 lifecycle/concurrency/idempotency conflict, 422 typed business-input validation, 429 rate limit, and 503 safe temporary unavailability.

The error envelope

Waiotech must keep errors machine-readable and non-disclosing.

The error envelope contains stable code, category, safe message, correlation identity, operation identity where available, field violations, typed blockers, named safe references, and retryability. Safe detail fields are closed and explicit; arbitrary metadata is prohibited. It never exposes stack traces, SQL, secrets, or infrastructure topology.

Keep client behavior independent from message wording

Waiotech must keep client behavior independent from message wording.

Domain errors use `<module>.<subject>.<condition>`. Technical cross-cutting errors use a stable platform namespace.

Expose one stable validation contract

WaioTech must expose one stable validation contract.

Each violation identifies a typed input path and stable code. Framework-specific error structures are mapped into the WaioTech envelope. Transport locations such as `body`, `query`, `path`, `header`, and `cookie` are not part of the input-model path exposed to clients.

Each field violation exposes a stable `code`, an input-model `path`, a localization `message_key`, and an optional closed `parameters` object containing only schema-declared safe scalar display values. Clients localize from `message_key` and `parameters`; they must not branch on English message text. The Server may also provide a safe fallback message for unsupported clients. Arbitrary validation metadata and raw rejected payloads are prohibited.

An available action

WaioTech must use authoritative action metadata for client usability.

An available action is a backend-evaluated statement that one named action is available to the Actor for the subject at evaluation time. It may include label key, method, route, input schema, idempotency requirement, and concurrency requirement.

Revalidate every action at execution time

Revalidate every action at execution time.

The command re-evaluates authority, state, concurrency, Readiness, policy, and references because facts may change.

Related documents

- [API queries, files, batches, and revisions](#)
- [OpenAPI and generated clients](#)
- [Application surface architecture](#)

API queries, files, batches, and revisions

Query, search, sorting, pagination, freshness, conditional access, offline synchronization, file transfer, batch commands, cancellation, revisions, catalogues, and localization use bounded explicit contracts rather than generic expression languages.

Treat reads as governed Application contracts

Waiotech must treat reads as governed Application contracts.

Every query defines Tenant scope, Permission, filters, sorting, pagination, representation, freshness, authoritative sources, and limits. Relationship-scoped views such as the current principal's effective Teams are computed by the owning server module; clients do not infer completeness by loading an arbitrary page and filtering locally.

Keep query semantics resource-owned and bounded

Waiotech must keep query semantics resource-owned and bounded.

Ordinary resources expose typed allow-listed filters. A Report Type may define a closed typed analytical expression contract, but unrestricted query languages are prohibited.

Multi-value lifecycle filtering uses repeated, closed-enum query parameters such as `states`. Relationship filters remain resource-owned and explicit, such as Work Order responsible Team IDs or Failure Event linked Work Order IDs. A client that needs the complete filtered collection follows the published pagination contract until `has_more` is false; it must not load one arbitrary maximum page and treat it as complete.

Use deterministic sorting safe for pagination

Waiotech must use deterministic sorting safe for pagination.

Each endpoint allow-lists indexed meaningful sort keys, direction, null order, and immutable identity tie-breaker. Unsupported fields are rejected.

Use stable continuation semantics

Waiotech must use stable continuation semantics.

Mutable operational collections use opaque integrity-protected cursor pagination bound to Tenant, filters, sort, and last ordering values. Bounded stable collections may use offset pagination when the contract states its limitations.

Return counts only when justified

Return counts only when justified.

The endpoint declares exact, estimated, or absent count semantics based on cost and disclosure risk.

Treat search as a protected query contract

Waiotech must treat search as a protected query contract.

Search declares searchable facts, normalization, ranking, Tenant scope, sensitivity, limits, and authorization. A global search cannot reveal unauthorized existence.

Make consistency part of the API contract

Waiotech must make consistency part of the API contract.

A query is classified as authoritative transactional, derived, cache-backed, or replica-backed lag-tolerant. Commands and Readiness use authoritative facts.

Use explicit preconditions for stale-write protection

Waiotech must use explicit preconditions for stale-write protection.

Reads may return ETag. Mutable commands require one consistent expected-version mechanism, ordinarily `expected_version` or `If-Match`. Ambiguous duplicate preconditions are rejected.

Standardize retry identity at the HTTP boundary

Standardize retry identity at the HTTP boundary.

Externally retryable mutations use an `Idempotency-Key` header or exact equivalent. The generated client preserves it for the logical submission. GET does not use mutation receipts.

Make flows traceable without trusting diagnostic input as authority

Waiotech must make flows traceable without trusting diagnostic input as authority.

The server creates request identity and operation identity. Clients may provide a validated correlation identity. Responses return applicable request, correlation, operation, and replay metadata through documented headers.

Make client contract identity explicit without treating it as authority

Waiotech must make client contract identity explicit without treating it as authority.

Browser and Android clients send bounded application identity, application version, and contract version through approved headers. These values support telemetry and minimum-version enforcement but grant no authority and cannot replace authentication or Tenant context.

Expose offline field work through explicit versioned contracts

Waiotech must expose offline field work through explicit versioned contracts.

Synchronization uses typed endpoints for Work Package and Process Package retrieval, operation-envelope submission, receipt retrieval, conflict handling, and Attachment transfer. It does not use a generic entity replication or arbitrary change-feed endpoint.

Keep transfer mechanics separate from governed file meaning

Waiotech must keep transfer mechanics separate from governed file meaning.

Attachment APIs separate upload initiation, byte transfer, verification, Attachment acceptance, linking, and retrieval. Large bytes do not travel inside ordinary JSON commands.

Make batch semantics explicit

Waiotech must make batch semantics explicit.

Each batch action has a typed module-owned contract declaring atomic or item-independent behavior and item-level outcomes. A generic action dispatcher is prohibited.

Machine Process Reading batches are item-independent unless the Process contract explicitly defines an atomic source sequence. Each item preserves Measurement Point resolution, source event identity where available, effective time, value, unit, quality, provenance, and duplicate-safe outcome. One invalid Reading must not cause unrelated valid Readings to be falsely reported as accepted or silently discarded. Batch size, payload size, rate, and source scope are bounded.

Preserve domain vocabulary at the API boundary

Waiotech must preserve domain vocabulary at the API boundary.

Named Product Authority actions are used, such as `cancel`, `retire`, `withdraw`, or `revoke`. HTTP DELETE does not disguise lifecycle.

Do not expose revision administration as CRUD

Waiotech must not expose revision administration as CRUD.

Stable definitions are the ordinary resources. Published revisions are read-only history and evidence. Editing uses named stable-object actions.

Reflect catalogue governance in API shape

Reflect catalogue governance in API shape.

Through generated schema values or read-only metadata endpoints. Tenant CRUD is prohibited. Tenant-governed catalogues use explicit domain routes.

Localize presentation without changing identity

Localize presentation without changing identity.

Standard language negotiation may select presentation labels. Canonical codes and machine contracts remain unchanged. The governed fallback locale applies.

Related documents

- [API contracts](#)
- [OpenAPI and generated clients](#)
- [File ingestion and storage](#)

OpenAPI and generated clients

OpenAPI is generated from module-owned API contracts. Supported browser and Android clients are generated from that canonical artifact and are never edited manually.

Use backend-generated OpenAPI as the transport reference

Waiotech must use backend-generated OpenAPI as the transport reference.

The Server generates one canonical OpenAPI document from registered routes, typed request and response schemas, errors, authentication, headers, action metadata, and owned catalogues.

Correct implementation when OpenAPI conflicts with authority

OpenAPI is a generated engineering reference, not an independent source of Product Authority.

An incorrect contract that has not been published as part of a supported release is corrected directly, and every affected generated client and caller changes atomically. The incorrect operation, field, schema, client module, and documentation are removed in the same change.

Once a contract version is part of a supported release, an incompatible public-contract change uses an explicit new contract version and governed consumer transition. Waiotech does not retain aliases, translation shims, dual request shapes, or competing models inside one canonical contract version.

Make operations complete enough for generation and verification

Every operation defines a stable `operationId`, owner, summary, request and response schemas, errors, authentication, Tenant context, idempotency, concurrency behavior, tags, and content types.

Incomplete operations are not published to OpenAPI and are not represented by handwritten frontend clients.

Maintain one contract chain for every operational surface

Each application generates only the client required by its accepted server surface. A frontend without an accepted operational API does not carry an unused copy of another application's client.

Generation is deterministic. Verification regenerates into a temporary directory and fails on file-set or content drift.

Verify the complete contract and rejection behavior

Contract tests cover schemas, status codes, errors, Tenant isolation, authentication, authorization, idempotency, concurrency, filters, sorting, pagination, files, safe non-disclosure, and generated-client parity.

Related documents

- [API contracts](#)
- [Application surface architecture](#)
- [Testing and software release](#)

Security

Authentication, authorization, Tenant isolation, secrets, enforcement, and operational security.

Contents

- [Authorization architecture](#)
- [Authorization enforcement](#)
- [Authentication, sessions, and secrets](#)

Authorization architecture

Authorization uses explicit human, Integration Principal, internal-system, and platform paths. Each path remains bounded to its own Product Authority responsibility and cannot imply another path.

IAM ownership

IAM owns User, Membership, Team, Team Membership, Permission, Access Profile, Access Profile Revision, Access Grant, Team Access Profile Assignment, Integration Principal, browser session, Android installation identity, mobile session, invitation, password verification, account recovery, and machine credential identity required by Product Authority.

Platform authority remains a separate explicit path.

Canonical human Tenant authority

- User
 - active Membership in selected Tenant
 - effective Team Membership
 - active Team
 - effective Team Access Profile Assignment
 - active Access Profile
 - published Access Profile Revision
 - Access Grants
 - Permission
 - module-owned action guards

There is no Role entity, direct User Access Profile assignment, or direct Membership Access Profile assignment.

Permissions are Tenant-wide capabilities. Functional Location, Process Unit, Asset, Inventory, or responsibility relationships do not create hidden resource-scope authority.

Canonical Integration Principal authority

- Integration Principal
 - active machine identity in one Tenant
 - active machine credential
 - explicit machine Permission
 - Data Source authorization where required
 - module-owned source and subject guards

An Integration Principal is not a User and has no Membership or Team authority path.

For Alpha Process Reading ingress, authorization must verify at least:

- active Tenant and principal;
- machine Permission for Reading submission;
- active Data Source authority;
- active Measurement Point;
- active Measurement Source Mapping between Data Source and Measurement Point;
- input and unit contract;
- any source-specific limits.

Machine authority must not imply Process Condition mutation, Maintenance actions, Inventory posting, Reliability Decisions, or Tenant administration unless separate Product Authority explicitly grants such a machine action.

Canonical platform authority

```
platform User
→ authenticated platform session
→ explicit platform Permission
→ platform-owned Application command
```

A platform command never becomes ordinary Tenant work and never derives Tenant Permission from platform authority.

When platform action affects one Tenant, Tenant scope, Actor, reason, operation identity, before/after evidence, and outcome remain explicit and audited.

Permission design

Permissions are atomic, module-qualified, allow-only capability identities.

They do not encode lifecycle state, responsibility, data-source mapping, separation-of-duty expressions, or arbitrary resource filters. Those guards remain with the owning action.

Assignment and history

Human authority is assigned through Team Access Profile Assignments with effective periods and revocation evidence. Several effective assignments compose by union of allowed Permissions.

Integration Principal authority uses its own explicit machine grant path and never shares Team assignment semantics.

Every evaluation uses durable effective facts. Ended or revoked grants remain history but cannot authorize future action.

Internal system actions

A Waiotech-owned Worker may use only one explicit product-defined internal-system action for the exact job being performed.

Infrastructure identity alone grants no Product Permission and must not become a generic system super-user.

Related documents

- [Integration Principals](#)
- [Authorization enforcement](#)
- [Authentication, sessions, and secrets](#)

Authorization enforcement

Authorization is enforced in the Application layer, fails closed, preserves Actor type and separation of duties, and cannot be replaced by route guards, hidden controls, Workers, stale caches, or platform role names.

Human Tenant authority

Ordinary human Tenant actions use an authenticated User acting through one active Membership in the selected Tenant.

Effective authority is derived from active Team Memberships, effective Access Profile assignments, published revisions, exact Permissions, Tenant condition, responsibility, and action-specific product guards.

A platform User does not gain Tenant Permission from platform authority. Waiotech does not use support sessions, hidden impersonation, or alternate Tenant Actor paths.

Machine Tenant authority

A machine request acts through one active Tenant-bound Integration Principal.

Integration Principal authority is deliberately narrower than human Membership authority. The backend evaluates:

- active Integration Principal;
- exact machine Permission;
- Tenant condition;
- authorized Data Source;
- Measurement Source Mapping and lifecycle;
- source-scoped restrictions;
- request authenticity, replay protection, idempotency, and rate limits;
- command-specific validation.

Alpha machine authority exists for explicitly governed Process Reading ingestion and any other action named by Product Authority. It cannot inherit Team Membership, Access Profiles intended for Users, broad Dashboard authority, or generic Tenant mutation capability.

A Data Source identifies provenance. It is not an Actor and cannot authorize a request by itself.

Platform authority

Platform Admin commands use explicit platform-owned actions and dedicated Application use cases.

A platform command may manage platform facts or invoke a narrow owning-module recovery interface, but it must not enter ordinary Tenant workflows or mutate Tenant records through a generic bypass.

Every privileged platform action records the real Actor, action, reason, scope, correlation, outcome, and affected Tenant where applicable.

Backend enforcement

Backend Application services evaluate authenticated identity, Actor type, Tenant scope, lifecycle, authority, separation of duties, Decisions, and command-specific guards inside the owning use case.

Routes and clients assist usability only. Every protected command and query is authorized by the backend.

Asynchronous execution

A Worker preserves the initiating Actor and authority basis when processing work on behalf of a command.

A Waiotech-owned system action uses an explicit product-defined system Actor and exact job contract. Infrastructure trust alone never grants product Permission.

A Worker must not upgrade an Integration Principal, User, or system action into broader authority than the initiating contract permits.

Separation of duties

The owning command evaluates both authority and independence.

Different Team names do not make the same User independent. Decisions, authorship, execution, verification, and prior Actor relationships are evaluated from preserved facts. Machine Actors do not satisfy a human-independent-verification requirement unless Product Authority explicitly defines such a machine decision.

Caching and failure

Authority may be cached only with Tenant-scoped keys, explicit authoritative sources, complete invalidation triggers, bounded maximum staleness, and fail-closed behavior.

Invalidation covers applicable User or Membership condition, Team Membership, Team condition, Access Profile assignment and publication, Permission catalogue, browser sessions, Android installation and mobile sessions, offline field packages, Integration Principal condition and credentials, Data Source authorization, and affected source mappings.

High-impact actions and machine-ingress checks may require direct authoritative evaluation.

When required identity, Tenant, authority, source, or security data cannot be established, the action is denied. Missing context, unavailable data, and expired cache never produce a permissive fallback.

Related documents

- [Responsibility and actors](#)
- [Integration Principals](#)

- Authorization architecture
- Authentication, sessions, and secrets

Authentication, sessions, and secrets

Authentication establishes attributable identity and protected credentials. Authorization, Tenant scope, lifecycle, and action-specific guards are evaluated separately.

Browser sessions

Browser access uses short-lived access credentials held only in memory and rotating refresh credentials in secure backend-managed cookies. JavaScript-readable persistent storage must not contain access tokens, refresh credentials, passwords, recovery secrets, or CSRF secrets.

Authentication establishes User and session identity. Tenant selection and per-request authorization are separate. A session never receives Tenant Permission merely because a Tenant identifier is present.

Mutations use same-origin or explicitly governed credentialed CORS, anti-CSRF protection, idempotency where required, and strict origin validation.

Platform sessions

Platform Admin uses an explicit platform session and platform authority. Platform refresh credentials use a separate backend-managed cookie and platform access tokens carry an explicit platform surface without a Tenant claim. Tenant and platform refresh families cannot be exchanged.

The session resolves current durable platform Permissions on each protected request. It does not create hidden Tenant context, impersonate a Tenant User, or imply Tenant Permission. User, Tenant lifecycle, and Tenant Export commands use this platform session. Export artifact retrieval additionally requires that the current platform refresh family was authenticated within the configured sensitive-action window; it uses a distinct retrieval Permission, fresh idempotency key, explicit reason, and audited retrieval evidence. High-impact platform recovery, protected export retrieval, credential reset, and equivalent sensitive actions require recent password re-authentication within the configured sensitive-action window. Alpha does not require a second authentication factor; adding one may strengthen the mechanism without changing the product authority path.

Integration Principal authentication

Integration Principals use a machine authentication mechanism distinct from browser, Android, and platform User sessions.

Machine credentials are bound to one Integration Principal and one Tenant. Authentication resolves only machine identity; authorization separately evaluates active principal state, explicit machine Permission, Data Source authority, Measurement Source Mapping, and owning-module guards.

Machine credentials must support rotation and revocation without changing Integration Principal identity. Secret material is stored only as protected verifier material or through an approved secret-management mechanism and is never returned after the one-time credential-establishment response where applicable.

A credential replay, rotation, or compromise response must not rewrite previously accepted Reading provenance.

Machine endpoints use replay resistance, rate limits, idempotency or source-event identity where required, and transport protections appropriate to automated ingestion.

Android sessions and installations

Android uses system-browser authorization code with PKCE, short-lived access credentials, rotating protected refresh credentials, server-side revocation, and Android Keystore-backed local protection. It never stores a password.

Each installation has a stable server-recognized identity bound to one User while active. Installation identity is distinct from physical hardware identifiers and browser sessions. User reassignment requires revocation and secure clearing; it must not transfer credentials, offline field packages, local evidence, or synchronization state to another User.

Mobile credentials remain subordinate to current User, Membership, Tenant, installation, offline field package, and action authority. Offline acceptance cannot be inferred from possession of a credential.

Passwords and recovery

Waiotech stores approved non-recoverable password verification material, never recoverable passwords. Hash parameters are versioned and upgraded only after successful verification.

Account recovery is a protected credential lifecycle with non-enumerating public responses, bounded attempts, expiry, one-time use, session revocation, audit, and safe delivery. The complete delivery and operator contract must exist before an application exposes it.

Admin User creation and password reset generate a server-created temporary password with bounded expiry. It is returned only by the first successful command execution, never persisted in command receipts or audit evidence, and cannot be recovered on an idempotent replay. All existing sessions are revoked on reset. The User must replace the temporary password before Dashboard or Admin work can continue; successful replacement revokes the temporary session as well. Alpha uses one-time operator handoff as the governed delivery contract. The temporary password is displayed exactly once to the authorized platform Actor performing the command, must be handed to the User through an organization-approved channel outside Waiotech, and is never recoverable from Waiotech afterward. Waiotech does not claim to secure the external handoff channel.

Session invalidation

User disablement, Membership end, Tenant lifecycle, credential compromise, password recovery, platform account changes, Android installation revocation, Integration Principal revocation or credential rotation, and mobile-session revocation invalidate all affected sessions and cached authority paths. Uncertainty fails closed.

Secret handling

Credentials, hashes, cookies, tokens, signing keys, encryption keys, delegated URLs, storage credentials, and database credentials remain outside source code, frontend bundles, logs, traces, metrics, error messages, generated documentation, and release manifests.

Audit

Audit covers authentication, recovery, User and Membership lifecycle, Team Membership changes, Access Profile publication and assignment, session revocation, Android installation and mobile-session changes, Integration Principal and machine-credential changes, protected platform recovery, and security-policy changes.

Related documents

- [Authorization architecture](#)
- [Authorization enforcement](#)
- [Android installation and offline authority](#)

Background processing

Durable internal jobs, Reports, Notifications, scheduling, cleanup, retries, and reconciliation.

Contents

- [Background work](#)
- [Report and Notification processing](#)

Background work

Background work is represented by durable PostgreSQL obligations with explicit identity, schema, claims, leases, retries, dead-lettering, fairness, and recovery. Redis coordinates delivery but does not own business truth.

Background work

Waitech must use background execution only for work with explicit durable semantics.

Background work is durable execution outside the initiating HTTP request because it requires substantial processing, retry, scheduling, artifact generation, or reconciliation.

Treat Worker execution as a mechanism, not another product model

Waitech must treat Worker execution as a mechanism, not another product model.

Authority, lifecycle, evidence, atomicity, failure, and Tenant isolation remain identical. The Worker invokes module-owned Application contracts.

Make PostgreSQL recover every required background action

Waitech must make PostgreSQL recover every required background action.

They provide queue delivery, Worker wake-up, routing, delay, and technical coordination. They do not own accepted process state or retry obligations.

A durable work identity

Waitech must separate logical work from execution attempts.

It identifies one logical background obligation across queue redelivery, Worker restart, lease expiry, retry, and dead-letter recovery. Each attempt has a separate attempt identity.

A job payload contents

Waitech must keep queue payloads small and reference PostgreSQL state.

The payload contains work type, schema version, durable record identity, applicable scope, correlation, and delivery attempt metadata. It does not contain a complete authoritative entity or secret.

Version durable job envelopes separately from work type

Version durable job envelopes separately from work type.

A queued payload may cross a deployment boundary. The Worker must identify supported structure and fail safely when a payload is incompatible.

Use PostgreSQL claims rather than queue visibility as execution authority

Waiotech must use PostgreSQL claims rather than queue visibility as execution authority.

The Worker atomically claims eligible PostgreSQL work with Worker identity, attempt identity, claim time, lease expiry, and attempt number. Only one unexpired claim controls the item.

A Worker lease

Waiotech must prevent a failed Worker from holding work indefinitely.

A lease is a bounded technical claim. It may be renewed during demonstrable execution, expires after Worker loss, and never proves product completion.

Resume from durable state and preserve uncertainty truthfully

Resume from durable state and preserve uncertainty truthfully.

The last committed PostgreSQL state controls. Uncommitted work rolls back and becomes claimable. Committed steps remain. Uncertain external effects enter reconciliation rather than blind replay.

Use explicit work-type retry policy

Waiotech must use explicit work-type retry policy.

Each work type defines retryable failures, attempt limit, backoff, jitter, maximum delay, expiry, dead-letter condition, and reconciliation. Automatic retry is bounded.

Dead-lettering

Waiotech must not discard exhausted obligations.

Dead-lettering stops automatic attempts while preserving obligation, attempts, failure class, payload schema, destination, and required intervention. Authorized replay creates another attempt for the same durable work identity.

Protect workload isolation as well as data isolation

Waiotech must protect workload isolation as well as data isolation.

Worker scheduling uses bounded per-Tenant concurrency, fair queueing, rate limits, and batch size so one Tenant cannot consume all execution capacity.

Avoid database locks across external latency

Avoid database locks across external latency.

It claims and loads state in bounded transactions, performs uncontrolled I/O outside long locks, and records acknowledgement or failure in another bounded transaction.

Waiotech Alpha worker inventory

Waiotech Alpha defines exactly five scheduled job types:

- Attachment cleanup;
- Scheduled Work generation;
- Process Round generation;
- authentication cleanup;
- Tenant Export processing.

This is a closed operational inventory. Adding another job type requires an owned Application use case, explicit policy, durable evidence, configuration, recovery, and verification. It does not justify a generic workflow engine.

Persist logical runs, attempts, and item failures

Each scheduled slot has one deterministic logical run identity. PostgreSQL records every claim and attempt, its lease, classified counts, bounded failure evidence, retry availability, and terminal outcome. Tenant-owned item failures are keyed by Tenant, job type, subject identity, and subject version so exclusion and recovery cannot cross Tenant boundaries. They prevent one poison record or Tenant from consuming each later batch indefinitely. Redis and ARQ wake the Worker but do not own retry state.

Separate liveness, readiness, and operational health

Worker liveness proves the process and event loop are responsive. Readiness proves required dependencies and the complete accepted job inventory are available. Operational health is evaluated per job from last successful scheduled slot, consecutive failures, active or expired claims, retry backlog, pending age, and unresolved terminal item failures. A Redis heartbeat alone cannot report the Worker healthy.

Make recovery explicit

Authorized operators recover work through explicit commands that retry one durable run or resolve one failed candidate after corrective action. Recovery preserves the original logical identity and creates another attempt; it does not rewrite prior attempt evidence.

Related documents

- [Audit, events, and transactional outbox](#)
- [Report and Notification processing](#)

Report and Notification processing

Report Runs and Notifications are persisted before asynchronous processing. Authority, cutoff, artifact integrity, recipient resolution, preference, duplicate prevention, and external delivery remain explicit.

Prohibit arbitrary SQL and unrestricted report expressions

WaioTech must prohibit arbitrary SQL and unrestricted report expressions.

A Report Type is a product-defined analytical contract declaring purpose, sources, metrics, dimensions, parameters, temporal semantics, output formats, sensitivity, authorization, and artifact behavior.

Make report output reproducible and attributable

WaioTech must make report output reproducible and attributable.

A Report Run is one durable execution preserving Report Type, contract schema, exact parameters, Actor, Tenant, temporal rule, data cutoff, format, sensitivity, execution lifecycle, artifact, failure, and retry relationship.

Use truthful terminal process states

WaioTech must use truthful terminal process states.

The lifecycle is:

```
queued -> running -> completed
queued -> cancelled
queued / running -> failed
running -> cancelled
```

Completed requires a valid Report Artifact. Cancellation succeeds only when no completed artifact exists.

Separate execution authorization from retrieval authorization

WaioTech must separate execution authorization from retrieval authorization.

Authority and parameter access are evaluated at request acceptance. Execution does not use broader Worker authority. Artifact retrieval re-evaluates Tenant participation, Permission, sensitivity, and governing report access.

A Report Artifact

Waiotech must separate report meaning from file storage identity.

It is the immutable domain-owned output of one completed Report Run and references exactly one Attachment. It preserves report meaning, parameters, cutoff, format, sensitivity, and content digest.

Prevent reports from combining incompatible source times

Waiotech must prevent reports from combining incompatible source times.

The Report Type declares temporal semantics and one coherent cutoff or period. Execution uses an appropriate snapshot, repeatable read, as-of source, staging, or reconciliation mechanism.

Distinguish equivalent report meaning from renderer bytes

Waiotech must distinguish semantic report equivalence from byte-for-byte renderer identity. Byte-identical regeneration is required only when the Report Type explicitly requires it; semantic reproducibility and exact integrity of each completed artifact are always mandatory.

Prohibit arbitrary Tenant-defined notification rule engines

Waiotech must prohibit arbitrary Tenant-defined notification rule engines.

A Notification Type is a product-defined communication contract declaring trigger, recipients, content, mandatory or optional behavior, channels, deduplication, subject, navigation, and sensitivity.

Keep preferences within product-defined limits

Waiotech must keep preferences within product-defined limits.

Preferences control only Product Authority options such as optional enablement, channel, digest, quiet period, locale, or destination. They cannot disable mandatory communication.

Use Notification for awareness rather than duplicate workflow state

Waiotech must use Notification for awareness rather than duplicate workflow state.

A Notification is a recipient-visible communication record with recipient, type, related subject, creation, stable content or rendering context, unread/read state, navigation, and deduplication identity.

Keep communication state separate from product actions

Waiotech must keep communication state separate from product actions.

Reading does not approve, complete, acknowledge, resolve, cancel, or satisfy a Requirement.

Make recipient determination explicit and testable

Waitech must make recipient determination explicit and testable.

The Notification Type uses typed product-defined rules and authoritative facts. The created Notification preserves the resolved recipient. Arbitrary audience queries are prohibited.

Make communication generation duplicate-safe

Waitech must make communication generation duplicate-safe.

A stable identity based on type, recipient, triggering event, subject, occurrence, or escalation stage is protected by a durable uniqueness mechanism.

Separate recipient-visible state from transport state

Waitech must separate recipient-visible state from transport state.

Channel delivery has separate technical attempts and states. Provider failure does not remove the Notification or mark it read.

Related documents

- [Reports](#)
- [Notifications](#)
- [Background work](#)

Application surfaces

Public Website, browser applications, Android, documentation, help, and learning implementation contracts.

Contents

- [Application surface architecture](#)
- [Public Website engineering](#)
- [Android installation and offline authority](#)
- [Android synchronization and release](#)
- [Documentation, help, and learning engineering](#)
- [Browser application contracts](#)
- [Browser interaction, accessibility, and localization](#)

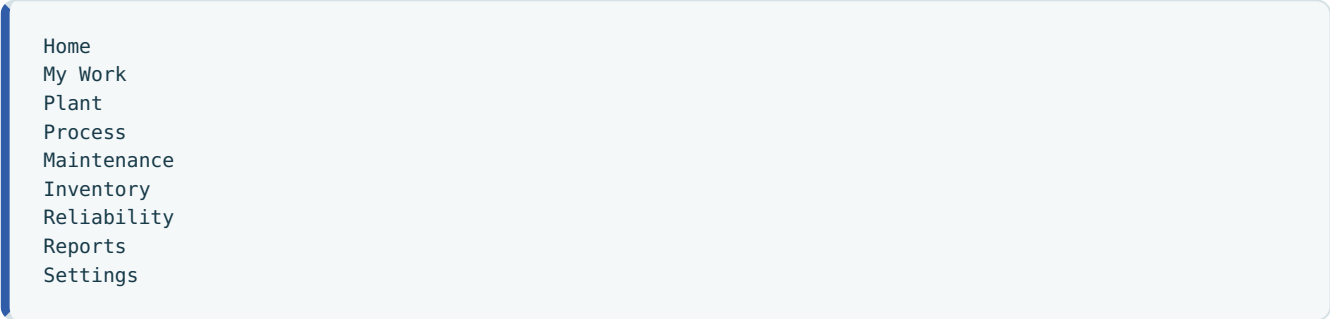
Application surface architecture

Waiotech uses distinct Public Website, Tenant Dashboard, Platform Admin, and Android Work App surfaces over one authoritative Server. Each surface serves a different interaction context without becoming an alternate product authority.

Tenant Dashboard

Tenant Dashboard is the complete browser workspace for one Tenant plant.

Its principal feature areas align with user experience rather than route-module names:



- Home
- My Work
- Plant
- Process
- Maintenance
- Inventory
- Reliability
- Reports
- Settings

The Dashboard uses generated API contracts, purpose-specific read models, Server-provided action availability, URL-backed workspace state where appropriate, and contextual cross-domain navigation.

It must not become a generic entity CRUD renderer.

Platform Admin

Platform Admin uses a separately generated client and explicit platform routes.

It performs only platform-owned provisioning, lifecycle, recovery, preservation, and support functions. It must not reuse Tenant Dashboard mutation components to bypass Tenant authority.

Android Work App

Android is the field surface for governed Process and Maintenance work.

It supports human Process Readings and Observations, bounded Process Condition interaction, Maintenance execution, evidence capture, scanning, and explicitly authorized Inventory interactions. Offline behavior requires governed package authority.

Android does not expose broad configuration, Plan publication, Data Source configuration, Integration Principal administration, or Platform Admin actions.

Public Website

Public Website remains static-first, bilingual, public-safe, and outside authenticated Tenant operations.

One Server authority

All application surfaces rely on the same Product Authority and Application commands. Differences in presentation or connectivity do not create alternate lifecycle, authorization, evidence, or correction rules.

Related documents

- [Browser application contracts](#)
- [Tenant Dashboard experience](#)
- [Android Work App](#)

Public Website engineering

The Public Website is a static-first bilingual Astro application with governed content, localized URLs and metadata, explicit SEO and performance contracts, minimal client JavaScript, privacy-bounded analytics, and narrow public forms.

Give every public page an explicit stable language identity

Give every public page an explicit stable language identity.

Persian is the default public edition. The canonical Persian home is `/`, `/fa/` remains a stable Persian alias, and the English home is `/en/`. All non-root localized routes use explicit locale prefixes:

```
/
/fa/
/fa/...
/en/
/en/...
```

The root path must render the Persian home directly and must not show a language-choice gateway. The `/fa/` alias declares `/` as its canonical URL and is excluded from the sitemap as a duplicate home URL.

Every localized page declares its locale and translated counterpart. English pages use left-to-right presentation. Persian pages use right-to-left presentation and Persian-appropriate typography and layout.

Keep language choice explicit and prevent redirect loops or hidden localized pages

Waiotech must keep language choice explicit and prevent redirect loops or hidden localized pages.

The header provides a direct language switch between the corresponding Persian and English pages. The site does not use automatic locale redirection. Both language editions remain directly accessible, and canonical localized pages remain indexable.

Treat localized metadata as part of the page contract

Waiotech must treat localized metadata as part of the page contract.

Every indexable public page provides localized:

- title;
- description;
- heading structure;
- canonical URL;

- reciprocal `hreflang` references;
- `x-default` where applicable;
- Open Graph metadata;
- social preview text and image alternative text;
- structured-data fields;
- breadcrumb labels where applicable.

Technically complete SEO

Waiotech must enforce complete SEO inputs while making no unsupported ranking guarantee.

It means Waiotech implements every controllable technical and editorial discoverability requirement, including:

- semantic HTML;
- useful original content;
- stable readable URLs;
- correct titles and descriptions;
- canonical links;
- reciprocal locale links;
- XML sitemaps;
- robots directives;
- structured data matching visible content;
- Open Graph and social metadata;
- descriptive internal links;
- optimized media;
- redirect management;
- correct not-found behavior;
- accessible heading hierarchy;
- strong measured performance;
- no duplicate, empty, or misleading localized pages.

External search ranking is not treated as a controllable test result.

Use structured data as truthful machine-readable content, not promotional markup abuse

Waiotech must use structured data as truthful machine-readable content, not promotional markup abuse.

It may publish schema types such as `Organization`, `SoftwareApplication`, `WebSite`, `BreadcrumbList`, `Article`, `FAQPage`, and `ContactPage` only when the visible page content truthfully satisfies the type.

Generate sitemaps from the governed publication state

WaioTech must generate sitemaps from the governed publication state.

Sitemaps are generated from accepted indexable content. They include canonical localized URLs and exclude drafts, private content, redirects, error pages, and non-indexable utility routes.

Control indexing explicitly while protecting private content through authentication and network boundaries

Control indexing explicitly while protecting private content through authentication and network boundaries.

Robots directives are source-controlled and environment-aware. Non-public deployments block indexing. The canonical public deployment permits only intended public content.

Robots directives are not a security boundary.

Preserve public link integrity through governed redirects

WaioTech must preserve public link integrity through governed redirects.

Replaced public URLs receive explicit permanent redirects to the closest semantically equivalent canonical page. Removed content with no replacement returns the correct terminal response and is removed from navigation and sitemap.

Redirect chains and redirect loops are prohibited.

Make public performance a build and release concern

WaioTech must make public performance a build and release concern.

Public pages are static-first, cacheable, media-optimized, and free of unnecessary client JavaScript. Performance budgets cover HTML, CSS, JavaScript, fonts, images, layout stability, and interaction latency.

Measured thresholds belong in the website Experience Contract and release tests.

Optimize public media without weakening accessibility or content meaning

Optimize public media without weakening accessibility or content meaning.

Images use responsive sizes, modern supported formats, intrinsic dimensions, meaningful alternative text, lazy loading where appropriate, and content-derived filenames or identifiers. Decorative images use empty alternative text.

Provide stable bilingual typography without avoidable tracking or rendering risk

Waiotech must provide stable bilingual typography without avoidable tracking or rendering risk.

English and Persian typography use licensed, locally governed or privacy-safe font delivery with bounded font files, subset strategy where valid, fallback stacks, and no layout-breaking dependency on third-party font services.

Treat public content as typed source-controlled material

Waiotech must treat public content as typed source-controlled material.

Content uses governed Astro collections with typed metadata such as:

- content identity;
- locale;
- translated counterpart;
- title;
- description;
- canonical path;
- publication condition;
- content class;
- structured-data type;
- social image;
- review ownership;
- public visibility.

Publish public content through the same attributable review and release discipline as software

Waiotech must publish public content through the same attributable review and release discipline as software.

Content changes use source control, typed content validation, language review, product-claim review, link and metadata checks, Astro build, preview verification, and immutable artifact release. Security, privacy, legal, or compliance claims require review by the applicable owner.

Keep public content versioned, reviewable, and reproducible

Waiotech must keep public content versioned, reviewable, and reproducible.

A contribution interface may prepare source changes, but accepted public content must resolve to a versioned reviewed source state and immutable website artifact. Runtime content must not bypass bilingual, authority, security, and SEO validation.

Make public product claims traceable to canonical meaning

Waiotech must make public product claims traceable to canonical meaning.

Product-significant pages identify their owning content review and applicable Product Authority concepts through metadata or review evidence. The relationship need not be displayed publicly, but it must be available during review and correction.

Prevent accidental partial bilingual publication

Waiotech must prevent accidental partial bilingual publication.

The build fails for content classes requiring bilingual parity. An explicitly language-specific article may be published only when its metadata and user experience identify that limitation without creating a broken counterpart.

Keep public claims authority-aligned

Waiotech must keep public claims authority-aligned.

Public content validation and review must identify the Product Authority basis for product-significant claims.

Measure public content without creating an unnecessary identity graph

Measure public content without creating an unnecessary identity graph.

Analytics must be privacy-minimized and limited to public content performance, navigation, campaign attribution, and technical quality. It must not create cross-surface user tracking, ingest Tenant data, or weaken consent and privacy obligations.

Keep public measurement minimal, transparent, and separate from product identity

Waiotech must keep public measurement minimal, transparent, and separate from product identity.

The Public Website may store language choice, accessibility preferences, and other bounded presentation settings. Privacy-safe aggregate analytics may operate under the applicable privacy contract.

Advertising trackers, cross-site behavioral profiling, Tenant identity correlation, and collection of authenticated product activity are prohibited. Non-essential storage or measurement requiring consent must remain disabled until valid consent exists.

Separate public browsing from authenticated product sessions

Waiotech must separate public browsing from authenticated product sessions.

It may link to the Dashboard authentication entry, but public-origin scripts and storage do not receive or inspect authenticated Dashboard session credentials or Tenant context.

Keep public submission bounded and non-authoritative

Waiotech must keep public submission bounded and non-authoritative.

Contact or support-entry forms use typed narrow endpoints with input validation, size limits, abuse controls, safe acknowledgement, privacy notice, and no product authority. Form failure must not expose provider or infrastructure details.

Related documents

- [Public Website](#)
- [Application surface architecture](#)
- [Documentation, help, and learning engineering](#)

Android installation and offline authority

Android Work App uses native Kotlin architecture, protected local storage, installation identity, mobile authentication, Tenant context, server-issued offline field packages, and one append-only local operation journal. Offline authority is package-specific and never turns the device into an alternate Tenant authority.

Make mobile participation attributable without unstable hardware identity

Each app installation receives an opaque Server-recognized installation identity bound to one User at a time.

Hardware serial, IMEI, advertising identifier, phone number, or another unstable/device-global identifier must not become product identity or authorization authority.

Installation identity participates in authentication, package issuance, synchronization, revocation, audit, and support diagnostics.

Prohibit shared active mobile application state

One active Android application state belongs to one User and one Tenant context.

Changing User or Tenant requires the governed authentication and local-data transition contract. Credentials, packages, journals, cached Tenant data, and protected files must never become shared between Users.

Use mobile-native delegated authentication

Android uses system-browser authorization with authorization-code flow and PKCE against the approved identity service, short-lived access credentials, protected refresh-credential rotation, Server-side revocation, and Android Keystore-backed protection.

Biometrics may unlock protected local credentials or encrypted local data. Biometrics are not independent Server authority.

Enforce Tenant isolation locally and during synchronization

Tenant identity is explicit in navigation, downloaded data, package envelopes, journals, query ownership, files, Notifications, and synchronization requests.

Tenant switch cancels in-flight Tenant work, clears or securely transitions caches according to the recovery contract, and cannot expose records from another Tenant context.

Minimize local data

Android stores only the plant context, Process context, Work Orders, evidence, material references, offline packages, and support data required for its field responsibilities.

It does not replicate the full Tenant database or broad configuration catalogues without a field-use requirement.

Offline field package types

The canonical package types are:

- **work** : exactly one Work Order and its bounded execution authority;
- **process** : a bounded set of Process subjects, Measurement Points, optional active Conditions, and explicitly permitted field-evidence actions.

The shared technical envelope contains:

- package identity and type;
- schema version;
- User;
- installation;
- Tenant;
- exact subject identities and authoritative versions where applicable;
- permitted action codes;
- evidence contract;
- issuance and expiry;
- package digest;
- Server authenticity proof.

Package-specific payload is typed and independently interpretable. Arbitrary extension JSON must not become business authority.

Work Package payload

A Work Package additionally preserves the exact Work Order basis, concurrency version, Procedure Revision, Task and Requirement snapshot, material allocation references where needed, and field execution contract.

Process Package payload

A Process Package additionally preserves the exact included Process Units or Streams, any included Process Round identity and deterministic occurrence identity, the exact immutable Process Routine Revision entry snapshot needed for that Round, Measurement Points available for manual entry, applicable active Process Conditions, bounded context, and Process action contract.

For an included Process Round, the package snapshot is sufficient to establish entry order, required-entry meaning, configured Measurement Points or Observation subjects, reference guidance, evidence requirements, and completion eligibility without consulting mutable Routine configuration. The resulting Readings and Observations remain canonical Process records rather than copied Round values.

A Process Package may authorize only the actions that Product and Experience Authority permit offline. Lifecycle actions that require fresh ownership, duplicate, or current-state evaluation remain online-only unless the package contains a sufficient explicit preauthorization contract.

Offline Work Request drafts

Android may preserve an encrypted local Work Request draft independent of package authority.

A draft is not a Domain record, audit record, or accepted maintenance need. It is submitted only through explicit User action after connectivity returns, using a stable operation identity and the canonical Work Request command.

Make issued authority durable and auditable

PostgreSQL preserves package issuance identity, type, User, installation, Tenant, subjects, contract version, digest, issuance, expiry, revocation, synchronization summary, and audit.

The exact package payload or a reproducible immutable snapshot is retained according to the evidence contract.

Fail closed on package verification

Android verifies Server authenticity, package integrity, type, version, User, installation, Tenant, and expiry before allowing another offline action.

Invalid, altered, unknown-version, wrong-User, wrong-installation, wrong-Tenant, revoked-known, or expired packages cannot authorize another operation.

Preserve local evidence in one ordered journal

Offline Process and Maintenance operations share one encrypted append-only technical journal while preserving package type and owning domain action.

Each entry contains stable local operation identity, package identity and type, action schema version, local sequence, typed input, evidence references, observed times, correction relationship, and synchronization state.

Sharing the journal mechanism does not make Process and Maintenance actions one generic business event model.

Use additive correction

Synced evidence is never mutated locally. A correction references the affected entry and provides the typed corrected value, replacement, or withdrawal reason permitted by the owning domain.

Unsubmitted drafts may be edited because they have not yet been represented as accepted field evidence.

Make offline operations duplicate-safe

The app generates collision-resistant operation identities scoped by installation and package. Identity generation does not depend on connectivity or device time.

Synchronization maps each operation into the same owning Application command used by connected execution and establishes one canonical Command Receipt.

Preserve causal order without trusting the device clock

Each package journal uses a monotonic local sequence and explicit causation where needed. Device timestamps remain evidence but do not replace Server acceptance time or owning-domain effective-time validation.

Make offline eligibility explicit

Package generation classifies required inputs as:

- immutable referenced fact;
- package snapshot;
- locally verifiable field input;
- Server-dynamic online-only fact;
- reconciliation-sensitive exception explicitly permitted by Product Authority.

The Server refuses to issue an offline action when a required input has no safe classification.

Key rotation

Package authenticity identifies the verification-key version.

Android retains the bounded trusted verification set required for valid unexpired packages. Key rotation preserves verification or revokes affected packages explicitly. Compromised keys trigger package revocation and incident response.

Related documents

- [Android Work App](#)
- [Offline field packages](#)

- [Android synchronization and release](#)

Android synchronization and release

Android synchronization preserves field evidence while establishing canonical Server authority operation by operation. Synchronization is resumable, duplicate-safe, package-aware, conflict-explicit, and compatible with both Process and Maintenance field work.

Prevent the mobile journal from becoming an alternate event source

Android journal entries are field operations awaiting or recording synchronization. They are not canonical Domain Events and do not bypass owning Application commands.

After acceptance, the Server emits ordinary audit and Domain Events from the canonical command transaction.

Use checkpointed resumable synchronization

Synchronization exchanges bounded operation batches and durable checkpoints.

A retry after network loss reuses stable operation identities. The client must be able to resume without replaying accepted operations as new work or discarding unresolved evidence.

Establish one canonical result per operation

Every synchronized operation establishes or returns one canonical Command Receipt.

The receipt preserves action, idempotency scope, request fingerprint, result, resulting product identities, and typed rejection where applicable. Mobile metadata links the result to package identity and type, installation, local sequence, and reconciliation state without creating another command-outcome authority.

Keep synchronization inside Application authority

Synchronization resolves the authenticated User and installation, validates the package basis, and invokes the owning Process, Maintenance, Inventory, Attachment, or other explicit Application command.

A synchronization handler must not write domain tables directly or downgrade validation because the action occurred offline.

Synchronize field files separately from Attachment acceptance

File bytes may transfer before or after the related operation according to the typed evidence contract.

Transfer completion does not establish Attachment acceptance. Canonical Attachment identity, classification, subject link, safety checks, and audit are established only through the file/evidence authority.

Preserve time evidence

Synchronization preserves at least the field-observed time, local sequence, Server receipt time, and owning-domain effective/recorded time semantics required by the action.

Clock skew is diagnostic evidence. The Server does not silently replace the User's observed time merely because device time differs.

Typed synchronization outcomes

The protocol distinguishes at least:

- accepted;
- already established;
- rejected with correctable input;
- denied or expired authority;
- authoritative-state conflict;
- needs explicit review;
- retryable infrastructure failure;
- update required.

The Android experience must not present all non-success results as a generic sync error.

Package revocation and expiry

When the Server knows a package is revoked or expired, no new action is accepted under it unless Product Authority explicitly allows reconciliation of evidence already recorded while authority was valid.

Recorded field evidence is not erased because authority later changed. Its final relationship to canonical state is preserved explicitly.

Process synchronization

Manual Process Readings and Operational Observations synchronize through canonical Process commands and preserve Measurement Point, Process subject, User, provenance, effective time, quality, reported unit, canonical normalization, and correction semantics.

Process Round operations synchronize against the exact Round and Process Routine Revision snapshot carried by the Process Package. Starting, entry progress, cancellation, and completion invoke canonical Process Round commands. Round entry synchronization establishes or references canonical Reading or Observation identities; it never creates a second copy of the evidence value inside the Round. A completion operation is accepted only when the canonical Round contract can establish that every required entry and required evidence is satisfied. Retry must preserve the deterministic Round occurrence and stable local operation identities so one logical scheduled occurrence cannot become several Rounds.

An offline Process lifecycle or Operational Action command is accepted only when its Process Package explicitly authorized the action and the canonical command confirms that the issuance basis still permits establishment. Otherwise the field evidence remains preserved and the result requires online review.

Maintenance synchronization

Work Order actions preserve Work Package identity, exact Procedure Revision, Task lineage, Requirements, field observations, Maintenance measurements, Findings, completion evidence, and material evidence according to their owning contracts.

Offline completion submission never performs independent verification or closeout unless Product Authority separately permits that action.

Inventory synchronization

The device never derives Inventory posting authority from a cached Stock Balance.

A Work Package or explicit field contract contains typed references to reserved, issued, kit-custody, or otherwise authorized material context. Synchronization invokes canonical Inventory commands and preserves reconciliation if authoritative custody prevents the intended posting.

Scanning

Scanning resolves typed identifiers such as Functional Location, Asset, Measurement Point, Item, Stock Location, or supported record code. A scan identifies context; it does not grant Permission or offline authority.

Push delivery

Push is a delivery hint for canonical Notification or field refresh. Push payloads contain no unnecessary protected plant data and never become the source of Notification truth.

App upgrades and local schema

Local Room schema and protected-file migrations preserve unsynchronized operations transactionally or through resumable controlled steps.

Supported app releases must remain able to interpret active package and journal versions. An incompatible version fails safely and retains protected evidence for governed recovery.

Mobile version overlap

Every request identifies app and mobile-contract version. The release manifest declares supported API, package, journal, and synchronization versions. The Server returns a typed update-required result when safe operation is no longer possible.

Release and distribution

Android artifacts are signed, identifiable, reproducible according to release policy, and distributed only through approved channels. Release verification includes authentication, package verification, offline operation, synchronization, conflict, upgrade, accessibility, Persian/English, and data-removal behavior.

Performance and field usability

Android uses bounded startup work, lazy loading, paged lists, compressed field media where evidence rules permit, background synchronization, restrained animation, and no unnecessary network request.

The app remains usable on the supported device class under intermittent connectivity.

Sign-out, revocation, and evidence preservation

Secure sign-out, device reassignment, administrative installation revocation, or protected reset removes credentials and readable Tenant data according to the security contract.

Voluntary sign-out must not silently discard unsynchronized evidence. The app identifies unresolved journal entries, attempts synchronization when possible, and requires an explicit safe resolution path before destructive local cleanup.

Related documents

- [Android Work App](#)
- [Offline field packages](#)
- [Android installation and offline authority](#)

Documentation, help, and learning engineering

Governed content is rendered consistently across Help Center, contextual help, Android field documentation, and guided missions. Learning-purpose Tenants run in an isolated data plane with synthetic identities, blocked side effects, resettable state, and release-aligned content.

The governed documentation source

Waiotech must generate all help surfaces from one governed documentation source.

Documentation uses one source-controlled bilingual content system with typed metadata for:

- content identity;
- locale;
- translated counterpart;
- title and summary;
- audience;
- product surface;
- public, authenticated, field, or platform visibility;
- related canonical concepts;
- deep-link targets;
- publication condition;
- applicable product-contract version;
- review owner.

Prevent documentation drift across surfaces

Waiotech must prevent documentation drift across surfaces.

A guide has one content identity with surface-specific rendering or explicitly related variants. Divergent duplicate content is a build failure unless the variants intentionally address different tasks or audiences.

Render governed help safely inside Dashboard

Render governed help safely inside Dashboard.

Tenant Dashboard provides searchable server- or build-indexed content, concept and task navigation, related guides, contextual entry, language selection, and secure deep links. Content rendering uses sanitized approved Markdown or MDX output and does not execute arbitrary embedded scripts.

Publish documentation as a verified versioned product artifact

WaioTech must publish documentation as a verified versioned product artifact.

Help content uses source-controlled review, bilingual validation, product-owner review for behavior and terminology, technical review for links and UI anchors, accessibility review, mission compatibility checks, and release-manifest identity.

A content-only release may publish compatible guidance without rebuilding the backend, but it remains an immutable identified artifact validated against the supported product contract.

Prevent operational edits from creating unreviewed product guidance

WaioTech must prevent operational edits from creating unreviewed product guidance.

Operational storage may hold search indexes, caches, publication manifests, and progress, but canonical help content comes from reviewed governed source.

Make help searchable without bypassing content access

WaioTech must make help searchable without bypassing content access.

Search indexes accepted documentation fields such as localized title, summary, headings, canonical terms, synonyms, error codes, roles, actions, and body text. Results respect content visibility and locale, avoid leaking protected content, and provide deterministic ranking signals.

Decouple contextual guidance from page URL changes

Decouple contextual guidance from page URL changes.

Product routes, forms, actions, lifecycle codes, blocker codes, and error codes map to stable documentation identities. Feature code references documentation identity rather than hard-coded external URLs.

Use deep links for safe navigation only

WaioTech must use deep links for safe navigation only.

A deep link contains a typed destination and permitted context. Navigation re-evaluates authentication, Tenant, Permission, lifecycle, and subject existence. It cannot contain credentials, protected payloads, or an instruction to execute a command automatically.

Provide offline field guidance with the same content identity as online help

WaioTech must provide offline field guidance with the same content identity as online help.

Offline field package generation may include or reference an integrity-checked field-documentation bundle containing only applicable accepted articles and media. It is versioned, localized, installation-scoped, and non-authoritative.

A guided-mission definition technically

Waitech must use typed missions rather than a generic workflow scripting engine.

A guided mission is a typed source-controlled definition containing:

- mission identity and schema version;
- locale content;
- learning objective;
- test-Tenant seed identity;
- permitted product surfaces;
- typed navigation and observation steps;
- expected authoritative facts;
- completion evaluator;
- reset contract.

Mission definitions cannot contain arbitrary executable scripts or bypass product commands.

Isolate learning execution at the data-plane boundary while preserving identical application behavior

Isolate learning execution at the data-plane boundary while preserving identical application behavior.

Learning-purpose Tenants run in a dedicated learning data plane using the same verified application release and product contracts as operational execution, with separately scoped:

- PostgreSQL database or database cluster;
- Redis and Worker queues;
- file storage;
- encryption and signing keys;
- runtime and deployment identities;
- provider adapters;
- search indexes;
- observability namespace;
- backup policy.

A deployment may share physical infrastructure only when an ADR proves equivalent credential, data, network, provider, deletion, and incident isolation. Shared tables or storage prefixes alone are insufficient proof.

Federate learner identity without copying operational credentials or data

Federate learner identity without copying operational credentials or data.

The authenticated platform User receives a short-lived learning-session exchange bound to the same stable User identity, mission, and private test Tenant. The learning plane resolves that identity through a protected mapping without copying password material, operational sessions, broad Membership data, or operational Tenant records.

Make synthetic counterpart Actors non-authenticating and learning-bound

Waiotech must make synthetic counterpart Actors non-authenticating and learning-bound.

Synthetic learning Users exist only in the learning data plane, carry an immutable synthetic classification, have no login credential or external delivery address, and may be invoked only by typed guided-mission simulation actions. Their identifiers are never accepted by operational APIs or migration paths.

Enforce one-way contract compatibility without cross-data-plane product access

Waiotech must enforce one-way contract compatibility without cross-data-plane product access.

It may use public version and compatibility metadata, but it cannot query or mutate operational Tenant, User, Membership, Attachment, report, audit, or integration data.

Provision practice through canonical Tenant isolation and explicit learner authority

Provision practice through canonical Tenant isolation and explicit learner authority.

The onboarding service creates or resets a learner-scoped Tenant with canonical `learning` purpose. It establishes synthetic data through canonical Application commands or a verified seed mechanism that produces equivalent authoritative records.

The learner receives explicit test-Tenant Membership and authority required by the mission.

Keep onboarding practice private per User

Waiotech must keep onboarding practice private per User.

A private test Tenant belongs to one learner. A separately defined instructor-led training contract would require explicit participant isolation and is not implied by guided missions.

Teach against canonical product execution

Teach against canonical product execution.

Dashboard and Android invoke the same APIs, Application commands, Domain rules, lifecycle, Permission evaluation, Readiness, revisions, persistence constraints, and UI components used by operational Tenants.

The implementation does not branch into simplified training rules.

Make training isolation a server-side invariant

Waiotech must make training isolation a server-side invariant.

The canonical `learning` purpose is enforced at durable delivery boundaries. Notification providers, push delivery, billing, public export delivery, and other external effects use non-delivering training adapters or explicit simulated outcomes.

A side-effect fence is enforced in the backend, not only hidden in the UI.

Keep synthetic learning activity outside operational interpretation

Waiotech must keep synthetic learning activity outside operational interpretation.

Operational searches, reports, billing, support queues, integration scans, and global operational metrics exclude test-purpose Tenants unless a protected platform diagnostic explicitly requests them.

Make training data safe and unmistakably synthetic

Waiotech must make training data safe and unmistakably synthetic.

Synthetic seed definitions use stable test-only identities and visible labels. They contain no copied operational personal information, credentials, secrets, file evidence, or organization-specific data.

Keep learning state independent from simulated business state

Waiotech must keep learning state independent from simulated business state.

A separate onboarding module stores User, mission identity, mission-definition version, resume step, completed checkpoints, completion, assessment result where permitted, and reset choices. It does not derive completion solely from mutable test-Tenant records.

Base guided-mission completion on product outcomes

Base guided-mission completion on product outcomes.

The mission evaluator reads authoritative test-Tenant facts through typed queries and compares them with the mission's expected contract. UI clicks alone do not prove completion.

Make reset deterministic, Tenant-bounded, and independently verifiable

Waiotech must make reset deterministic, Tenant-bounded, and independently verifiable.

Reset uses a versioned deterministic seed manifest. It fences mission activity, cancels synthetic processes, removes or supersedes synthetic records according to the reset contract, cleans generated test Attachments and artifacts, restores seed data, and verifies expected state.

It never addresses another Tenant and does not alter onboarding progress unless a separate progress-reset command is accepted.

Execute learning reset through a purpose-checked, locked, idempotent, and verified backend process

Waiotech must execute learning reset through a purpose-checked, locked, idempotent, and verified backend process.

Reset is a named idempotent backend process that:

- authenticates the learner or governed support Actor;
- verifies `learning` purpose;
- acquires a Tenant-scoped reset lock;
- fences mission commands and Workers;
- records the reset request and seed version;
- removes or replaces only the learning Tenant's synthetic state and files;
- recreates the deterministic seed;
- verifies expected state and side-effect fences;
- records the terminal result;
- leaves onboarding progress unchanged unless a separate command requests progress reset.

An operational Tenant identifier is rejected before destructive work begins.

Keep training reset separate from recovery infrastructure

Waiotech must keep training reset separate from recovery infrastructure.

Test reset is a product-level synthetic seed operation, not database restoration or Tenant Import.

Release documentation and onboarding as verified product contracts

Release documentation and onboarding as verified product contracts.

A release manifest identifies compatible documentation and mission content revisions. A guide or mission that references absent routes, actions, codes, or UI anchors fails validation. Public Website, Dashboard, and Android builds consume only compatible accepted content.

Combine structural and linguistic verification for English and Persian parity

Combine structural and linguistic verification for English and Persian parity.

Automated checks compare required counterparts, metadata, headings, canonical terms, deep links, mission steps, safety text, error-code coverage, and visibility. Human language review verifies meaning, readability, Persian directionality, and terminology.

Treat documentation content as untrusted until validated and published

WaioTech must treat documentation content as untrusted until validated and published.

Content rendering sanitizes HTML, validates links and media, prohibits embedded secrets and executable scripts, applies content security policy, and enforces visibility before search indexing or retrieval. Public content must not link to protected raw assets.

Related documents

- [Documentation and Help Center](#)
- [Learning environment](#)
- [Public Website engineering](#)

Browser application contracts

Tenant Dashboard and Platform Admin are non-authoritative browser applications built from generated API clients, purpose-specific read models, explicit action handlers, stable navigation, and shared interaction infrastructure.

Generated clients are mandatory

Browser application code must use generated canonical clients for Server contracts. Hand-written undocumented HTTP requests are prohibited.

Generated wire types are adapted into presentation models only where the adapter adds user-facing meaning without changing Product Authority.

Feature organization

Tenant Dashboard feature ownership should align with the canonical experience:

```
features/  
├── home/  
├── my-work/  
├── plant/  
│   ├── functional-locations/  
│   ├── assets/  
│   ├── process-model/  
│   └── measurement-points/  
├── process/  
├── maintenance/  
├── inventory/  
├── reliability/  
├── reports/  
└── settings/
```

Exact source folders may differ, but browser architecture must not reproduce obsolete Product Authority or hide Process and Reliability inside Maintenance.

Purpose-specific read models

The Server should expose read models suited to user tasks rather than forcing the browser to assemble operational meaning from dozens of low-level entity queries.

Examples include:

- Plant subject summary with connected context;
- Process Condition workspace row and detail;
- Work Order readiness and blocker summary;

- Inventory availability summary;
- Failure Event operational story;
- My Work queue item;
- Home attention projection.

Read models remain projections. Mutation always invokes owning domain actions.

Action registry

Every Server-provided available action shown by the browser must map to a registered typed interaction.

Unknown action descriptors are contract defects and generate diagnostics rather than being silently hidden.

The action handler owns interaction shape such as dialog, full page, confirmation, form section, idempotency behavior, and post-success refresh. It does not re-evaluate Server authority locally.

Contextual creation

Browser routing and creation flows should preserve known canonical context.

Examples:

- Process Condition → Work Request carries Condition and relevant Plant references;
- Asset → Finding carries Asset identity;
- Process Unit → Reading selects compatible Measurement Points;
- Work Order → material Reservation carries Work Order and requirement context.

The browser must not ask users to re-enter canonical relationships already known from the initiating context.

Workspace state

Search, filter, ordering, page size, queue mode, and selected record should be URL-backed where this improves repeated operational use and browser back/forward behavior.

Tenant identity and Permission are never encoded as trusted URL authority.

Process Reading entry

Human Reading entry must optimize for minimal input.

When Measurement Point context is known, the browser uses generated metadata to show quantity and canonical unit, default effective time safely, and submit only legitimate user input. It never invents unit conversion or source provenance.

Machine Reading ingestion is not exposed through ordinary Dashboard data-entry forms. Data Source and Integration Principal configuration belongs in Settings.

Operational timelines

The browser may render cross-domain chronological projections but every entry retains source record type, canonical identity, time semantics, Actor, and authorized navigation to source detail.

Timeline composition must not create a generic Event mutation model.

Home and My Work

Home and My Work use Server-owned projections for attention, executable actions, blockers, due state, and responsibility.

The browser may filter and order within explicit contract parameters but must not infer hidden business states from raw records.

Errors and concurrency

Browser error handling uses stable error codes and typed blockers. Concurrency conflict causes authoritative refetch and an understandable user recovery path.

Uncertain retries reuse the same idempotency identity when the command contract requires it.

Platform Admin separation

Platform Admin must use explicit platform contracts and components appropriate to platform operation. Sharing technical UI primitives with Tenant Dashboard is permitted; sharing plant authority or hidden Tenant mutation paths is prohibited.

Related documents

- [API contracts](#)
- [OpenAPI and generated clients](#)
- [Tenant Dashboard experience](#)

Browser interaction, accessibility, and localization

Forms, revisions, Plan impact, Access Profile authoring, high-impact actions, lifecycle presentation, tables, long-running processes, artifacts, Notifications, uploads, localization, time, quantities, accessibility, untrusted content, storage, telemetry, and feature flags follow explicit interaction contracts.

Keep form edits distinct from authoritative resources

Waiotech must keep form edits distinct from authoritative resources.

Forms separate server initial values, unaccepted local changes, local structural validation, server validation, submission, and accepted result. Every business field maps to a typed request contract.

Avoid a second Domain implementation in the browser

Avoid a second Domain implementation in the browser.

It improves usability for required fields, formatting, and generated schema. Backend validation controls authority, lifecycle, policy, relationships, evidence, and calculations.

Preserve recoverable work without preserving unsafe data

Waiotech must preserve recoverable work without preserving unsafe data.

Recoverable user input remains available after ordinary validation or transport failures except when retention creates a security risk, authorization scope changed, the session ended, the subject disappeared, or the field contains a secret that must be cleared.

Restrict autosave to explicitly mutable unaccepted content

Restrict autosave to explicitly mutable unaccepted content.

Only for mutable working drafts and non-authoritative preferences. Autosave does not publish, approve, post, release, complete, revoke, or perform another protected transition.

Expose stable-object workflows

Waiotech must expose stable-object workflows.

The interface uses Edit, Save changes, Unpublished changes, Review changes, Publish changes, Discard changes, Change history, and Restore content. It does not present branches, merge, manual supersession, or revision activation.

Make publication coverage exact and understandable

Waiotech must make publication coverage exact and understandable.

The Plan editor shows published and proposed targets, additions, removals, exclusions, ineligible targets, missing trigger facts, evaluation time, and effect on occurrence recognition. Raw selector JSON is not exposed.

Simplify authoring without hiding grants

Simplify authoring without hiding grants.

The interface provides Permission search, module grouping, selected Permissions, dependencies, template comparison, additions, removals, blockers, and impact on assigned Teams. Groups and templates grant no authority themselves.

Use consequence-specific confirmation according to risk

Waiotech must use consequence-specific confirmation according to risk.

Confirmation names the exact action, subject, material consequence, reversibility, reason, and resulting evidence or lifecycle. Generic “Are you sure?” text is insufficient for high-impact actions.

Do not present projections as editable lifecycle facts

Waiotech must not present projections as editable lifecycle facts.

Canonical lifecycle codes receive localized labels and accessible visual treatment. Readiness, overdue condition, available stock, target drift, and other projections are labelled as derived and include evaluation time where material.

Keep operational queries server-owned and bounded

Waiotech must keep operational queries server-owned and bounded.

Tables use typed server filters, deterministic sorting, cursor pagination, accessible semantics, clear states, stable row identity, and bounded presentation preferences. They do not download unbounded Tenant datasets or expose arbitrary filter builders.

Present process execution without fabricated progress

Waiotech must present process execution without fabricated progress.

The UI displays backend states such as queued, running, completed, failed, and cancelled. Measured byte or item progress may be shown when truthful. Time-based invented completion percentages are prohibited.

Reauthorize sensitive file access

Reauthorize sensitive file access.

Every retrieval invokes backend authorization. Temporary download URLs are not persisted, shared, or logged. Generating an artifact does not grant permanent retrieval authority.

Keep communication separate from workflow

Waitech must keep communication separate from workflow.

Notifications show type, related subject, creation, unread/read state, safe content, navigation, and applicable product actions. Reading does not execute the related action. Channel failures remain diagnostic delivery state.

Present exact file state

Waitech must present exact file state.

The UI distinguishes selected locally, transferring, transferred, verifying, safety evaluation, accepted Attachment, failure, quarantine, rejection, and unavailability. Transfer progress does not prove Attachment acceptance.

Localize presentation without changing contracts

Localize presentation without changing contracts.

Stable localization keys and canonical API codes drive labels. Locale-aware formatting applies to dates, time, decimal, quantity, and money. Translated labels never become identity.

Keep operational time independent from device settings

Waitech must keep operational time independent from device settings.

Tenant operational scheduling uses the Tenant timezone. Browser timezone may support personal display only where the Experience Contract permits it. Ambiguous time presentation includes the applicable timezone.

Prevent JavaScript floating-point from changing product meaning

Waitech must prevent JavaScript floating-point from changing product meaning.

Frontend code uses decimal-safe parsing and formatting. Authoritative calculations come from the backend. A client preview is labelled non-authoritative.

Treat accessibility as a mandatory functional requirement

Waitech must treat accessibility as a mandatory functional requirement.

Applications support semantic structure, keyboard operation, visible focus, labels, accessible errors, dialogs, tables, status announcements, contrast, text resizing, reduced motion, and non-color status meaning.

Treat all external and uploaded content as hostile until verified

Waitech must treat all external and uploaded content as hostile until verified.

Untrusted HTML is not rendered directly. Rich content requires approved sanitization, safe URLs, script removal, isolation, content security policy, and media-type controls.

Keep credentials and private service authority outside browser code

Waitech must keep credentials and private service authority outside browser code.

Every frontend build value is public.

Use browser persistence only for non-authoritative presentation data

Waitech must use browser persistence only for non-authoritative presentation data.

Persistent browser storage is limited to bounded low-risk presentation preferences. Tenant and User scope are explicit. Authentication secrets, authoritative aggregates, sensitive evidence, and unrestricted payloads are prohibited.

The browser offline contract

Waitech must limit browser offline behavior to safe preparation and resumable technical state.

Tenant Dashboard and Platform Admin remain online-authoritative. A browser may preserve permitted unsent draft input, presentation preferences, and resumable transfer state, but it cannot execute a Work Order, post Inventory, publish, approve, close out, or perform another protected command without backend confirmation.

Governed offline field execution belongs only to Android Work App through the Work Package contract.

Require deliberate submission of recovered business input

Waiotech must require deliberate submission of recovered business input.

Not for protected commands without explicit user confirmation. Resumable byte transfer may continue under its specific contract.

Minimize diagnostic data and keep product metrics server-authoritative

Minimize diagnostic data and keep product metrics server-authoritative.

Telemetry contains safe route performance, latency, error code, request and correlation identity, and client exception classification. It excludes secrets, form values, file content, unrestricted API payloads, and personal information without an explicit contract.

Use flags as deployment controls only

Waiotech must use flags as deployment controls only.

Feature flags control technical exposure of already governed behavior. They do not create hidden product rules, weaken Permission, change lifecycle meaning, create Tenant forks, or maintain two permanent vocabularies.

Verify every user-facing surface without relying on client-side product authority

Waiotech must verify every user-facing surface without relying on client-side product authority.

Tests cover:

- Dashboard and Platform Admin component, integration, accessibility, and protected end-to-end behavior;
- generated browser and Android clients;
- Public Website bilingual routes, metadata, structured data, sitemaps, redirects, accessibility, performance, and content parity;
- Android User and installation binding, Room migrations, Work Package verification, offline journal ordering, file capture, scanning, material scope, synchronization, conflicts, revocation, upgrade, and evidence recovery;
- Help Center search, contextual links, visibility, bilingual parity, mission definitions, private test-Tenant isolation, side-effect fences, progress separation, and deterministic reset;
- separation of public, Tenant, platform, field, and learning responsibilities.

Related documents

- [Browser application contracts](#)
- [Tenant Dashboard and Platform Admin](#)

- [Testing and software release](#)

Related experience

- [Tenant Dashboard experience](#)
- [Dashboard design conventions](#)

Files and artifacts

Attachment ingestion, immutable storage, access, integrity, generated artifacts, and recovery.

Contents

- [File ingestion and storage](#)
- [File access, integrity, and lifecycle](#)
- [Generated artifacts and file recovery](#)

File ingestion and storage

Attachments use explicit upload sessions, Tenant and subject binding, server-owned storage keys, verified media type and size, streaming transfer, immutable storage, and recoverable acceptance across storage and PostgreSQL boundaries.

The Attachments module ownership

Waiotech must keep file identity and handling in Attachments and business evidence meaning in the governing module.

Attachments owns accepted file identity, upload sessions, file integrity, classification, safety and quarantine, storage availability, storage ports, transfer contracts, retrieval coordination, file-level audit, and typed Attachment Links. The governing product module owns why the file matters.

Create Attachment identity only after bytes satisfy the acceptance contract

Waiotech must create Attachment identity only after bytes satisfy the acceptance contract.

An Attachment is the stable identity of one accepted immutable file. It preserves Tenant, filename, declared and verified media type, size, integrity algorithm and digest, uploader or generating process, acceptance time, classification, safety state, availability, immutable storage reference, and governing link.

Separate transfer state from accepted file identity

Waiotech must separate transfer state from accepted file identity.

It is temporary technical state for receiving and verifying bytes. It does not prove accepted evidence, file availability, or business linkage.

Use typed links to assign product meaning to files

Waiotech must use typed links to assign product meaning to files.

An Attachment Link connects an Attachment to one typed governed subject and role. It preserves link identity, Tenant, subject, role, governing or additional status, adding Actor, time, lifecycle, and correction relationship.

Keep evidence roles domain-owned

Waiotech must keep evidence roles domain-owned.

The module owning the linked subject defines permitted roles, evidence effect, authority, correction, and retrieval behavior. Attachments provides mechanism only.

Prohibit generic file association as a product API

Waiotech must prohibit generic file association as a product API.

Links are created through module-owned typed commands. Internal polymorphic references may exist technically but cannot authorize unrestricted linking.

The governing subject

Waiotech must never allow an accepted Attachment without governing context.

Every Attachment has exactly one governing subject that determines minimum access, sensitivity, primary meaning, preservation dependency, and evidence context. Acceptance and governing-link creation commit together.

Permit controlled reuse without access escalation

Permit controlled reuse without access escalation.

One accepted Attachment may be linked to additional governed subjects within the same Tenant when the owning-domain contracts permit reuse. Additional links cannot weaken governing-subject protection and may impose stronger restrictions.

Keep evidence sufficiency under the owning domain

Waiotech must keep evidence sufficiency under the owning domain.

The governing domain verifies required role, availability, integrity, safety, metadata, and any domain-specific acceptance. Upload or link presence alone is insufficient.

The canonical ingestion flow

Waiotech must separate transport, verification, storage, and business acceptance.

The flow is:

```
create upload session
-> transfer temporary bytes
-> complete transfer
-> verify size and digest
-> evaluate media type
-> complete required safety processing
-> finalize immutable storage object
-> commit Attachment and governing Link
-> expose according to accepted state
```

Bind temporary transfer to one authority and intended use

Waiotech must bind temporary transfer to one authority and intended use.

It preserves Tenant, principal, intended subject and role, filename, declared type and size, limits, transfer method, temporary object, transferred bytes, expiry, completion, and safe failure.

Create another session when the intended governed context changes

Waiotech must create another session when the intended governed context changes.

Prevent abandoned upload authority from remaining valid

Waiotech must prevent abandoned upload authority from remaining valid.

Every session has a bounded expiry. After expiry, transfer is rejected, no Attachment is created, and temporary bytes become eligible for controlled cleanup.

Keep storage addressing private and non-semantic

Waiotech must keep storage addressing private and non-semantic.

The server or storage adapter creates an opaque collision-resistant key independent from filename, product code, or user path. Clients never provide or receive authoritative storage paths.

Prevent path injection and unsafe download metadata

Waiotech must prevent path injection and unsafe download metadata.

Filenames are untrusted presentation metadata. Path components and control characters are removed, unsafe length and header content are rejected, and the name is never used as a physical path or content-type proof.

Treat file content as untrusted

Waiotech must treat file content as untrusted.

The contract compares declared type, extension, detected type, and file signature where supported. Mismatch may reject or quarantine. Filename extension alone is insufficient.

Bound file resource use at every ingestion stage

Bound file resource use at every ingestion stage.

Limits apply before transfer where declared, during streaming, and before acceptance. Aggregate subject or command limits may also apply. Exceeding a limit creates no Attachment.

Keep file processing memory-bounded

Waiotech must keep file processing memory-bounded.

No file operation may require loading the complete object into memory when the permitted size exceeds a safe bounded allocation. Transfer, hashing, scanning, and retrieval use streaming or bounded buffering.

Delegate byte transfer without delegating Attachment authority

Delegate byte transfer without delegating Attachment authority.

Where direct object-store transfer is used, Waiotech grants only short-lived authorization bound to one upload session, Tenant, object, operation, size, and content limits where supported. The backend still verifies and accepts the object.

Choose disposable orphan bytes over false accepted evidence

Choose disposable orphan bytes over false accepted evidence.

If byte transfer succeeds but governed Attachment acceptance does not, no Attachment exists. The unaccepted object remains private and becomes eligible for cleanup after the diagnostic interval.

Never commit Attachment metadata before immutable bytes exist

Waiotech must never commit Attachment metadata before immutable bytes exist.

No Attachment exists. The finalized unreferenced object is an orphan eligible for conservative cleanup.

Bind one Attachment identity permanently to one byte sequence

Waiotech must bind one Attachment identity permanently to one byte sequence.

The exact byte sequence cannot be overwritten under the accepted reference. Object versioning, write-once keys, digest verification, or equivalent controls prevent silent mutation and key reuse.

Define provider conformance by capability

Waiotech must define provider conformance by capability.

The storage port supports temporary upload, streaming, exact finalization, immutable read, metadata, integrity support, bounded delegated transfer where used, cleanup of disposable objects, health, backup, and recovery.

Related documents

- [Attachments and evidence](#)
- [File access, integrity, and lifecycle](#)
- [Generated artifacts and file recovery](#)

File access, integrity, and lifecycle

Accepted file bytes remain private, immutable, integrity-protected, subject-authorized, quarantine-aware, correction-based, and preserved according to governing evidence rules.

Keep storage replaceable without changing product evidence

Waiotech must keep storage replaceable without changing product evidence.

Changing the storage mechanism requires an ADR and migration that preserves every Attachment identity, byte sequence, digest, link, classification, availability meaning, access rule, and recovery guarantee.

Keep accepted file storage private

Waiotech must keep accepted file storage private.

Retrieval occurs through backend authorization or bounded delegated read authority.

Permit bounded storage delegation without bypassing governing-subject access

Permit bounded storage delegation without bypassing governing-subject access.

They are short-lived, read-only, bound to one exact immutable object, issued after authorization, excluded from logs and persistent storage, and treated as temporary secrets. Classification may prohibit them.

Re-evaluate file authority at retrieval time

Re-evaluate file authority at retrieval time.

Every retrieval evaluates principal, Tenant, governing-subject access, applicable Permission, classification, safety, integrity, storage availability, and artifact sensitivity.

Freeze bytes, not access rights

Waiotech must freeze bytes, not access rights.

Uploading does not survive Membership suspension, Permission loss or governing-subject access change.

Deliver accepted bytes safely and consistently

Waiotech must deliver accepted bytes safely and consistently.

Downloads use streaming, safe content disposition, accepted filename, verified type or safe fallback, byte size, sensitivity-aware cache controls, `nosniff`, and interruption handling. Storage references remain private.

Treat preview as an active security boundary

Waiotech must treat preview as an active security boundary.

Download is the safe default. Inline rendering requires an allow-listed media type, integrity and safety checks, sandboxing, content security controls, size limits, and no credential-bearing embedded requests.

Make accepted bytes cryptographically identifiable

Waiotech must make accepted bytes cryptographically identifiable.

Attachment metadata stores approved cryptographic algorithm and digest, accepted size, exact object identity, and verification evidence. SHA-256 is the minimum unless a stronger security standard applies.

Establish integrity in a trusted boundary

Establish integrity in a trusted boundary.

The backend or trusted adapter computes it and validates the result. Client-provided digest may assist transport but is not sole authority.

Verify at trust and recovery boundaries

Waiotech must verify at trust and recovery boundaries.

Before acceptance, after storage migration, during restoration verification, upon suspected corruption, and when a sensitive retrieval contract requires it. Equivalent provider integrity controls may avoid rehashing every read.

Treat mismatch as an evidence-integrity incident

Waiotech must treat mismatch as an evidence-integrity incident.

Normal use and download stop. The Attachment enters integrity-failure quarantine or unavailability, remains historically visible, triggers operations alert, and may be restored only with bytes exactly matching the accepted digest.

Quarantine

Restrict unresolved or unsafe files without deleting history.

Quarantine means safety, integrity, content type, or processing result does not permit ordinary use. The Attachment remains visible but cannot satisfy evidence or normal retrieval.

Make safety disposition explicit and auditable

Waiotech must make safety disposition explicit and auditable.

An idempotent protected action preserves reviewer or system process, reason, evidence, time, prior state, and outcome: release, retain quarantine, or reject from normal use.

Storage unavailability

Waiotech must preserve meaning while reporting technical loss truthfully.

It means Attachment identity and links exist but accepted bytes cannot be retrieved. It is distinct from quarantine. Historical existence remains visible, but the file cannot satisfy retrievable evidence.

Correct by addition, never overwrite

Correct by addition, never overwrite.

Different content creates another Attachment and typed link with correction or supersession reason, Actor, and time. The original remains immutable.

Prevent metadata edits from rewriting evidence

Waiotech must prevent metadata edits from rewriting evidence.

Minor non-evidential presentation correction may use a protected action. Material changes to governing subject, role, attribution, classification, filename with evidential meaning, type, size, digest, or identity require additive correction.

Keep file relationships append-oriented

Waiotech must keep file relationships append-oriented.

They may be superseded, revoked for new reliance, or corrected while preserving history.

Limit deletion to disposable technical data and explicitly disposable learning-Tenant state

Waiotech must limit deletion to disposable technical data and explicitly disposable learning-Tenant state.

Automatic deletion is permitted for expired unaccepted uploads, abandoned sessions, failed temporary generation output, rebuildable previews, and verified orphan objects.

Accepted Attachments belonging to a learning-purpose test Tenant may be deleted only through its deterministic reset contract because the learning-environment contract classifies the complete synthetic business state as disposable. Governed operational Attachment destruction requires a separate Product Authority and legal contract.

Avoid deleting bytes during transient database or transaction delay

Avoid deleting bytes during transient database or transaction delay.

An orphan has no Attachment, active upload, or active generation reference and has exceeded the diagnostic interval. Detection reconciles PostgreSQL and storage conservatively before deletion.

Keep logical evidence identities separate from storage optimization

WaioTech must keep logical evidence identities separate from storage optimization.

Only when logical Attachment identities, Tenant isolation, classification, access, audit, and retention remain independent. Cross-Tenant deduplication requires a security ADR proving no existence side channel.

Related documents

- [File ingestion and storage](#)
- [Generated artifacts and file recovery](#)
- [Authentication, sessions, and secrets](#)

Generated artifacts and file recovery

Generated artifacts commit through explicit durable generation workflows. Previews and extracted text remain non-authoritative, and backup, restoration, archive handling, encrypted files, and file tests preserve accepted evidence.

A generated artifact

Share file infrastructure without creating a generic artifact domain.

A generated artifact is a domain-owned immutable output of a governed process. Report Artifact and Tenant Export Artifact retain distinct meaning and each references exactly one Attachment.

Expose only validated complete output

Waiotech must expose only validated complete output.

The process loads exact inputs, creates temporary bytes, validates format and completeness, computes digest, performs safety processing, finalizes storage, and commits artifact, Attachment, governing link, and completed process state consistently.

Never present partial output as complete

Waiotech must never present partial output as complete.

No completed artifact exists. Partial bytes remain unavailable and disposable; the process preserves safe failure and retry evidence.

Prevent derived file processing from becoming hidden product truth

Waiotech must prevent derived file processing from becoming hidden product truth.

They are derived, rebuildable, access-controlled representations linked to the source Attachment. Extracted facts become authoritative only through an explicit domain action.

Fail closed on content that cannot be inspected safely

Fail closed on content that cannot be inspected safely.

Archive processing enforces expansion, nesting, traversal, file-type, and encryption limits. Password-protected or encrypted files are rejected unless a complete Product Authority and security handling contract exists.

Restore metadata and bytes as one evidence system

Restore metadata and bytes as one evidence system.

Recovery verifies PostgreSQL Attachment records, links, artifact records, object existence, size, digest, classification, and Tenant partitioning. Missing bytes leave the Attachment unavailable rather than erased.

Verify ingestion, evidence, security, storage, and recovery independently

Waiotech must verify ingestion, evidence, security, storage, and recovery independently.

Tests cover hostile filenames, size, streaming, media spoofing, digests, safety failure, quarantine, Tenant isolation, governing access, additional links, direct upload, delegated URL, immutable storage, correction, orphan cleanup, artifacts, backup, and adapter conformance.

Related documents

- [File ingestion and storage](#)
- [Backup and recovery](#)
- [Reports](#)

Delivery and operation

Deployment, migration, recovery, observability, security, testing, and release.

Contents

- [Deployment and migrations](#)
- [Backup and recovery](#)
- [Observability](#)
- [Operational security](#)
- [Testing and software release](#)

Deployment and migrations

Waiotech deploys immutable versioned artifacts through serialized, verified activation. Schema and data changes use forward-only migrations, expand-and-contract compatibility, explicit verification, and non-destructive application rollback.

The canonical release unit

Release identifiable compatible server, web, content, and Android artifacts.

One coordinated server-and-web release set contains server image, Dashboard image, Platform Admin image, Public Website artifact, migration set, OpenAPI, generated clients, catalogues and registries, documentation content identity, mission content identity, release manifest, and build provenance. API and Worker may use one server image with separate entry points.

Android Work App is an independently distributed signed artifact whose release manifest declares compatible API, Work Package, journal, synchronization, documentation, and minimum-server contract versions.

Use immutable role-specific server and web artifacts plus a separately signed Android artifact

Waiotech must use immutable role-specific server and web artifacts plus a separately signed Android artifact.

The canonical deployable web and server artifacts are `waiotech-server`, `waiotech-dashboard`, `waiotech-admin`, and `waiotech-website`. The server image provides explicit API, Worker, migration, and controlled CLI entry points. Android Work App is a signed application package rather than a runtime image.

Share code artifacts without sharing runtime responsibility

Share code artifacts without sharing runtime responsibility.

They share Domain, Application, persistence, migrations, generated definitions, and schema assumptions. One image reduces interpretive drift while role-specific commands and privileges remain separate.

Deploy the exact artifact that passed verification

Deploy the exact artifact that passed verification.

Digests, tags, and bytes do not change after acceptance. Environment configuration is external. Promotion does not rebuild.

Make software supply chain reconstructable

Waiotech must make software supply chain reconstructable.

Release identity includes image digest, source commit, build identity, dependency provenance, software bill of materials, and signature or equivalent provenance where required.

Specify orchestration capabilities rather than one vendor

Specify orchestration capabilities rather than one vendor.

Docker Compose is conforming when it provides immutable images, separate roles, private networks, secrets, health, resources, persistent storage, locking, and compatible rollback. Another adapter requires equivalent guarantees.

Separate immutable software, configuration, and credentials

Waiotech must separate immutable software, configuration, and credentials.

Configuration is external, schema-validated, environment-scoped, and technical. Secrets use approved secret management and never appear in images, frontend bundles, logs, migrations, or release manifests.

A release manifest

Waiotech must make the compatible deployment set explicit.

It identifies source commit, image digests, migration head, OpenAPI digest, generated-client and catalogue digests, configuration schema, supported PostgreSQL and Redis versions, and compatibility constraints.

Separate deployment authority from product administration

Waiotech must separate deployment authority from product administration.

A narrowly authorized engineering or automation identity may activate verified images, run approved migrations, update approved configuration, inspect bounded health, and perform compatible rollback. It has no ordinary business-record authority.

Ensure one controller for release activation

Ensure one controller for release activation.

An attributable recoverable lock prevents overlapping deployment and schema change in one environment.

The deployment sequence

Waiotech must use one repeatable verified activation process.

The sequence is:

```
verify manifest
-> validate configuration and secrets
-> verify infrastructure compatibility
-> acquire deployment lock
-> establish required recovery protection
-> run migrations
-> activate compatible backend roles
-> activate Public Website, Tenant Dashboard, Platform Admin, and compatible help content
-> verify Android compatibility policy and distribution metadata
-> verify readiness
-> run external smoke tests
-> record deployment
-> release lock
```

Use explicit operational fencing rather than inconsistent service

Waiotech must use explicit operational fencing rather than inconsistent service.

Maintenance mode or write fencing is required whenever online compatibility cannot preserve one authoritative meaning. It safely rejects mutations, pauses affected Workers, and does not alter Tenant lifecycle.

Do not mix versions that interpret durable records differently

Waiotech must not mix versions that interpret durable records differently.

Only when the release manifest declares compatibility with the active schema, event and job envelopes, receipts, process records, and generated contracts.

Fail closed on schema incompatibility

Fail closed on schema incompatibility.

API, Worker, migration, and CLI verify compatible migration head before performing responsibilities.

Use one governed schema evolution mechanism

Waiotech must use one governed schema evolution mechanism.

Alembic or an approved equivalent provides ordered immutable migration identity, history, transactional execution where supported, and head verification.

Keep schema change explicit and separately privileged

Waiotech must keep schema change explicit and separately privileged.

Startup verifies compatibility. Only the protected migration entry point changes schema.

Preserve authoritative schema and data history

Waiotech must preserve authoritative schema and data history.

Relied-upon migration files are immutable. Recovery uses rollback of uncommitted transactions, forward correction, compatible application rollback, or verified restoration rather than destructive down migration.

Make migration semantics and operational risk explicit

Waiotech must make migration semantics and operational risk explicit.

It defines identity, owner, predecessor, affected structures, data transformation, compatibility, locking, recovery, verification, and removal eligibility for obsolete structures.

Expand-and-contract

Waiotech must use bounded compatibility for zero- or low-downtime change.

The sequence is:

```
expand compatible structure
-> deploy code supporting the compatibility contract
-> migrate or backfill data
-> switch canonical reads and writes
-> verify
-> remove obsolete compatibility
```

Only one write meaning is authoritative throughout.

Prevent dual data authority

Waiotech must prevent dual data authority.

A compatibility stage declares canonical write, deterministic projection, or transactionally constrained dual representation. Conflicting writes are prohibited.

Make large transformations resumable and isolated

Waiotech must make large transformations resumable and isolated.

They use durable engineering jobs, bounded Tenant-scoped batches, deterministic ordering, checkpoints, idempotency, safe pause, verification, and completion evidence. They do not use one unbounded transaction.

Preserve truthful history during transformation

WaioTech must preserve truthful history during transformation.

Unknown source meaning remains unresolved, uses only an explicitly authorized migration-origin value, or blocks affected action. Database nullability does not justify invention.

Remove old representation after its authority has ended and verification passes

WaioTech must remove old representation after its authority has ended and verification passes.

Only after canonical data is complete, all writers and required readers use canonical contracts, constraints and counts reconcile, compatibility use is absent, recovery supports the canonical shape, and rollback no longer depends on the obsolete representation.

Fail deployment on migration uncertainty

Fail deployment on migration uncertainty.

Deployment is not accepted. Engineering uses transaction rollback, forward correction, or restoration. Ordinary service does not continue against an unverified authoritative shape.

Application rollback

WaioTech must keep software rollback non-destructive and compatibility-based.

It activates a preceding image set without reversing schema or product history. It is permitted only when that release supports the active schema, event and job envelopes, receipts, configuration, and artifact formats.

Separate software recovery from business correction

WaioTech must separate software recovery from business correction.

It does not reverse Work Orders, Inventory Movements, Decisions, revisions, audit, Notifications, or Tenant lifecycle.

Recover durable learning progress and definitions while rebuilding disposable synthetic Tenant state

Waiotech must recover durable learning progress and definitions while rebuilding disposable synthetic Tenant state.

Learning-Tenant state is reproducible from versioned seeds and may be excluded from operational recovery objectives. Onboarding progress, mission definitions, and seed definitions remain durable and recoverable.

A backup containing learning data must still protect it as Tenant data, but restoration does not require preserving a learner's synthetic business state when deterministic reset is available.

Related documents

- [Backup and recovery](#)
- [Testing and software release](#)

Backup and recovery

A recovery set combines PostgreSQL, accepted file bytes, encryption material, manifests, and required configuration. Backup reliability is proven through isolated restore rehearsal and explicit disaster-recovery sequencing.

A backup

Waiotech must keep recovery infrastructure separate from portability.

A backup is a protected recovery copy of the installation's durable state. It is not Tenant Export, report export, Tenant Import, database migration, or product undo.

A recovery set

Back up structured truth and accepted bytes as one recoverable system.

A recovery set contains PostgreSQL backup or point-in-time chain, accepted Attachment and artifact bytes, integrity metadata, backup manifest, migration head, release identity, configuration schema, and required encryption-key references.

Keep recovery independent from transient delivery state

Waiotech must keep recovery independent from transient delivery state.

Recovery must succeed with empty Redis by reconstructing queues, leases, and caches from PostgreSQL. Redis backup may be an operational convenience only.

Capture database and file state under one consistency contract

Capture database and file state under one consistency contract.

A coordinated pause, snapshots, transaction-log boundary plus storage snapshot, or another verified protocol ensures the restored database does not reference absent accepted bytes.

Make recovery sets self-describing and verifiable

Waiotech must make recovery sets self-describing and verifiable.

It contains backup identity, source environment, creation time, database recovery position, migration head, release identity, storage snapshot, artifact list, sizes, checksums, encryption metadata, and component outcomes.

Treat recovery copies as highly sensitive assets

Waiotech must treat recovery copies as highly sensitive assets.

Backups are encrypted in transit and at rest, integrity-protected, privately stored, access-controlled, audited, and isolated from the active host and applicable administrative failure domains.

Make accepted data-loss and recovery-time tolerances explicit

Waiotech must make accepted data-loss and recovery-time tolerances explicit.

Each durable environment declares recovery point objective, recovery time objective, method, isolation, rehearsal cadence, and operational authority in an Operations Standard or ADR.

Treat restoration as the proof of recoverability

Waiotech must treat restoration as the proof of recoverability.

An isolated restore rehearsal proves that PostgreSQL, files, application release, authentication, Tenant isolation, Workers, outbox, and governed artifacts can be reconstructed. Successful backup creation alone is insufficient.

Prevent rehearsal from creating operational consequences

Waiotech must prevent rehearsal from creating operational consequences.

It uses isolated credentials, storage, database, Redis, and network. External Notification delivery, public downloads, and other non-local side effects are disabled or redirected.

Test the complete system, not only database import

Test the complete system, not only database import.

It verifies manifest integrity, schema compatibility, database records, Attachment existence and digest, Report and Tenant Export Artifacts, authentication, Tenant isolation, representative protected behavior, Worker recovery, outbox recovery, and rebuildable projections.

The disaster-recovery sequence

Restore one authoritative installation and prevent split brain.

The sequence is:

```
declare incident and fence writes  
-> select verified recovery position
```

- > establish replacement infrastructure
- > restore PostgreSQL and accepted files
- > verify integrity and schema
- > activate compatible release
- > keep external delivery controlled
- > reconcile outbox, processes, and integrations
- > run recovery checks
- > authorize service
- > preserve evidence

Make recovery-point loss and external divergence explicit

Waiotech must make recovery-point loss and external divergence explicit.

The selected PostgreSQL transaction-log position is reconciled with file state, outbox, process records, artifacts, and external acknowledgements. Commands after the recovery point are absent and require incident reconciliation rather than invention.

Recover processes from PostgreSQL truth

Waiotech must recover processes from PostgreSQL truth.

Durable state determines resume, retry, fail, cancel, or reconcile. Expired Worker lease does not prove product failure. Redis may be empty.

Do not use backup as Tenant movement or creation

Waiotech must not use backup as Tenant movement or creation.

Injecting backup data into another Tenant or active installation requires a separate Product Authority contract for identities, Users, credentials, Attachments, integrations, events, and collision handling.

Include key recoverability in disaster-recovery design

Waiotech must include key recoverability in disaster-recovery design.

Key identity, protected backup, separation from encrypted data, authorization, and rotation compatibility must preserve decryptability for the recovery lifetime. Keys are not stored inside ordinary backup payloads.

Liveness, startup, and readiness

Waiotech must keep health meanings distinct and responsibility-aware.

Liveness proves process health, startup proves initialization and compatibility, and readiness proves the process can safely perform its assigned responsibility.

Remove unsafe API instances from service

Waiotech must remove unsafe API instances from service.

Configuration, PostgreSQL connectivity, schema compatibility, security initialization, and required storage access. Redis is required only for API responsibilities that cannot safely operate without it.

Prevent unqualified Workers from claiming obligations

Waiotech must prevent unqualified Workers from claiming obligations.

Configuration, PostgreSQL, schema, Redis delivery, claim capability, file storage, and supported job schemas.

Base activation on verified responsibility

Base activation on verified responsibility.

Only after manifest, migrations, verification, readiness, smoke tests, and deployment identity all succeed with no unresolved critical compatibility defect.

Verify release and recovery as complete operational contracts

Waiotech must verify release and recovery as complete operational contracts.

Tests cover immutable server and web artifacts, signed Android artifacts, secret absence, locking, migrations, overlap compatibility, website and help activation, Android compatibility policy, readiness, smoke tests, non-destructive rollback, coordinated backup, encryption, isolated restoration, file integrity, Worker and outbox recovery, learning-progress recovery, and integration and learning-side-effect fencing.

Related documents

- [Deployment and migrations](#)
- [Generated artifacts and file recovery](#)
- [Testing and software release](#)

Observability

Observability provides protected, non-authoritative evidence through structured logs, metrics, traces, exception reporting, alerts, runbooks, and surface-specific monitoring without exposing secrets or Tenant data.

Observability

Diagnose implementation without creating another source of business truth.

Observability is protected technical evidence used to understand Waitech behavior, health, performance, and failure through structured logs, metrics, traces, exceptions, health, deployment identity, and alerts. It does not replace product evidence or audit.

Operational security

Waitech must protect Product Authority guarantees throughout operation.

Operational security protects build, deployment, runtime, support, monitoring, backup, recovery, and incident handling through least privilege, runtime identity, secret management, network boundaries, vulnerability management, containment, and evidence.

Keep observability non-authoritative while preserving mandatory audit

Waitech must keep observability non-authoritative while preserving mandatory audit.

External logging, metrics, tracing, or exception-reporting failure does not invalidate a safe command. Required domain evidence and security audit remain durable in PostgreSQL and do not depend on telemetry delivery.

A structured log

Emit machine-readable bounded diagnostic records.

A structured log uses a stable event name and typed bounded fields such as timestamp, severity, service, runtime role, release identity, safe message, request, correlation, operation, scope, route or work type, outcome, duration, and safe exception class.

Prevent observability from becoming a sensitive-data replica

Waiotech must prevent observability from becoming a sensitive-data replica.

Passwords, hashes, cookies, tokens, reset credentials, keys, authorization headers, delegated URLs, database credentials, unrestricted request or response bodies, Attachment content, sensitive report or export content, scanner detail, and unnecessary personal information are prohibited.

Stop sensitive data at the source

Stop sensitive data at the source.

Redaction occurs before emission through typed secret wrappers, header and query filtering, safe exception serialization, payload suppression, size limits, and automated tests. Post-ingestion cleanup is not the primary control.

Treat telemetry leakage as credential exposure

Waiotech must treat telemetry leakage as credential exposure.

The condition is a security incident: access is restricted, the secret is revoked or rotated, exposed records are handled under the incident contract, the logging path is corrected, and evidence is preserved.

Preserve diagnosis without creating enumeration and cardinality risk

Waiotech must preserve diagnosis without creating enumeration and cardinality risk.

A safe stable Tenant reference may appear when necessary. Names and business content are minimized. Tenant identifiers are not general metric labels.

Make one flow reconstructable across synchronous and asynchronous boundaries

Waiotech must make one flow reconstructable across synchronous and asynchronous boundaries.

Requests, operations, receipts, events, durable work, attempts, external messages, audit, logs, and traces preserve applicable request, correlation, operation, causation, and trace identities.

Protect metric systems from data leakage and cardinality failure

Waiotech must protect metric systems from data leakage and cardinality failure.

Labels are bounded, low-cardinality, stable, non-secret, and non-personal. Record IDs, User IDs, Tenant IDs, filenames, arbitrary URLs, exception messages, and raw error text are prohibited.

Measure traffic, latency, errors, saturation, and software identity

Measure traffic, latency, errors, saturation, and software identity.

Metrics cover request rate, latency, errors, active work, resource saturation, database pools and locks, Redis latency, storage latency, dependency latency, readiness, restart, and release identity.

Detect stalled, duplicated, and failing execution

Detect stalled, duplicated, and failing execution.

Metrics cover command attempts, outcomes, replays, conflicts, transaction duration, durable work, queue delay, claims, leases, retries, dead letters, execution duration, and payload incompatibility.

Observe obligations without replacing governed reporting

Observe obligations without replacing governed reporting.

Metrics cover outbox age, Report Runs, artifact failures, Notification delivery, integration acknowledgements and reconciliation, Attachment processing and integrity, backup, restoration, and migration health. They exclude product content and recipient identity.

Distributed tracing used for

Diagnose cross-component latency without collecting unrestricted payloads.

Tracing follows technical execution across API, Application, PostgreSQL, outbox, Worker, storage, and approved external providers. Sampling is controlled and biased toward errors or latency where appropriate.

Apply the same data-minimization rules to traces and logs

Waiotech must apply the same data-minimization rules to traces and logs.

Bodies, secret values, SQL parameters, file content, sensitive report parameters, high-cardinality span names, and unnecessary personal identity.

Keep business rejection separate from technical defects

Waiotech must keep business rejection separate from technical defects.

Unexpected technical failures belong in exception aggregation. Expected validation, Permission denial, life-cycle rejection, concurrency conflict, and idempotency conflict use typed results and metrics rather than exceptions.

An alert

Alert on actionable threats rather than raw noise.

An alert is an actionable signal identifying affected responsibility, severity, condition, release, environment, safe diagnostics, runbook, and escalation. Individual expected errors do not each create an alert.

Alert on risks to truth, security, and recoverability

Alert on risks to truth, security, and recoverability.

Alerting covers Tenant isolation, authentication and authorization failure, integrity mismatch, durable obligation risk, outbox and Worker recovery, database and storage capacity, backup and restore failure, integration uncertainty, secret exposure, deployment incompatibility, and monitoring-path failure.

Connect detection to safe response

Connect detection to safe response.

Recurring actionable alerts reference protected runbooks defining verification, containment, recovery, escalation, evidence, and prohibited actions.

Monitor public reliability and discoverability without expanding data collection

Monitor public reliability and discoverability without expanding data collection.

Monitoring covers availability, response status, redirect failure, certificate validity, deployment identity, page-template performance, asset errors, sitemap and robots availability, public-form failure, and privacy-safe aggregate traffic. It excludes cross-surface identity tracking and protected Tenant information.

Make field synchronization diagnosable without copying field evidence into telemetry

Waiotech must make field synchronization diagnosable without copying field evidence into telemetry.

Protected telemetry covers application version, installation-safe identity, authentication failure class, Work Package issuance and verification, local journal count, synchronization delay, accepted, rejected, and reconciliation-required operations, file-transfer failure, revocation delivery, crash class, and update-required results.

Telemetry excludes Work Order narrative, measurements, photographs, file content, credentials, and unrestricted local records.

Alert on mobile conditions that threaten authority, evidence, or recoverability

Alert on mobile conditions that threaten authority, evidence, or recoverability.

Alerts cover abnormal package-signing failure, package-verification failure, widespread synchronization delay, elevated reconciliation or rejection, loss of mobile authentication, incompatible app-version concentration, push-registration failure affecting required field communication, and evidence-transfer backlog threatening package or local-storage limits.

Detect any failure that could make learning unsafe, misleading, or operationally visible

Detect any failure that could make learning unsafe, misleading, or operationally visible.

Monitoring covers private test-Tenant provisioning, seed verification, mission completion evaluation, reset, side-effect-fence violations, real-provider delivery attempts, cross-purpose query attempts, documentation compatibility, and onboarding-progress persistence.

Preserve operational metric meaning by excluding synthetic learning activity

Waiotech must preserve operational metric meaning by excluding synthetic learning activity.

Learning-purpose activity is separated by default. A protected engineering dashboard may inspect learning metrics explicitly, but operational service and business metrics do not silently include synthetic activity.

An incident

Waiotech must use one accountable incident process for material operational and security failures.

An incident threatens or causes unacceptable availability, integrity, confidentiality, Tenant isolation, authority enforcement, recoverability, or external consistency. It preserves identity, timeline, affected scope, containment, evidence, recovery, reconciliation, and closure basis.

Containment

Waiotech must use explicit reversible controls to contain risk.

Containment limits harm through readiness removal, credential or session revocation, write fencing, Worker pause, external-delivery pause, quarantine, endpoint restriction, or maintenance mode. It does not rewrite product history.

Preserve evidence and product correction semantics during incidents

Waiotech must preserve evidence and product correction semantics during incidents.

Only through a protected correction or recovery procedure with authorization, recovery point, exact statements, audit, verification, and reconciliation. Urgency does not create a hidden authority path.

Treat telemetry platforms as protected data systems

Waiotech must treat telemetry platforms as protected data systems.

Individually authenticated engineering, security, audit, or support operators receive least privilege, environment scope, Tenant-sensitive restrictions, strong authentication, expiry, audit, and revocation. Shared accounts are prohibited.

Keep infrastructure diagnosis separate from product authority

Waiotech must keep infrastructure diagnosis separate from product authority.

Diagnostic access does not authorize Tenant actions, file retrieval through product contracts, Decisions, IAM changes, or ordinary commands.

Bound Tenant diagnosis by purpose and least access

Bound Tenant diagnosis by purpose and least access.

An authorized operator may filter by safe Tenant reference for an explicit support or incident purpose, with unrelated data excluded, access audited, and exported diagnostics minimized.

Keep operational telemetry retention separate from product retention

Waiotech must keep operational telemetry retention separate from product retention.

An Operations Standard defines retention for logs, metrics, traces, alerts, and incident evidence based on diagnostic need, security, privacy, cost, and obligations. It does not authorize deletion of product audit or governed evidence.

Related documents

- [Operational security](#)
- [Background work](#)
- [Audit, events, and transactional outbox](#)

Operational security

Runtime identities, database roles, networks, transport, browser controls, secrets, dependencies, time, shell access, break-glass actions, monitoring, denial-of-service controls, environment separation, drift detection, and user-facing failures are explicitly governed.

Separate operational authority by responsibility

WaioTech must separate operational authority by responsibility.

API, Worker, Public Website delivery, migration, backup, restore, deployment, monitoring collection, file processing, onboarding control, learning-data-plane services, and other separated responsibilities use distinct technical identities and least privileges. Android application installations use User-bound mobile identity rather than infrastructure credentials.

Prohibit schema and superuser authority in application processes

WaioTech must prohibit schema and superuser authority in application processes.

API and Worker use role-specific application privileges. Migration and recovery privileges are isolated and unavailable to ordinary runtime.

Keep infrastructure outside the public trust boundary

WaioTech must keep infrastructure outside the public trust boundary.

Ingress exposes only Public Website, approved Dashboard and Platform Admin delivery, and approved API surfaces. Android Work App reaches only the approved API and public distribution endpoints. The learning data plane uses separate ingress and cannot reach operational product APIs or data stores. PostgreSQL, Redis, private storage, monitoring, backup, and administrative interfaces remain private. Network policy is deny-by-default and permits only required paths.

Encrypt sensitive transport and authenticate endpoints

Encrypt sensitive transport and authenticate endpoints.

Authenticated encryption protects Public Website, browser applications, Android API traffic, learning-session exchange, integrations, telemetry forwarding, backup transfer, secret retrieval, and administration across trust boundaries.

Use browser platform controls as layered defense

Waiotech must use browser platform controls as layered defense.

Serving components apply content security, frame restrictions, MIME-sniff prevention, referrer policy, secure transport, permissions policy, cache controls, and cross-origin protections as applicable.

Disable unsafe debug behavior in protected environments

Disable unsafe debug behavior in protected environments.

Interactive consoles, stack traces, source, configuration, environment variables, SQL, and internal routes remain unavailable to untrusted users.

Separate secret lifecycle from software artifacts

Waiotech must separate secret lifecycle from software artifacts.

Secrets are injected or retrieved through protected channels, scoped to one identity, excluded from command-line exposure and logs, and rotatable without rebuilding images. Rotation defines overlap, activation, revocation, reload, verification, and audit.

Treat third-party code as part of the security boundary

Waiotech must treat third-party code as part of the security boundary.

Server, browser, Public Website, Android, build, and operational dependencies use manifests, lock files, verified sources, scanning, software bills of materials, reproducible builds, licence controls where applicable, update testing, and removal of unused components.

Require explicit time-bounded risk acceptance

Waiotech must require explicit time-bounded risk acceptance.

Only through a bounded deviation recording vulnerability, exposure, compensating controls, owner, expiry, and verification. Product Authority guarantees cannot be waived.

Preserve ordering, expiry, and evidence time integrity

Waiotech must preserve ordering, expiry, and evidence time integrity.

Application hosts, PostgreSQL, Redis, logging, and monitoring use controlled synchronization and drift monitoring. Security expiry, effective authority, leases, and recorded time use trusted backend clocks.

Prevent operational tools from creating hidden product paths

Waiotech must prevent operational tools from creating hidden product paths.

Controlled CLI tools use attributable identity, explicit environment and scope, Application contracts where possible, reasons, audit, and secret redaction. Shell access is exceptional and cannot become ordinary product administration.

Break-glass access

Waiotech must make emergency authority explicit and accountable.

Break-glass is exceptional time-bounded infrastructure authority used when ordinary recovery paths cannot protect the system. It requires named identity, strong authentication, purpose, scope, approval where feasible, complete audit, credential revocation, and review.

Detect attempts to bypass identity, isolation, evidence, and recovery controls

Detect attempts to bypass identity, isolation, evidence, and recovery controls.

Monitoring detects authentication abuse, revoked credential use, suspicious support activity, cross-Tenant attempts, denial spikes, download anomalies, secret access anomalies, superuser use, unauthorized migration, integrity mismatch, signature failure, and backup or telemetry access anomalies.

Do not create hidden automated authority

Waiotech must not create hidden automated authority.

Only through an explicit security-response contract defining trigger, false-positive handling, scope, duration, restoration authority, and audit. Otherwise it alerts authorized responders.

Fail safely under resource pressure

Fail safely under resource pressure.

Ingress rate limits, authentication limits, request and upload limits, query and pagination bounds, Worker concurrency, Tenant fairness, timeouts, and safe circuit breakers bound resource consumption. Overload never becomes silent acceptance.

Prevent one environment from affecting another

Waiotech must prevent one environment from affecting another.

Development, test, rehearsal, and protected operation use separate credentials, databases, Redis, storage, external destinations, secrets, deployment identities, and observability access.

Detect unreviewed operational change

Detect unreviewed operational change.

Active service configuration, secret references, network policy, database roles, storage policy, monitoring rules, release identity, and migration head are compared with approved configuration sources.

Separate supportable user information from infrastructure disclosure

Waiotech must separate supportable user information from infrastructure disclosure.

Users receive stable safe error code, message, correlation reference, operation identity where applicable, and retry guidance. Technical detail remains in protected diagnostics.

Make operational controls executable and rehearsed

Waiotech must make operational controls executable and rehearsed.

Tests cover log structure, redaction, correlation, bounded metrics, tracing, alerts, monitoring failure, runtime privileges, private networks, TLS, security headers, debug absence, secret rotation, break-glass audit, environment separation, and incident exercises.

Related documents

- [Authentication, sessions, and secrets](#)
- [Observability](#)
- [Testing and software release](#)

Testing and software release

Testing verifies Product and Engineering Authority across Domain, Application, PostgreSQL, API, generated contracts, architecture, lifecycle, isolation, idempotency, concurrency, background work, integrations, files, applications, security, performance, recovery, and release activation.

The primary testing principle

Test contracts at their owning layer and verify their composition.

Each guarantee is tested at the lowest layer that can prove it completely, and cross-layer composition is tested at the boundaries where several guarantees interact. Tests verify authority and do not create product meaning.

Maintain complementary tests across product, implementation, operation, and recovery

WaioTech must maintain complementary tests across product, implementation, operation, and recovery.

The required layers are static validation, Domain, Application, PostgreSQL, API, generated contract, architecture, Worker and outbox, migration, security, frontend unit and component, frontend integration, accessibility, end-to-end, performance, deployment, backup and recovery, and release-artifact verification.

Align test ownership with contract ownership

Align test ownership with contract ownership.

The module or engineering concern owning the contract owns its assertions. Shared infrastructure may provide PostgreSQL, Redis, storage, clock, identity, API, browser, and assertion helpers but cannot hide domain expectations.

Test actual product behavior without mocking the Domain

Test actual product behavior without mocking the Domain.

Domain tests use real aggregates, entities, value objects, and services without framework infrastructure. They verify lifecycle, invariants, Decisions, calculations, evidence, correction, and code interpretation.

Test use-case behavior at the Application boundary

Test use-case behavior at the Application boundary.

Application tests verify commands and queries, Tenant context, authorization, Domain invocation, transaction orchestration, cross-module consequences, audit, receipts, outbox, results, and errors.

Use the production database engine for persistence guarantees

WaioTech must use the production database engine for persistence guarantees.

They use PostgreSQL whenever correctness depends on transactions, constraints, locks, isolation, receipts, outbox, Tenant-aware foreign keys, repositories, or migration-created structures.

Test authoritative persistence on PostgreSQL

Test authoritative persistence on PostgreSQL.

SQLite cannot prove PostgreSQL types, constraints, locking, transactions, concurrency, indexing, or SQL semantics.

Test the deployable schema

Test the deployable schema.

Test schema comes from the authoritative migration chain or an automatically verified snapshot proven equivalent to it. ORM table creation alone is insufficient.

Test the public contract through the real API boundary

Test the public contract through the real API boundary.

API tests exercise actual authentication integration, Tenant context, authorization, routes, schemas, unknown-field rejection, idempotency, concurrency, status, errors, headers, information disclosure, and OpenAPI compatibility.

Treat negative behavior as a first-class contract

WaioTech must treat negative behavior as a first-class contract.

Tests cover absent or invalid authentication, missing or invalid Tenant, cross-Tenant identity, denied Permission, invalid input, unknown field, retired code, stale version, idempotency conflict, unavailable action, unsafe file access, malformed cursor, unsupported query, rate limit, and safe unexpected failure.

Fail the build on generated drift

Fail the build on generated drift.

They compare OpenAPI, generated clients, catalogues, Permissions, actions, events, localization, documentation, and database reference representations with their canonical sources.

Make architecture executable

Waiotech must make architecture executable.

They enforce layer dependencies, module privacy, API and Worker invocation of Application contracts, generated client use, application separation, schema-change boundaries, absence of generic engines, catalogue governance, storage ports, and acyclic module dependencies.

Keep lifecycle behavior fully enumerated

Waiotech must keep lifecycle behavior fully enumerated.

Each transition has successful execution, incompatible-state rejection, missing-authority rejection, missing-requirement rejection, concurrency behavior, audit, and event verification where applicable.

Verify one authoritative action-specific Readiness evaluation

Waiotech must verify one authoritative action-specific Readiness evaluation.

Tests cover authoritative inputs, every blocker class, expiry, policy, evidence availability, reevaluation after fact changes, and distinction from historical lifecycle authorization.

Prove isolation even with globally unique identifiers

Prove isolation even with globally unique identifiers.

Every Tenant-owned resource family is tested against valid identifiers belonging to another Tenant for reads, writes, links, receipts, cursors, files, audit, selectors, processes, reports, and errors.

Verify platform authority as a separate bounded path

Waiotech must verify platform authority as a separate bounded path.

Tests prove dedicated platform scope, exact platform Permissions, bounded Tenant inspection, separation of duties, real-Actor attribution, no Membership creation, no Tenant impersonation, no ordinary Tenant action, and no infrastructure-access grant.

Test system actions independently from human IAM

Product-defined internal system actions are tested against their exact Worker job, initiating evidence, Tenant scope, idempotency, audit, retry, failure, and prohibition on reuse as generic automation authority.

Prove one result for one logical attempt

Prove one result for one logical attempt.

Tests cover first execution, equivalent replay, conflicting reuse, concurrent duplicate, response loss after commit, failure before commit, abandoned claim, retryable failure, rejected replay, uncertainty, and receipt retention boundary.

Test real races rather than sequential approximations

Test real races rather than sequential approximations.

Tests use actual parallel PostgreSQL transactions for stale versions, simultaneous updates, unique races, row locks, advisory locks, deadlocks, serialization retry, process claims, publication, stock posting, and Plan publication drift.

Make time deterministic and explicit

WaioTech must make time deterministic and explicit.

Time-dependent behavior uses an injected trusted clock or controlled database boundary. Tests cover effective periods, Tenant timezone, local schedules, daylight-saving transitions where applicable, leases, sessions, report cutoff, and occurred versus recorded time.

Prevent machine-speed-dependent tests

WaioTech must prevent machine-speed-dependent tests.

Use barriers, condition polling, controlled clock advancement, transaction observation, or explicit event synchronization.

Keep test determinism without weakening production entropy

WaioTech must keep test determinism without weakening production entropy.

Production uses secure randomness. Tests inject deterministic sources where repeatability is needed and use property or statistical verification where distribution matters.

Test interruption and recovery as primary behavior

Test interruption and recovery as primary behavior.

Tests cover queue loss, duplicate delivery, Worker death before and after commit, lease expiry and renewal, retries, dead letters, replay, Tenant context, payload schema compatibility, source/outbox atomicity, consumer deduplication, and ordering where required.

Test files as hostile input and immutable evidence

Test files as hostile input and immutable evidence.

Tests cover upload claims, hostile filenames, streaming, limits, type spoofing, digest, scanner failure, quarantine, governing access, correction, unavailable bytes, storage conformance, Report Artifact, Tenant Export Artifact, partial-generation rejection, and recovery.

Test migration as transformation of authoritative state

Test migration as transformation of authoritative state.

Tests cover empty installation, upgrade from every supported schema state, populated data, mapping, Tenant constraints, lifecycle, revisions, catalogues, receipts, outbox, files, batching, failure resume, compatibility overlap, and post-migration product behavior.

Verify every user-facing surface without moving product authority into a client

Waiotech must verify every user-facing surface without moving product authority into a client.

Dashboard and Platform Admin tests cover formatting and view models, generated clients, session and Tenant context, cache invalidation, forms, actions, concurrency, idempotency recovery, files, reports, localization, accessibility, and protected browser journeys.

Public Website tests cover English and Persian routes, directionality, metadata, canonical and locale links, structured data, sitemap, robots directives, redirects, not-found behavior, content parity, accessibility, security headers, and performance budgets.

Android tests cover User and installation binding, authentication, encrypted storage, Room migration, disconnected Work Request drafts, Work Package authenticity and expiry, journal sequencing and correction, scanning, file capture, bounded material use, synchronization retry, duplicate submission, conflicts, revocation, app upgrade, accessibility, and unsynchronized-evidence recovery.

Help and onboarding tests cover content visibility, bilingual parity, contextual links, mission schemas, authoritative completion evaluation, private test-Tenant isolation, side-effect fencing, progress separation, and deterministic reset.

Treat public discoverability, accessibility, privacy, and performance as release contracts

Waiotech must treat public discoverability, accessibility, privacy, and performance as release contracts.

The release gate verifies Astro static generation, bilingual content completeness, no accidental private content, canonical and `hreflang` correctness, sitemap and robots output, structured-data validity, redirects, accessibility, security headers, media optimization, JavaScript and asset budgets, and measured page performance on representative templates.

Verify offline field execution under conflict, interruption, and hostile input

Waiotech must verify offline field execution under conflict, interruption, and hostile input.

Tests cover loss before and after local commit, duplicate synchronization, out-of-order delivery, expired and revoked package, altered package, wrong User or installation, wrong Tenant, clock anomaly, Work Order cancellation, release withdrawal, Procedure change, another Actor completion, material conflict, file-transfer interruption, app termination, device restart, app upgrade, token revocation, and server-version incompatibility.

Prove learning isolation through negative and side-effect tests

Prove learning isolation through negative and side-effect tests.

Tests prove that guided missions cannot access or affect operational Tenant data, unrelated Users, real email or push, billing, operational Reports, search, exports, platform queues, or external files. Reset is tested against wrong-Tenant identifiers and concurrent mission activity.

Combine automated and manual accessibility verification

Combine automated and manual accessibility verification.

Keyboard completion, focus order, labels, announcements, cognitive clarity, and screen-reader workflows require human interaction testing.

Test security across application and operational boundaries

Test security across application and operational boundaries.

Tests cover authentication, fixation, CSRF, CORS, cookies, authorization, Tenant isolation, support, credentials, enumeration, rate limits, redaction, file attacks, injection, XSS, redirects, headers, signatures, dependencies, runtime privileges, backup, and observability access.

Apply expert review to security boundaries

Waiotech must apply expert review to security boundaries.

It is required for changes affecting authentication, authorization, Tenant isolation, cryptography, credentials, files, integrations, support, audit, backup, recovery, privileged operation, or network exposure.

Make declared performance limits executable

Waiotech must make declared performance limits executable.

Performance tests cover declared latency, transaction duration, query behavior, pools, Worker throughput, outbox recovery, selector resolution, Plan publication, reports, exports, files, Notifications, integrations, and frontend responsiveness.

Keep environment-specific numbers scoped while requiring measurable guarantees

Waiotech must keep environment-specific numbers scoped while requiring measurable guarantees.

Numerical objectives belong in Experience Contracts, Operations Standards, or scoped ADRs. Mandatory limits identified there become release checks.

Verify truthful behavior under infrastructure failure

Waiotech must verify truthful behavior under infrastructure failure.

Isolated resilience tests cover PostgreSQL, Redis, Worker, storage, scanner, external timeout, partial acknowledgement, network interruption, capacity pressure, lease expiry, telemetry outage, and backup-component failure.

Do not accept backup-job success as recovery proof

Waiotech must not accept backup-job success as recovery proof.

A complete isolated restore rehearsal verifies database, files, integrity, schema, authentication, Tenant isolation, Workers, outbox, processes, integrations, projections, and service activation.

Keep tests isolated from protected operational data

Waiotech must keep tests isolated from protected operational data.

Test data is synthetic, purpose-specific, deterministic where required, Tenant-separated, secret-free, and safe for logs and screenshots. Operational data requires a protected handling procedure.

Make tests independent and reproducible

Waiotech must make tests independent and reproducible.

Each test establishes prerequisites. Parallel execution isolates databases or schemas, Redis keys, storage objects, stubs, ports, browser sessions, and Tenant identities.

A test double

Mock mechanisms, not authoritative behavior.

A fake, stub, spy, mock, or emulator replaces an outbound mechanism. It must preserve the behavior relevant to the contract. Product rules are not mocked away.

Prevent easier test semantics than production

WaioTech must prevent easier test semantics than production.

When Application tests rely on a test double for a defined port, the same contract test suite verifies the test double and the production adapter.

A flaky test

WaioTech must treat flakiness as a release-quality defect.

A flaky test passes or fails without a relevant contract or implementation change. It is a test defect or evidence of a race. Rerunning until pass does not erase the failure.

Prohibit suppression of required release tests

WaioTech must not suppress a required release test.

A failing or unavailable required test blocks the release until the test environment or implementation is corrected. A test may be replaced only in the same change by stronger executable verification of the same guarantee.

Combine quantitative coverage with explicit critical-path tests

Combine quantitative coverage with explicit critical-path tests.

Coverage detects untested code but does not prove correctness. Repository thresholds may apply to statements, branches, critical packages, and changed code. Mandatory scenarios remain required regardless of percentage.

Require direct evidence for high-impact guarantees

WaioTech must require direct evidence for high-impact guarantees.

They include Tenant isolation, IAM, account recovery, Work Request acceptance, Work Order lifecycle, emergency execution, Plan publication and drift, Scheduled Work duplicate prevention, Inventory posting, reservations, Stocktaking, Access Profile publication, idempotency, transactions, outbox, Worker recovery, Attachments, reports, Tenant Export, migration, and restoration.

Reject detectable defects before runtime tests

WaioTech must reject detectable defects before runtime tests.

Formatting, linting, strict backend and frontend typing, import boundaries, unsafe pattern checks, generated drift, secrets, dependencies, migration graph, API schemas, localization, and builds are mandatory.

Treat code, schema, contracts, security, and docs as one release surface

Waiotech must treat code, schema, contracts, security, and docs as one release surface.

CI verifies static checks, all applicable test layers, PostgreSQL integration, generated artifacts, migrations, security, accessibility automation, frontend and container builds, dependency provenance, documentation, and release-manifest consistency.

Optimize feedback without skipping relevant verification

Optimize feedback without skipping relevant verification.

Only when dependency analysis proves the omitted tests cannot be affected. Protected release branches satisfy the complete gate for the change class.

Require human review and executable verification

Waiotech must require human review and executable verification.

Review evaluates authority alignment, design, security, migration, test sufficiency, and operations. Tests provide repeatable evidence.

A release candidate

Waiotech must evaluate exact server, web, content, and Android artifacts rather than an unfixed branch.

It is one immutable coordinated server-and-web artifact set with image digests, Public Website output, migration set, manifest, OpenAPI, generated clients and registries, documentation and mission identities, software bill of materials, test evidence, and security evidence. Each Android release candidate is one signed immutable application artifact with its own contract-version manifest and evidence.

The software-release gate

Waiotech must not deploy an artifact set whose contracts or recovery path are unverified.

The gate requires Product and Engineering Authority alignment, mandatory CI, migration rehearsal, generated parity, security verification and review, container verification, deployment rehearsal, critical end-to-end journeys, affected performance checks, recovery verification where required, rollback assessment, manifest verification, and attributable approval.

Preserve artifact continuity from build to operation

WaioTech must preserve artifact continuity from build to operation.

Reproducible build comparison is permitted, but deployment promotes the exact artifact that passed the gate.

Reverify recovery when its contract changes

Reverify recovery when its contract changes.

Changes to schema, storage adapter, backup format, encryption, Attachment representation, artifact format, restoration tooling, deployment topology, or durable event and job storage require recovery verification.

Prohibit release-gate bypass

WaioTech must not bypass the release gate.

A missing or failed required check blocks activation. Emergency urgency may reduce unrelated scope, but it does not convert an unverified artifact into an accepted release.

An emergency release

Accelerate response without removing release accountability.

It is an accelerated immutable release used to contain or correct an active incident. It still requires attributable source, focused review, applicable tests, security checks, migration and recovery assessment, release evidence, and activation verification.

Make release acceptance reconstructable

WaioTech must make release acceptance reconstructable.

Evidence includes release and source identity, artifact digests, migration head, manifest, CI results, tests, migration, security, performance, restoration, approvals, rollback evidence, and activation result.

Do not operate an unverified deployment

WaioTech must not operate an unverified deployment.

The deployment is not accepted. Unsafe services leave readiness and Engineering uses a verified non-destructive rollback, roll-forward correction, or recovery under the deployment standard.

Correct existing effects as well as recurrence risk

Correct existing effects as well as recurrence risk.

Engineering separately performs implementation correction, data correction, audit, external reconciliation, and regression verification. A test addition alone is insufficient.

Related documents

- [Deployment and migrations](#)
- [Backup and recovery](#)

30-EXPERIENCE

Experience

How people encounter Waiotech as a professional plant operating workspace.

10 source documents

Experience

Experience Authority defines how people encounter Waiotech product meaning across public, browser, field, and platform surfaces without creating alternate business authority.

Contents

- [Product surfaces](#)
- [Public Website](#)
- [Tenant Dashboard and Platform Admin](#)
- [Android Work App](#)
- [Offline field packages](#)
- [Documentation and Help Center](#)
- [Learning environment](#)
- [Tenant Dashboard experience](#)
- [Dashboard design conventions](#)

Product surfaces

Waiotech provides a Public Website, Tenant Dashboard, Platform Admin, Android Work App, and Documentation and Help Center. Each surface has a distinct audience, trust boundary, and responsibility while exposing one Product Authority.

One product, distinct interaction surfaces

Waiotech must not reproduce the same application at different screen sizes or create alternate authority paths for different surfaces.

The canonical user-facing surfaces are:

- **Public Website:** public understanding, trust, discovery, and product entry;
- **Tenant Dashboard:** the primary professional operating workspace for the plant;
- **Android Work App:** the governed field workspace for Process and Maintenance work;
- **Platform Admin:** protected platform provisioning, support, preservation, and recovery;
- **Documentation and Help Center:** contextual guidance embedded in authenticated work, with selected public and field guidance where appropriate.

A private learning Tenant may expose real product behavior with synthetic data under the same Product Authority. Learning purpose changes side-effect and reset policy, not domain meaning.

Tenant Dashboard

Tenant Dashboard owns the complete browser experience for ordinary plant operation and configuration, including:

- Plant exploration and configuration;
- Process Operations;
- Maintenance planning, control, and analysis;
- Inventory operation and administration;
- Reliability work and analysis;
- Reports and operational history;
- Users, Teams, Permissions, policy, Data Sources, Integration Principals, and other Tenant-owned settings;
- Documentation and Help Center.

The Dashboard must feel like an operating workspace, not a collection of database administration screens.

Android Work App

Android Work App owns focused field interaction where mobility, scanning, evidence capture, and intermittent connectivity matter.

It supports:

- My Work across authorized Process and Maintenance obligations;
- human Process Readings and Operational Observations;
- bounded Process Condition and Operational Action interaction;
- Work Order execution;
- Findings and field evidence;
- contextual Inventory interactions;
- Plant identification and context;
- governed offline authority through explicit field packages.

Broad configuration, publication, planning, access administration, Data Source configuration, Integration Principal configuration, and unrestricted reporting remain Dashboard responsibilities.

Platform Admin

Platform Admin owns only platform authority.

It may provision and suspend Tenants, perform governed support and preservation actions, inspect platform-owned operational state, and invoke protected recovery contracts. It must not become a privileged plant-operations interface and must not provide a generic bypass around Tenant authority.

Public Website

Public Website explains Waiotech truthfully in English and Persian and provides public product, trust, legal, support, and product-entry content. It performs no authenticated Tenant operation.

Documentation and Help Center

Documentation is part of the product experience. Guidance should be reachable from the context where a User needs it and should explain product meaning, decisions, and safe workflows rather than mirror code or database structure.

Surface boundary rules

Across all surfaces:

- the Server remains authoritative for product state, permissions, lifecycle, validation, and available actions;
- surface-specific convenience must not create alternate domain meaning;
- a User never gains authority because a control is visible;

- field or offline authority is explicit, bounded, attributable, and revocable;
- public information cannot expose Tenant data or protected product authority;
- Platform Admin cannot impersonate an ordinary Tenant Actor;
- English and Persian experiences preserve equivalent meaning.

Related documents

- [Public Website](#)
- [Tenant Dashboard and Platform Admin](#)
- [Android Work App](#)
- [Offline field packages](#)
- [Tenant Dashboard experience](#)

Public Website

The Public Website communicates Waiotech publicly in English and Persian, supports discoverability, and contains only governed public-safe content. It is not an authenticated product surface or documentation authority.

The Public Website

Waiotech must use the Public Website for public understanding and access, not Tenant operation.

The Public Website is Waiotech's public information, trust, discovery, support-entry, and product-entry surface.

It explains Waiotech to prospective customers, partners, applicants, procurement reviewers, security reviewers, and users seeking public information.

Maintain English and Persian as equal public content editions

Waiotech must maintain English and Persian as equal public content editions.

The Public Website provides complete English and Persian editions identified by `en` and `fa`.

Persian is the default public edition at `/`. `/fa/` remains a stable Persian alias, while English is served under `/en/`. The root path renders Persian content directly rather than presenting a language-choice page.

Both editions must communicate equivalent product meaning, security claims, legal information, support information, and calls to action. One language must not contain materially stronger promises, missing limitations, or incomplete safety information.

Publish truthful public content that explains the governed Waiotech product

Waiotech must publish truthful public content that explains the governed Waiotech product.

Public Website content may include:

- product purpose and capabilities;
- Plant Model, Process Operations, Maintenance, Inventory, and Reliability scope;
- industry and use-case explanations;
- security, privacy, reliability, accessibility, and recovery information;
- public contact, sales, support, legal, and policy pages;
- public-safe articles and guides;
- Android Work App distribution links;

- Tenant Dashboard sign-in entry;
- a link to a separately governed service-status capability.

Every product statement must remain consistent with Product Authority.

Keep the Public Website outside authenticated product operation and authority

Waiotech must keep the Public Website outside authenticated product operation and authority.

The Public Website must not:

- expose Tenant records;
- perform Work Order, Inventory, IAM, policy, publication, or governance actions;
- expose Platform Admin;
- grant product authority;
- implement an alternate product workflow;
- present marketing language as normative authority;
- publish protected documentation or Tenant evidence;
- make unsupported capability, security, performance, or compliance claims.

Treat the Public Website as explanatory content derived from authority

Waiotech must treat the Public Website as explanatory content derived from authority.

Public pages may explain Waiotech and may present public-safe guidance, but Product Authority and Engineering Authority remain controlling. Public content must be corrected when it diverges from authority.

Guarantee high-quality public discoverability controls without promising external ranking

Guarantee high-quality public discoverability controls without promising external ranking.

Public content must be technically discoverable, accessible, stable, and understandable in both supported languages.

Waiotech must provide meaningful page identity, navigable information architecture, durable public URLs, appropriate public metadata, and content that matches the subject represented by each page.

Search-engine placement is not a product guarantee because ranking is controlled by external systems.

Related documents

- [Product surfaces](#)
- [Public Website engineering](#)

- [Documentation and Help Center](#)

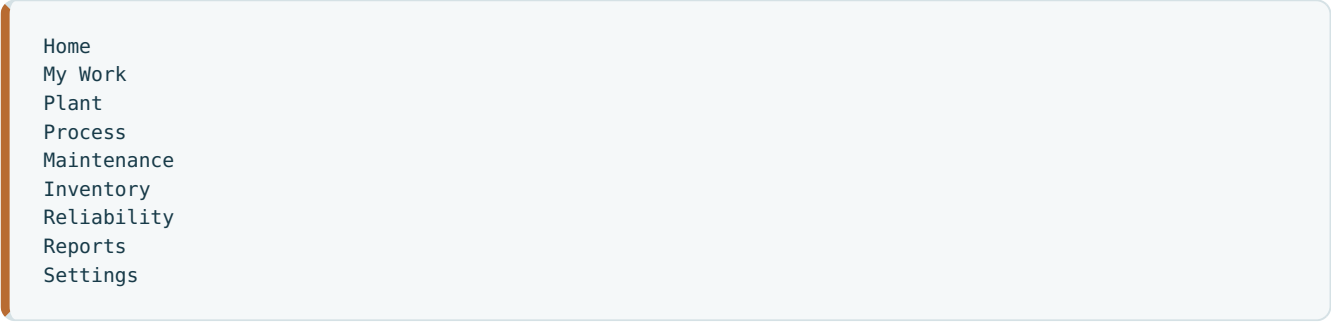
Tenant Dashboard and Platform Admin

Tenant Dashboard is the professional operating workspace of one Tenant plant. Platform Admin is a separate platform-control surface and must not become an alternate plant-operations interface.

Tenant Dashboard

Tenant Dashboard serves plant managers, operators, engineers, maintenance planners, supervisors, technicians using browser workflows, warehouse personnel, reliability users, and Tenant administrators according to Permission.

Its primary navigation is:



- Home
- My Work
- Plant
- Process
- Maintenance
- Inventory
- Reliability
- Reports
- Settings

Permission restrictions remove inaccessible destinations without changing the conceptual information architecture for authorized users.

Home

Home provides a concise operational orientation. It prioritizes what needs attention, why it matters, current blockers, responsibilities, overdue obligations, important Process Conditions, maintenance readiness, Inventory constraints, failures, and significant recent activity.

Home is not a configurable wall of charts.

My Work

My Work is the cross-domain personal operational queue. It may include Process Conditions, Maintenance actions, Inventory obligations, Reliability investigation or review, and other records where the current principal has explicit responsibility or an executable decision-bearing action.

My Work preserves record identity and owning domain. It does not merge different domain records into a generic Task aggregate.

Plant

Plant is the shared navigation and understanding surface for:

- Functional Locations;
- Assets and installation;
- Process Units and Streams;
- Measurement Points;
- connected operational history and relationships.

Plant should feel like navigating the plant, not administering database tables.

Process

Process is the operational workspace for:

- human Process Reading entry;
- Operational Observations;
- open Process Conditions;
- Operational Actions and outcomes;
- focused Measurement Point trends;
- process-to-Maintenance coordination;
- operational continuity and handover views.

Machine-submitted Readings enrich the same Process experience rather than creating a separate machine-data product.

Maintenance

Maintenance contains Work Requests, Findings, Work Orders, planning, readiness, Procedures, Maintenance Plans, Scheduled Work, and maintenance execution and verification experiences.

Planning is a Maintenance workspace rather than a separate top-level product domain.

Inventory

Inventory contains stock, custody, Reservations, Movements, replenishment, Stocktaking, and material-availability experiences.

Reliability

Reliability contains Failure Events, restoration, investigation, Cause Assessments, recurrence, and derived reliability learning.

Reports

Reports provide governed reproducible views and artifacts. Reports do not become a generic analytics builder or replace operational workspaces.

Settings

Settings contains Tenant-owned configuration such as Users and Teams, Access Profiles, plant classifications and configuration, Data Sources, Integration Principals, Process configuration, Maintenance configuration, Inventory configuration, and Tenant policy according to Permission.

Configuration must remain visually and conceptually distinct from operational work.

Platform Admin

Platform Admin is a separate application for platform-owned functions such as:

- Tenant provisioning and lifecycle control;
- platform User and support administration explicitly defined by Product Authority;
- preservation, recovery, import, export, and platform readiness actions;
- operational support evidence and diagnostics where authorized.

Platform Admin must not expose ordinary Plant, Process, Maintenance, Inventory, or Reliability mutation merely because platform operators can technically access the infrastructure.

A platform User cannot impersonate a Tenant User.

Keep cross-domain context connected

The Dashboard should carry known context forward between domains.

Examples include:

- Process Condition → request Maintenance;
- Functional Location → view installed Assets, open work, Process context, and failures;
- Asset → view installation, Measurement Points, work, failures, and material history;
- Failure Event → open related Process evidence and Maintenance response;
- Work Order → issue or return Inventory;
- Inventory Item → inspect usage by Maintenance or Process where authorized.

A related workflow should not ask the user to reselect plant context that Waiotech already knows.

Related documents

- [Tenant Dashboard experience](#)
- [Dashboard design conventions](#)
- [Browser application contracts](#)

Android Work App

Android Work App is Waiotech's governed native field surface for individually authenticated plant Users performing Process and Maintenance work near the plant, equipment, materials, and physical evidence. It prioritizes fast human evidence capture, dependable execution, scanning, clear synchronization, and bounded offline authority under intermittent connectivity.

Make Android a field product rather than mobile administration

Android must optimize for Users who are moving through the plant, working with gloves or limited attention, scanning identifiers, following Tasks or Process Rounds, recording evidence, and operating with unreliable connectivity.

The primary users are technicians, operators, inspectors, field supervisors, and other authorized Users who perform or record plant work.

Android is not the primary surface for planners, Tenant administrators, IAM administrators, Process configuration, report administration, publication, or Platform Admin.

It must not reproduce the complete Tenant Dashboard on a small screen.

Bind each active installation to one attributable User

Each active Android installation belongs to exactly one User at a time and preserves its own installation identity, credentials, local Tenant partitions, offline packages, encrypted files, journal, synchronization state, and revocation state.

A User may have several separately authorized installations. Offline authority is never shared between them.

A physical device may be reassigned only after the earlier User's authenticated context is ended or administratively revoked and readable Waiotech data is securely cleared or preserved through the governed unresolved-evidence contract.

Keep Tenant context explicit

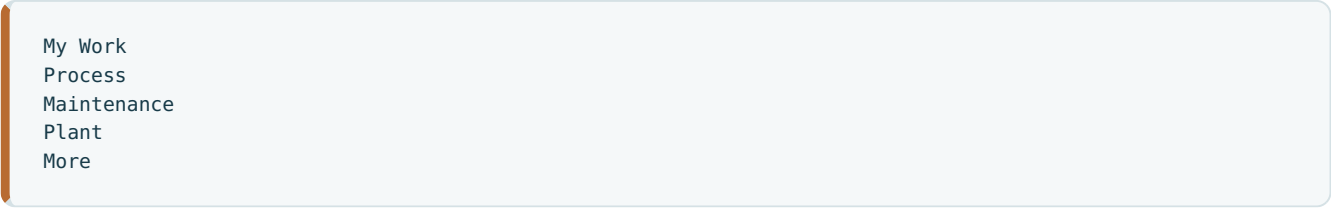
Tenant is the plant boundary on Android as everywhere else.

Every cached record, offline package, journal entry, file, scan result, Notification, synchronization operation, and command belongs to an explicit Tenant partition.

Changing Tenant context must not expose, submit, relabel, or clear another Tenant's unresolved field evidence incorrectly. Unsynchronized work remains associated with the Tenant under which it was created until governed synchronization or recovery resolves it.

Primary Android areas

Canonical mobile navigation is:



My Work
Process
Maintenance
Plant
More

More may contain Notifications, Sync Issues, profile/device status, contextual help, and other field-support functions that do not deserve permanent primary navigation.

Inventory interactions appear contextually where field custody or material use is authorized. Broad Inventory administration, Maintenance planning, Procedure or Plan publication, Process-model configuration, Data Source and Integration Principal configuration, IAM, policy, Reports, Tenant lifecycle, and Platform Admin remain outside ordinary Android workflows.

My Work

My Work is the User's bounded cross-domain field queue.

It may include authorized:

- Process Rounds;
- Process Conditions requiring field attention;
- Work Orders;
- field evidence or synchronization obligations;
- other explicitly supported field responsibilities.

The queue prioritizes what genuinely requires the User or one of the User's Teams, due and overdue meaning, current responsibility, offline availability, important blockers, synchronization condition, and the next supported action.

Mobile discovery must remain relevant, authorized, bounded, paged where needed, and Tenant-scoped. Broad read Permission must not automatically turn Android into a Tenant-wide administration browser.

Process field workflows

Android must support the human Process workflows that are naturally performed in the plant, including:

- navigate or scan to Process Unit, Process Stream, Measurement Point, Functional Location, or Asset context;
- discover, start, continue, and complete authorized Process Rounds;
- move through Round entries sequentially without re-selecting known context;
- record manual Process Readings;
- record Operational Observations;
- inspect recent relevant Readings and open Process Conditions;
- interact with a Process Condition where the action is authorized for mobile use;

- record Operational Actions and Outcome Assessment evidence where appropriate;
- capture photographs and other supported evidence;
- request Maintenance from Process context through the canonical Maintenance contract.

Human Reading entry must preserve Measurement Point, quantity/unit semantics, effective time, User, provenance, quality, correction, and audit meaning identical to Dashboard authority.

Machine-ingestion configuration does not appear in ordinary Android Process workflows.

Process Round experience

Process Round execution is optimized for rapid, accountable human collection.

The field experience shows:

- Routine and Round identity;
- current entry and progress;
- required versus optional meaning;
- configured Measurement Point or Observation subject;
- concise reference guidance;
- recent useful context where authorized;
- missing required entries;
- offline and synchronization state.

When the Round already defines Measurement Point, subject, unit, Tenant, and User, the app must carry those facts into entry rather than asking the User to select them again.

Completing a Round means required collection work was performed. Android must not present Round completion as proof that the plant is healthy or a Process Condition is resolved.

Process Condition field interaction

Process Condition handling on Android must preserve the same ownership, attention, lifecycle, action, and evidence meaning as Dashboard.

Mobile may emphasize the field-relevant subset: current operational situation, attention, responsible Team/User, latest evidence, action already taken, next review, and available action.

The app must not reduce Process Conditions to an editable status dropdown or a generic note thread.

Maintenance field workflows

Android must support the governed Work Order field-execution loop, including:

- assigned and otherwise authorized Work Order discovery;
- Work Target and connected Plant/Process/Failure context;
- exact Procedure Revision and Task instructions;
- execution-relevant Work Requirements;

- hazards, permits, isolations, blockers, and field-visible Readiness;
- start, hold, resume, and other supported execution actions;
- maintenance measurements and observations;
- Findings;
- photographs, Attachments, and other evidence;
- authorized material issue, return, or usage capture;
- completion submission;
- visibility of verification, rework, or follow-up outcome where useful.

Planning results required for safe execution must be visible without moving planning authority into Android.

Keep planning, approval, administration, and governance outside Android

Android must not provide ordinary workflows for:

- Work Request triage;
- Work Order preparation or release;
- emergency authorization unless a distinct mobile Product contract explicitly permits it;
- Procedure or Maintenance Plan authoring/publication;
- Process Unit, Stream, Measurement Point, Data Source, or machine-ingress configuration;
- Access Profile, Team, Membership, or Tenant Control administration;
- broad Inventory configuration or unrestricted adjustment;
- Stocktaking approval/posting unless separately authorized by Product Authority;
- independent verification requiring another Actor where the field User cannot satisfy separation of duties;
- final administrative closeout where Dashboard context is required;
- Report administration;
- Tenant export/lifecycle;
- Platform Admin.

Displaying planning or governance results does not transfer the authority to change them.

Plant identification and scanning

Android may search or scan governed identifiers to identify:

- Functional Location;
- Asset;
- Measurement Point;
- Work Order or supported Process/Reliability record;
- Item;
- Stock Location;
- supported external identifiers.

A scan identifies candidate context; it never grants Permission or offline authority.

Unknown, retired, ambiguous, invalid, or cross-Tenant identifiers fail safely and explain the result.

Offline authority

Disconnected Process or Maintenance actions are permitted only where Offline field-package Product and Engineering Authority explicitly define the exact action, subject, evidence basis, expiry, synchronization, conflict, and safety semantics.

Android local persistence is not an alternate Tenant database and a downloaded record is not disconnected write authority by itself.

Offline Process authority

Offline Process support must remain deliberately bounded.

A Process Package may authorize execution of an explicitly identified Process Round Revision snapshot, manual Readings, Operational Observations, evidence, or other exact Process actions only for pre-identified subjects and Measurement Points.

It must not grant arbitrary offline creation or mutation of plant configuration, Data Sources, machine mappings, Integration Principals, or unrestricted Process Condition lifecycle authority.

Where a Process action cannot safely be pre-authorized, Android may preserve typed local evidence or a draft and establish the canonical action only after synchronization and fresh Server validation.

Offline Maintenance authority

A Work Package may authorize only the field actions explicitly issued for one Work Order.

Downloaded Work Order information without a valid applicable Work Package is read context, not disconnected execution authority.

Offline completion submission never implies independent verification, closeout, or another Actor's Decision unless Product Authority separately defines that action.

Restrict offline material recording to pre-authorized custody and allocation

Offline material recording is limited to explicit package or field-contract authority such as:

- material already issued to the Work Order;
- material reserved for the Work Order;
- a controlled User kit or custody location authorized for offline use;
- another product-defined bounded allocation.

The app must never treat a downloaded Stock Balance as authority for unrestricted offline issue, transfer, return, or adjustment.

Preserve field evidence without falsifying Inventory posting

A User may record intended or observed material use under the applicable field contract, but Android must not invent a successful Inventory Movement, create negative stock without authority, silently substitute another Stock Location, or erase the field evidence when central custody validation rejects the intended posting.

Synchronization establishes the canonical Inventory result and preserves any reconciliation obligation.

Support disconnected maintenance-need capture as a draft

Android may preserve an encrypted local Work Request draft while disconnected even when no Work Package grants Maintenance command authority, because the draft does not yet create a Work Request.

The draft may preserve candidate Functional Location or Asset, Process or Work Order context, narrative, occurred time, evidence, and stable local operation identity.

The User deliberately submits it when canonical synchronization is possible. Rejection preserves the draft and explains required correction.

Separate local file capture from accepted Attachment identity

Android may capture protected local photos/files and link them to pending journal operations.

A local file is not yet an accepted Attachment.

Synchronization transfers and establishes canonical file identity, integrity, classification, safety/quarantine behavior, subject relationship, and audit through the Attachment contract.

Local evidence must remain attributable and must not be silently discarded after uncertain synchronization.

Keep mobile communication separate from workflow authority

Android may receive field-relevant Notifications and minimal push hints.

Opening, reading, dismissing, or receiving a Notification never executes the related Process, Maintenance, Inventory, Reliability, or governance action. Sensitive details are retrieved through ordinary authenticated authority.

Protect local field data

Local plant data must be:

- User-scoped;
- installation-scoped;
- Tenant-partitioned;

- encrypted through platform-backed protection;
- minimized to field need;
- bounded by package/session/retention rules;
- revocable where centrally enforceable;
- non-authoritative except for preserved local evidence awaiting canonical synchronization.

Readable Tenant data must not remain available after secure logout, User reassignment, required local destruction, or acknowledged installation revocation according to Engineering Authority.

Contain device loss

Installation revocation, credential/session revocation, encryption, package expiry, bounded local retention, and central preservation of already-synchronized evidence limit the consequence of a lost device.

A disconnected device cannot be guaranteed to receive revocation immediately. Offline package scope and expiry therefore form part of the security boundary.

Optimize for physical work

Android must prioritize:

- fast startup;
- clear current subject and responsibility;
- obvious connected/offline/synchronizing condition;
- large practical touch targets;
- strong contrast;
- minimal typing;
- fast camera and scanning access;
- predictable Task/Round sequence;
- useful recent context without dense dashboards;
- visible local-unsynced evidence;
- recoverable failure and conflict handling;
- usability in bright, noisy, moving field environments;
- accessibility;
- complete Persian and English behavior.

Decorative animation, broad analytics, and complex nested navigation must not interfere with execution.

Provide focused field help

Android provides contextual help for the field workflows it supports, including Process Rounds, Readings, Observations, Process Condition interaction, Work Order execution, Tasks, measurements, Attachments, scanning, offline packages, synchronization, conflict handling, material recording, and completion submission.

Help required to execute a downloaded package safely may be included in the package or otherwise available offline under the package contract.

Android must not copy the complete administrative Help Center into the app.

Related documents

- [Offline field packages](#)
- [Process Routines and Rounds](#)
- [Process Readings and Operational Observations](#)
- [Android installation and offline authority](#)
- [Work Order lifecycle and Readiness](#)

Offline field packages

Offline authority is explicit, server-issued, User- and installation-bound, narrow, expiring, and auditable. Waiotech uses distinct package contracts for disconnected Maintenance execution and bounded Process field evidence rather than treating the Android database as an alternate Tenant authority.

Use offline authority only where safe meaning can be established before disconnection

Disconnected work is permitted only when the Server can issue an immutable package that contains or references every fact needed to bound the authorized action safely.

A package never grants general Tenant Permission. It grants only the exact offline actions declared by its package type, subject set, User, installation, Tenant, issuance basis, and expiry.

The two canonical Alpha package types are:

- **Work Package:** offline authority for one Work Order;
- **Process Package:** offline authority for an explicit bounded set of Process subjects and field-evidence actions.

These package types may share technical envelope mechanisms but must not lose their distinct product meaning.

Work Package

A Work Package is an immutable server-issued offline execution snapshot for exactly one Work Order.

It may include:

- Work Order identity and concurrency basis;
- Work Target and relevant Plant context;
- exact Procedure Revision and Task snapshot;
- field-visible Requirements and Readiness basis;
- permitted offline action codes;
- evidence contract;
- material allocation or reservation references where authorized;
- issuance and expiry;
- User and Android installation;
- package schema and authenticity information.

A Work Package never authorizes planning, release, verification, closeout, unrestricted Inventory posting, or another Work Order unless Product Authority explicitly says otherwise.

Process Package

A Process Package is an immutable server-issued offline authority snapshot for pre-identified Process field work.

It may include:

- one or more explicit Process Units or Process Streams;
- zero or more explicit Process Rounds together with their exact immutable Process Routine Revision entry snapshots, occurrence identity, execution window, and required-entry contract;
- explicit Measurement Points available for manual Reading entry;
- relevant active Process Conditions when their evidence may be updated offline;
- recent bounded context needed to interpret field work;
- permitted offline action codes;
- evidence contract;
- issuance and expiry;
- User and Android installation;
- package schema and authenticity information.

Canonical offline Process actions may include:

- start or continue an included Process Round;
- satisfy included Process Round entries by recording their canonical Reading or Observation evidence;
- complete an included Process Round only when the package contains its exact immutable entry/evidence contract and local validation can establish that every required entry is represented;
- record a manual Process Reading for an included Measurement Point;
- record an Operational Observation against an included subject;
- attach field evidence to an included subject or Condition;
- record an Operational Action or outcome evidence against an included Process Condition only when the package explicitly authorizes that action and contains the required authoritative basis.

Process Round completion is independent of plant-condition resolution: completing a Round establishes only that the required collection work was performed.

Opening, resolving, reopening, reassigning, or otherwise changing Process Condition lifecycle while disconnected is not implicitly authorized. When the Server cannot safely pre-authorize the lifecycle change, Android preserves a typed local draft or field evidence and establishes the canonical command only after synchronization and fresh authorization.

Offline Work Request draft

Android may preserve a typed encrypted Work Request draft while disconnected.

The draft is not a Work Request, audit event, or accepted maintenance need. It may contain candidate Functional Location or Asset context, narrative, occurred time, evidence, and a stable local operation identity. On synchronization, an explicit User submission invokes the canonical Work Request command and receives the authoritative result.

Package issuance

The Server issues a package only after authoritative evaluation of:

- authenticated User and active Membership;
- Android installation;
- Tenant condition;
- exact Permission and responsibility;
- subject lifecycle;
- action prerequisites;
- applicable Readiness or Process safety constraints;
- material authority where applicable;
- package expiry and security policy.

Issuance is an attributable auditable action and is duplicate-safe.

Expiry and renewal

Every package expires at the earliest safe limit established by product, security, lifecycle, source freshness, or operational constraints.

Renewal creates a new issued package after fresh authoritative evaluation. Renewal does not mutate historical package evidence or extend authority silently.

Expiry prevents new offline actions but does not destroy already recorded field evidence.

Device and User binding

A package is bound to exactly one User, one Android installation, and one Tenant.

It cannot be transferred, shared, exported as a reusable credential, or used after installation revocation, package revocation, or expiry. Several Users collaborating on the same plant subject receive independent authority and independent journals.

Offline action journal

Accepted local field actions are appended to an encrypted ordered journal.

Each entry preserves:

- stable local operation identity;
- package identity and type;
- action code and schema version;
- local sequence;
- typed input;
- evidence references;
- device-observed time;

- correction relationship where applicable;
- synchronization condition.

Synced entries are not edited. Corrections are additive.

Time and ordering

Device time is evidence, not authoritative system time.

The journal preserves local causal order independently from wall-clock accuracy. Synchronization records Server acceptance time and preserves both the field-observed time and authoritative recorded time required by the owning domain.

Synchronization

Each offline journal operation synchronizes through the same owning Application command used by connected operation, with a stable operation identity and canonical Command Receipt.

Synchronization is operation-by-operation, resumable, idempotent, and ordered. One package upload is not one unbounded transaction.

Accepted earlier operations remain accepted if a later dependent operation cannot be established safely.

Conflict handling

The Server never silently rewrites field evidence to make it fit changed authoritative state.

When authoritative state changed after package issuance, synchronization returns a typed result that distinguishes accepted, already established, rejected, needs review, or conflict conditions. Field evidence remains attributable even when the intended domain action cannot be accepted.

The User experience explains what changed and what action is required next.

Offline Inventory boundary

Offline material evidence may be captured only when the Work Package or other explicit contract identifies the authorized allocation or custody basis.

Android cannot infer successful issue, return, transfer, Reservation release, or Stock Balance from cached values. Synchronization invokes canonical Inventory authority and preserves reconciliation if authoritative custody prevents posting.

Evidence and files

Local files remain field evidence until canonical Attachment acceptance occurs.

Synchronization preserves file integrity, Actor, subject, package, operation identity, and relationship to the canonical action. Failed transfer or rejected action must not silently discard captured evidence.

Revocation

Revocation stops future authority when the app learns of it. A disconnected device cannot be guaranteed to learn revocation before reconnecting, so package scope and expiry are the primary bound on disconnected risk.

Revocation never erases already recorded local evidence. Synchronization evaluates it truthfully against the package and authoritative state.

Related documents

- [Android Work App](#)
- [Work Order lifecycle and Readiness](#)
- [Process Readings and Operational Observations](#)
- [Android installation and offline authority](#)

Documentation and Help Center

Documentation and Help Center provides bilingual, persistent, contextual product guidance and governed deep links. Guides explain product use without becoming independent Product or Engineering Authority.

The Documentation and Help Center

WaioTech must provide an integrated permanent Help Center rather than a link directory.

The Documentation and Help Center is the permanent bilingual learning, explanation, contextual-help, and troubleshooting capability inside Tenant Dashboard.

It replaces a page that merely lists documentation links.

Maintain bilingual documentation parity

WaioTech must maintain bilingual documentation parity.

It provides complete English and Persian content.

Both language editions must communicate equivalent concepts, workflows, blockers, safety information, and mission instructions. Canonical codes remain unchanged; presentation text is localized.

Organize help around concepts, tasks, roles, and problems

Organize help around concepts, tasks, roles, and problems.

The Help Center includes:

- concept explanations;
- canonical terminology;
- task-based guides;
- role-relevant learning paths;
- action and lifecycle explanations;
- Permission and responsibility explanations;
- evidence guidance;
- troubleshooting;
- error-code guidance;
- field-work guidance;
- links to related subjects and actions.

Keep user documentation explanatory and authority-aligned

Waiotech must keep user documentation explanatory and authority-aligned.

Documentation explains Product Authority and supported product behavior. It must remain aligned with authority but cannot introduce or override a product rule.

Keep documentation permanently available as product help

Waiotech must keep documentation permanently available as product help.

Documentation availability is independent from mission completion, assessment result, Tenant state, and onboarding progress, subject only to ordinary authentication and content-access rules.

Contextual help

Waiotech must make guidance available at the point of work without losing user input.

Contextual help connects a product page, form, field, action, lifecycle condition, blocker, or error code to the applicable guide.

It must preserve the User's unfinished work when opened and permit navigation back to the relevant product context.

Use deep links for navigation, never authority

Waiotech must use deep links for navigation, never authority.

A guide may link to the relevant Dashboard page, record type, action, or form. Android field guides may link to the applicable Work App screen.

A guide link does not bypass authentication, Permission, Tenant context, lifecycle, or Readiness.

Interactive onboarding

Teach product work through guided practice.

Interactive onboarding consists of task-based guided missions that teach concrete Waiotech workflows through safe product interaction.

It is not limited to passive introduction screens or completion checklists.

Related documents

- [Public Website](#)
- [Learning environment](#)
- [Documentation, help, and learning engineering](#)

Learning environment

Guided missions run inside each learner's private learning-purpose Tenant using synthetic identities and data. Learning state is isolated from operations, may be reset, and never grants operational authority or converts into operational evidence.

A guided mission

Waiotech must represent onboarding as governed task-based learning.

A guided mission defines:

- learning objective;
- required starting data;
- ordered or conditional learning steps;
- relevant product surface;
- contextual instructions;
- completion evidence;
- reset behavior;
- language content.

A mission guides the User but must not bypass product rules.

Give each learner an isolated practice environment

Give each learner an isolated practice environment.

Guided missions run in a private test Tenant containing safe synthetic data.

The test Tenant is assigned to the learner and is not shared with unrelated Users.

Use one reusable private learning environment per User

Waiotech must use one reusable private learning environment per User.

A User has at most one active private test Tenant in one Waiotech product environment.

The Tenant may be reset for another mission while onboarding progress for completed or paused missions remains separate. A mission switch that replaces synthetic state requires explicit confirmation.

A synthetic learning User

Waiotech must support realistic multi-Actor training without creating operational identities.

A synthetic learning User is a non-operational identity used only inside learning-purpose Tenants to demonstrate collaboration, assignment, approval, or separation of duties.

A synthetic learning User:

- cannot authenticate independently;
- cannot have Membership in an operational Tenant;
- cannot receive real communication;
- cannot own operational credentials;
- cannot be converted into a real User;
- is visibly identified as synthetic;
- may act only through a guided mission's explicit learning simulation contract.

The learner's own authenticated User remains the Actor for actions the learner performs.

Use real product rules without treating synthetic practice as operational truth

WaioTech must use real product rules without treating synthetic practice as operational truth.

It is a Tenant with `learning` purpose.

It uses canonical product behavior for practice, but its records are explicitly synthetic and have no operational, financial, compliance, Process-history, maintenance-history, Reliability-history, Inventory-custody, or evidential effect outside the learning environment.

Preserve realistic in-mission behavior while allowing deterministic disposal of synthetic state

WaioTech must preserve realistic in-mission behavior while allowing deterministic disposal of synthetic state.

Within a mission, lifecycle, immutability, authority, and evidence rules behave as they do in the product so the User learns the correct workflow. The complete synthetic Tenant state remains disposable under the explicit reset contract.

Onboarding progress is a separate durable User learning record.

Prohibit training-data promotion into operational authority

WaioTech must prohibit training-data promotion into operational authority.

Synthetic records, identifiers, Attachments, Decisions, audit, revisions, Inventory facts, and artifacts cannot be promoted into operational use.

Confine destructive reset authority to learning-purpose synthetic state

Confine destructive reset authority to learning-purpose synthetic state.

Reset is an explicit product action over a learning-purpose Tenant whose complete business state is synthetic and disposable. It does not authorize destruction, rollback, or replacement of operational Tenant evidence.

Teach the real product rather than a simplified imitation

Teach the real product rather than a simplified imitation.

The test Tenant uses the same product concepts, Domain rules, Application actions, lifecycle, Permission evaluation, Readiness, validation, revision behavior, APIs, and user-interface behavior as an operational Tenant.

Simulation is limited to external effects and synthetic data.

Prevent training activity from affecting operational systems or people

WaioTech must prevent training activity from affecting operational systems or people.

The test Tenant is isolated from:

- operational Tenant data;
- unrelated Users;
- real communications;
- operational integrations;
- billing;
- externally visible files;
- operational reports and search;
- platform support activity unrelated to the mission.

Use safe, recognizable synthetic training data

WaioTech must use safe, recognizable synthetic training data.

It contains clearly identified synthetic Users, Teams, Functional Locations, Assets, Process Units, Measurement Points, Process Conditions, Items, Work Orders, Maintenance Plans, Findings, Failure Events, Attachments, report inputs, and other mission data.

Synthetic records must not impersonate real organizations, people, credentials, or evidence.

Keep guided missions free from real external consequences

Waiotech must keep guided missions free from real external consequences.

Email, push, export-delivery, and other external effects are blocked or simulated inside the test environment.

Separate learning progress from simulated product state

Waiotech must separate learning progress from simulated product state.

Onboarding progress is a separate learner record containing mission identity, resume position, completed learning steps, completion, and permitted assessment result.

Test-Tenant product records remain separate simulated business data.

Support resumable learning without conflating progress and product records

Waiotech must support resumable learning without conflating progress and product records.

The User may exit and resume from preserved onboarding progress. The mission must verify that its test-Tenant state remains compatible with the recorded learning step.

Support repeatable practice

Waiotech must support repeatable practice.

A mission may be repeated without removing permanent documentation access or changing operational Tenant data.

Make test-environment reset deterministic and isolated

Waiotech must make test-environment reset deterministic and isolated.

Reset restores the mission's declared synthetic starting state.

It clears or replaces unfinished synthetic business activity, cancels synthetic processes, and removes generated test files and artifacts according to the mission definition.

Reset does not alter operational Tenants.

Make learning reset an explicit attributable action controlled by the learner or governed support

Waiotech must make learning reset an explicit attributable action controlled by the learner or governed support.

The learner may reset the private test Tenant through the guided-mission interface. A governed support action may reset it when the learner requests assistance or the environment is unusable.

Reset requires explicit confirmation when unsaved mission work will be removed. It preserves a reset record and does not imply onboarding-progress reset.

Preserve learning progress independently from test data reset

WaioTech must preserve learning progress independently from test data reset.

A test-Tenant reset does not reset onboarding progress unless the User explicitly chooses a separate onboarding-progress reset.

Keep learning completion separate from IAM and workforce capability

WaioTech must keep learning completion separate from IAM and workforce capability.

Mission completion does not:

- grant Permission;
- create Team Membership;
- assign an Access Profile;
- prove Qualification;
- replace required training, approval, or verification;
- authorize operational work.

Related documents

- [Tenant and operational scope](#)
- [Documentation and Help Center](#)
- [Documentation, help, and learning engineering](#)

Tenant Dashboard experience

The Waiotech Dashboard is the professional operating workspace of the plant. It helps users understand plant context, identify what requires attention, act with confidence, and verify outcomes. It must expose operational meaning rather than database structure.

Product character

The Dashboard must feel:

- calm, clear, and dependable;
- professional without feeling cold;
- operational rather than administrative;
- structured without becoming bureaucratic;
- approachable for users who are not software specialists;
- efficient for expert users who spend hours in the product;
- useful during both routine work and urgent situations.

Warmth comes from clear language, meaningful context, visible people and equipment, useful feedback, confident workflows, and respectful information design rather than decorative effects.

Organize experience around understand, decide, act, and verify

The dominant interaction model is:

```
Understand
↓
Decide
↓
Act
↓
Verify
```

The Dashboard must not default to:

```
List
↓
Record
↓
Generic edit form
↓
Save
```

Database entities and fields remain implementation details unless their product meaning is itself operationally important.

Four experience modes

Orient

Orienting surfaces answer:

- What is happening?
- What needs attention?
- What changed?
- What is blocked?
- Who owns the next action?
- What is due or overdue?

Home, My Work, domain overviews, and attention views use this mode.

Orientation must prioritize actionable meaning over passive metrics.

Work

Operational workspaces help users process repeated records efficiently.

Examples include:

- Process Rounds;
- Process Conditions;
- Work Requests;
- Work Orders;
- Findings;
- Failure Events;
- Replenishment Requests;
- Stocktaking obligations.

High-volume workspaces should use the task-appropriate combination of search, filters, queue or table, master-detail inspection, saved or URL-backed views, and direct available actions.

Users must not be forced through repeated `list → open → back → find previous position` navigation when a master-detail or equivalent workspace can preserve context.

Understand

An operational detail experience tells the story of one subject or record before exposing all metadata.

A detail page should normally communicate:

1. human operational identity and current state;
2. important context, consequence, and warnings;

3. current responsibility and next meaningful action;
4. operational facts and relationships;
5. related Process, Maintenance, Inventory, and Reliability records where relevant;
6. evidence and attachments;
7. chronological operational history;
8. technical metadata and identifiers.

Technical UUIDs, revision identifiers, source keys, and secondary fields remain available but do not dominate the page.

Configure

Configuration experiences define the model and rules that operational work uses.

Examples include Functional Location structure, Asset Classification, Asset installation, Process Units and Streams, Measurement Points, Data Sources, Users, Teams, Access Profiles, Maintenance definitions, Inventory configuration, and Tenant policy.

Configuration may use denser registries and forms. Operational work must not inherit configuration-style CRUD merely because both manipulate records.

Work before data

Screens must be organized around the work users need to complete, not around database tables or server modules.

The interface should answer:

- What needs attention?
- What should I do next?
- Why does this matter?
- What is blocking progress?
- What changed?
- Who is responsible?
- What evidence supports the current interpretation or decision?

Server-owned business behavior

The Dashboard must not invent Permissions, lifecycle rules, available actions, state transitions, action prerequisites, validation, code generation, taxonomy, or audit behavior.

Server authority is exposed through generated contracts and action availability. The Dashboard makes those contracts understandable and efficient; it does not reproduce them as competing browser business logic.

Connected operational context

Plant, Process, Maintenance, Inventory, and Reliability must feel connected even though their authority remains separate.

Users should begin related work from the context they are already viewing.

Examples include:

- Process Unit → record Reading or Observation;
- Process Condition → record Operational Action;
- Process Condition → request Maintenance;
- Functional Location → view installed Assets and current work;
- Asset → report Finding or create Work Request;
- Work Order → reserve, issue, or return material;
- Failure Event → inspect Process evidence and maintenance response;
- Inventory Item → inspect relevant usage and availability.

Contextual creation must carry forward known relationships and evidence rather than opening a blank form that asks users to reconstruct them.

Plant experience

Plant is a connected explorer, not a set of unrelated registries.

Users must be able to navigate distinct but related plant dimensions:

- Functional Location hierarchy;
- Asset physical composition and installation;
- Process Unit hierarchy;
- Process Stream flow;
- Measurement Points.

The interface must not pretend these are one tree.

A selected Functional Location, Asset, Process Unit, Stream, or Measurement Point should reveal useful connected context such as current Process Conditions, relevant Readings, installed Assets, Functional Locations serving the selected Process context, open Maintenance, Failure Events, and material history where authorized.

A simple process-flow view derived from Process Units and Process Streams is part of Plant exploration. A P&ID-oriented or geographic view, when provided, is another projection over the same canonical Plant Model and must not become a separate data authority.

Process experience

Human Process entry must be fast enough for routine plant operation.

When the user is already in a Process Unit or Measurement Point context, Waiotech should already know the Tenant, subject, Measurement Point, canonical unit, current User, and normal time default. The user should enter only the information that is genuinely unknown.

A Process workspace should prioritize:

- open Conditions;
- Process attention;
- current handling state and responsibility;
- latest relevant Readings and Observations;
- actions already taken;
- next review obligation;
- outcome;
- related Maintenance and Reliability activity.

Machine and human Readings appear in one coherent timeline and trend experience with provenance visible when useful.

Process Rounds provide a fast sequential field or control-room workflow for routine human collection. The Round keeps the operator in context, advances through required Reading and Observation entries, shows reference guidance without manufacturing alarms, and makes missing required entries obvious before completion.

Machine ingestion configuration belongs in Settings, not in ordinary operator workflow.

Maintenance experience

Maintenance should present why work exists, whether it is ready, what blocks it, what must be done, what was found, what material is needed, what evidence is required, and whether the technical result was verified.

A Work Order must not feel like a long editable database row.

Available domain actions such as assign, release, start, record Finding, issue material, complete, verify, cancel, or close should be explicit and state-aware.

Inventory experience

Inventory should make custody trustworthy and fast to understand.

Users need to see on-hand, reserved, available, unavailable, required, and replenishment meaning without reconstructing balances from Movement rows.

Posted Movements remain visible as history, but operational pages should lead with current custody meaning and the action the user needs to take.

Reliability experience

Reliability should tell a failure story:

- what function failed;
- where or which Asset failed;
- process consequence;

- when it occurred and was detected;
- restoration attempts;
- maintenance response;
- material constraints;
- cause assessment;
- recurrence;
- operational recovery.

Reliability analytics must lead back to source Failure Events and evidence rather than becoming disconnected KPI cards.

Home

Home is a concise server-owned operational orientation surface, not a wall of charts or a duplicate of every module overview.

It should prioritize, according to the User's effective authority:

- actions that genuinely require the User or one of the User's Teams;
- due or overdue Process Rounds assigned to the User or the User's Teams;
- urgent or critical Process Conditions and due monitoring reviews;
- active Failure Events and significant restoration or investigation obligations;
- overdue, urgent, or materially blocked Maintenance work;
- Inventory shortages or replenishment obligations that affect current plant work;
- recent significant operational changes worth understanding;
- direct routes into the normal owning workspace.

Home summaries are produced by server-owned cross-domain read models. The browser must not reconstruct global attention by downloading partial registries or inventing attention rules.

A summary count is shown only when the User may inspect the underlying records and the count is complete for its declared scope. An unavailable metric is omitted rather than rendered as a misleading zero.

Home never executes consequential commands directly. It opens the normal authorized context where the User can understand and perform the action.

My Work

My Work is the user's cross-domain operational queue.

It may include records assigned to the current User, records owned by the User's Teams, and records exposing an executable decision-bearing action for the User.

The queue preserves each record's owning domain and identity.

Users must be able to filter and search, understand priority and due state, see why an item requires attention, open complete context, perform the next available action, and return without losing filters, sorting, selection, or position.

Available actions

Available actions are a core Dashboard contract.

An action is displayed only when:

- Server authority returns it as available;
- the Dashboard has a complete registered interaction for it;
- required inputs and known prerequisites are understood.

An action must have clear human wording, appropriate confirmation or input, visible requirements, specific validation feedback, predictable success behavior, and clear handling of changed record state.

Unknown action descriptors are contract defects. They must not be silently ignored.

Prefer domain actions over generic editing

Operational state changes must be represented primarily through named domain actions.

Examples include:

- assign;
- release;
- start;
- complete;
- verify;
- resolve;
- reopen;
- record Reading;
- record Operational Action;
- install Asset;
- remove Asset;
- transfer stock;
- adjust stock.

Generic edit is appropriate for descriptive or configuration information. It must not conceal lifecycle transitions or consequential actions.

Forms

Forms are organized around user intent.

They must:

- ask for the minimum required information first;
- never ask for information WaioTech already knows from context;
- use operational defaults where safe;
- group fields into meaningful sections;
- progressively reveal optional detail;

- use direct operational language;
- use proper selectors rather than raw identifiers;
- preserve user input after recoverable failure;
- explain important consequences;
- avoid raw enum and database-field vocabulary.

Complex workflows should use meaningful sections or steps rather than one unstructured wall of controls.

Operational timelines

WaioTech should present consequential history as a chronological operational story.

Timeline entries may compose source events from several domains while preserving their original authority and identity.

Examples include Readings, Observations, Process Condition actions, Work Request creation, Work Order execution, Inventory usage, Failure Event restoration, verification, and resolution.

Raw audit field changes belong in audit detail where required; ordinary users should see the business event and attributable Actor.

Readings and trends

Charts and trends exist to answer an operational question.

Useful examples include:

- a Measurement Point trend around an active Process Condition;
- runtime since the last maintenance occurrence;
- process performance before and after an intervention;
- selected Readings around a Failure Event.

The default interface should communicate current meaning first. A trend should not consume the page merely because data exists.

WaioTech must not become a generic dashboard builder.

Professional information density

WaioTech must optimize for useful information density.

The product should be compact enough for planners, operators, engineers, and warehouse personnel to work efficiently on large displays while remaining clear and touch-usable on tablets and supported mobile widths.

Avoid both extremes:

- dense ungrouped enterprise screens where everything competes for attention;
- decorative SaaS layouts with excessive whitespace and too little operational information.

Tables and master-detail workspaces

Tables are appropriate where comparison, prioritization, scanning, and bulk operational understanding matter.

The problem is not the table; the problem is a table that only leads to generic record editing.

Where suitable, a table or queue should support selection, contextual detail, visible blockers, available actions, related history, and preserved workspace state.

Search

Search must operate on human operational identity rather than requiring users to know which database registry to search.

Plant search should resolve canonical codes, names, and supported external identifiers and surface understandable matches across Functional Locations, Assets, Process Units, Measurement Points, and relevant operational records according to Permission.

Search results preserve the owning record type and open its normal authorized context.

Status and attention

The Dashboard must distinguish:

- lifecycle state;
- handling state;
- priority or urgency;
- due or overdue state;
- responsibility;
- action readiness;
- blockers;
- material availability;
- verification or approval requirements;
- Reading quality;
- Process attention.

Colour may reinforce meaning but text and icons must remain sufficient.

Attention colour is scarce. Neutral states should remain visually neutral so true operational attention remains visible.

Language and tone

Dashboard language is concise, human, and operational.

Prefer phrases such as:

- “What needs to be done?”

- “Why is this work blocked?”
- “Record dissolved oxygen.”
- “Three required items are unavailable.”
- “The record changed after you opened it.”

Avoid internal implementation terminology, generic technical errors, unexplained codes, excessive administrative wording, and labels copied from database fields.

Persian and English are equal supported languages. Neither is a translated afterthought.

Responsive, accessible, and bidirectional behavior

All supported Tenant workflows must remain usable at supported desktop, tablet, and mobile widths.

Responsive behavior preserves information, actions, validation, workflow progress, related context, and safe confirmation behavior.

The Dashboard supports keyboard navigation, visible focus, semantic labels, screen-reader-compatible controls, sufficient contrast, reduced reliance on colour, correct RTL and LTR layout, and appropriate icon direction.

RTL behavior is designed into shared layouts and components.

Errors, conflicts, and uncertain results

Errors explain what failed, what the user can do next, whether input was preserved, whether retry is safe, and whether authoritative data changed elsewhere.

Concurrency conflicts must not appear as generic failure. The Dashboard refreshes authoritative state and helps the user reconsider or safely repeat the action.

For idempotent commands, uncertain retry reuses the same operation identity according to Engineering Authority.

Performance and feedback

The Dashboard responds immediately to user input while preserving Server authority.

It provides useful loading states, background-refresh indication without unnecessarily clearing current content, immediate submission feedback, duplicate-command protection, specific success confirmation, and updated authoritative state.

Optimistic updates are used only when they cannot misrepresent consequential operational truth.

Acceptance standard

A Dashboard capability is ready only when:

- Server and generated-client contracts align;
- Tenant isolation is preserved;

- supported actions are complete end to end;
- the primary workflow is understandable without specialist software guidance;
- English and Persian behavior is correct;
- desktop, tablet, and supported mobile behavior is usable;
- loading, empty, validation, conflict, error, refreshing, and success states are handled;
- consequential actions and evidence remain attributable;
- critical workflows have automated tests;
- the result feels like one coherent WaioTech operating product rather than a collection of technical screens.

Related documents

- [Tenant Dashboard and Platform Admin](#)
- [Dashboard design conventions](#)

Dashboard design conventions

Dashboard design conventions translate the Waiotech experience contract into consistent layout, density, hierarchy, interaction, state, and responsive behavior. They support operational clarity rather than defining product authority.

Visual hierarchy

Every viewport should make the following order visually obvious where applicable:

1. where the user is and which operational subject is in context;
2. current state and material attention;
3. one primary next action;
4. the information needed to decide or act;
5. related history and secondary detail.

Pages must not use visual decoration to compete with operational meaning.

Spacing scale

Use one shared spacing scale. Components must use design tokens rather than one-off page values for ordinary padding, margin, and gap.

Token	Value	Typical use
space-1	4 px	icon-to-label, very tight inline separation
space-2	8 px	compact control and key-value gaps
space-3	12 px	compact rows, filter groups, related controls
space-4	16 px	ordinary section and surface padding
space-5	20 px	prominent object context and comfortable forms
space-6	24 px	major section separation
space-8	32 px	page-level separation where hierarchy benefits

Operational registries use compact spacing; detail and decision surfaces use comfortable spacing. Dense information remains grouped rather than compressed into indistinguishable rows. New spacing tokens require a system-wide need rather than a one-page exception.

Typography

Typography must create unmistakable page, section, record-identity, field-label, value, and supporting-text hierarchy. The ordinary Dashboard body size is 14 px; text smaller than 12 px is not used for operational content.

Token	Size	Typical use
<code>text-xs</code>	12 px	metadata, compact timestamps, low-emphasis labels
<code>text-sm</code>	13 px	dense table content and secondary text
<code>text-body</code>	14 px	ordinary body and form content
<code>text-lg</code>	16 px	section and workspace headings
<code>text-xl</code>	20 px	object identity and narrow-layout page titles
<code>text-2xl</code>	24 px	primary desktop page titles

Font weight, spacing, and placement should carry hierarchy before font-size inflation. Codes, quantities, units, timestamps, and technical identifiers use readable tabular or monospace treatment only when it improves scanning. Machine-oriented values remain directionally isolated in both English and Persian layouts.

Navigation hierarchy

Primary navigation follows the canonical Dashboard areas. Secondary navigation belongs inside the selected area and must not reproduce the complete application tree.

The interface should prefer contextual links between related records over forcing users to navigate back to top-level modules.

Page headers

An operational page header normally includes:

- human operational identity;
- concise context breadcrumb or relationship;
- lifecycle or handling state where important;
- one primary available action;
- secondary actions in a labelled menu or grouped control.

A page should not display several equally dominant primary buttons.

Master-detail workspace

High-volume workspaces may use a master-detail layout on suitable desktop widths.

The master region contains queue identity, search, filters, ordering, and records. The detail region contains selected-record context, important state, blockers, actions, related evidence, and history.

Selection, filters, ordering, page size, and relevant workspace mode should be URL-backed where practical so browser navigation restores the working context.

At narrower widths, master-detail becomes a focused list-to-detail flow while preserving return state.

Operational detail composition

Plant, Process, Maintenance, Inventory, and Reliability detail experiences use a common conceptual order:

1. operational identity and state;
2. primary available action;
3. immediate context and attention;
4. working content;
5. connected cross-domain records;
6. evidence and operational timeline;
7. technical details.

The exact sections vary by subject. The common order must not force irrelevant tabs onto every record type.

Plant explorer

Plant views may combine hierarchy, process topology, selected-object detail, and related records.

Functional Location hierarchy and Process Unit hierarchy must remain visually distinguishable. Asset installation is shown as a relationship to Functional Location, not as a fake shared tree level. Process Streams show direction.

Simple diagram or map lenses use the same canonical selection and detail patterns as list and hierarchy lenses.

Process Reading entry

Reading entry should be exceptionally low-friction.

When context already determines Measurement Point, unit, Tenant, User, and normal current time, the primary form should emphasize only value and any genuinely optional note or time correction.

Source and provenance are shown after acceptance and when needed for interpretation. Machine source configuration never intrudes into routine human Reading entry.

Process Round workspace

A Process Round workspace optimizes rapid sequential human entry. It shows Routine identity, current entry, remaining required entries, recent accepted values, concise reference guidance, and completion state without making the User navigate to each Measurement Point separately.

Keyboard and touch progression must be predictable. Accepting one entry should move naturally to the next required entry while preserving a clear path to review earlier entries.

Process Condition workspace

Process Condition queues emphasize:

- condition description;
- Process Unit or Stream;
- handling state;
- owner Team and responsible User;
- latest significant Reading or Observation;
- next review obligation;
- related Maintenance or Failure Event status.

Selected-condition detail emphasizes the operational story and available actions rather than configuration metadata.

Work Order workspace

Work Order views emphasize why work exists, Work Target, current readiness, blockers, responsibility, schedule, Tasks, material availability, Process or Failure context, evidence, and next action.

Lifecycle changes use explicit actions. Editable status dropdowns are prohibited.

Inventory workspace

Inventory registries use tables where scanning quantity, location, availability, reservation, and replenishment state is more effective than cards.

Inventory detail emphasizes current custody position before Movement history.

Reliability workspace

Failure Event detail emphasizes failed function, primary target, actual consequence, operational condition, investigation condition, restoration history, related Process evidence, Maintenance response, Cause Assessments, and recurrence.

Derived metrics must link back to supporting events where feasible.

Button hierarchy

Use these visual roles consistently:

- **primary** — one dominant next action in the current context;
- **secondary** — important neutral action;

- **outline** — lower-weight navigation or inline action;
- **ghost** — contextual control that should not compete with content;
- **danger** — destructive or irreversible action with explicit confirmation or guard.

A screen may contain several actions but should rarely contain several simultaneous primary visual actions.

Forms and dialogs

Short focused actions use dialogs or compact inline flows when doing so preserves context.

Long or consequential workflows use full pages or structured step experiences when users need sustained context, evidence review, or several meaningful sections.

Dialogs must not become miniature scroll-heavy record-edit pages.

Empty, loading, error, refreshing, and success states

Every data-dependent view handles:

- first loading;
- successful empty state;
- recoverable and non-recoverable error;
- background refresh while current content remains visible;
- submitted or completed success where acknowledgement is useful.

State containers use the same visual system as ordinary content and avoid unnecessary layout jump.

Error presentation includes a human explanation, safe next action, and correlation reference when support diagnostics require one.

Radius and control scale

Waiotech uses restrained geometry. Ordinary surfaces and controls should not look excessively soft or decorative.

Token	Value	Typical use
<code>radius-sm</code>	4 px	badges and compact controls
<code>radius-md</code>	6 px	fields, tables, cards, dialogs
<code>radius-lg</code>	8 px	floating action panels or exceptional grouped surfaces
<code>radius-pill</code>	999 px	status or compact identity pills only

Desktop compact controls should normally fit a 32-36 px visual rhythm. Touch-oriented controls at tablet and mobile widths must provide at least a 44 px effective touch target even when the visible control is smaller.

Layout breakpoints

Shared layout primitives use these reference ranges unless content testing proves a component needs an earlier transition:

- **narrow:** below 640 px;
- **medium:** 640-1023 px;
- **wide:** 1024 px and above.

The permanent labelled primary navigation is available at wide widths. At narrower widths it becomes an accessible labelled drawer; Waiotech must not collapse primary navigation into an unlabeled icon rail.

Master-detail layouts normally require wide width. Tables may retain horizontal comparison, reduce secondary columns, or become structured rows according to the task; they must not mechanically convert every table into cards at one universal breakpoint.

Responsive behavior

The Dashboard uses responsive behavior based on content needs rather than assuming one desktop table layout everywhere.

Typical behavior:

- desktop: master-detail, full comparison tables, multi-column summaries where useful;
- tablet: reduced split layouts, touch-friendly actions, compact but complete information;
- mobile widths: focused single-column workflows, cards or structured rows where tables no longer scan reliably.

Important fields and actions must not disappear merely because width is constrained.

Density

Operational registries default to compact density. Detail, decision, and form experiences default to comfortable density.

Density changes control spacing, row height, and section rhythm; they do not remove information or alter product behavior.

Cards and surfaces

Use cards only when a group benefits from a distinct visual surface. Do not wrap every section and every metric in an identical card.

Avoid excessive borders, nested boxes, decorative gradients, competing colours, and passive KPI tiles.

Status and colour

Colour reinforces meaning but never carries meaning alone.

Neutral lifecycle states remain restrained. Warning and danger colours are reserved for genuine attention. Domain status badges must not become a rainbow of equally prominent labels.

Operational timeline

Timeline presentation uses concise business events with Actor, time, subject, and result. It may group low-value repetitive events while preserving access to complete history.

Raw audit diffs remain available where authorized but are not the default operational story.

Accessibility and bidirectionality

All components support keyboard interaction, visible focus, semantic names, screen readers, sufficient contrast, and correct RTL/LTR behavior.

Persian and English use the same information hierarchy and workflow completeness. Codes, quantities, and technical tokens remain isolated LTR where required.

Related documents

- [Tenant Dashboard experience](#)
- [Browser interaction, accessibility, and localization](#)

70-REFERENCE

Reference

Canonical vocabulary and consolidated product and lifecycle references.

4 source documents

Reference

Canonical vocabulary and derived consolidated views of the product model and lifecycles.

Contents

- [Canonical terminology](#)
- [Canonical product model](#)
- [Lifecycle summary](#)

Canonical terminology

Canonical terminology is used consistently across Product Authority, Engineering Authority, Experience Contracts, APIs, user interfaces, Reports, help, and generated references. A familiar plant synonym may appear as explanatory copy, but it must not create alternate product meaning.

Tenant

A **Tenant** is one independently governed wastewater plant and Waiotech's hard operational, security, configuration, and data-isolation boundary.

Waiotech has no subordinate Site concept.

Tenant is not a billing account or generic SaaS organization concept.

Plant Model

The **Plant Model** is Waiotech's canonical structural understanding of one Tenant plant.

It contains distinct connected concepts:

- Functional Location;
- Asset;
- Asset installation;
- Asset Classification;
- Operational Criticality;
- Process Unit;
- Process Stream;
- Measurement Point;
- Data Source and external mappings.

Plant Model is not one universal hierarchy and not a generic Plant Object graph.

Plant code

A **Plant code** is an optional human-facing operational identifier on a Functional Location, Asset, Process Unit, Process Stream, or Measurement Point. It is unique case-insensitively within the Tenant and that entity category.

Code changes are explicit and history-preserving. A used code is not silently reassigned to a different retained entity in the same category.

Functional Location

A **Functional Location** is a stable installed position, structure, or maintainable place whose identity may continue while physical Assets are installed, removed, or replaced.

Functional Location hierarchy expresses stable installed containment. Hierarchy changes preserve effective history where retained evidence depends on prior containment.

Use Functional Location for the stable plant position. Do not use the generic capitalized term `Location` as a competing canonical entity.

Asset

An **Asset** is an individually identified physical object or assembly whose own lifecycle, maintenance, reliability, or installation history matters.

Asset parentage means physical composition only. Composition changes preserve effective history, and a composed child derives installed Functional Location through its containing Asset rather than carrying a competing direct installation.

`Unit`, `Equipment`, and `Component` are not canonical structural Asset types. The lowercase words may appear in ordinary language, `Process Unit` is a distinct canonical concept, and `unit` may refer to a measurement unit where context is clear.

Asset installation

Asset installation is the historical relationship placing one physical Asset at one Functional Location for an effective period.

Installation is not Asset identity and is not represented only by overwriting a current location field.

Asset Classification

Asset Classification describes what a physical Asset technically is through governed taxonomy.

Classification does not determine physical hierarchy, Functional Location, or Process Unit membership.

Operational Criticality

Operational Criticality represents the potential plant consequence of losing the stable function at a Functional Location.

Operational Criticality is assigned to Functional Location, not to the replaceable physical Asset. An installed Asset may display the criticality of its Functional Location as context.

Operational Criticality is separate from actual Failure Event consequence, Process Condition attention, Work Order urgency, priority, execution risk, and deferral consequence.

Process Unit

A **Process Unit** is a stable functional boundary in which a defined part of the wastewater process occurs. Process hierarchy changes preserve effective history where retained evidence depends on prior decomposition.

Examples include screening, aeration train, secondary clarification, sludge dewatering, and disinfection.

Process Unit is separate from Functional Location and Asset.

Process Stream

A **Process Stream** is a stable directional logical flow of process medium between Process Units or between a Process Unit and the modelled plant boundary.

Stream medium and directional endpoints define stable operational meaning after retained use; materially changing that meaning requires retirement and a new Stream identity.

A Process Stream is not a physical pipe or channel.

Measurement Point

A **Measurement Point** is the stable semantic identity of an operational quantity that Waiotech understands can be observed on one supported plant subject.

It defines quantity, canonical unit, subject, and optional instrument/source relationships.

Measurement Point is not a Reading and is not an external SCADA tag. Subject and measured quantity define semantic identity after operational use; instrument and machine-source relationships may change with preserved effective history.

Data Source

A **Data Source** is one governed external source or namespace relevant to plant data.

Data Source identifies where machine evidence comes from. It is not the machine Actor.

Measurement Source Mapping

A **Measurement Source Mapping** is the governed effective relationship that maps one Data Source signal or key to one Measurement Point for machine Process Reading ingestion.

It is distinct from a general External Identifier Mapping and does not replace Measurement Point identity. Alpha permits at most one active machine-ingress mapping per Measurement Point at one effective instant.

External identifier mapping

An **external identifier mapping** associates a source-scoped external identifier with one canonical supported Plant Model object while preserving canonical Waiotech identity.

Process Reading

A **Process Reading** is immutable structured Process evidence that one value was observed for one Measurement Point.

It preserves reported and canonical unit meaning, effective time, recorded time, Actor, source/provenance, and quality.

Use `Reading` in Process context where unambiguous. Do not use `meter reading` as a separate Maintenance authority.

Reading quality

Canonical Reading quality values are:

- `good` ;
- `uncertain` ;
- `bad` ;
- `unknown` .

Quality describes confidence in the value, not whether the value is operationally desirable.

Operational Observation

An **Operational Observation** is immutable qualitative or contextual Process evidence about plant operation.

Observation is distinct from Process Reading and Process Condition.

Process Routine

A **Process Routine** is the stable identity of a reusable human operating round or log-sheet definition.

A **Process Routine Revision** is immutable published content defining ordered Reading and Observation collection entries and optional calendar recurrence. One effective published revision governs subsequent Round creation for an active Routine.

Process Round

A **Process Round** is the accountable execution record for one Process Routine Revision at one scheduled occurrence or authorized ad-hoc start. Readings and Observations created through it remain their own canonical Process evidence.

Process Condition

A **Process Condition** is a meaningful operational situation concerning one Process Unit or Process Stream that requires active handling, continued monitoring, or explicit resolution.

A Process Condition is not every alarm, abnormal Reading, defect, Finding, or Failure Event.

Process attention

Process attention describes the required operational response intensity and timeliness for one active or monitoring Process Condition.

Canonical values are `routine`, `elevated`, `urgent`, and `critical`.

Process attention is distinct from Failure Event consequence, Maintenance urgency, Maintenance priority, Operational Criticality, and Reading quality.

Operational Action

An **Operational Action** is immutable evidence of what Operations deliberately changed or did in response to plant context or a Process Condition.

It is not physical maintenance work merely because equipment participates.

Outcome Assessment

An **Outcome Assessment** is immutable Process evidence of the observed operational result attributable to one Operational Action.

Canonical values are `improved`, `no_meaningful_change`, `worsened`, and `inconclusive`.

Outcome Assessment is separate from Process Condition resolution.

Integration Principal

An **Integration Principal** is a Tenant-owned machine Actor with narrowly granted external automation authority.

It is not a User, Membership, Team, platform User, or Data Source.

Actor

An **Actor** is the attributable identity under which a consequential accepted action occurred.

Canonical Actor categories include human User, Integration Principal, exact internal system Actor, and platform User acting through their separate authority paths.

Work Request

A **Work Request** is an unaccepted proposed maintenance need subject to accountable triage.

It may originate from a person, Process Condition, Finding, Failure Event, external evidence, or another governed workflow.

Accepted maintenance need

An **accepted maintenance need** is represented by one or more Work Orders. There is no second generic accepted-maintenance aggregate.

Work Order

A **Work Order** is the authoritative record of accepted maintenance work and its planning, execution, completion, and maintenance verification.

Work Target

A **Work Target** is the one primary canonical subject of a Work Order before release or emergency start.

Supported Work Target types are:

- Functional Location;
- Asset.

Finding

A **Finding** is an independently identifiable maintainable condition discovered through Maintenance, inspection, completion, verification, or another governed evidence context and requiring explicit Maintenance disposition.

Finding is not automatically a Failure Event or Work Order.

Failure Event

A **Failure Event** is the authoritative Reliability record of actual loss or materially unacceptable degradation of required function.

Its primary target is one Functional Location or Asset.

Failure Event is separate from Process Condition, Finding, Work Request, and Work Order.

Cause Assessment

A **Cause Assessment** is immutable Reliability evidence of one accepted conclusion about why a Failure Event occurred. It may be confirmed, probable, contributory, inconclusive, or no-cause-found according to evidence.

Procedure

A **Procedure** is the stable identity of a reusable governed maintenance method.

A **Procedure Revision** is immutable published method content.

Maintenance Plan

A **Maintenance Plan** is the stable identity of governed recurring maintenance intent.

A **Maintenance Plan Revision** is immutable published policy defining exact targets, trigger, method, timing, and generation defaults.

Measurement Point trigger

A **Measurement Point trigger** is a deterministic Maintenance Plan trigger based on accepted Process Readings and a governed threshold sequence. Each resolved Plan target preserves exactly one compatible trigger Measurement Point bound immutably by the published Plan Revision.

Do not use `meter` as a separate canonical Maintenance source concept.

Scheduled Work

Scheduled Work is the durable identity of one recognized Maintenance Plan occurrence for one resolved target and one nominal trigger instance.

It is not executable maintenance; the resulting Work Order owns execution.

Readiness

Readiness is a derived evaluation of whether one named Work Order action can proceed under current authoritative facts.

Readiness is not a Work Order lifecycle state.

Prepared and released

Prepared means the Work Order has reached the defined planning state required before release evaluation.

Released means execution has been authorized according to the governing Maintenance rules.

Neither term is interchangeable with Readiness.

Priority and urgency

Urgency describes time sensitivity or immediacy of response.

Priority orders work relative to other work under the product's prioritization rules.

They are distinct.

Execution risk

Execution risk describes hazards and consequences of performing a specific activity in its execution context.

It is distinct from Operational Criticality and deferral consequence.

Deferral consequence

Deferral consequence describes the consequence of delaying a specific maintenance obligation.

Inventory Item

An **Item** is a Tenant-owned material identity controlled by Inventory.

Stock Location

A **Stock Location** is an Inventory-controlled custody node.

It is distinct from Functional Location even when it optionally references one for physical placement.

Inventory Movement

An **Inventory Movement** is an immutable posted custody fact changing tracked stock quantity or condition according to Inventory rules.

Reservation

A **Reservation** protects a quantity of an Item at a Stock Location for an authorized demand without moving physical stock.

Material usage

The consuming domain owns why material was used; Inventory owns physical custody.

Canonical usage concepts include Maintenance Material Usage and Process Material Usage where defined.

Team, Permission, Access Profile, and Access Grant

A **Team** is a Tenant-owned durable group used for responsibility and human authority assignment.

A **Permission** is an atomic allow-only capability identity.

An **Access Profile** is a stable Tenant-owned collection of human authority backed by immutable Access Profile Revisions.

An **Access Grant** is one Permission included in an Access Profile Revision.

A **Team Access Profile Assignment** is the ordinary human path assigning one Access Profile to one Team for an effective period.

Waiotech has no canonical `Role` entity.

Decision

Decision is immutable evidence of one governed judgment owned by its business context. Waiotech does not force all Decisions into one generic polymorphic aggregate.

Attachment and evidence

An **Attachment** is governed accepted file evidence with immutable accepted bytes, classification, source, subject relationship, and lifecycle according to evidence Product Authority.

Tenant Dashboard and Platform Admin

Tenant Dashboard is the professional operating workspace of one plant.

Platform Admin is the separate platform-control application and must not become an alternate Tenant authority path.

Android Work App

Android Work App is the governed field surface for supported Process and Maintenance workflows and bounded offline authority.

Related documents

- [Canonical product model](#)

- [Lifecycle summary](#)

Canonical product model

This reference consolidates the authoritative Waiotech V1 product model. Owning Product Authority remains controlling when a summary is incomplete.

Product surfaces

```
Waiotech
├─ Server and workers
├─ Tenant Dashboard
├─ Platform Admin
├─ Android Work App
├─ Public Website
└─ Documentation and Help Center
```

One Tenant represents one wastewater plant.

Top-level Tenant model

```
Tenant
├─ IAM and policy
├─ Plant Model
├─ Process Operations
├─ Maintenance
├─ Inventory
├─ Reliability
├─ Evidence and communications
└─ Tenant lifecycle and portability
```

Plant Model

```
Plant Model
├─ Functional Location
│   └─ child Functional Location    effective containment history where required
├─ Asset
│   └─ child Asset                physical composition with effective history
├─ Asset Installation
│   ├── directly installed root Asset
│   └─ Functional Location
│       └─ composed child Assets derive installed context through Asset composition
├─ Asset Classification
└─ Operational Criticality        Functional Location context
```

- └─ Process Unit
 - └─ child Process Unit effective process-decomposition history where required
- └─ Process Stream
 - └─ governed process medium
 - └─ upstream Process Unit or plant boundary
 - └─ downstream Process Unit or plant boundary
- └─ Functional Location serves Process Unit
- └─ Measurement Point
 - └─ measured quantity
 - └─ canonical unit
 - └─ subject
 - └─ Functional Location
 - └─ Asset
 - └─ Process Unit
 - └─ Process Stream
 - └─ optional instrument Asset
 - └─ Measurement Source Mappings
- └─ Data Source
- └─ Measurement Source Mapping
 - └─ effective Data Source signal → Measurement Point relationship
 - └─ at most one active machine-ingress mapping per Measurement Point
- └─ External Identifier Mapping

Functional Location, Asset, Process Unit, Process Stream, and Measurement Point are distinct identities. Waiotech does not use one universal Plant Object aggregate.

Each has a required name and may have an optional plant code unique case-insensitively within the Tenant and that entity category. Code history remains separate from immutable identity.

Process Operations

- Process Operations
 - └─ Process Reading
 - └─ Measurement Point
 - └─ reported value and unit
 - └─ canonical normalized value where required
 - └─ effective time
 - └─ recorded time
 - └─ Actor
 - └─ provenance / Data Source
 - └─ quality
 - └─ additive correction
 - └─ Operational Observation
 - └─ exactly one primary Process Unit / Process Stream / Functional Location / Asset
 - └─ narrative evidence
 - └─ related Readings / Measurement Points / plant context / attachments
 - └─ Process Routine
 - └─ immutable Process Routine Revision
 - └─ Reading / Observation collection entries
 - └─ Process Round
 - └─ scheduled occurrence or ad-hoc execution
 - └─ links to canonical Readings / Observations
 - └─ Process Condition
 - └─ Process Unit or Process Stream

- |— active / monitoring / resolved
- |— routine / elevated / urgent / critical attention
- |— owner Team
- |— optional responsible User
- |— evidence
- |— resolution
- |— Operational Action
 - |— exactly one primary Process Unit or Process Stream
 - |— optional Process Condition
- |— Outcome Assessment
 - |— exactly one Operational Action
 - |— improved / no_meaningful_change / worsened / inconclusive
- |— Process Material Usage

Human Reading entry is first-class. Machine submission uses Integration Principal and Data Source authority but creates the same Process Reading concept.

Maintenance

- Maintenance
 - |— Work Request
 - |— Triage Decision
 - |— Finding
 - |— explicit disposition
 - |— Procedure
 - |— Procedure Revision
 - |— Maintenance Plan
 - |— Maintenance Plan Revision
 - |— Functional Location or Asset target kind
 - |— immutable resolved targets
 - |— calendar or Measurement Point trigger
 - |— per-target trigger Measurement Point binding when applicable
 - |— Scheduled Work
 - |— Work Order
 - |— Work Target
 - |— Functional Location
 - |— Asset
 - |— Tasks
 - |— Work Requirements
 - |— responsibility
 - |— Readiness projections
 - |— Maintenance Material Requirement / Usage
 - |— execution evidence
 - |— completion submission
 - |— Verification Decisions
 - |— closeout

Work Order is the only representation of accepted maintenance work.

Process Conditions and Failure Events may originate or support Maintenance work but remain source-domain authority.

Inventory

```

Inventory
├── Item
│   ├── Item Category
│   ├── Item Policy
│   └── allowed Unit Conversions
├── Stock Location
├── Inventory Movement
├── Stock Balance                derived
├── Reservation
├── availability                derived
├── Replenishment Request
├── Stocktaking Run
├── Maintenance Material Usage relationship
└── Process Material Usage relationship
  
```

Inventory owns physical custody. The consuming domain owns why material was required or used.

Reliability

```

Reliability
├── Failure Event
│   ├── primary Functional Location or Asset
│   ├── required function
│   ├── failure mode
│   ├── occurrence and detection timing
│   ├── operational condition
│   ├── investigation condition
│   ├── consequence
│   ├── restoration history
│   ├── Process evidence
│   ├── supporting Findings
│   ├── linked Work Requests / Work Orders
│   ├── Cause Assessments
│   └── recurrence / duplicate relationships
└── reliability analytics        derived
  
```

Failure Event records what failed and its consequence. Work Order records maintenance response. Process Condition records operational handling. They remain distinct.

IAM

```

User
├── Membership
│   ├── Team Membership
│   │   └── Team
│   │       ├── Team Access Profile Assignment
│   │       │   └── Access Profile
│   │       │       ├── published Access Profile Revision
│   │       │       │   ├── Access Grant
│   │       │       │   └── Permission
│   └── Team
  
```

```
Integration Principal
├─ Tenant
├─ explicit machine Permission
├─ machine credential history
├─ authorized Data Source relationship

platform User
├─ explicit platform Permission
```

Human, machine, internal-system, and platform authority paths are separate.

Policy and Decisions

```
Tenant Control Policy
├─ immutable published Tenant Control Policy Revision

Domain-owned Decisions
├─ triage
├─ verification
├─ waiver
├─ stocktaking approval
├─ Failure Event investigation requirement
├─ other explicitly defined governed judgments
```

Waiotech does not use generic resource-scoped policy or one universal Decision lifecycle.

Evidence and communications

```
Attachment
Report Type / Report Run
Notification
Audit access and operational history projections
```

Accepted evidence remains attributable, historically interpretable, and bounded by owning-domain subject relationships.

Tenant portability

Tenant export and import preserve canonical identities, relationships, source history, installation history, Process evidence, Maintenance, Inventory, Reliability, files, authority evidence, and required integrity metadata according to portability Product Authority.

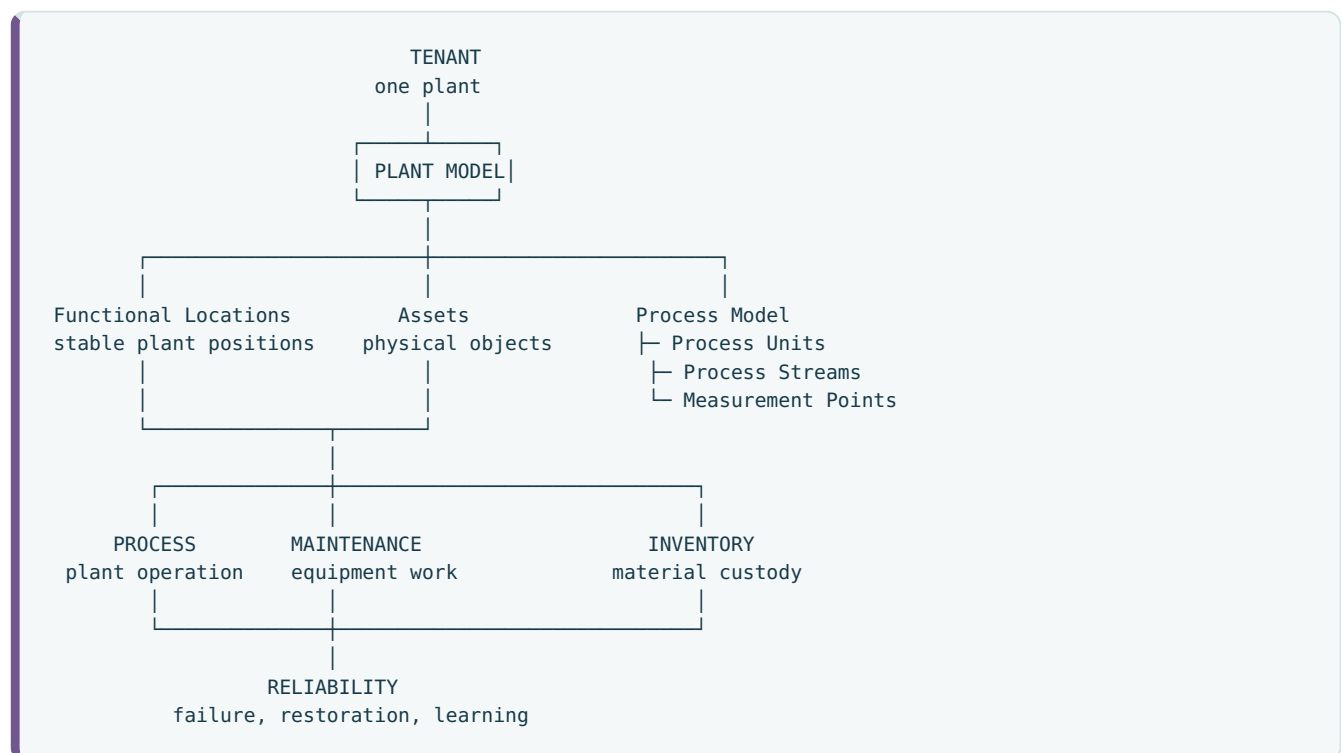
Explicitly excluded generic concepts

The canonical model excludes:

- Site inside Tenant;

- Unit / Equipment / Component as mandatory Asset structural types;
- generic Plant Object;
- generic Entity Relationship authority;
- generic Measurement or Event aggregate spanning all domains;
- generic Activity or Workflow engine;
- generic Condition engine;
- arbitrary policy scope;
- generic dashboard builder;
- SCADA control or historian replacement.

High-level model



Related documents

- [Canonical terminology](#)
- [Lifecycle summary](#)
- [Domain and plant model](#)

Lifecycle summary

Lifecycle states represent durable business conditions of specific entities. Derived meaning such as Readiness, availability, attention, overdue state, effective authority, and overall failure resolution remains separate unless Product Authority explicitly defines a state.

Lifecycle principle

Use:

- lifecycle states for durable entity condition;
- Decisions for governed judgment;
- dispositions for explicit resolution;
- effective periods for time-bounded relationships;
- immutable records for evidence;
- derived conditions for calculated meaning.

Tenant

```
provisioning → active  
active → suspended → active  
provisioning / active / suspended → deactivated
```

Deactivated is terminal for ordinary operation and does not delete retained data.

Functional Location

```
active → retired
```

Retirement preserves all installation, Maintenance, Process, Inventory-reference, and Reliability history while preventing new ordinary current use.

Asset

```
inactive → active  
active → inactive  
active / inactive → decommissioned
```

Asset lifecycle is separate from installation and temporary operating condition. Decommissioned is terminal.

Asset installation

Asset installation uses effective periods rather than a lifecycle status. Only root physical Assets are directly installed; composed children derive installed Functional Location through effective physical-composition history.

installed_at t1 — removed_at t2

The relationship is immutable historical evidence after it ends. Reinstallation creates another effective period.

Asset Classification and Operational Criticality

Governed catalogue values generally use:

active → retired

Operational Criticality changes preserve effective history where required for historical decisions. It is not an Asset lifecycle.

Process Unit

active → retired

Retirement preserves Process history and requires active child units, Streams, Measurement Points, and open Conditions to be resolved according to their owning rules.

Process Stream

active → retired

Retirement prevents new ordinary Process use while preserving historical Readings, Conditions, and topology evidence.

Measurement Point

active → retired

Retirement prevents new ordinary Readings while preserving prior Readings, Measurement Source Mappings, Maintenance trigger evidence, and Reports.

Data Source

active → retired

Retirement prevents new machine source use but preserves prior mapping and Reading provenance.

Process Reading

Process Reading is immutable evidence and has no lifecycle.

Correction is additive. A correction may invalidate an earlier Reading and establish a replacement without editing the original.

Process Routine

inactive → active
active → paused → active
inactive / active / paused → retired

Only active Routines generate ordinary scheduled Rounds. Paused intervals do not create retroactive obligations. Retired is terminal.

Process Routine Revision uses:

draft → published → superseded
draft → discarded

Published revisions are immutable.

Process Round

open → in_progress → completed
open / in_progress → cancelled

Due and overdue are derived, not lifecycle states. Completion means required collection work was accepted; it does not resolve Process Conditions automatically.

Operational Observation

Operational Observation is immutable evidence and has no lifecycle. Correction is additive.

Process Condition

```
active → monitoring → resolved
active → resolved
monitoring → active
resolved → active
```

- **Active:** immediate intervention, decision, or coordination remains required.
- **Monitoring:** no immediate intervention is required, but explicit continued observation and review remain.
- **Resolved:** no active Process obligation remains.

Reopen preserves prior resolution evidence.

Process attention is a separately governed current condition attribute, not a lifecycle state. Canonical values are `routine`, `elevated`, `urgent`, and `critical`; changes preserve attributable history.

Operational Action and Outcome Assessment

Operational Action and Outcome Assessment are immutable evidence records, not lifecycle entities.

Work Request

```
submitted → under_review
under_review → more_information_required
more_information_required → under_review
under_review → deferred
deferred → under_review
under_review → accepted
under_review → rejected
under_review → duplicate
submitted / under_review / more_information_required / deferred → cancelled
```

Terminal states are accepted, rejected, duplicate, and cancelled.

Acceptance establishes or links the Work Orders representing accepted maintenance need.

Procedure

Stable Procedure identity:

active → retired

Procedure Revision:

draft → published → superseded
draft → discarded
published → withdrawn

Published revisions are immutable.

Maintenance Plan

inactive → active
active → paused → active
inactive / active / paused → retired

Maintenance Plan Revision

draft → published → superseded
draft → discarded

Published revisions are immutable.

Scheduled Work

recognized → deferred → recognized
recognized / deferred → generated
recognized / deferred → skipped
recognized / deferred → cancelled

Generated, skipped, and cancelled are terminal. Overdue and missed are derived.

Work Order

draft → prepared → released → in_progress → completed → closed
prepared → draft

```
released → prepared
in_progress → on_hold → in_progress
completed → in_progress
draft / prepared / released / in_progress / on_hold → cancelled
```

`completed → in_progress` occurs through distinct `withdraw_completion` or `return_for_rework` actions.

A governed emergency-start action may move `draft`, `prepared`, or `released` directly to `in_progress` under the defined emergency contract.

Work Order Readiness

Readiness is derived for a named action and is not a Work Order lifecycle state.

Finding

Finding uses explicit disposition rather than a broad lifecycle.

Canonical dispositions include:

- resolved within governing work;
- no action required;
- monitor;
- Work Request created;
- Work Order created or linked;
- duplicate;
- invalid or corrected.

Failure Event operational condition

```
active_failure → temporarily_restored → operationally_resolved
active_failure → operationally_resolved
temporarily_restored → active_failure
```

Failure Event investigation condition

```
not_required → required
required → in_progress → concluded
concluded → in_progress
```

New evidence may change `not_required → required`. A concluded investigation may be explicitly reopened while preserving the prior conclusion.

Overall Failure Event open/resolved is a derived projection over operational condition, investigation, follow-up, and blockers.

Cause Assessment

Cause Assessment is immutable evidence. A later assessment may explicitly supersede an earlier assessment without editing it.

Team

active → retired

Integration Principal

active → revoked

Revocation is terminal for that machine identity. New credentials do not change principal identity; a new machine identity requires a new Integration Principal.

Access Profile

Stable Access Profile identity:

active → retired

Access Profile Revision:

draft → published → superseded
draft → discarded

Tenant Control Policy Revision

draft → published → superseded
draft → discarded

Stock Location

active → retired

Retirement requires active custody obligations to be resolved.

Reservation

active → fulfilled
active → released
active → expired
active → cancelled

Partial consumption changes remaining quantity without another lifecycle state.

Replenishment Request

active → fulfilled
active → cancelled

Partial fulfillment is derived from linked receipts.

Stocktaking Run

counting → submitted → approved → posted
submitted → returned_for_recount → counting
counting / submitted / returned_for_recount → cancelled

Report Run

queued → running → completed
queued → cancelled
queued / running → failed
running → cancelled

Tenant Export

```
queued → running → completed  
queued → cancelled  
queued / running → failed  
running → cancelled
```

Notification

```
unread → read
```

Related business action belongs to the subject, not Notification lifecycle.

Work Requirement outcome

```
unsatisfied  
satisfied  
waived  
not_applicable  
expired
```

These are control outcomes, not an independent workflow lifecycle.

Derived conditions

Do not store these as independently editable lifecycle truth:

- Work Order Readiness;
- overdue Work Order;
- missed Scheduled Work;
- Stock Balance;
- available stock;
- effective Permissions;
- effective Access Profile Revision;
- Failure Event overall open/resolved projection;
- Report metrics;
- reliability analytics;
- maintenance compliance projections.

Related documents

- [Canonical terminology](#)
- [Canonical product model](#)